Cubical Cohomological Type Theory for Program Semantics

Algorithm Equivalence, Specification Checking,
Bug Finding, and Automatic Spec Inference
via Fibered Categories, Čech Cohomology,
and Custom SMT Theories

Deppy Project

April 13, 2026

Abstract

We develop *Cubical Cohomological Type Theory* (CTTT), a framework for program semantics that unifies algorithm equivalence, specification checking, bug finding, and automatic specification inference under a single mathematical theory.

Programs are compiled to terms in a *semantic algebra* — a Z3 algebraic theory equipped with equational axioms that serve as cubical path constructors. The type fibration $\pi : \mathsf{PyObj} \to \mathsf{TypeTag}$ decomposes the input space into fibers, and genuine Čech cohomology (the full complex $C^0 \to C^1 \to C^2$ with coboundary operators δ^0, δ^1 and GF(2) rank computation) determines whether local results glue to global verdicts.

The key insight is that neither cubical type theory nor cohomology theory alone suffices for program semantics. Cubical paths bridge structural gaps between programs (recursion $\leftrightarrow$ iteration, comprehension $\leftrightarrow$ loop, etc.) but cannot decompose by type. Cohomology decomposes by type fiber but cannot bridge structural gaps. Their synthesis yields eleven new results, including automatic specification inference from unannotated code, cohomological bug severity quantification, semantic distance metrics, repair localization, and compositional verification via sheaf gluing.

We extend the theory to non-deterministic specifications via quotient types and set-level truncation, and identify twenty-four computational dichotomies that generate the cubical path space.

Contents

I Cubical Type Theory **13**

1 The Interval, Faces, and Connections **14**
1.1 The Abstract Interval $\mathbb{I}$ . . . 14
1.2 Faces and Face Lattice . . . 15
1.3 The Face Map and Degeneracy . . . 16
1.4 Connections and Their Computational Meaning . . . 17
1.5 Cubical Sets and the Program Cubical Set . . . 17

2 Path Types and Function Extensionality **20**
2.1 Path Types . . . 20
2.2 Function Extensionality . . . 21
2.3 Dependent Paths and Transport . . . 24
2.4 Higher Paths and Truncation . . . 25

3 The Univalence Axiom **26**
3.1 Type Equivalences . . . 26
3.2 The Univalence Axiom . . . 26
3.3 Transport and Its Computational Content . . . 27
3.4 Univalence for Program Semantics . . . 27

4 Higher Inductive Types for Programs **29**
4.1 Definition and Motivation . . . 29
4.2 The HIT PyComp . . . 29
4.3 The 0-Truncation Condition . . . 32
4.4 The Elimination Principle . . . 33
4.5 The Free Algebra Interpretation . . . 34

5 Kan Operations and Composition **35**
5.1 Open Boxes and Fillers . . . 35
5.2 Composition . . . 35
5.3 Kan Filling as Multi-Step Rewriting . . . 35
5.4 Bidirectional BFS as Kan Search . . . 36
5.5 Composition for Multi-Step Proofs . . . 37

II Cohomology Theory **38**

6 Sites, Presheaves, and Sheaves **39**
6.1 The Type Site . . . 39
6.2 Duck Type Inference and the Grothendieck Topology . . . 39
6.3 Tag Constraints and Z3 Sections . . . 40
6.4 Value Presheaves . . . 40

6.5 The Sheaf Condition . . . 41
6.6 The Isomorphism Sheaf . . . 41
6.7 Worked Example: Sheaf-Theoretic Proof . . . 42

7 The Čech Complex 43
7.1 Cochain Groups . . . 43
7.2 Coboundary Operators . . . 43
7.3 Cohomology Groups . . . 44
7.4 Computation over GF(2) . . . 45

8 The Mayer–Vietoris Sequence 46
8.1 Setup and Statement . . . 46
8.2 The Connecting Homomorphism . . . 46
8.3 Application: Type-Fiber Splitting . . . 47

9 Stalks and the Stalk Criterion 48
9.1 Stalks of the Value Presheaf . . . 48
9.2 The Stalk Criterion . . . 48
9.3 Symbolic Stalks . . . 49

III The Cubical–Cohomological Synthesis 50

10 Why Neither Alone Suffices 51
10.1 What Cubical Type Theory Alone Can Do . . . 51
10.2 What Čech Cohomology Alone Can Do . . . 52
10.3 Example: eq02 — Different OTerms, $H^1 = 0$ . . . 52
10.4 What the Combination Achieves . . . 53

11 New Results Enabled by the Synthesis 55
11.1 Strict Power of the Synthesis . . . 55
11.2 The Decidability Stratification . . . 56
11.3 The 7-Phase Normalizer as a Convergent Rewrite System . . . 56
11.4 Soundness of the Full Pipeline . . . 58
11.5 The Trust Hierarchy . . . 59
11.6 Result 1: Automatic Dependent Type Inference Across Fibers . . . 59
11.7 Result 2: Specification Inference Without Annotations . . . 60
11.8 Result 3: Equivalence of Observationally Identical Programs . . . 61
11.9 Result 4: Compositional Verification via the Spectral Sequence . . . 61
11.10 Result 5: Automatic Specification Inference . . . 62
11.11 Result 6: Bug Severity Quantification . . . 63
11.12 Result 7: Program Semantic Distance . . . 64
11.13 Result 8: Automatic Repair Localization . . . 64
11.14 Result 9: Cross-Program Invariant Discovery . . . 65
11.15 Result 10: Semantic Refactoring Safety . . . 65
11.16 Result 11: Compositional Semantics via Sheaf Gluing . . . 66
11.17 False Positive Analysis . . . 66
11.18 Benchmark Case Studies: Where Synthesis Matters . . . 67
11.19 Summary: The CTTT Advantage . . . 68
11.20 Python as a Fibered Category . . . 69
11.21 Duck Typing as Cubical Transport . . . 69
11.22 Effects as Non-Trivial Cohomology . . . 69

11.23 Why This Could Not Be Done Before . . . 70

IV Advanced CTTT Machinery **72**

12 Galois Connections for Abstract Specification Inference **73**
12.1 Abstract Domains as Presheaves . . . 73
12.2 The Formal Galois Connection . . . 73
12.3 Duck Type Inference as the Abstraction Map . . . 74
12.4 Spec Identification as the Concretization Map . . . 74
12.5 How This Increases CTTT's Power . . . 75
12.6 The Galois–Čech Interaction . . . 76

13 The Descent Spectral Sequence **77**
13.1 Filtrations of the Type Site . . . 77
13.2 The Normalizer Filtration . . . 77
13.3 The E_2 Page: What Survives Normalization . . . 78
13.4 The Verification Strategy . . . 79
13.5 Convergence and Degeneration . . . 79
13.6 Non-Degenerate Cases: Type Polymorphism . . . 79

14 Operad Theory for Higher-Order Programs **81**
14.1 The Operad of Python Operations . . . 81
14.2 Operadic Composition and Phase 4 . . . 82
14.3 Operadic Path Constructors . . . 82
14.4 The Operadic Nerve and Higher Coherence . . . 83
14.5 Application: Higher-Order Equivalence . . . 83

15 The Semantic Enrichment Theorem **85**
15.1 Enrichment of the Program Category . . . 85
15.2 Enriched Cohomology . . . 85
15.3 The Enrichment–Degeneration Tradeoff . . . 86

16 Inductive Families and Dependent Pattern Matching **88**
16.1 Inductive Families in CTTT . . . 88
16.2 OTerm as an Inductive Family . . . 88
16.3 The Elimination Principle . . . 89
16.4 Application: Invariant-Indexed Specifications . . . 90
16.5 Automatic Precondition Inference via Inductive Families . . . 90
16.6 Dependent Pattern Matching as Fiber Restriction . . . 91

17 The Computable Fragment of CTTT **92**
17.1 The Three Decidability Classes . . . 92
17.1.1 What is Decidable: Canonical Forms and Structural Equality . . . 92
17.1.2 What is Semi-Decidable: Path Search . . . 92
17.1.3 What is Undecidable: General Equivalence . . . 93
17.2 Decidability Map . . . 93
17.3 The Role of Z3 . . . 94
17.4 The Decidable Core . . . 94
17.5 Extending the Decidable Fragment . . . 95

18 Automated Path Search **96**
18.1 The Path Search Problem . . . 96
18.2 The Path Graph . . . 96
18.3 The Normalization Strategy . . . 96
18.4 The Path Search Algorithm . . . 98
18.5 Path Search on Each Fiber . . . 98
18.6 Path Composition and the Proof Certificate . . . 99
18.7 Path Search Strategies . . . 100
18.7.1 Bidirectional Search . . . 100
18.7.2 Fiber-Guided Search . . . 100
18.7.3 Axiom Relevance Filtering . . . 101
18.8 The HIT Structural Prover . . . 101
18.8.1 Congruence Rules . . . 102
18.8.2 Cross-Type Path Constructors . . . 102
18.9 The Axiom Registry . . . 103
18.10 Fiber Context and Fiber-Aware Commutativity . . . 103
18.11 The Spec Identification Engine (Yoneda Embedding) . . . 104
18.12 The Bidirectional BFS Algorithm . . . 105
18.12.1 Benchmark Performance by Axiom . . . 106
18.13 Sheaf Descent for Tractability of Undecidable Problems . . . 106
18.13.1 The Undecidability Barrier . . . 106
18.13.2 How Descent Exploits Structure . . . 106
18.13.3 The Descent Decomposition Theorem . . . 107
18.13.4 Complexity Analysis . . . 107
18.13.5 Comparison with Prior Approaches . . . 108
18.13.6 The Refinement Principle . . . 108
18.13.7 Iterated Descent: The Descent Tower . . . 109
18.13.8 The Effective Descent Condition . . . 109
18.13.9 Extensions: Beyond Type Fibers . . . 110
18.14 Soundness and Completeness . . . 110
18.15 Path Search for Spec Checking . . . 111
18.16 Path Search for Bug Finding . . . 111
18.17 Path-Based Automatic Repair Suggestion . . . 112

V The Twenty-Four Dichotomies **113**

19 Overview of the Dichotomies **114**
19.1 Complete Dichotomy Table . . . 114

20 Control Flow Dichotomies (D1–D8) **116**
20.1 D1: Recursive ↔ Iterative . . . 116
20.2 D2: Nested Helpers ↔ Flat Code . . . 117
20.3 D3: `while` ↔ `for` . . . 118
20.4 D4: Comprehension ↔ Loop . . . 119
20.5 D5: `reduce` ↔ Loop . . . 120
20.6 D6: Generator ↔ Eager . . . 121
20.7 D7: Tail-Recursive ↔ Loop . . . 122
20.8 D8: Multi-Return ↔ Single Return . . . 123

21 Data Structure Dichotomies (D9–D14) 125
21.1 D9: Stack ↔ Recursion . . . 125
21.2 D10: `heapq` ↔ `bisect` . . . 126
21.3 D11: `OrderedDict` ↔ LinkedList LRU . . . 126
21.4 D12: `defaultdict` ↔ `setdefault` . . . 126
21.5 D13: `Counter` ↔ Manual Counting . . . 126
21.6 D14: Array ↔ Dict Table . . . 127

22 Algorithm Strategy Dichotomies (D15–D20) 128
22.1 D15: BFS ↔ DFS . . . 128
22.2 D16: Memoized ↔ Bottom-Up DP . . . 128
22.3 D17: Divide-and-Conquer ↔ Iterative . . . 129
22.4 D18: Greedy ↔ DP . . . 129
22.5 D19: Sort-Then-Scan ↔ Sweep . . . 129
22.6 D20: Different Algorithms for Same Specification . . . 130

23 Language Feature Dichotomies (D21–D24) 132
23.1 D21: `isinstance` ↔ Dispatch Table . . . 132
23.2 D22: `try/except` ↔ Conditional . . . 132
23.3 D23: Sorted-Then-Process ↔ Direct Processing . . . 134
23.4 D24: Lambda ↔ Named Function . . . 134

24 Deep Theory: Algorithm A ↔ Algorithm B 136
24.1 The Problem . . . 136
24.2 Specifications as Propositional Types . . . 136
24.3 Examples of Deterministic Specifications . . . 137
24.3.1 Convex Hull . . . 137
24.3.2 Shortest Paths . . . 137
24.3.3 Strongly Connected Components . . . 137
24.3.4 Edit Distance . . . 137
24.3.5 Binomial Coefficients . . . 138
24.4 The Specification as a Higher Inductive Type . . . 138
24.5 Z3 Encoding of Specifications . . . 138
24.6 Factorial Variants: The D1 Path . . . 139
24.7 Fibonacci Variants: D1 and D16 Paths . . . 140
24.8 Sorting-Then-Processing: The D19 Path . . . 141
24.9 GCD Variants: The D1 Path with Recurrence Matching . . . 141
24.10 Algorithm Equivalence via Yoneda . . . 142
24.11 The Matroid/Greedy Theorem and D20 . . . 142
24.12 Constructive Witness for D20 . . . 143
24.13 Limits of D20 . . . 144
24.14 Quotient Types for Non-Deterministic Specifications . . . 144
24.14.1 Z3 Encoding of Quotient Types . . . 145
24.14.2 Example: Spanning Tree . . . 146
24.14.3 Example: Topological Sort . . . 146
24.15 Propositional Extensionality . . . 146
24.16 Set-Level Truncation . . . 147
24.17 Z3 Encoding of Truncated Specifications . . . 147
24.17.1 Fiber-Wise Truncated Spec Checking . . . 148
24.18 The "Hard 8" Analysis . . . 148
24.18.1 Why Structural Paths Fail . . . 148
24.18.2 The Common Pattern . . . 149

24.19 The Structural/Extensional Boundary . . . 149
24.20 The Full Benchmark Classification . . . 150

VI The Semantic Algebra 153

25 The PyObj Z3 Theory: A Complete Semantic Model of Python 154
25.1 The Value Universe . . . 154
25.1.1 The Seven Constructors . . . 154
25.1.2 Constructor–Destructor Pairs . . . 155
25.1.3 Why Seven Constructors Suffice . . . 156
25.1.4 Decidability Preservation . . . 156
25.2 The Type Fibration . . . 156
25.2.1 TypeTag Sort . . . 156
25.2.2 Fiber Projection Lemmas . . . 157
25.3 Operations: Full Python Semantics in Z3 . . . 157
25.3.1 Binary Operations . . . 157
25.3.2 Unary Operations . . . 159
25.3.3 The Truthiness Protocol . . . 160
25.3.4 The Canonical Fold (Nat-Recursor) . . . 160
25.4 The RecFunction System . . . 161
25.4.1 From Python AST to Z3 RecFunction . . . 161
25.4.2 Canonical Fold Detection (Nat-Eliminator Extraction) . . . 162
25.4.3 RecFunction Limitations . . . 162
25.5 The Uninterpreted Opacity Zone . . . 163
25.5.1 The Interpreted–Uninterpreted Spectrum . . . 163
25.5.2 The Opacity Boundary . . . 163
25.5.3 Example: `sorted(list(set(x)))` in the Opacity Zone . . . 164
25.5.4 Item Access (GETITEM) . . . 164
25.6 Container Semantics via Shared Functions . . . 164
25.6.1 Dictionary Operations . . . 165
25.6.2 List Operations . . . 165
25.6.3 Set Operations . . . 166
25.7 The Heap Model . . . 166
25.7.1 Heap Representation . . . 166
25.7.2 Aliasing and the Effect Discipline . . . 166
25.8 Python Control Flow in Z3 . . . 167
25.8.1 Conditional Expressions . . . 167
25.8.2 Short-Circuit Boolean Operators . . . 167
25.8.3 Exception Semantics via Bottom . . . 167
25.8.4 isinstance as Fiber Projection . . . 167
25.8.5 Comprehensions . . . 168
25.8.6 Attribute Access . . . 168
25.9 Shared Function Symbols (Univalence) . . . 168
25.9.1 The Registry . . . 168
25.9.2 Builtin Classification . . . 168
25.9.3 Why Sharing Preserves Decidability . . . 168
25.9.4 The Extensibility Principle . . . 170

26 The Semantic Algebra: Complete Axiomatization 171
26.1 The Integer Ring . . . 171
26.1.1 Ring Axioms as Path Constructors . . . 171
26.1.2 Z3 Encoding . . . 172
26.1.3 What the Ring Axioms Enable . . . 173
26.2 The Lattice of Min and Max . . . 173
26.3 The Sequence Algebra . . . 174
26.3.1 The Free Monoid . . . 174
26.3.2 Sorting as a Retraction . . . 175
26.3.3 Reversal as an Involution . . . 175
26.3.4 Length as a Monoid Homomorphism . . . 175
26.3.5 Fold as Universal Catamorphism . . . 176
26.3.6 Sum and Product as Specific Folds . . . 176
26.4 The Boolean Algebra . . . 176
26.5 The Idempotent Semiring . . . 177
26.6 The Effect Algebra . . . 178
26.7 The Composed Algebra . . . 178
26.8 Interaction Between Sub-Algebras . . . 179
26.8.1 Ring–Fold Interaction . . . 179
26.8.2 Lattice–Fold Interaction . . . 179
26.8.3 Sequence–Boolean Interaction . . . 179
26.9 Soundness of the Z3 Theory . . . 179
26.9.1 The Supported Fragment . . . 180
26.9.2 The Denotational Semantics . . . 180
26.9.3 The Soundness Theorem . . . 180
26.10 The General Extraction Algorithm . . . 181
26.11 From `for i in range(n)` to OFold . . . 182
26.12 Formal Extraction Theorem . . . 182
26.13 Commutativity and Argument Order . . . 182

27 Iteration Beyond Range: Collection Catamorphisms 183
27.1 The List Catamorphism . . . 183
27.1.1 The Map-via-Append Pattern . . . 184
27.2 The General Catamorphism . . . 184
27.3 Dict Iteration . . . 184
27.4 String Iteration . . . 184
27.5 Composed Iterators . . . 185
27.6 Generator Consumption . . . 185

28 The Recursion Zoo: Beyond Nat Descent 186
28.1 Primitive Recursion (Nat Descent) . . . 186
28.2 Course-of-Values Recursion . . . 186
28.3 Tree Recursion . . . 186
28.4 Multi-Parameter Recursion (2D) . . . 187
28.5 Mutual Recursion . . . 187
28.6 Well-Founded Recursion with Fuel . . . 188

29 Higher-Order Functions: Compilation and Normalization 189
29.1 The OLam and OMap Constructors . . . 189
29.2 Compilation Rules . . . 189
29.3 Function Composition Normalization . . . 190
29.4 HOF Normalization Rules . . . 190

30 Complete Normalization Traces **191**
30.1 The Seven Phases . . . 191
30.2 Trace 1: Factorial (Rec → Fold) . . . 192
30.3 Trace 2: Sum of Squares . . . 192
30.4 Trace 3: Edit Distance (2D Recurrence) . . . 193
30.5 Trace 4: all vs any (De Morgan) . . . 193
30.6 Trace 5: Sorted Unique . . . 193
30.7 Trace 6: Absolute Value (Phase 7 Piecewise) . . . 193

VII Specification Checking and Bug Finding via CTTT **195**

31 The Three Problems, One Framework **196**
31.1 The Unification Theorem . . . 196

32 Specification Checking **198**
32.1 Specifications as Dependent Types . . . 198
32.2 Specifications from Natural Language . . . 198
32.3 Fiber-Wise Spec Checking . . . 199
32.4 Example: Sorting Specification . . . 199
32.5 Example: Fibonacci Specification . . . 200
32.6 Spec Checking via Equivalence . . . 200

33 Bug Finding via Cohomological Obstruction **201**
33.1 Bugs as H^1 Obstructions . . . 201
33.2 Bug Localization . . . 202
33.3 Bug Classification by Cohomological Degree . . . 202
33.4 The Bug-Finding Pipeline . . . 203
33.5 Example: Off-by-One in Range . . . 203
33.6 Example: Mutation Bug (False Aliasing) . . . 203
33.7 Extended Example: Off-by-One via H^1 on the Int Fiber . . . 204

34 Unified Theory: Equivalence, Specs, and Bugs **205**
34.1 The Verification Triangle . . . 205
34.2 The Universal Verification Query . . . 205
34.3 Formal Unification . . . 206
34.4 Composition: Spec-Guided Equivalence . . . 206
34.5 Composition: Equivalence-Guided Bug Finding . . . 206
34.6 The Graded Verification Verdict . . . 207
34.7 Confidence Scoring . . . 207

VIII Implementation Architecture **208**

35 The Verification Pipeline **209**
35.1 Pipeline Overview . . . 209
35.2 Strategy S0: Closed-Term Evaluation . . . 209
35.3 Strategy S1: Denotational OTerm Equivalence . . . 210
35.4 Strategy S2: Z3 Structural Equality . . . 210
35.5 Strategy S3: Semantic Fingerprints . . . 210
35.6 Strategy S4: Per-Fiber Z3 + Čech H^1 . . . 210
35.6.1 Step 4a: Duck Type Inference . . . 210
35.6.2 Step 4b: Site Category Construction . . . 211

35.6.3 Step 4c: Local Equivalence Queries . . . 211
35.6.4 Step 4d: Čech Cohomology . . . 211
35.6.5 Step 4e: Verdict and Confidence . . . 211
35.7 AST Compilation . . . 211
35.8 Section Extraction . . . 212
35.9 Site Category Construction . . . 212
35.10 Local Equivalence Check . . . 212
35.11 Čech Cohomology Computation . . . 213

36 The AST Compiler in Detail 214
36.1 Compilation State . . . 214
36.2 The Section Representation . . . 215
36.3 Expression Compilation Rules . . . 215
36.3.1 Rule E1: Literals . . . 215
36.3.2 Rule E2: Variables . . . 215
36.3.3 Rule E3: Binary Operations . . . 215
36.3.4 Rule E4: Unary Operations . . . 216
36.3.5 Rule E5: Comparisons (Chained) . . . 216
36.3.6 Rule E6: Boolean Operations . . . 216
36.3.7 Rule E7: Conditional Expression . . . 216
36.3.8 Rule E8: Function Calls . . . 216
36.3.9 Rule E9: Subscript . . . 217
36.3.10 Rule E10: Tuple/List Display . . . 217
36.3.11 Rule E11: Comprehension . . . 217
36.3.12 Rule E12: Attribute Access . . . 217
36.3.13 Additional Expression Rules . . . 217
36.4 Statement Compilation Rules . . . 217
36.4.1 Rule S1: Assignment . . . 218
36.4.2 Rule S2: Augmented Assignment . . . 218
36.4.3 Rule S3: Return . . . 218
36.4.4 Rule S4: If/Elif/Else . . . 218
36.4.5 Rule S5: For Loop . . . 218
36.4.6 Rule S6: While Loop . . . 219
36.4.7 Rule S7: Function Definition . . . 219
36.4.8 Rule S8: Expression Statement (Mutation) . . . 219
36.4.9 Rule S9: Try/Except . . . 219
36.5 The Section Extractor . . . 220
36.6 The Nat-Eliminator Extraction . . . 220
36.7 What the Compiler Cannot Handle . . . 221

37 Duck Type Inference in Detail 222
37.1 The Duck Type Lattice . . . 222
37.2 The Inference Algorithm . . . 222
37.3 Operation Extraction from AST . . . 223
37.4 Formal Connection to Galois Connections . . . 224
37.5 Fiber Narrowing and the Site Category . . . 225

38 The Čech Complex Construction Algorithm 226
38.1 Data Structures . . . 226
38.2 Step 1: Local Equivalence Judgments . . . 226
38.3 Step 2: The Short-Circuit . . . 227
38.4 Step 3: Building C^0 . . . 228

38.5 Step 4: Computing $\delta^0 : C^0 \to C^1$ . . . 228
38.6 Step 5: Computing $\delta^1 : C^1 \to C^2$ . . . 228
38.7 Step 6: Computing H^1 via GF(2) Rank . . . 229
38.8 The GF(2) Rank Algorithm . . . 229
38.9 Correctness and Complexity . . . 229

IX Practical Engineering 231

39 End-to-End Pipeline Trace 232
39.1 Example 1: Recursive vs. Iterative Sum (EQUIVALENT) . . . 232
39.1.1 The Input Pair . . . 232
39.1.2 Step 1: Parse to AST . . . 232
39.1.3 Step 2: Strategy 0 — Closed-Term Evaluation . . . 232
39.1.4 Step 3: Strategy 1 — Denotational OTerm Equivalence . . . 232
39.1.5 Step 4: Create Theory and Compile . . . 233
39.1.6 Step 5: Unify Parameters and Structural Check . . . 233
39.1.7 Step 6: Duck Type Inference . . . 233
39.1.8 Step 7: Build Site Category . . . 233
39.1.9 Step 8: Local Equivalence Check . . . 233
39.1.10 Step 9: Čech Cohomology . . . 233
39.1.11 Step 10: Verdict . . . 234
39.2 Example 2: Sorting with Different Tie-Breaking (INEQUIVALENT) . . . 234
39.2.1 The Input Pair . . . 234
39.2.2 Compilation . . . 234
39.2.3 Duck Type Inference . . . 234
39.2.4 Local Checks . . . 234
39.2.5 Čech Result . . . 234

40 Bridging the Intensional–Extensional Gap 235
40.1 The Current Pipeline as a Typed Function Chain . . . 235
40.2 The 7-Phase Normalizer . . . 236
40.2.1 Confluence and Termination . . . 237
40.3 The HIT Prover as a Term-Rewriting System . . . 238
40.3.1 Congruence Rules . . . 238
40.3.2 Cross-Type Path Constructors . . . 238
40.3.3 Memoization and Termination . . . 239
40.4 The D20 Spec Identification Engine . . . 239
40.4.1 Spec Identification as a Morphism . . . 239
40.4.2 Formal Pattern Matching as Presheaf Morphism . . . 239
40.5 BFS Path Search and the 16 Axiom Functions . . . 240
40.5.1 The 16 Registered Axiom Functions . . . 240
40.5.2 Axiom Soundness . . . 240
40.5.3 BFS Structure . . . 241
40.6 Provable NEQ Detection . . . 242
40.7 The Intensional–Extensional Gap . . . 243
40.7.1 Where the Pipeline Fails . . . 243
40.7.2 Formal Gap Statement . . . 243
40.7.3 What the System Cannot Do . . . 243
40.8 Enriching the Path Space . . . 244
40.8.1 The Rewrite Groupoid . . . 244
40.8.2 Three Mechanisms for New Paths . . . 244

40.9 Strengthening the D20 Spec Engine . . . 244
40.9.1 The Spec Library as a Presheaf . . . 244
40.9.2 Observable Properties as Spec Discovery . . . 245
40.10 Improving the Denotational–Cohomological Interaction . . . 245
40.10.1 Shared Symbol Congruence . . . 245
40.10.2 Feeding Denotational Evidence into Fibers . . . 245
40.11 The Graded Verdict . . . 245
40.11.1 H^1 Suppression and the Uninterpreted-Depth Filter . . . 245
40.12 Soundness and Completeness . . . 246
40.13 The Programme for Improvement . . . 246
40.13.1 Axis 1: Richer Normalization (More Paths) . . . 246
40.13.2 Axis 2: Richer Specifications (More Yoneda) . . . 247
40.13.3 Axis 3: Z3 Compilation Fidelity (Fewer Spurious H^1) . . . 247
40.13.4 Concrete Improvement Theorems . . . 247
40.14 Summary: What the Theory Tells Us to Build Next . . . 247

41 OOP: Classes, Inheritance, and Method Resolution 248
41.1 Class Compilation . . . 248
41.2 Classes as OTerm Inhabitants . . . 248
41.3 Inheritance as Fiber Refinement . . . 249
41.4 Method Resolution as a Path in the Class HIT . . . 249
41.5 Virtual Dispatch as Fiber Case Split . . . 249
41.6 Descriptors and Properties . . . 250

42 Error Handling and Graceful Degradation 251
42.1 Degradation Cascade . . . 251
42.2 Unsupported Syntax . . . 251
42.3 Z3 Timeout Handling . . . 251
42.4 Per-Fiber Timeouts and Global Deadlines . . . 252
42.5 Memory Safety . . . 252
42.6 Combinatorial Explosion in Site Category . . . 253
42.7 Partial Verdicts and Confidence Levels . . . 253

43 Scalability 254
43.1 Complexity of the Full Pipeline . . . 254
43.2 Bottleneck Analysis . . . 254
43.3 Z3 Query Complexity . . . 254
43.4 Scaling Strategies . . . 255
43.5 Heuristics for Large Programs . . . 255

A Mathematical Prerequisites 256
A.1 Category Theory Essentials . . . 256
A.2 Sheaf Theory and Cohomology . . . 256
A.3 Homotopy Type Theory Essentials . . . 257
A.4 Algebraic Geometry Background . . . 257
A.5 SMT Solving Essentials . . . 257

B Benchmark Pair Catalog 259
B.1 Pipeline Resolution Stages . . . 259
B.2 Difficulty Classification . . . 259
B.3 Equivalent Pairs (EQ-01 through EQ-50) . . . 262
B.4 Non-Equivalent Pairs (NEQ-01 through NEQ-50) . . . 264

B.5 Summary Statistics . . . 265

C Proof Theory: Foundations 266
C.1 Motivation: Why a Proof Theory? . . . 266
C.2 The Language of Proof Terms . . . 266
C.2.1 Structural Rules . . . 266
C.2.2 Computation Rules . . . 267
C.2.3 Induction Principles . . . 267
C.2.4 Axiom Application . . . 267
C.2.5 Loop and Recursion Reasoning . . . 268
C.2.6 Algebraic Reasoning . . . 268
C.2.7 Z3 Discharge . . . 268
C.2.8 Cohomological Reasoning . . . 268
C.3 The Proof Checker . . . 268
C.4 Soundness Theorem . . . 269
C.5 Comparison with Other Proof Systems . . . 269
C.5.1 Martin-Löf Identity Types . . . 269
C.5.2 Cubical Path Types . . . 270
C.5.3 Lean Proofs . . . 270
C.6 Proof Term Serialization . . . 270
C.7 Integration with the CCTT Pipeline . . . 270

D Proof Theory: Worked Examples 272
D.1 Proof Strategies Overview . . . 272
D.2 Example 1: Recursive vs. Iterative Factorial . . . 272
D.2.1 The Programs . . . 272
D.2.2 OTerm Representation . . . 273
D.2.3 Proof Strategy: Natural Number Induction . . . 273
D.2.4 Proof Term . . . 273
D.3 Example 2: Merge Sort vs. Quick Sort . . . 273
D.3.1 The Programs . . . 273
D.3.2 Proof Strategy: Abstraction Refinement . . . 273
D.4 Example 3: Edit Distance DP vs. Memoized Recursive . . . 274
D.4.1 The Programs . . . 274
D.4.2 Proof Strategy: Simulation . . . 274
D.5 Example 4: BFS Flood Fill vs. Union-Find . . . 274
D.6 Example 5: Dijkstra vs. Bellman-Ford . . . 274
D.7 Example 6: Trial Division vs. Sieve Factorization . . . 275
D.8 Example 7: Powerset Recursive vs. Bitmask . . . 275
D.9 Example 8: LCS via DP vs. Hunt-Szymanski . . . 275
D.10 Example 9: Red-Black Tree vs. AVL Tree Insert . . . 275
D.11 Example 10: Regex NFA vs. DFA Matching . . . 275
D.12 Example 11: Map-then-Filter vs. Filter-then-Map . . . 276
D.13 Example 12: Insertion Sort is Stable . . . 276
D.14 Proof Complexity Analysis . . . 276

E Proof Theory: Automation and Tactics 278
E.1 Tactic Architecture . . . 278
E.2 The auto Tactic . . . 279
E.3 The ring Tactic . . . 280
E.4 The omega Tactic . . . 280
E.5 The simp Tactic . . . 280

E.6 Tactic Combinators 281
E.7 Integration with Z3 281
E.8 What Can and Cannot Be Automated 281
E.9 Semi-Automated Proof Construction 282
E.10 LLM-Assisted Proof Construction 282

Part I

Cubical Type Theory

Chapter 1

The Interval, Faces, and Connections

Cubical type theory (CCHM, 2017; Cohen et al., 2018) is a type theory where equality is witnessed by *paths* — functions from an abstract interval. We develop the theory in full, specialized to program semantics, and make precise the correspondence between the formal interval and the rewrite parameter that interpolates between equivalent programs in our implementation.

1.1 The Abstract Interval $\mathbb{I}$

Definition 1.1 (The Interval)**.** The *abstract interval* $\mathbb{I}$ is a type with:

1. Two *endpoints*: $\mathsf{i0}, \mathsf{i1} : \mathbb{I}$.
2. *Connections*: binary operations $\wedge, \vee : \mathbb{I} \times \mathbb{I} \to \mathbb{I}$ satisfying:

$$r \wedge \mathsf{i0} = \mathsf{i0} \qquad r \wedge \mathsf{i1} = r \tag{1.1}$$

$$r \vee \mathsf{i0} = r \qquad r \vee \mathsf{i1} = \mathsf{i1} \tag{1.2}$$

3. *Reversal*: $\neg : \mathbb{I} \to \mathbb{I}$ with $\neg\mathsf{i0} = \mathsf{i1}$ and $\neg\mathsf{i1} = \mathsf{i0}$.
4. The tuple $(\mathbb{I}, \wedge, \vee, \neg, \mathsf{i0}, \mathsf{i1})$ forms a *De Morgan algebra*.

Definition 1.2 (De Morgan Algebra — Axioms)**.** A *De Morgan algebra* $(L, \wedge, \vee, \neg, 0, 1)$ is a bounded distributive lattice equipped with an order-reversing involution $\neg$ satisfying:

(DM1) $\neg(r \wedge s) = (\neg r) \vee (\neg s)$ (De Morgan's law)

(DM2) $\neg(r \vee s) = (\neg r) \wedge (\neg s)$ (dual De Morgan)

(DM3) $\neg(\neg r) = r$ (involution)

(DM4) $\neg\mathsf{i0} = \mathsf{i1}, \ \neg\mathsf{i1} = \mathsf{i0}$ (endpoint swap)

The free De Morgan algebra on n generators has $3^n \cdot 2^{2^n}$ elements when quotiented by the DM axioms; for one generator the quotient is the three-element chain $\mathsf{i0} < r < \mathsf{i1}$ plus its reversal $\neg r$.

Proposition 1.3 (Interval as Free DM Algebra)**.** *The abstract interval $\mathbb{I}$ is (the carrier of) the free De Morgan algebra* $\mathrm{DM}(r)$ *on a single generator r. Its elements are exactly* $\{\mathsf{i0}, r, \neg r, \mathsf{i1}\}$*, and every term built from r using $\wedge, \vee, \neg$ reduces to one of these four.*

Proof. By the lattice identities and the involution axiom, $r \wedge r = r$, $r \vee r = r$, and $\neg(\neg r) = r$. The only non-trivial compound is $r \wedge (\neg r)$, which need not equal $\mathsf{i0}$ (DM algebras are not Boolean in general), but the CCHM interval takes the quotient by the additional relation $r \wedge (\neg r) = \mathsf{i0}$ and $r \vee (\neg r) = \mathsf{i1}$ (the *complementarity* or *Boolean DM* axiom). Under this additional axiom, the free algebra collapses to the four elements above. □

Remark 1.4 (Why Not the Unit Interval?)**.** The interval $\mathbb{I}$ is *not* the topological unit interval $[0, 1]$. It is a formal, syntactic object — a free De Morgan algebra on one generator. Its purpose is to parameterize *deformations* between terms. In our setting, a path $p : \mathbb{I} \to (A \to B)$ with $p(\mathsf{i0}) = f$ and $p(\mathsf{i1}) = g$ is a formal deformation from program f to program g. The key distinction is that $\mathbb{I}$ carries *no topology*: there is no notion of "nearby" points. Every element is either an endpoint or the formal parameter r (or $\neg r$). Continuity is replaced by *algebraic coherence*: the De Morgan laws ensure that face maps, connections, and reversals interact correctly.

Remark 1.5 (The Interval in the Implementation)**.** In the implementation (`cctt/path_search.py`), the interval does not appear as an explicit data type. Instead, a path between OTerms t_f and t_g is realized as a finite rewrite chain

$$t_f = h_0 \xrightarrow{\mathrm{ax}_1} h_1 \xrightarrow{\mathrm{ax}_2} \cdots \xrightarrow{\mathrm{ax}_k} h_k = t_g$$

where each h_j is an OTerm and each ax_j is a dichotomy axiom (D1–D24). The chain is a *piecewise-linear* path in $\mathbb{I}$: we subdivide $\mathbb{I}$ into k sub-intervals $[t_{j-1}, t_j]$ and assign h_j to the j-th segment. The face maps ∂^0 and ∂^1 simply read off $h_0 = t_f$ and $h_k = t_g$.

1.2 Faces and Face Lattice

A *face* is a constraint on interval variables.

Definition 1.6 (Face Formulas)**.** The *face lattice* $\mathbb{F}$ is the free distributive lattice generated by atoms $(i = 0)$ and $(i = 1)$ for each interval variable i. A face formula $\varphi \in \mathbb{F}$ is a conjunction/disjunction of such atoms:

$$\varphi ::= (i = 0) \mid (i = 1) \mid \varphi_1 \wedge \varphi_2 \mid \varphi_1 \vee \varphi_2$$

We write $\mathbf{1}_{\mathbb{F}}$ for the top element (the trivially satisfied constraint) and $\mathbf{0}_{\mathbb{F}}$ for the bottom (the inconsistent constraint, e.g. $(i = 0) \wedge (i = 1)$).

Definition 1.7 (Face Restriction)**.** Given a type A in context $\Gamma, i : \mathbb{I}$ and a face formula φ, the *face restriction* or *partial element* $A[\varphi]$ is the type of terms $a : A$ that are only defined when φ holds. The judgment

$$\Gamma \vdash u : A[\varphi]$$

says that u is a term of type A defined on the sub-cube specified by φ. A *system* (or *partial element*) is a compatible family of restrictions:

$$[\varphi_1 \mapsto u_1, \ldots, \varphi_n \mapsto u_n]$$

where $u_j = u_k$ whenever $\varphi_j \wedge \varphi_k$ is satisfiable.

Example 1.8 (Face of a Square)**.** A 2-dimensional cube (square) has four edges (1-faces) and four vertices (0-faces). The edge $i = 0$ is the left side; $j = 1$ is the top. The vertex $(i = 0) \wedge (j = 1)$ is the top-left corner. The system

$$[(i = 0) \mapsto a,\ (i = 1) \mapsto b,\ (j = 0) \mapsto p,\ (j = 1) \mapsto q]$$

specifies all four edges of a square; it is *compatible* if the corners agree: $p(\mathsf{i0}) = a(\mathsf{i0}), p(\mathsf{i1}) = b(\mathsf{i0})$, etc.

Example 1.9 (Faces as Type-Fiber Restrictions)**.** In program equivalence, faces correspond to *restrictions* of the input space. The face $\mathsf{tag}(p) = \mathsf{Int}$ restricts to the integer fiber. The `FiberCtx` class in `cctt/path_search.py` captures exactly this:

```
class FiberCtx:
    param_duck_types: Dict[str, str]  # p0 -> 'int', ...
    def is_numeric(self, term: OTerm) -> bool: ...
```

When the face formula $\mathsf{tag}(p_0) = \mathsf{int}$ holds, numeric axioms (commutativity, associativity) are available; on the complementary face $\mathsf{tag}(p_0) = \mathsf{str}$, they are not. This is *sheaf descent*: axiom applicability depends on which face we are restricting to.

1.3 The Face Map and Degeneracy

Definition 1.10 (Face Maps)**.** For a cubical term t depending on interval variable i, the *face maps* are:

$$\partial_i^0(t) := t[i \mapsto \mathsf{i0}] \tag{1.3}$$

$$\partial_i^1(t) := t[i \mapsto \mathsf{i1}] \tag{1.4}$$

These give the "boundary values" of t at each endpoint of i.

Definition 1.11 (Degeneracy Maps)**.** The *degeneracy map* σ_i adds a "trivial" interval dimension:

$$\sigma_i(t) = t \quad \text{(where } t \text{ does not depend on } i\text{)}$$

A degenerate cube is one that is constant in some dimension.

Proposition 1.12 (Face Maps on OTerms)**.** *In the OTerm algebra, face maps have a direct interpretation. Given a path $p : \mathbb{I} \to \mathrm{OTerm}$ realized as a rewrite chain $(h_0, \ldots, h_k)$:*

$$\partial^0(p) = h_0 \qquad \textit{(the source OTerm)} \tag{1.5}$$

$$\partial^1(p) = h_k \qquad \textit{(the target OTerm)} \tag{1.6}$$

The degeneracy map $\sigma(t)$ sends an OTerm t to the length-0 rewrite chain (t), corresponding to the reflexivity path refl_t.

In the implementation, `PathResult` stores these explicitly: the `path` field is $[s_1, \ldots, s_k]$ where each `PathStep` s_j records `before` and `after` canonical forms. The face maps are: $\partial^0 = s_1.$`before` and $\partial^1 = s_k.$`after`.

Face and degeneracy maps satisfy the *cubical identities*:

$$\partial_j^b \circ \partial_i^a = \partial_i^a \circ \partial_{j-1}^b \quad \text{for } j > i \tag{1.7}$$

$$\sigma_j \circ \sigma_i = \sigma_i \circ \sigma_{j+1} \quad \text{for } j \geq i \tag{1.8}$$

$$\partial_j^b \circ \sigma_i = \begin{cases} \sigma_{i-1} \circ \partial_j^b & j < i \\ \mathrm{id} & j = i \\ \sigma_i \circ \partial_{j-1}^b & j > i \end{cases} \tag{1.9}$$

These identities ensure that the cubical structure is coherent — different ways of computing the same face give the same result.

Theorem 1.13 (Cubical Identities for Rewrite Chains)**.** *Let $\mathcal{R}$ be the set of OTerm rewrite chains under the dichotomy axioms. Define ∂^0, ∂^1, and σ as in Proposition 1.12. Then the cubical identities* (1.7)–(1.9) *hold in $\mathcal{R}$.*

Proof. For one-dimensional chains (our primary case), the identities (1.7) and (1.9) are trivial: there is only one interval variable, so the $j > i$ and $j < i$ cases are vacuous, and $\partial^a \circ \sigma = \mathrm{id}$ holds because applying a face map to a degenerate (constant) chain returns the original term.

For multi-dimensional chains arising from homotopies between rewrite paths (Section 2.4), identity (1.7) states that restricting a 2-path first on dimension j then on dimension i gives the same result as restricting in the other order (with index adjustment). This holds because our 2-paths are constructed by Kan filling, which guarantees boundary coherence by construction. □

1.4 Connections and Their Computational Meaning

Definition 1.14 (Connections)**.** The *connection maps* on the interval are the binary operations

$$\gamma^0(r,s) \coloneqq r \wedge s \qquad \text{(min-connection)} \tag{1.10}$$

$$\gamma^1(r,s) \coloneqq r \vee s \qquad \text{(max-connection)} \tag{1.11}$$

They induce maps $\gamma_i^\varepsilon : [n+1] \to [n]$ on the cube category by applying $\wedge$ (resp. $\vee$) to coordinates i and $i+1$.

Theorem 1.15 (Connection–Face Interaction)**.** *The connections interact with face maps via:*

$$\delta_i^0 \circ \gamma_i^0 = \sigma_i \qquad \delta_i^1 \circ \gamma_i^0 = \mathrm{id} \tag{1.12}$$

$$\delta_i^0 \circ \gamma_i^1 = \mathrm{id} \qquad \delta_i^1 \circ \gamma_i^1 = \sigma_i \tag{1.13}$$

These say: evaluating a min-connection at $\mathsf{i0}$ *degenerates (becomes constant), while evaluating it at* $\mathsf{i1}$ *is the identity (passes through). Dually for the max-connection.*

Proof. Consider $\delta_i^0 \circ \gamma_i^0$ applied to a term $t(r_1, \ldots, r_{n+1})$. First, γ_i^0 replaces coordinates i and $i+1$ by their meet: $t(\ldots, r_i \wedge r_{i+1}, \ldots)$. Then δ_i^0 substitutes $r_i \mapsto \mathsf{i0}$, yielding $t(\ldots, \mathsf{i0} \wedge r_{i+1}, \ldots) = t(\ldots, \mathsf{i0}, \ldots)$, which is $\sigma_i(t)$. The identity $\delta_i^1 \circ \gamma_i^0$ similarly gives $t(\ldots, \mathsf{i1} \wedge r_{i+1}, \ldots) = t(\ldots, r_{i+1}, \ldots) = \mathrm{id}(t)$. The γ^1 cases are dual by De Morgan. □

Proposition 1.16 (The Cube Category □)**.** *The* cube category □ *has objects* $[n] = \mathbb{I}^n$ *for* $n \geq 0$ *and morphisms generated by:*

- *Face maps* $\delta_i^\varepsilon : [n-1] \to [n]$ *for* $1 \leq i \leq n$ *and* $\varepsilon \in \{0,1\}$*, inserting* $\mathsf{i}\varepsilon$ *at position* i*.*
- *Degeneracy maps* $\sigma_i : [n+1] \to [n]$ *for* $1 \leq i \leq n+1$*, projecting away dimension* i*.*
- *Connection maps* $\gamma_i^\varepsilon : [n+1] \to [n]$ *for* $1 \leq i \leq n$*, applying* $\wedge$ *(for* $\varepsilon = 0$*) or* $\vee$ *(for* $\varepsilon = 1$*) to coordinates* i *and* $i+1$*.*

Subject to the cubical identities (1.7)–(1.9) *and the connection–face identities of Theorem 1.15.*

Remark 1.17 (Connections as Path Contraction and Expansion)**.** Connections have an important computational meaning for program equivalence proofs. The min-connection γ^0 "contracts" a path toward $\mathsf{i0}$: if $p : \mathsf{Path}_A(a,b)$, then $\lambda i.\, p(i \wedge j)$ is a homotopy from the constant path refl_a (when $j = \mathsf{i0}$) to p itself (when $j = \mathsf{i1}$). This contraction proves that every path is homotopic to a path that "starts slowly" from its source.

In the implementation, this corresponds to *path prefix extraction*: given a rewrite chain $(h_0, \ldots, h_k)$, the min-connection produces the prefix $(h_0, \ldots, h_{\lfloor k/2 \rfloor})$ — a partial rewrite that has "contracted" toward the source. The max-connection dually produces the suffix contracting toward the target.

1.5 Cubical Sets and the Program Cubical Set

Definition 1.18 (Cubical Sets)**.** A *cubical set* is a presheaf $X : \square^{\mathsf{op}} \to \mathbf{Set}$. Concretely, it consists of:

- For each $n \geq 0$, a set X_n of *n-cubes*.
- For each face map δ_i^ε, a restriction $\partial_i^\varepsilon : X_n \to X_{n-1}$.
- For each degeneracy map σ_i, an extension $s_i : X_n \to X_{n+1}$.

- For each connection, a diagonal map, all satisfying the dual cubical identities.

A *morphism of cubical sets* $f : X \to Y$ is a natural transformation. The category of cubical sets is written $\widehat{\square} := [\square^{\mathsf{op}}, \mathbf{Set}]$.

Example 1.19 (Programs as a Cubical Set)**.** Fix a function type $A \to B$. Define the cubical set $\mathcal{P}_{A\to B}$ by:

- $\mathcal{P}_0 = \{\text{programs } f : A \to B\}$ (0-cubes are programs).
- $\mathcal{P}_1 = \{\text{formal deformations } p : \mathbb{I} \to (A \to B) \mid p(\mathsf{i0}) = f, p(\mathsf{i1}) = g\}$ (1-cubes are equivalence witnesses).
- $\mathcal{P}_2$ consists of homotopies between equivalence proofs (coherence data).

The face maps $\partial_1^0, \partial_1^1 : \mathcal{P}_1 \to \mathcal{P}_0$ extract the source and target programs. The degeneracy $s_1 : \mathcal{P}_0 \to \mathcal{P}_1$ sends each program to its reflexivity path.

Definition 1.20 (The OTerm Cubical Set)**.** Let OT denote the set of OTerms modulo canonical form (Definition 1.18). We construct the cubical set $\mathcal{O}$ as follows:

- $\mathcal{O}_0 := \mathrm{OT}$, the set of normalized OTerms.
- $\mathcal{O}_1 := \{(t_0, \mathrm{ax}, t_1) \mid t_0 \xrightarrow{\mathrm{ax}} t_1 \text{ is a single axiom application}\}$, the set of one-step rewrites. Each element corresponds to one invocation of a root axiom from the list `_ROOT_AXIOMS` in `path_search.py`.
- $\mathcal{O}_2 := \{\text{coherence witnesses between different orderings of axiom application}\}$.

The face maps are $\partial^0(t_0, \mathrm{ax}, t_1) = t_0$ and $\partial^1(t_0, \mathrm{ax}, t_1) = t_1$. The degeneracy $s(t) = (t, \mathrm{refl}, t)$.

A `PathResult` with k steps is a composable chain of k elements of $\mathcal{O}_1$, and the path search algorithm (`search_path`) computes a morphism $\square[-, [1]] \to \mathcal{O}$ — that is, a 1-cube in $\mathcal{O}$ — by the Yoneda correspondence (Theorem 1.21).

Theorem 1.21 (The Yoneda Embedding for $\square$)**.** *The representable presheaf $\square[-, [n]]$ is the standard n-cube. By the Yoneda lemma, for any cubical set X:*

$$\mathrm{Hom}_{\widehat{\square}}(\square[-, [n]], X) \cong X_n$$

Thus the n-cubes of X are exactly the maps from the standard n-cube into X.

Proof. This is the standard Yoneda lemma applied to the functor category $[\square^{\mathsf{op}}, \mathbf{Set}]$. A natural transformation $\alpha : \square[-, [n]] \Rightarrow X$ is determined by $\alpha_{[n]}(\mathrm{id}_{[n]}) \in X_n$, and conversely every element of X_n determines such a transformation. Naturality of α encodes precisely the compatibility with face, degeneracy, and connection maps. $\square$

Remark 1.22 (Yoneda and Path Search)**.** Theorem 1.21 gives the key identification: a path from OTerm t_f to OTerm t_g in the cubical set $\mathcal{O}$ is the same thing as an element of $\mathcal{O}_1$ (a composable rewrite chain) with the correct face values. When `search_path` returns `PathResult(found=True, path=[...])`, it has constructed exactly such a Yoneda element.

Example 1.23 (D1 Rewrite as a Cubical Path)**.** Consider the D1 (recursion $\leftrightarrow$ iteration) dichotomy applied to factorial. The recursive version compiles to:

$$t_{\mathrm{rec}} = \mathsf{fix}[f](\mathsf{case}(\mathsf{le}(p_0, 0),\ 1,\ \mathsf{mul}(p_0, f(\mathsf{sub}(p_0, 1)))))$$

The iterative version compiles to:

$$t_{\mathrm{iter}} = \mathsf{fold}[\mathsf{mul}](1, \mathsf{range}(1, \mathsf{add}(p_0, 1)))$$

The axiom `D1_fix_to_fold` witnesses a 1-cube $(t_{\text{rec}}, \texttt{D1}, t_{\text{iter}}) \in \mathcal{O}_1$ with face maps:

$$\partial^0(t_{\text{rec}}, \texttt{D1}, t_{\text{iter}}) = t_{\text{rec}} \tag{1.14}$$

$$\partial^1(t_{\text{rec}}, \texttt{D1}, t_{\text{iter}}) = t_{\text{iter}} \tag{1.15}$$

This is a path $p : \mathsf{Path}_{\mathcal{O}}(t_{\text{rec}}, t_{\text{iter}})$ in the precise cubical sense. The path is *not* a continuous deformation — it is a single axiom application that the type theory treats as an atomic equality witness.

The inverse path p^{-1}, obtained by reversal ($\neg$), corresponds to applying the axiom in the opposite direction (`D1_fold_to_fix`), witnessing $\mathsf{Path}_{\mathcal{O}}(t_{\text{iter}}, t_{\text{rec}})$.

Chapter 2

Path Types and Function Extensionality

2.1 Path Types

Definition 2.1 (Path Type)**.** For a type A and terms $a, b : A$, the *path type* $\mathsf{Path}_A(a, b)$ is:

$$\mathsf{Path}_A(a, b) \coloneqq \{p : \mathbb{I} \to A \mid p(\mathsf{i0}) = a \text{ and } p(\mathsf{i1}) = b\}$$

A term $p : \mathsf{Path}_A(a, b)$ is a *path from a to b in A*.

Remark 2.2 (Path Type vs. Identity Type)**.** In Martin-Löf type theory, the identity type $\mathrm{Id}_A(a, b)$ is an *inductively defined* type with a single constructor refl. The path type $\mathsf{Path}_A(a, b)$ replaces this with a *function space*: a path is a function $\mathbb{I} \to A$ with fixed endpoints. The crucial advantage is that $\mathsf{Path}_A(a, b)$ can be *inhabited by construction*: to prove $a = b$, one exhibits a concrete function $p : \mathbb{I} \to A$. In our setting, each dichotomy axiom (D1–D24) provides such a function for specific pairs (a, b) of OTerms.

Definition 2.3 (Reflexivity)**.** For any $a : A$, the *constant path* $\mathsf{refl}_a : \mathsf{Path}_A(a, a)$ is:

$$\mathsf{refl}_a \coloneqq \lambda i.\, a$$

Remark 2.4 (Reflexivity in the Implementation)**.** When `search_path` finds that `nf_f.canon()` = `nf_g.canon()`, it returns `PathResult(found=True, path=[], reason='refl')`: the empty path list is exactly the reflexivity witness refl_{t_f}.

Definition 2.5 (Symmetry)**.** For $p : \mathsf{Path}_A(a, b)$, the *inverse path* $p^{-1} : \mathsf{Path}_A(b, a)$ is:

$$p^{-1} \coloneqq \lambda i.\, p(\neg i)$$

Proposition 2.6 (Symmetry of Rewrite Chains)**.** *Every axiom in the path search is invertible: if $(t_0, \mathrm{ax}, t_1) \in \mathcal{O}_1$, then $(t_1, \mathrm{ax}^{-1}, t_0) \in \mathcal{O}_1$. The inverse of a `PathResult` with steps $[s_1, \ldots, s_k]$ is the reversed list $[s_k^{-1}, \ldots, s_1^{-1}]$. This is the computational content of path symmetry.*

Proof. Each axiom in `_ROOT_AXIOMS` is an equational identity of the form $\ell = r$. The forward direction applies $\ell \mapsto r$; the reverse applies $r \mapsto \ell$. In the bidirectional BFS of `search_path`, the backward frontier expands from t_g using exactly these reversed axioms. When the frontiers meet, the backward portion of the path is reversed (line: `list(reversed(bpath))`), producing the composite path. □

Definition 2.7 (Path Composition via Kan Filling)**.** For $p : \mathsf{Path}_A(a, b)$ and $q : \mathsf{Path}_A(b, c)$, the *composite path* $p \cdot q : \mathsf{Path}_A(a, c)$ is constructed by the Kan filling operation.

Consider the open box:

$$\begin{tikzcd} a \arrow[r, "p"] \arrow[d, dashed, "p\cdot q"'] & b \arrow[d, "q"] \\ & c \end{tikzcd}$$

The Kan filler closes the box, producing the composite path from a to c.

Formally:

$$(p \cdot q)(i) \coloneqq \mathsf{hfill}^i_j \begin{cases} (j = \mathsf{i0}) & \mapsto a \\ (j = \mathsf{i1}) & \mapsto q(i) \\ (i = \mathsf{i0}) & \mapsto p(j) \end{cases}$$

Theorem 2.8 (Path Composition is Well-Defined)**.** *The Kan filling operation produces a well-defined term of type* $\mathsf{Path}_A(a, c)$*. The boundary conditions are:*

- $(p \cdot q)(\mathsf{i0}) = a$ *(by the* $i = \mathsf{i0}$ *face, which gives* $p(\mathsf{i0}) = a$*).*
- $(p \cdot q)(\mathsf{i1}) = c$ *(by the* $j = \mathsf{i1}$ *face, which gives* $q(\mathsf{i1}) = c$*).*

Proof. The system $[(j = \mathsf{i0}) \mapsto a,\ (j = \mathsf{i1}) \mapsto q(i),\ (i = \mathsf{i0}) \mapsto p(j)]$ is compatible: at the corner $(i = \mathsf{i0}, j = \mathsf{i0})$ both branches give a (since $p(\mathsf{i0}) = a$); at $(i = \mathsf{i0}, j = \mathsf{i1})$ both give $q(\mathsf{i0}) = b = p(\mathsf{i1})$. The Kan condition on the cubical set (or composition structure on the type) guarantees a unique filler. Its boundary at $i = \mathsf{i0}$ is $p(\mathsf{i0}) = a$ and at $i = \mathsf{i1}$ the filler agrees with $q(\mathsf{i1}) = c$ by the $(j = \mathsf{i1})$ face. □

Proposition 2.9 (Composition of Rewrite Chains)**.** *In the implementation, path composition is list concatenation: given* `PathResult` R_1 *with steps* $[s_1, \ldots, s_j]$ *and* R_2 *with steps* $[s_{j+1}, \ldots, s_k]$ *where* s_j`.after` $= s_{j+1}$`.before` *(i.e.,* R_1 *ends where* R_2 *begins), the composite is* $[s_1, \ldots, s_k]$*.*

This is the computational content of Kan filling: the open box has R_1 *on the left face and* R_2 *on the top face; the filler is the concatenated chain on the diagonal. The bidirectional BFS in* `search_path` *constructs exactly this composite when the forward and backward frontiers meet:* `full_path = new_path + list(reversed(bpath))`*.*

2.2 Function Extensionality

Theorem 2.10 (Function Extensionality — funext)**.** *For* $f, g : A \to B$*:*

$$\mathsf{Path}_{A\to B}(f, g) \simeq \prod_{x:A} \mathsf{Path}_B(f(x), g(x))$$

That is, a path between functions is equivalent to a family of pointwise paths.

Proof. We construct an explicit equivalence between the two types.

Step 1 (happly*: forward direction).* Given $p : \mathsf{Path}_{A\to B}(f, g)$ and $x : A$, define:

$$\mathsf{happly}(p, x) \coloneqq \lambda i.\, p(i)(x) : \mathsf{Path}_B(f(x), g(x))$$

This is well-typed because $p(i) : A \to B$ for each $i : \mathbb{I}$, so $p(i)(x) : B$. At the endpoints: $\mathsf{happly}(p, x)(\mathsf{i0}) = p(\mathsf{i0})(x) = f(x)$ and $\mathsf{happly}(p, x)(\mathsf{i1}) = p(\mathsf{i1})(x) = g(x)$.

Step 2 (funext*: backward direction).* Given a family $h : \prod_{x:A} \mathsf{Path}_B(f(x), g(x))$, define:

$$\mathsf{funext}(h) \coloneqq \lambda i.\, \lambda x.\, h(x)(i) : \mathsf{Path}_{A\to B}(f, g)$$

At $i = \mathsf{i0}$: $\mathsf{funext}(h)(\mathsf{i0}) = \lambda x.\, h(x)(\mathsf{i0}) = \lambda x.\, f(x) = f$. At $i = \mathsf{i1}$: $\mathsf{funext}(h)(\mathsf{i1}) = \lambda x.\, g(x) = g$.

Step 3 (Round-trip). $\mathsf{happly}(\mathsf{funext}(h), x)(i) = \mathsf{funext}(h)(i)(x) = h(x)(i)$, so $\mathsf{happly} \circ \mathsf{funext} = \mathrm{id}$ (judgmentally, by β-reduction of the λ-abstractions).

$\mathsf{funext}(\mathsf{happly}(p))(i)(x) = \mathsf{happly}(p, x)(i) = p(i)(x)$, so $\mathsf{funext}(\mathsf{happly}(p)) = \lambda i.\, \lambda x.\, p(i)(x)$. By η-expansion, $\lambda x.\, p(i)(x) = p(i)$, hence $\mathsf{funext} \circ \mathsf{happly} = \mathrm{id}$.

The two directions are mutually inverse, so the types are equivalent. □

Remark 2.11 (Why funext Is a Theorem, Not an Axiom)**.** In intensional Martin-Löf type theory, function extensionality is *unprovable* and must be postulated as an axiom. In cubical type theory, it is a *theorem*: the proof above uses only the λ-calculus structure of $\mathbb{I} \to A$ and the definitional $\beta\eta$-rules. This is one of the principal motivations for adopting cubical foundations. For our application, it means that pointwise program equivalence *automatically* implies function-level equivalence — no additional axiom is needed.

Corollary 2.12 (Program Equivalence as Pointwise Path Family)**.** *Two programs $f, g : A \to B$ are equivalent iff for every input $x : A$, there is a path $p_x : \mathsf{Path}_B(f(x), g(x))$. By funext, this family assembles into a single path $f \equiv g$.*

This is the *fundamental theorem* of our approach: to prove program equivalence, we construct pointwise paths (one per input) and assemble them via funext.

Theorem 2.13 (Funext for OTerm Equivalence)**.** *Let t_f, t_g be normalized OTerms depending on parameters $p_0, \ldots, p_{n-1}$. Suppose that for every type-consistent assignment $\vec{v} = (v_0, \ldots, v_{n-1})$ of the parameters, there is a path $q_{\vec{v}} : \mathsf{Path}_B(t_f[\vec{p} \mapsto \vec{v}],\ t_g[\vec{p} \mapsto \vec{v}])$. Then there is a path $\mathsf{Path}_{A\to B}(t_f, t_g)$.*

Proof. By funext (Theorem 2.10), the pointwise path family $\vec{v} \mapsto q_{\vec{v}}$ assembles into a function-level path. In the implementation, this is precisely the architecture of the checker: for each fiber (determined by `FiberCtx.param_duck_types`), the checker finds a path between the fiber-restricted OTerms, then combines the per-fiber paths into a global equivalence verdict via Čech cohomology (Chapter 1). □

Example 2.14 (Funext in Practice: Recursive vs. Iterative Factorial)**.** The recursive factorial $f(n) = \mathsf{fix}[h](\ldots)$ and iterative factorial $g(n) = \mathsf{fold}[\mathsf{mul}](\ldots)$ are compared as follows:

1. The type fiber is $\mathsf{tag}(p_0) = \mathsf{Int}$ (both functions operate on integers).
2. On this fiber, the D1 axiom (`D1_fix_to_fold`) applies to the OTerm, yielding a path $q : \mathsf{Path}_{\mathcal{O}}(t_f, t_g)$ at the *term* level.
3. Since the only non-trivial fiber is Int and the path exists there, funext assembles this into $\mathsf{Path}_{\mathsf{Int}\to\mathsf{Int}}(f, g)$.

This is a worked instance of Theorem 2.13: the per-fiber paths (here, a single fiber) lift to a function-level path.

Lemma 2.15 (Path Inversion is an Involution)**.** *For any path $p : \mathsf{Path}_A(a, b)$, we have $(p^{-1})^{-1} = p$.*

Proof. $(p^{-1})^{-1} = \lambda i.\, p^{-1}(\neg i) = \lambda i.\, p(\neg(\neg i)) = \lambda i.\, p(i) = p$, using the involution property $\neg\neg r = r$ of the De Morgan algebra $\mathbb{I}$ (Definition 1.2, axiom DM3). □

Theorem 2.16 (Groupoid Structure of Paths)**.** *For any type A, the path type structure forms a groupoid:*

1. Identity*: $\mathsf{refl}_a : \mathsf{Path}_A(a, a)$ for all a.*
2. Inverse*: $p^{-1} : \mathsf{Path}_A(b, a)$ for $p : \mathsf{Path}_A(a, b)$.*
3. Composition*: $p \cdot q : \mathsf{Path}_A(a, c)$ for $p : \mathsf{Path}_A(a, b)$, $q : \mathsf{Path}_A(b, c)$.*
4. Left unit*: $\mathsf{refl}_a \cdot p \sim p$.*
5. Right unit*: $p \cdot \mathsf{refl}_b \sim p$.*

6. Left inverse: $p^{-1} \cdot p \sim \mathsf{refl}_b$.

7. Right inverse: $p \cdot p^{-1} \sim \mathsf{refl}_a$.

8. Associativity: $(p \cdot q) \cdot r \sim p \cdot (q \cdot r)$.

Here $\sim$ denotes the existence of a 2-path (homotopy).

Proof. We give full Kan filling constructions for each law.

(Left unit.) $\mathsf{refl}_a \cdot p \sim p$. The 2-path $\alpha : \mathbb{I}^2 \to A$ is defined by:

$$\alpha(i,j) := \mathsf{hfill}^i_k \begin{cases} (k = \mathsf{i0}) & \mapsto a \\ (k = \mathsf{i1}) & \mapsto p(i) \\ (j = \mathsf{i0}) & \mapsto a \\ (j = \mathsf{i1}) & \mapsto p(i) \end{cases}$$

At $j = \mathsf{i0}$: the system gives $\mathsf{refl}_a \cdot p$. At $j = \mathsf{i1}$: the system collapses to $p(i)$ since all three specified faces agree with p.

(Right unit.) $p \cdot \mathsf{refl}_b \sim p$. The witness is:

$$\beta(i,j) := \mathsf{hfill}^i_k \begin{cases} (k = \mathsf{i0}) & \mapsto p(i \wedge j) \\ (k = \mathsf{i1}) & \mapsto b \\ (i = \mathsf{i0}) & \mapsto p(k \wedge j) \end{cases}$$

The min-connection $i \wedge j$ ensures that at $j = \mathsf{i0}$ we recover $p \cdot \mathsf{refl}_b$, and at $j = \mathsf{i1}$ we get p.

(Left inverse.) $p^{-1} \cdot p \sim \mathsf{refl}_b$. Define:

$$\gamma(i,j) := p(\neg i \vee j)$$

At $i = \mathsf{i0}$: $p(\mathsf{i1} \vee j) = p(\mathsf{i1}) = b$. At $i = \mathsf{i1}$: $p(\mathsf{i0} \vee j) = p(j)$. At $j = \mathsf{i0}$: $p(\neg i) = p^{-1}(i)$. At $j = \mathsf{i1}$: $p(\neg i \vee \mathsf{i1}) = p(\mathsf{i1}) = b$. So this is a 2-path from the composite $p^{-1} \cdot p$ (boundary at $j = \mathsf{i0}$) to refl_b (boundary at $j = \mathsf{i1}$), using the max-connection.

(Right inverse.) Dual to the left inverse, using $\gamma'(i,j) = p(i \wedge \neg j)$.

(Associativity.) $(p \cdot q) \cdot r \sim p \cdot (q \cdot r)$. This requires a 3-dimensional filling argument. The cube $\mathbb{I}^3 \to A$ is specified on five of its six faces; the Kan condition provides the sixth. The key ingredient is the connection-based reparametrization:

$$\delta(i,j,k) := \mathsf{hfill}^i_m \begin{cases} (m = \mathsf{i0}) & \mapsto a \\ (m = \mathsf{i1}) & \mapsto r(i) \\ (j = \mathsf{i0}) & \mapsto (p \cdot q)(m \vee k) \\ (k = \mathsf{i1}) & \mapsto q(m) \cdot r(i) \end{cases}$$

See Cohen et al. (2018, §6.1) for the full verification that this cube has consistent corners and edges. □

Example 2.17 (Groupoid of Program Rewrites)**.** Consider the type PyComp (Chapter 4). The groupoid structure on its path type has a direct computational reading:

- *Identity*: every program is equivalent to itself (no rewrite needed). In code: `PathResult(found=True, path=[], reason='refl')`.

- *Inverse*: if f can be rewritten to g, then g can be rewritten back to f. Each axiom in `_ROOT_AXIOMS` is an equational identity, so every application is reversible.

- *Composition*: rewrite chains can be concatenated. `search_path` builds composites via its bidirectional BFS: `full_path = fwd_path + reversed(bwd_path)`.
- *Associativity*: $(s_1 \cdot s_2) \cdot s_3 = s_1 \cdot (s_2 \cdot s_3)$ because list concatenation is associative. The 3-dimensional Kan filling in the proof above is implemented trivially by the fact that list concatenation is strictly (not just up to homotopy) associative.

The path search algorithm (`cctt/path_search.py`) constructs explicit witnesses of this groupoid structure: each `PathStep` records a single axiom application (fields: `axiom`, `position`, `before`, `after`), and a `PathResult` is a chain of such steps, which is exactly a composite path $p_1 \cdot p_2 \cdots p_k$ in PyComp.

Example 2.18 (Congruence as Functorial Action)**.** The `_congruence_rewrites` function in `path_search.py` applies axioms inside sub-terms — for instance, rewriting the `init` argument of an `OFold` while leaving the rest unchanged. Categorically, this is the *functorial action* of the OTerm constructors on paths: if $p : \mathsf{Path}_{\mathcal{O}}(s, s')$ is a path between sub-terms, and F is a term constructor (e.g., $\mathsf{OFold}(\mathsf{op}, -, \mathsf{coll})$), then $F(p) : \mathsf{Path}_{\mathcal{O}}(F(s), F(s'))$ is a path between the larger terms.

The annotation `f'{ax}@init'` in the implementation records which sub-term position the axiom was applied at — this is a trace of the functorial action through the OTerm syntax tree.

2.3 Dependent Paths and Transport

Definition 2.19 (Dependent Path Type)**.** For a type family $P : A \to \mathsf{Type}$ and a path $p : \mathsf{Path}_A(a, b)$, the *dependent path type* (path-over) is:

$$\mathsf{PathP}(P, p, u, v) := \{q : \prod_{i:\mathbb{I}} P(p(i)) \mid q(\mathsf{i0}) = u,\ q(\mathsf{i1}) = v\}$$

where $u : P(a)$ and $v : P(b)$.

Proposition 2.20 (PathP Reduces to Path When P is Constant)**.** *If $P \equiv \lambda a.\, B$ is a constant family, then* $\mathsf{PathP}(P, p, u, v) \simeq \mathsf{Path}_B(u, v)$.

Proof. When P is constant, $P(p(i)) = B$ for all i, so a dependent path $q : \prod_{i:\mathbb{I}} B$ with endpoints u, v is exactly an ordinary path in B. □

Remark 2.21 (Dependent Paths for Type-Changing Rewrites)**.** Dependent paths arise naturally in type-changing rewrites. Consider a refactoring that changes a function's return type from int to bool (e.g., replacing `return 0` with `return False`). The OTerm output changes from $\mathsf{IntObj}(0)$ to $\mathsf{BoolObj}(\mathsf{false})$. A dependent path over the type coercion $p : \mathsf{Path}_{\mathsf{Type}}(\mathsf{int}, \mathsf{bool})$ connects these two outputs: $\mathsf{PathP}(\lambda T.\, T,\ p,\ 0,\ \mathsf{false})$. In practice, this corresponds to the Python truthiness coercion $\mathsf{bool}(0) = \mathsf{False}$.

Definition 2.22 (Transport)**.** Given a type family $B : A \to \mathsf{Type}$ and a path $p : \mathsf{Path}_A(a, a')$, *transport* along p is:

$$\mathsf{transp}(B, p) : B(a) \to B(a')$$

defined by $\mathsf{transp}(B, p)(b) := \mathsf{comp}^i(B(p(i)), [\,], b)$.

Proposition 2.23 (Transport Respects Composition)**.** *For paths $p : \mathsf{Path}_A(a, b)$ and $q : \mathsf{Path}_A(b, c)$:*

$$\mathsf{transp}(B, p \cdot q) \sim \mathsf{transp}(B, q) \circ \mathsf{transp}(B, p)$$

Proof. By the Kan filling construction for $p \cdot q$, transporting along the composite path factors through the midpoint b. The filler provides the 2-path witnessing this factorization. □

Transport is essential for *type-changing* equivalences. When a program returns int in one version and bool in another, transport along the coercion path $\mathsf{int} \to \mathsf{bool}$ (via Python's truthiness) relates the two outputs.

2.4 Higher Paths and Truncation

Definition 2.24 (Higher Paths)**.** A *2-path* (homotopy) between paths $p, q : \mathsf{Path}_A(a, b)$ is:

$$\alpha : \mathsf{Path}_{\mathsf{Path}_A(a,b)}(p, q)$$

This is a "path between paths" — a square $\alpha : \mathbb{I} \times \mathbb{I} \to A$ with boundary: $\alpha(\mathsf{i0}, -) = p$, $\alpha(\mathsf{i1}, -) = q$, $\alpha(-, \mathsf{i0}) = \mathsf{refl}_a$, $\alpha(-, \mathsf{i1}) = \mathsf{refl}_b$.

Remark 2.25 (2-Paths as Alternative Rewrite Strategies)**.** When two different sequences of axiom applications both convert t_f to t_g — say $(D1, D4)$ and $(D5, D17)$ — the two rewrite chains are paths with the same endpoints. A 2-path between them witnesses the fact that the two strategies produce the same equivalence. In the propositionally truncated setting (Definition 2.26), we identify all such 2-paths, since we care only about the existence of *some* rewrite chain, not which one.

Definition 2.26 (Propositional Truncation)**.** A type A is *propositional* (or a *mere proposition*) if any two elements are connected by a path:

$$\mathsf{isProp}(A) \coloneqq \prod_{a,b:A} \mathsf{Path}_A(a, b)$$

The *propositional truncation* $\|A\|$ makes any type propositional by adding a path between every pair of elements.

Remark 2.27 (Truncation for Equivalence)**.** Program equivalence is a *proposition*: either $f \equiv g$ or $f \not\equiv g$, and there is at most one proof (up to higher paths). We work in the propositionally truncated path type $\|\mathsf{Path}_{A \to B}(f, g)\|$. This means we don't distinguish between different proofs of equivalence — we only care about the *existence* of a path. Computationally, `search_path` returns a single `PathResult` (the first path found by BFS), not the set of all possible rewrite chains.

Chapter 3

The Univalence Axiom

3.1 Type Equivalences

Definition 3.1 (Equivalence of Types)**.** A function $f : A \to B$ is an *equivalence* if there exists $g : B \to A$ with homotopies:

$$\eta : \prod_{a:A} \mathsf{Path}_A(g(f(a)),\ a) \tag{3.1}$$

$$\varepsilon : \prod_{b:B} \mathsf{Path}_B(f(g(b)),\ b) \tag{3.2}$$

satisfying the coherence condition

$$\prod_{a:A} \mathsf{Path}\big(\mathsf{ap}_f(\eta(a)),\ \varepsilon(f(a))\big).$$

We write $A \simeq B$ and call the quadruple $(f, g, \eta, \varepsilon)$ an *equivalence datum.*

Lemma 3.2 (Equivalence is an equivalence relation)**.** *For types $A, B, C : \mathsf{Type}$:*

(i) $A \simeq A$ (reflexivity)*, via* $(\mathrm{id}_A, \mathrm{id}_A, \mathsf{refl}, \mathsf{refl})$.

(ii) $A \simeq B \Rightarrow B \simeq A$ (symmetry)*, via* $(g, f, \varepsilon, \eta)$.

(iii) $A \simeq B$ *and* $B \simeq C \Rightarrow A \simeq C$ (transitivity)*, via composition of the underlying maps and concatenation of homotopies.*

Proof. Parts (i) and (ii) are immediate from Definition 3.1. For (iii), given $(f_1, g_1, \eta_1, \varepsilon_1) : A \simeq B$ and $(f_2, g_2, \eta_2, \varepsilon_2) : B \simeq C$, the composite equivalence is $(f_2 \circ f_1,\ g_1 \circ g_2)$ with section homotopy $\eta(a) \coloneqq \eta_1(a) \cdot \mathsf{ap}_{g_1}(\eta_2(f_1(a)))$ and retraction homotopy $\varepsilon(c) \coloneqq \varepsilon_2(c) \cdot \mathsf{ap}_{f_2}(\varepsilon_1(g_2(c)))$. Coherence follows from naturality of homotopies. □

3.2 The Univalence Axiom

Definition 3.3 (The canonical map $\mathsf{idToEquiv}$)**.** For $A, B : \mathsf{Type}$, define

$$\mathsf{idToEquiv} : (A =_{\mathsf{Type}} B) \to (A \simeq B)$$

by path induction: $\mathsf{idToEquiv}(\mathsf{refl}_A) \coloneqq (\mathrm{id}_A, \mathrm{id}_A, \mathsf{refl}, \mathsf{refl})$.

Axiom 3.4 (Univalence)**.** *For types $A, B : \mathsf{Type}$, the canonical map $\mathsf{idToEquiv} : (A =_{\mathsf{Type}} B) \to (A \simeq B)$ is itself an equivalence. Equivalently, there exists a map*

$$\mathsf{ua} : (A \simeq B) \to (A =_{\mathsf{Type}} B)$$

with $\mathsf{idToEquiv} \circ \mathsf{ua} \sim \mathrm{id}$ *and* $\mathsf{ua} \circ \mathsf{idToEquiv} \sim \mathrm{id}$.

Theorem 3.5 (Univalence in Cubical Type Theory)**.** *In cubical type theory, univalence is a* theorem *rather than an axiom. Given an equivalence $e : A \simeq B$ with underlying map f and inverse g, the path $\mathsf{ua}(e) : A =_{\mathsf{Type}} B$ is constructed as*

$$\mathsf{ua}(e)(i) \coloneqq \mathsf{Glue}\left[\,(i = \mathsf{i0}) \hookrightarrow (A, \mathrm{id}_A),\ (i = \mathsf{i1}) \hookrightarrow (A, e)\,\right] B$$

where Glue *is the glueing type former of CCHM cubical type theory. The Kan operations (composition and filling) on* Glue *types provide the computational content:* $\mathsf{transp}^{\mathsf{ua}(e)}(a) \equiv f(a)$.

Proof sketch. We verify that $\mathsf{ua}(e)$ satisfies the universal property. At $i = \mathsf{i0}$ the glueing reduces to A; at $i = \mathsf{i1}$ it reduces to B. The transport operation along $\mathsf{ua}(e)$ reduces to f by the computation rule for Glue: transport pushes through the equivalence datum. The inverse direction $\mathsf{idToEquiv}(\mathsf{ua}(e))$ recovers $(f, g, \eta, \varepsilon)$ by the β-rule for glueing, and $\mathsf{ua}(\mathsf{idToEquiv}(\mathsf{refl}))$ reduces to refl by the η-rule. See Cohen–Coquand–Huber–Mörtberg (2018) for the full construction. □

3.3 Transport and Its Computational Content

Definition 3.6 (Transport)**.** Given a type family $P : A \to \mathsf{Type}$ and a path $p : \mathsf{Path}_A(a, b)$, *transport* is the function

$$\mathsf{transp}^P(p) : P(a) \to P(b), \qquad \mathsf{transp}^P(p)(x) \coloneqq \mathsf{transp}\ \big(\lambda i.\, P(p(i))\big)\ \mathsf{i0}\ x.$$

When $p = \mathsf{ua}(f, g, \eta, \varepsilon)$, transport computes as $\mathsf{transp}^P(p)(x) \equiv f(x)$.

Proposition 3.7 (Transport respects composition)**.** *For paths $p : a =_A b$ and $q : b =_A c$ in a type family $P : A \to \mathsf{Type}$,*

$$\mathsf{transp}^P(p \cdot q) \sim \mathsf{transp}^P(q) \circ \mathsf{transp}^P(p).$$

Proof. By the composition operation (hcomp) on the interval: $\mathsf{transp}^P(p \cdot q)(x) = \mathsf{comp}^P\,[\,(0 = 1) \hookrightarrow \mathsf{transp}^P(q)\,]\,(\mathsf{transp}^P(p)(x))$, which reduces to $\mathsf{transp}^P(q)(\mathsf{transp}^P(p)(x))$ by the Kan filling equations. □

3.4 Univalence for Program Semantics

For our purposes, univalence has two computational consequences:

1. **Shared function symbols**: If two programs both call Python's `sorted`, these references denote the *same* function. In the universe Type, the type "Python's sorted function" has a unique representative. By univalence, any two copies of this type are equal. Computationally, we implement this as a shared Z3 function symbol: both programs use py_sorted.

2. **Type coercions as paths**: Python's type coercions (e.g., bool → int via `int(True)` = 1) are *equivalences* between types. By univalence, they become *paths* in Type. Transport along these paths relates the behaviors of programs that use different but coercible types.

Theorem 3.8 (Univalence for program types)**.** *Let $\mathcal{U}_{\mathsf{Py}}$ be the sub-universe of Python runtime types (int, float, bool, str, list, dict, ...). For Python types $\tau_1, \tau_2 : \mathcal{U}_{\mathsf{Py}}$, if there exists a Python coercion $c : \tau_1 \to \tau_2$ that is an equivalence (i.e., injective with a computable retraction), then $\mathsf{ua}(c) : \tau_1 =_{\mathcal{U}_{\mathsf{Py}}} \tau_2$.*

In particular, for any property $P : \mathcal{U}_{\mathsf{Py}} \to \mathsf{Type}$ that is universe-polymorphic *(depends only on the type's structure, not its name), a proof $P(\tau_1)$ transports to a proof $P(\tau_2)$ via $\mathsf{transp}^P(\mathsf{ua}(c))$.*

Proof. By Axiom 3.4 (or Theorem 3.5 in the cubical setting), it suffices to exhibit the equivalence datum. The coercion $c : \tau_1 \to \tau_2$ is the forward map. The retraction $r : \tau_2 \to \tau_1$ exists by hypothesis (e.g., $\mathsf{bool}(n)$ for $n \in \{0, 1\}$). The homotopies $r \circ c \sim \mathrm{id}_{\tau_1}$ and $c \circ r \sim \mathrm{id}_{\tau_2}$ hold on the relevant sub-type. The path $\mathsf{ua}(c, r, \eta, \varepsilon)$ is the required equality. Transport along this path computes as c itself, so $\mathsf{transp}^P(\mathsf{ua}(c))(p) = P(c)(p)$ carries the proof forward. □

Example 3.9 (Shared `sorted`)**.** Program f contains `sorted(x)` and program g contains `sorted(y)`. In the Z3 model, both compile to $\mathsf{py_sorted}([\![x]\!])$ and $\mathsf{py_sorted}([\![y]\!])$ respectively — the *same* Z3 function. Z3 can then focus on whether $[\![x]\!] = [\![y]\!]$, rather than comparing two independent uninterpreted functions.

Formally: the type $\mathsf{Sorted} \coloneqq \mathsf{list}(\alpha) \to \mathsf{list}(\alpha)$ has a unique inhabitant (up to equivalence) satisfying the sorting specification. By univalence, any two realizations of this type that satisfy the spec are identified in $\mathcal{U}_{\mathsf{Py}}$. The Z3 function symbol $\mathsf{py_sorted}$ is the image of this unique representative under the denotation functor $[\![\cdot]\!]$.

Example 3.10 (Bool–Int Coercion Path)**.** The coercion $c : \mathsf{bool} \simeq \{0, 1\} \hookrightarrow \mathsf{int}$ gives a path $p : \mathsf{Path}_{\mathsf{Type}}(\mathsf{bool}, \mathsf{int})$ (in the subuniverse of types with ≤ 2 values). If program f returns a `bool` and program g returns `0` or `1`, transport along p relates them.

Concretely: let $P(\tau) \coloneqq (\tau \to \mathsf{str})$ be the type of formatting functions. A formatter $\mathsf{fmt}_{\mathsf{bool}} : P(\mathsf{bool})$ transports to $\mathsf{transp}^P(p)(\mathsf{fmt}_{\mathsf{bool}}) = \mathsf{fmt}_{\mathsf{bool}} \circ c^{-1} : P(\mathsf{int})$, which formats $0 \mapsto$ `"False"`, $1 \mapsto$ `"True"`.

Example 3.11 (Transporting correctness via D1)**.** Consider iterative and recursive factorial:

$$f_{\mathsf{iter}}(n) \coloneqq \mathsf{fold}(\times, \mathsf{lit}(1), \mathsf{range}(1, n+1))$$
$$f_{\mathsf{rec}}(n) \coloneqq \mathsf{rec}(\mathsf{lit}(1), \lambda k.\, k \times (n - k + 1))$$

The D1 path constructor (Definition 4.2 below) provides $\mathsf{D1} : \mathsf{Path}_{\mathsf{PyComp}}(f_{\mathsf{rec}}(n),\ f_{\mathsf{iter}}(n))$. Suppose we have a correctness proof for the iterative version:

$$p_{\mathsf{iter}} : \prod_{n:\mathbb{N}} [\![f_{\mathsf{iter}}(n)]\!] = n!$$

Define the type family $P(t) \coloneqq ([\![t]\!] = n!)$. Then transport gives:

$$p_{\mathsf{rec}} \coloneqq \mathsf{transp}^P(\mathsf{D1}^{-1})(p_{\mathsf{iter}}) : \prod_{n:\mathbb{N}} [\![f_{\mathsf{rec}}(n)]\!] = n!$$

The correctness proof transfers automatically because the denotation functor $[\![\cdot]\!]$ respects the D1 path (both sides denote the same Z3 expression after the path axiom is applied to the solver). This is the key payoff of univalence: *prove once, transport everywhere.*

Chapter 4

Higher Inductive Types for Programs

4.1 Definition and Motivation

A *higher inductive type* (HIT) is a type defined by:

1. **Point constructors**: elements of the type.
2. **Path constructors**: equalities between elements.
3. **Higher path constructors**: equalities between equalities (optional — for coherence).

For program equivalence, we define the HIT of Python computations. The point constructors are the syntactic forms (literals, operations, conditionals, loops, etc.). The path constructors are the *equational axioms* that identify syntactically different but semantically equivalent expressions.

Remark 4.1 (Why a HIT and not a quotient type?)**.** A standard quotient A/R only identifies elements. A HIT additionally gives *names* to the identifications (the path constructors), making them available for computation. When we transport a proof along the D1 path, we need to know *which* axiom justifies the equality — the path constructor carries this information. In the implementation, each `PathStep` records the axiom name, position, and before/after terms.

4.2 The HIT PyComp

Definition 4.2 (The HIT of Python Computations)**.** PyComp is the higher inductive type generated by the following data.

Point constructors (corresponding to OTerm variants in the implementation):

$$\mathsf{lit} : \mathbb{Z} \to \mathsf{PyComp} \qquad \text{(OLit)} \tag{4.1}$$
$$\mathsf{binop} : \mathsf{OpTag} \times \mathsf{PyComp} \times \mathsf{PyComp} \to \mathsf{PyComp} \qquad \text{(OOp, arity 2)} \tag{4.2}$$
$$\mathsf{unop} : \mathsf{OpTag} \times \mathsf{PyComp} \to \mathsf{PyComp} \qquad \text{(OOp, arity 1)} \tag{4.3}$$
$$\mathsf{cond} : \mathsf{PyComp} \times \mathsf{PyComp} \times \mathsf{PyComp} \to \mathsf{PyComp} \qquad \text{(OCase)} \tag{4.4}$$
$$\mathsf{fold} : \mathsf{OpTag} \times \mathsf{PyComp} \times \mathsf{PyComp} \to \mathsf{PyComp} \qquad \text{(OFold)} \tag{4.5}$$
$$\mathsf{fix} : \mathsf{Name} \times \mathsf{PyComp} \to \mathsf{PyComp} \qquad \text{(OFix)} \tag{4.6}$$
$$\mathsf{seq} : \mathsf{PyComp}^* \to \mathsf{PyComp} \qquad \text{(OSeq)} \tag{4.7}$$
$$\mathsf{dict} : (\mathsf{PyComp} \times \mathsf{PyComp})^* \to \mathsf{PyComp} \qquad \text{(ODict)} \tag{4.8}$$
$$\mathsf{lam} : \mathsf{Name}^* \times \mathsf{PyComp} \to \mathsf{PyComp} \qquad \text{(OLam)} \tag{4.9}$$
$$\mathsf{map} : \mathsf{PyComp} \times \mathsf{PyComp} \to \mathsf{PyComp} \qquad \text{(OMap)} \tag{4.10}$$
$$\mathsf{catch} : \mathsf{PyComp} \times \mathsf{PyComp} \to \mathsf{PyComp} \qquad \text{(OCatch)} \tag{4.11}$$
$$\mathsf{quot} : \mathsf{PyComp} \times \mathsf{EqClass} \to \mathsf{PyComp} \qquad \text{(OQuotient)} \tag{4.12}$$
$$\mathsf{abstract} : \mathsf{Spec} \times \mathsf{PyComp}^* \to \mathsf{PyComp} \qquad \text{(OAbstract)} \tag{4.13}$$

Path constructors. We list all 24 dichotomy paths and the algebraic paths. Each path constructor is an element of an identity type in PyComp.

Arithmetic paths:

$$\mathsf{add_comm} : \prod_{a,b} \mathsf{Path}(\mathsf{binop}(+, a, b),\ \mathsf{binop}(+, b, a)) \tag{4.14}$$
$$\mathsf{mul_comm} : \prod_{a,b} \mathsf{Path}(\mathsf{binop}(\times, a, b),\ \mathsf{binop}(\times, b, a)) \tag{4.15}$$
$$\mathsf{add_assoc} : \prod_{a,b,c} \mathsf{Path}(\mathsf{binop}(+, \mathsf{binop}(+, a, b), c),\ \mathsf{binop}(+, a, \mathsf{binop}(+, b, c))) \tag{4.16}$$
$$\mathsf{mul_assoc} : \prod_{a,b,c} \mathsf{Path}(\mathsf{binop}(\times, \mathsf{binop}(\times, a, b), c),\ \mathsf{binop}(\times, a, \mathsf{binop}(\times, b, c))) \tag{4.17}$$
$$\mathsf{add_zero} : \prod_{a} \mathsf{Path}(\mathsf{binop}(+, a, \mathsf{lit}(0)),\ a) \tag{4.18}$$
$$\mathsf{mul_one} : \prod_{a} \mathsf{Path}(\mathsf{binop}(\times, a, \mathsf{lit}(1)),\ a) \tag{4.19}$$
$$\mathsf{mul_zero} : \prod_{a} \mathsf{Path}(\mathsf{binop}(\times, a, \mathsf{lit}(0)),\ \mathsf{lit}(0)) \tag{4.20}$$
$$\mathsf{neg_neg} : \prod_{a} \mathsf{Path}(\mathsf{unop}(-, \mathsf{unop}(-, a)),\ a) \tag{4.21}$$
$$\mathsf{abs_idem} : \prod_{a} \mathsf{Path}(\mathsf{unop}(|\cdot|, \mathsf{unop}(|\cdot|, a)),\ \mathsf{unop}(|\cdot|, a)) \tag{4.22}$$
$$\mathsf{min_comm} : \prod_{a,b} \mathsf{Path}(\mathsf{binop}(\min, a, b),\ \mathsf{binop}(\min, b, a)) \tag{4.23}$$
$$\mathsf{max_comm} : \prod_{a,b} \mathsf{Path}(\mathsf{binop}(\max, a, b),\ \mathsf{binop}(\max, b, a)) \tag{4.24}$$

Additional algebraic paths (comparison flips, Boolean involution, identity elements):

$$\mathsf{comp_flip} : \prod_{a,b} \mathsf{Path}(\mathsf{binop}(<, a, b),\ \mathsf{binop}(>, b, a)) \tag{4.25}$$

$$\mathsf{not_invol} : \prod_{a} \mathsf{Path}(\mathsf{unop}(\neg, \mathsf{unop}(\neg, a)),\ a) \tag{4.26}$$

$$\mathsf{zero_add} : \prod_{a} \mathsf{Path}(\mathsf{binop}(+, \mathsf{lit}(0), a),\ a) \tag{4.27}$$

$$\mathsf{one_mul} : \prod_{a} \mathsf{Path}(\mathsf{binop}(\times, \mathsf{lit}(1), a),\ a) \tag{4.28}$$

The 24 dichotomy paths (D1–D24):

Control flow dichotomies (D1–D8):

$$\mathsf{D1} : \prod_{\mathsf{op},\mathsf{init},\mathsf{coll}} \mathsf{Path}\big(\mathsf{fix}[h](\mathsf{body}),\ \mathsf{fold}(\mathsf{op}, \mathsf{init}, \mathsf{coll})\big) \qquad \text{rec} \leftrightarrow \text{iter} \tag{4.29}$$

$$\mathsf{D2} : \prod_{f,a} \mathsf{Path}\big(\mathsf{binop}(\mathsf{apply}, \mathsf{lam}(\vec{x}, \mathsf{body}), a),\ \mathsf{body}[\vec{x} \mapsto a]\big) \qquad \beta\text{-reduction} \tag{4.30}$$

$$\mathsf{D3} : \prod_{\mathsf{init},\mathsf{step},n} \mathsf{Path}\big(\mathsf{while}(\mathsf{init}, \mathsf{step}, n),\ \mathsf{for}(\mathsf{init}, \mathsf{step}, n)\big) \qquad \text{while} \leftrightarrow \text{for} \tag{4.31}$$

$$\mathsf{D4} : \prod_{f,g,c} \mathsf{Path}\big(\mathsf{map}(f, \mathsf{map}(g, c)),\ \mathsf{map}(f \circ g, c)\big) \qquad \text{map fusion} \tag{4.32}$$

$$\mathsf{D5} : \prod_{\mathsf{op},e,c} \mathsf{Path}\big(\mathsf{fold}(\mathsf{op}, e, \mathsf{map}(f, c)),\ \mathsf{fold}(\mathsf{op} \circ f, e, c)\big) \qquad \text{fold universality} \tag{4.33}$$

$$\mathsf{D6} : \prod_{c} \mathsf{Path}\big(\mathsf{list}(\mathsf{gen}(c)),\ \mathsf{comp}(c)\big) \qquad \text{lazy} \leftrightarrow \text{eager} \tag{4.34}$$

$$\mathsf{D7} : \prod_{\mathsf{op},e,c} \mathsf{Path}\big(\mathsf{fix}[\mathsf{tail}](\mathsf{body}),\ \mathsf{fold}(\mathsf{op}, e, c)\big) \qquad \text{tail-rec} \leftrightarrow \text{loop} \tag{4.35}$$

$$\mathsf{D8} : \prod_{g,A,B,C} \mathsf{Path}\big(\mathsf{cond}(g, A, \mathsf{cond}(\neg g, B, C)),\ \mathsf{cond}(g, A, C)\big) \qquad \text{section merge} \tag{4.36}$$

Data structure dichotomies (D9–D14):

$$\mathsf{D9} : \prod_{\mathsf{body}} \mathsf{Path}\big(\mathsf{fix}[\mathsf{stack}](\mathsf{body}),\ \mathsf{fix}[\mathsf{rec}](\mathsf{body}')\big) \qquad \text{stack} \leftrightarrow \text{recursion} \tag{4.37}$$

$$\mathsf{D10} : \prod_{\mathsf{pq}} \mathsf{Path}\big(\mathsf{heapq}(\mathsf{pq}),\ \mathsf{bisect}(\mathsf{pq})\big) \qquad \text{PQ abstraction} \tag{4.38}$$

$$\mathsf{D11} : \prod_{\mathsf{ops}} \mathsf{Path}\big(\mathsf{OrderedDict}(\mathsf{ops}),\ \mathsf{LinkedList}(\mathsf{ops})\big) \qquad \text{ADT refinement} \tag{4.39}$$

$$\mathsf{D12} : \prod_{d,k,v} \mathsf{Path}\big(\mathsf{defaultdict}(d, k, v),\ \mathsf{setdefault}(d, k, v)\big) \qquad \text{default maps} \tag{4.40}$$

$$\mathsf{D13} : \prod_{c} \mathsf{Path}\big(\mathsf{Counter}(c),\ \mathsf{fold}(\mathsf{count}, \mathsf{defaultdict}(0), c)\big) \qquad \text{histogram} \tag{4.41}$$

$$\mathsf{D14} : \prod_{a} \mathsf{Path}\big(\mathsf{array_idx}(a),\ \mathsf{dict_lookup}(a)\big) \qquad \text{dense indexing} \tag{4.42}$$

Algorithm strategy dichotomies (D15–D20):

$$\mathsf{D15}: \prod_{\substack{\mathsf{op}\ \text{comm.}\\ G,s}} \mathsf{Path}\big(\mathsf{fold}(\mathsf{op}, s, \mathsf{BFS}(G)),\ \mathsf{fold}(\mathsf{op}, s, \mathsf{DFS}(G))\big) \qquad \text{BFS} \leftrightarrow \text{DFS} \tag{4.43}$$

$$\mathsf{D16}: \prod_{\mathsf{rec}} \mathsf{Path}\big(\mathsf{fix}[\mathsf{memo}](\mathsf{rec}),\ \mathsf{fix}[\mathsf{dp}](\mathsf{rec})\big) \qquad \text{memo} \leftrightarrow \text{DP} \tag{4.44}$$

$$\mathsf{D17}: \prod_{\substack{\mathsf{op}\ \text{assoc.}\\ e,c}} \mathsf{Path}\big(\mathsf{fold}(\mathsf{op}, e, \mathsf{tree_split}(c)),\ \mathsf{fold}(\mathsf{op}, e, c)\big) \qquad \text{D\&C} \leftrightarrow \text{iter} \tag{4.45}$$

$$\mathsf{D18}: \prod_{\substack{M\ \text{matroid}\\ w}} \mathsf{Path}\big(\mathsf{greedy}(M, w),\ \mathsf{dp}(M, w)\big) \qquad \text{greedy} \leftrightarrow \text{DP} \tag{4.46}$$

$$\mathsf{D19}: \prod_{\substack{\mathsf{op}\ \text{comm.+assoc.}\\ e,c}} \mathsf{Path}\big(\mathsf{fold}(\mathsf{op}, e, \mathsf{sorted}(c)),\ \mathsf{fold}(\mathsf{op}, e, c)\big) \qquad \text{sort-scan} \leftrightarrow \text{sweep} \tag{4.47}$$

$$\mathsf{D20}: \prod_{\substack{t_1,t_2\ \text{s.t.}\\ \mathsf{spec}(t_1)=\mathsf{spec}(t_2)}} \mathsf{Path}\big(t_1,\ t_2\big) \qquad \text{Yoneda / spec unify} \tag{4.48}$$

Language feature dichotomies (D21–D24):

$$\mathsf{D21}: \prod_{\mathsf{cases}} \mathsf{Path}\big(\mathsf{isinstance_chain}(\mathsf{cases}),\ \mathsf{dispatch_table}(\mathsf{cases})\big) \qquad \text{dispatch} \tag{4.49}$$

$$\mathsf{D22}: \prod_{b,d} \mathsf{Path}\big(\mathsf{catch}(b, d),\ \mathsf{cond}(\mathsf{guard}(b), b, d)\big) \qquad \text{try} \leftrightarrow \text{cond} \tag{4.50}$$

$$\mathsf{D23}: \prod_{\substack{\mathsf{op}\ \text{order-inv.}\\ c}} \mathsf{Path}\big(\mathsf{process}(\mathsf{sorted}(c)),\ \mathsf{process}(c)\big) \qquad \text{sort-process} \tag{4.51}$$

$$\mathsf{D24}: \prod_{f} \mathsf{Path}\big(\mathsf{lam}(x,\ \mathsf{binop}(\mathsf{apply}, f, x)),\ f\big) \qquad \eta\text{-expansion} \tag{4.52}$$

Truncation constructor:

$$\mathsf{trunc}: \prod_{a,b:\mathsf{PyComp}} \prod_{p,q:\mathsf{Path}(a,b)} \mathsf{Path}_{\mathsf{Path}(a,b)}(p, q) \tag{4.53}$$

4.3 The 0-Truncation Condition

Definition 4.3 (0-Truncated type (set))**.** A type A is *0-truncated* (or a *set*) if for all $a, b : A$ and all $p, q : a =_A b$, we have $p = q$. Equivalently, $\mathsf{isSet}(A) := \prod_{a,b:A} \prod_{p,q:a=_A b} p =_{a=_A b} q$.

Proposition 4.4 (Why PyComp must be 0-truncated)**.** *The truncation constructor in Definition 4.2 is necessary for the following reasons:*

(i) ***Decidable equality.*** *We want program equivalence to be a* proposition *(mere truth value), not a structured type. The Z3 solver returns* sat/unsat*, not a witness of higher structure.*

(ii) ***Unique elimination.*** *The elimination principle (Theorem 4.5) requires the codomain to be a set. Without truncation, we would need to verify that h respects path-between-paths, which is vacuously true when all such higher paths are trivial.*

(iii) ***Coherence of axiom composition.*** *Multiple axiom sequences may connect the same two programs. For instance, $f \xrightarrow{\mathsf{D1}} g$ and $f \xrightarrow{\mathsf{D5}} h \xrightarrow{\mathsf{D4}} g$ yield two paths $f = g$. The truncation condition ensures these are equal, reflecting that we care about* whether *an equivalence holds, not* which *proof witness we chose.*

Proof. For (i): the path type $a =_{\mathsf{PyComp}} b$ must be a proposition ((−1)-truncated) for elimination into propositions (e.g., the Boolean output of Z3) to be well-defined. A 0-truncated type has propositional identity types by definition.

For (ii): given $h : \mathsf{PyComp} \to X$ with X a set, the path-respecting condition requires $\mathsf{ap}_h(p) = \mathsf{ap}_h(q)$ for parallel paths p, q. When PyComp is 0-truncated, $p = q$ by the truncation constructor, so $\mathsf{ap}_h(p) = \mathsf{ap}_h(q)$ by congruence.

For (iii): the bidirectional BFS in the implementation may discover multiple paths. The truncation ensures that all discovered paths yield the same transported result, making the verifier deterministic. □

4.4 The Elimination Principle

Theorem 4.5 (Elimination for PyComp). *Let X be a 0-truncated type (set). To define a function $h : \mathsf{PyComp} \to X$, it suffices to give:*

(a) Point cases. *A value for each point constructor:*

$$\begin{aligned} h_{\mathsf{lit}} &: \mathbb{Z} \to X \\ h_{\mathsf{binop}} &: \mathsf{OpTag} \times X \times X \to X \\ h_{\mathsf{unop}} &: \mathsf{OpTag} \times X \to X \\ h_{\mathsf{cond}} &: X \times X \times X \to X \\ h_{\mathsf{fold}} &: \mathsf{OpTag} \times X \times X \to X \\ h_{\mathsf{fix}} &: \mathsf{Name} \times X \to X \\ &\vdots \quad \textit{(one case per point constructor)} \end{aligned}$$

(b) Path equations. *For each path constructor, a proof that h respects it. For example, for D1:*

$$h_{\mathsf{D1}} : \prod_{\mathsf{op},\mathsf{init},\mathsf{coll}} h_{\mathsf{fix}}(\mathsf{name}, h(\mathsf{body})) = h_{\mathsf{fold}}(\mathsf{op}, h(\mathsf{init}), h(\mathsf{coll}))$$

and similarly for all 24 dichotomy paths and the algebraic paths.

(c) Truncation coherence. *The condition that X is a set (which is a property of X, not additional data).*

Then there exists a unique *function $h : \mathsf{PyComp} \to X$ satisfying these equations.*

Proof. We construct h by structural recursion on the HIT generators.

Point constructors. Define h on each point constructor by the given cases (a). For $\mathsf{lit}(n)$, set $h(\mathsf{lit}(n)) \coloneqq h_{\mathsf{lit}}(n)$. For $\mathsf{binop}(\mathsf{op}, a, b)$, set $h(\mathsf{binop}(\mathsf{op}, a, b)) \coloneqq h_{\mathsf{binop}}(\mathsf{op}, h(a), h(b))$, and similarly for all constructors.

Path constructors. For each path constructor $p : \mathsf{Path}(t_1, t_2)$ in PyComp, we must show $\mathsf{ap}_h(p) : h(t_1) =_X h(t_2)$. By the path equations (b), the left and right sides are equal in X. Concretely, for $\mathsf{add_comm}(a, b) : \mathsf{binop}(+, a, b) = \mathsf{binop}(+, b, a)$, we need $h_{\mathsf{binop}}(+, h(a), h(b)) = h_{\mathsf{binop}}(+, h(b), h(a))$, which is exactly the commutativity equation for $h_{\mathsf{binop}}(+, -, -)$.

Truncation. For the truncation constructor $\mathsf{trunc}(a, b, p, q) : p =_{a=b} q$, we must show $\mathsf{ap}_h(p) =_{h(a)=h(b)} \mathsf{ap}_h(q)$. Since X is a set (condition (c)), all parallel paths in X are equal, so this holds automatically.

Uniqueness. Suppose $h, h' : \mathsf{PyComp} \to X$ both satisfy the equations. By induction on PyComp: on point constructors, h and h' agree by the equations; on path constructors, the agreement is a path-between-paths in X, which exists because X is a set. Hence $h = h'$. □

Corollary 4.6 (The denotation functor respects PyComp)**.** *The Z3 denotation functor* $[\![\cdot]\!] : \mathsf{PyComp} \to \mathsf{Z3Term}$ *is the unique function satisfying:*

(i) $[\![\mathsf{lit}(n)]\!] = n$ *(Z3 integer literal),*

(ii) $[\![\mathsf{binop}(\mathsf{op}, a, b)]\!] = \mathsf{z3_op}([\![a]\!], [\![b]\!])$*,*

(iii) $[\![\mathsf{fold}(\mathsf{op}, e, c)]\!]$ *encodes the fold as a Z3 recursive function,*

(iv) for each path constructor p*, the axiom* $[\![p(\mathsf{i0})]\!] = [\![p(\mathsf{i1})]\!]$ *is added to the Z3 solver context.*

Proof. Z3Term with equality modulo the solver's axiom set is a set (Z3 terms have decidable equality). The point cases (i)–(iii) define the action on constructors. The path equations (iv) are discharged by adding each axiom to the solver: when the solver assumes $[\![\mathsf{binop}(+, a, b)]\!] = [\![\mathsf{binop}(+, b, a)]\!]$, it exactly validates the commutativity path. Uniqueness follows from Theorem 4.5. □

4.5 The Free Algebra Interpretation

PyComp is the *free algebra* generated by the Python operations modulo the equational axioms:

$$\mathsf{PyComp} \cong \mathsf{Free}_{\Sigma/E}(X)$$

where Σ is the algebraic signature (point constructors), E is the equational theory (path constructors), and X is the set of program variables.

Theorem 4.7 (PyComp as set-truncated free algebra)**.** *The type* PyComp *is equivalent to the 0-truncation of the free* Σ*-algebra on* X *quotiented by* E*:*

$$\mathsf{PyComp} \simeq \left\| T_\Sigma(X)/E \right\|_0$$

where $T_\Sigma(X)$ *is the free term algebra and* E *is the smallest congruence containing all path constructor equations.*

Proof. By the universal property (Theorem 4.5), for any set A equipped with a Σ-algebra structure satisfying the equations E, there is a unique homomorphism $\mathsf{PyComp} \to A$. This is exactly the universal property of $\|T_\Sigma(X)/E\|_0$. By the Yoneda lemma (applied in the category of sets with Σ-algebra structures satisfying E), the two types represent the same functor and are therefore equivalent. □

Corollary 4.8 (Completeness of path search)**.** *Two programs* f, g *are equivalent in* PyComp *if and only if they are connected by a finite sequence of path constructor applications (in either direction), possibly under congruence (applying an axiom inside a sub-term).*

Proof. The equivalence $\mathsf{PyComp} \simeq \|T_\Sigma(X)/E\|_0$ shows that paths in PyComp correspond exactly to equivalence classes in $T_\Sigma(X)/E$. Two terms are in the same equivalence class iff they are connected by a finite chain of E-rewrites (applied under congruence), which is exactly the path search algorithm's bidirectional BFS. □

Chapter 5

Kan Operations and Composition

5.1 Open Boxes and Fillers

The *Kan condition* is the defining property of cubical types: every open box has a filler. In our setting, "open boxes" arise naturally from partial rewrite sequences: we know a term rewrites along several directions but have not yet found the connecting lid.

Definition 5.1 (Open Box)**.** An *open n-box in direction i and side b* in a type A consists of:

- A *bottom face*: $u : A$ (the base of the box).
- Faces for all directions $j \neq i$ and sides $c \in \{0, 1\}$: $\alpha_{j,c} : \mathbb{I} \to A$ with boundary conditions matching u and each other on shared edges.
- The face (i, b) is *missing* — this is the "open" side.

Definition 5.2 (Kan Filler)**.** The *Kan filler* $\mathsf{hfill}^i_b(\alpha)$ is a term $\mathsf{hfill}^i_b(\alpha) : \mathbb{I} \to A$ that:

1. Agrees with u at $i = b$: $\mathsf{hfill}^i_b(\alpha)(b) = u$.
2. Agrees with each $\alpha_{j,c}$ at $j = c$: $\mathsf{hfill}^i_b(\alpha)(j = c) = \alpha_{j,c}$.
3. The missing face $i = 1 - b$ is the *lid* of the box.

5.2 Composition

Composition is the operation that composes two paths:

Definition 5.3 (Homogeneous Composition)**.** Given $A : \mathsf{Type}$, $a : A$, and a partial path $\varphi \vdash u : A$ with $u[\varphi] = a$:

$$\mathsf{comp}^i(A, [\varphi \mapsto u], a) : A$$

This is the "lid" of the Kan filling of the open box with base a and sides u.

For program equivalence, composition means: if we can transform f into h (by one axiom application) and h into g (by another), we can compose these transformations to get $f \equiv g$.

5.3 Kan Filling as Multi-Step Rewriting

The path search implementation (`path_search.py`) makes the Kan filling condition computationally explicit. Each rewrite step constructs one face of a cubical box; the filling operation composes them into a connected path.

Definition 5.4 (Rewrite Frontier)**.** Let t be a term in PyComp. The *rewrite frontier* of t is

$$\mathsf{Rew}(t, \Gamma) := \left\{ (t', \mathsf{ax}) \;\middle|\; \mathsf{ax} \in \Sigma_E,\ t \to_{\mathsf{ax}} t' \right\}$$

where Σ_E is the set of all path constructors (axioms D1–D24 plus algebraic identities) and Γ is the fiber context restricting which axioms are applicable.

The implementation computes this frontier in two phases:

1. **Root axioms**: apply each path constructor at the top-level node of the term.
2. **Congruence rewrites**: for each sub-term position, recursively compute the frontier and lift the result by reconstructing the surrounding context.

Definition 5.5 (Congruence Lifting)**.** Given a term $t = C[s]$ (context C with hole filled by sub-term s) and a one-step rewrite $s \to_{\mathsf{ax}} s'$, the *congruence lifting* is:

$$C[s] \to_{\mathsf{ax@pos}} C[s']$$

This corresponds to the congruence rule of cubical type theory: if $p : \mathsf{Path}_A(s, s')$ then $\mathsf{ap}_C(p) : \mathsf{Path}_A(C[s], C[s'])$.

Remark 5.6 (Congruence as `ap`)**.** The function `_congruence_rewrites` iterates over all sub-term positions (arguments of OOp, branches of OCase, body of OFold, etc.) and recursively calls `_all_rewrites` on each sub-term. Each returned rewrite is lifted by rebuilding the parent node. The annotation $\mathsf{ax@arg}i$ records both the axiom and the position, exactly mirroring the cubical ap_C combinator applied at a specific sub-expression.

Theorem 5.7 (Soundness of Multi-Step Composition)**.** *Let $t_0, t_1, \ldots, t_k$ be terms in PyComp with one-step rewrite paths $p_i : \mathsf{Path}(t_{i-1}, t_i)$ for $i = 1, \ldots, k$. Then the composite*

$$p_1 \cdot p_2 \cdot \ldots \cdot p_k \;:\; \mathsf{Path}(t_0, t_k)$$

is a well-formed path in PyComp, and hence $t_0 \equiv t_k$ in the quotient algebra Σ/E.

Proof. Each p_i is a path constructor (axiom application or congruence lifting thereof), so it inhabits $\mathsf{Path}(t_{i-1}, t_i)$ by construction. Homogeneous composition $\mathsf{comp}^i(A, [\varphi \mapsto p_i], t_{i-1})$ produces a filler whose lid is t_i. Iterating k times yields a composite path from t_0 to t_k. The resulting path lies in PyComp because the quotient algebra is closed under the equational theory. □

5.4 Bidirectional BFS as Kan Search

The path search algorithm uses bidirectional breadth-first search (BFS), expanding frontiers from both the source t_{src} and target t_{tgt} simultaneously.

Definition 5.8 (Bidirectional Frontier Search)**.** The *forward frontier* F_d at depth d and *backward frontier* B_d are defined inductively:

$$F_0 := \{t_{\mathsf{src}}\}, \qquad B_0 := \{t_{\mathsf{tgt}}\} \tag{5.1}$$

$$F_{d+1} := F_d \cup \bigcup_{t \in F_d} \pi_1(\mathsf{Rew}(t, \Gamma)), \qquad B_{d+1} := B_d \cup \bigcup_{t \in B_d} \pi_1(\mathsf{Rew}(t, \Gamma)) \tag{5.2}$$

A path is found when $F_d \cap B_d \neq \emptyset$ for some d. The meeting point $m \in F_d \cap B_d$ yields a path $t_{\mathsf{src}} \to^* m \leftarrow^* t_{\mathsf{tgt}}$, which by path reversal and composition gives $t_{\mathsf{src}} \equiv t_{\mathsf{tgt}}$.

Proposition 5.9 (Completeness within Depth Bound)**.** *If t_{src} and t_{tgt} are connected by a path of length $\leq 2d$ in the rewrite graph, bidirectional BFS with depth bound d will find a connecting path.*

Proof. At depth d, F_d contains all terms reachable from t_{src} in $\leq d$ steps, and B_d contains all terms reachable from t_{tgt} in $\leq d$ steps. If the shortest path has length $\leq 2d$, some intermediate term lies in both frontiers. □

5.5 Composition for Multi-Step Proofs

Example 5.10 (Three-Step Equivalence)**.** Consider programs:

$$f = \mathsf{sum}(\mathsf{sorted}(\mathsf{reversed}(x))) \tag{5.3}$$

$$g = \mathsf{sum}(x) \tag{5.4}$$

The equivalence proof is a three-step path:

$$p_1 : \mathsf{Path}(f,\ \mathsf{sum}(\mathsf{sorted}(x))) \quad \text{(by } \mathsf{reversed} \text{ involution + } \mathsf{sorted} \text{ perm-invariance)} \tag{5.5}$$

$$p_2 : \mathsf{Path}(\mathsf{sum}(\mathsf{sorted}(x)),\ \mathsf{sum}(x)) \quad \text{(by } \mathsf{sum} \text{ perm-invariance)} \tag{5.6}$$

The composite $p_1 \cdot p_2 : \mathsf{Path}(f, g)$ is obtained by Kan composition.

In the BFS implementation, the forward frontier starting from f expands to include $\mathsf{sum}(\mathsf{sorted}(x))$ at depth 1 (via axiom `QUOT_sorted_absorb` applied at argument position 0, yielding annotation `QUOT_sorted_absorb@arg0`). The backward frontier starting from g expands to include $\mathsf{sum}(\mathsf{sorted}(x))$ at depth 1 (via axiom `D19_sort_irrelevant`). The frontiers meet, and the composed path is returned.

Example 5.11 (Fiber-Aware Kan Composition)**.** Consider proving $\mathsf{add}(a, b) \equiv \mathsf{add}(b, a)$. The axiom `ALG_commute` applies only when the fiber context Γ satisfies $\Gamma.\mathsf{is_numeric}(t) = \mathsf{true}$. If a has duck type str, then add is string concatenation and commutativity does not hold — the path constructor is *not available* on the str fiber.

This fiber restriction ensures that the Kan filling operation respects the sheaf structure: a path $p : \mathsf{Path}(t_0, t_1)$ on fiber U may not extend to a path on a wider fiber $V \supseteq U$. The `FiberCtx` parameter threads this restriction through every call to `_all_rewrites`.

Part II

Cohomology Theory

Chapter 6

Sites, Presheaves, and Sheaves

6.1 The Type Site

Definition 6.1 (Type Site $\mathcal{S}$)**.** The *type site* $\mathcal{S}$ is the category whose:

- **Objects** are *type fiber assignments* $U = (\tau_1, \ldots, \tau_n) \in \mathsf{TypeTag}^n$, specifying the type of each parameter. We write $\mathsf{TypeTag} = \{\mathsf{int}, \mathsf{bool}, \mathsf{str}, \mathsf{pair}, \mathsf{ref}, \mathsf{none}\}$.
- **Morphisms** $U \to V$ are *type widenings*: $(U \to V)$ iff τ_i^U is a subtype of τ_i^V for all i.
- **Covering families**: for each object U, the covering $\{U_\tau\}_{\tau \in T}$ consists of all single-fiber restrictions.

Remark 6.2 (Enumeration of Objects)**.** For a function with n parameters, each having $|\mathsf{TypeTag}| = 6$ possible tags, the site has at most 6^n objects. The implementation caps this at 12 fiber combinations to prevent Z3 memory blow-up: $|\mathrm{Ob}(\mathcal{S})| \leq \min(6^n, 12)$.

Definition 6.3 (Overlap Relation)**.** Two fiber assignments $U_i, U_j \in \mathrm{Ob}(\mathcal{S})$ *overlap* if they share at least one parameter axis:

$$U_i \cap U_j \neq \emptyset \quad \iff \quad \exists\, k \in \{1, \ldots, n\}.\ \tau_k^{U_i} = \tau_k^{U_j}$$

The implementation computes this as: `any(ci[k] == cj[k] for k in range(len(ci)))`.

Example 6.4 (Site for a Unary Function)**.** For $f(x)$ where x can be int, str, or list, the site $\mathcal{S}$ has objects U_{int}, U_{str}, U_{list}, and a "top" object $U_{\mathsf{any}} = U_{\mathsf{int}} \cup U_{\mathsf{str}} \cup U_{\mathsf{list}}$. Since $n = 1$, every pair of distinct fibers has trivially empty overlap (their single axis differs), so no overlap edges exist and the Čech complex is concentrated in degree 0.

Example 6.5 (Site for a Binary Function)**.** For $g(x, y)$ with $x \in \{\mathsf{int}, \mathsf{str}\}$ and $y \in \{\mathsf{int}, \mathsf{str}\}$, the site has four objects:

$$U_{(\mathsf{int},\mathsf{int})}, \quad U_{(\mathsf{int},\mathsf{str})}, \quad U_{(\mathsf{str},\mathsf{int})}, \quad U_{(\mathsf{str},\mathsf{str})}$$

The overlap relation pairs fibers sharing an axis: $U_{(\mathsf{int},\mathsf{int})}$ overlaps $U_{(\mathsf{int},\mathsf{str})}$ (shared first axis int) and $U_{(\mathsf{str},\mathsf{int})}$ (shared second axis int). This yields a non-trivial Čech complex with potentially non-zero H^1.

6.2 Duck Type Inference and the Grothendieck Topology

The Grothendieck topology on $\mathcal{S}$ determines which coverings are "sufficient" for gluing. In our setting, *duck type inference* (`duck.py`) narrows the coverings by eliminating irrelevant fibers.

Definition 6.6 (Duck Sieve)**.** For a parameter p of a function f, the *duck sieve* $\mathsf{Duck}(f,p)$ is the set of type tags τ compatible with all operations applied to p in f:

$$\mathsf{Duck}(f,p) \coloneqq \left\{\tau \in \mathsf{TypeTag} \;\middle|\; \mathsf{ops}(f,p) \subseteq \mathsf{ops}_\tau\right\}$$

where $\mathsf{ops}(f,p)$ is the set of operations applied to p in f's AST, and ops_τ is the set of operations supported by type τ.

Example 6.7 (Duck Sieve Narrowing)**.** If parameter p is used in $p-1$ and $p\%2$, then $\mathsf{ops}(f,p) = \{\mathsf{sub}, \mathsf{mod}\}$. Since $\mathsf{sub}, \mathsf{mod} \in \mathsf{ops}_{\mathsf{int}}$ but $\mathsf{sub} \notin \mathsf{ops}_{\mathsf{str}}$, the duck sieve narrows to $\mathsf{Duck}(f,p) = \{\mathsf{int}\}$. The site collapses to a single fiber, and the equivalence check reduces to a single Z3 query on the int fiber.

Proposition 6.8 (Duck Sieve is a Covering Sieve)**.** *Let f, g be two programs with matching parameter arities. Define the duck sieve for each parameter as $T_k \coloneqq \mathsf{Duck}(f,p_k) \cup \mathsf{Duck}(g,p_k)$. Then $\{U_{(\tau_1,\ldots,\tau_n)}\}_{(\tau_1,\ldots,\tau_n)\in T_1\times\cdots\times T_n}$ is a covering of U_{any} in the Grothendieck topology on $\mathcal{S}$.*

Proof. The duck sieve for each parameter p_k includes every type tag on which f or g can be evaluated without type error. Inputs whose type tags lie outside the duck sieve produce runtime type errors in at least one program, so both programs are equally undefined on those inputs. The remaining fibers cover all well-typed inputs. □

6.3 Tag Constraints and Z3 Sections

For each type tag τ, the implementation associates a Z3 constraint that restricts the tagged union to the corresponding fiber.

Definition 6.9 (Tag Constraint Map)**.** The *tag constraint map* $\mathsf{tc} : \mathsf{TypeTag} \to (p \mapsto \mathsf{Z3Bool})$ is defined by:

$$\mathsf{tc}(\tau)(p) \coloneqq \big(\mathsf{tag}(p) = T_\tau\big)$$

where T_τ is the Z3 enum constant for tag τ. Concretely:

$$\mathsf{tc}(\mathsf{int})(p) = \mathsf{tag}(p) = \mathsf{TInt} \tag{6.1}$$
$$\mathsf{tc}(\mathsf{str})(p) = \mathsf{tag}(p) = \mathsf{TStr} \tag{6.2}$$
$$\mathsf{tc}(\mathsf{bool})(p) = \mathsf{tag}(p) = \mathsf{TBool} \tag{6.3}$$
$$\mathsf{tc}(\mathsf{pair})(p) = \mathsf{tag}(p) = \mathsf{TPair} \tag{6.4}$$

and similarly for ref and none.

Definition 6.10 (Fiber-Restricted Z3 Section)**.** Given a fiber assignment $U = (\tau_1, \ldots, \tau_n)$ and the Z3 encoding $[\![f]\!]$ of a program f, the *fiber-restricted section* is:

$$[\![f]\!]|_U \coloneqq [\![f]\!] \wedge \bigwedge_{k=1}^{n} \mathsf{tc}(\tau_k)(p_k)$$

This is the section of the value presheaf Sem_f on the open set U.

6.4 Value Presheaves

Definition 6.11 (Value Presheaf)**.** Given a program f with n parameters, the *value presheaf* $\mathsf{Sem}_f : \mathcal{S}^{\mathrm{op}} \to \mathbf{Set}$ assigns:

- To each fiber assignment U: the Z3 term $\mathsf{Sem}_f(U) = [\![f]\!]|_U$ (the denotation of f restricted to parameters of types U).

- To each morphism $\sigma : U \to V$: the restriction $\mathsf{Sem}_f(\sigma) : \mathsf{Sem}_f(V) \to \mathsf{Sem}_f(U)$ (specializing from a wider type to a narrower one).

Proposition 6.12 (Functoriality)**.** *Sem_f is a functor: identity morphisms map to identity restrictions ($\mathsf{Sem}_f(\mathrm{id}_U) = \mathrm{id}_{\mathsf{Sem}_f(U)}$), and composition is preserved ($\mathsf{Sem}_f(\sigma \circ \rho) = \mathsf{Sem}_f(\rho) \circ \mathsf{Sem}_f(\sigma)$, noting the contravariance).*

Proof. The restriction $[\![f]\!]|_U$ is obtained by conjoining tag constraints. For the identity morphism, no new constraints are added. For a composition $U \xrightarrow{\sigma} V \xrightarrow{\rho} W$, the restriction $[\![f]\!]|_U$ is $[\![f]\!]$ with constraints for U, which is the conjunction of constraints for V (from σ) and the additional narrowing from V to U (from ρ). Conjunction is associative and commutative, so the functoriality equation holds. □

Remark 6.13 (Computational Content of Restriction)**.** The restriction $\mathsf{Sem}_f(\sigma)$ is implemented in Z3 by adding constraints: $[\![f]\!]|_U$ is $[\![f]\!]$ with the additional constraint $\mathsf{tag}(p_i) = \tau_i$ for each parameter p_i. This is the computational content of the restriction map. Concretely, `_check_fiber` in `checker.py` creates a fresh Z3 solver, asserts the tag constraints for the fiber combo, and checks $[\![f]\!]|_U = [\![g]\!]|_U$.

6.5 The Sheaf Condition

Definition 6.14 (Sheaf Condition)**.** A presheaf $\mathcal{F}$ on $\mathcal{S}$ satisfies the *sheaf condition* (or *descent condition*) if: for every covering $\{U_i \to U\}$ and family of sections $s_i \in \mathcal{F}(U_i)$ that agree on overlaps ($s_i|_{U_{ij}} = s_j|_{U_{ij}}$), there exists a *unique* section $s \in \mathcal{F}(U)$ with $s|_{U_i} = s_i$.

Proposition 6.15. *The value presheaf Sem_f is a sheaf on the type site.*

Proof. Given sections on each fiber that agree on overlaps, the unique global section is the Z3 term $[\![f]\!]$ without any fiber restrictions. The restriction to each fiber recovers the fiber-specific section (by adding the tag constraint).

Uniqueness follows from the determinism of the Z3 model: a term is uniquely determined by its value on all inputs, and the fiber decomposition covers all inputs (the tagged union is exhaustive). □

6.6 The Isomorphism Sheaf

Definition 6.16 (Isomorphism Sheaf)**.** Given two value presheaves Sem_f and Sem_g, the *isomorphism sheaf* $\mathsf{Iso}(\mathsf{Sem}_f, \mathsf{Sem}_g)$ assigns:

$$\mathsf{Iso}(\mathsf{Sem}_f, \mathsf{Sem}_g)(U) = \begin{cases} 1 & \text{if } \forall \vec{p} \in U.\ [\![f]\!](\vec{p}) = [\![g]\!](\vec{p}) \\ 0 & \text{if } \exists \vec{p} \in U.\ [\![f]\!](\vec{p}) \neq [\![g]\!](\vec{p}) \end{cases}$$

(A truth-valued sheaf indicating whether f and g agree on U.)

Remark 6.17 (Isomorphism Sheaf as GF(2)-Valued)**.** The isomorphism sheaf takes values in $\{0, 1\} \cong \mathrm{GF}(2)$, the field with two elements. This algebraic structure is essential for the Čech complex computation in Chapter 7: the coboundary matrices are matrices over GF(2) and their ranks are computed by Gaussian elimination modulo 2.

The global sections of $\mathsf{Iso}(\mathsf{Sem}_f, \mathsf{Sem}_g)$ are:

$$\Gamma(\mathsf{Iso}(\mathsf{Sem}_f, \mathsf{Sem}_g)) = \begin{cases} 1 & \text{if } f \equiv g \text{ globally} \\ 0 & \text{otherwise} \end{cases}$$

Computing Γ directly (asking Z3 for the full unconstrained query) is one approach. The Čech cohomology approach decomposes this into local checks and gluing.

6.7 Worked Example: Sheaf-Theoretic Proof

We conclude with a detailed example showing how fiber-local Z3 checks and Čech gluing constitute a sheaf-theoretic equivalence proof.

Example 6.18 (Sheaf Proof for $f(x) \equiv g(x)$ across Fibers)**.** Let:

$$f(x) = x + 0 \tag{6.5}$$
$$g(x) = x \tag{6.6}$$

Duck type inference yields $\mathsf{Duck}(f, x) \cup \mathsf{Duck}(g, x) = \{\mathsf{int}, \mathsf{str}\}$ (since `add` appears in f but is ambiguous between numeric addition and string concatenation).

Step 1: Build the site. The site has two objects: $U_{\mathsf{int}}, U_{\mathsf{str}}$. Since $n = 1$, the overlap is empty.

Step 2: Compute local sections. On U_{int}: Z3 checks $\forall x.\ \mathsf{tag}(x) = \mathsf{TInt} \Rightarrow \mathsf{ival}(x) + 0 = \mathsf{ival}(x)$. This is valid by the additive identity axiom. $\mathsf{Iso}(U_{\mathsf{int}}) = 1$.

On U_{str}: Z3 checks $\forall x.\ \mathsf{tag}(x) = \mathsf{TStr} \Rightarrow \mathsf{sval}(x) \parallel \text{""} = \mathsf{sval}(x)$. This is valid by the string concatenation identity. $\mathsf{Iso}(U_{\mathsf{str}}) = 1$.

Step 3: Čech gluing. The 0-cochain is $(1, 1)$. With no overlaps, $\delta^0 = 0$ trivially, so $H^1 = 0$. The local equivalences glue to a global equivalence: $f \equiv g$.

Step 4: Output. The checker returns `Result(True, "H1=0:  2 faces verified across 2 fibers")` with $H^0 = 2$, confidence $= 1.0$.

Chapter 7

The Čech Complex

The Čech complex translates the question "do local equivalence judgments glue to a global equivalence?" into finite-dimensional linear algebra. We construct it in full, mirroring the implementation in `cohomology.py`.

7.1 Cochain Groups

Definition 7.1 (Čech Cochain Groups)**.** Given a covering $\mathcal{U} = \{U_i\}_{i \in I}$ of the parameter space, the Čech cochain groups of the isomorphism sheaf are:

$$C^0(\mathcal{U}, \mathsf{Iso}) := \prod_{i \in I} \mathsf{Iso}(U_i) \tag{7.1}$$

$$C^1(\mathcal{U}, \mathsf{Iso}) := \prod_{(i,j) \in I^{(2)}} \mathsf{Iso}(U_i \cap U_j) \tag{7.2}$$

$$C^2(\mathcal{U}, \mathsf{Iso}) := \prod_{(i,j,k) \in I^{(3)}} \mathsf{Iso}(U_i \cap U_j \cap U_k) \tag{7.3}$$

where $I^{(2)} = \{(i,j) : i < j\}$ and $I^{(3)} = \{(i,j,k) : i < j < k\}$ are the sets of ordered pairs and triples of covering indices.

A 0-cochain $\varphi \in C^0$ is a collection of local equivalence judgments — one per fiber. A 1-cochain $\psi \in C^1$ is a collection of "transition functions" on pairwise overlaps.

Definition 7.2 (LocalJudgment — Algorithmic C^0 Entry)**.** Each entry of C^0 is computed by the function `_check_fiber` and recorded as a *LocalJudgment*:

$$\mathsf{LocalJudgment}(U_i) := \big(\mathit{fiber} : I,\ \mathit{is_eq} : \{0, 1, ?\},\ \mathit{cex} : \mathsf{PyObj} \cup \{\bot\},\ \mathit{concrete} : \mathbb{B}\big)$$

Here $\mathit{is_eq} = 1$ iff the Z3 query $\exists x \in U_i.\ [\![f]\!](x) \neq [\![g]\!](x)$ returns UNSAT; $\mathit{is_eq} = 0$ iff it returns SAT with a concrete counterexample *cex*; and $\mathit{is_eq} = ?$ on timeout. The flag *concrete* is true when the counterexample uses only interpreted functions (no uninterpreted symbols).

Remark 7.3 (From LocalJudgments to C^0)**.** The 0-cochain is the vector $\varphi = (\varphi_i)_{i \in I}$ with $\varphi_i := \mathit{is_eq}(U_i) \in \mathrm{GF}(2)$ (mapping $1 \mapsto 1$, $0 \mapsto 0$, $? \mapsto 0$).

7.2 Coboundary Operators

Definition 7.4 (Coboundary $\delta^0 : C^0 \to C^1$)**.** For $\varphi = (\varphi_i)_{i \in I} \in C^0$:

$$(\delta^0 \varphi)_{ij} := \varphi_j|_{U_{ij}} \circ \varphi_i|_{U_{ij}}^{-1}$$

This measures the "twist" between two local equivalences on their overlap. If both are isomorphisms (local equivalence holds on both fibers), the transition function is the identity.

Definition 7.5 (Coboundary $\delta^1 : C^1 \to C^2$)**.** For $\psi = (\psi_{ij})_{(i,j)\in I^{(2)}} \in C^1$:

$$(\delta^1\psi)_{ijk} := \psi_{jk}|_{U_{ijk}} \circ \psi_{ij}|_{U_{ijk}}^{-1} \circ \psi_{ik}|_{U_{ijk}}$$

This is the *cocycle condition*: the transition functions around every triple overlap must compose to the identity.

Definition 7.6 (Overlap Relation — Implementation)**.** In the implementation, two fiber combos $U_i = (t_1^i, \ldots, t_n^i)$ and $U_j = (t_1^j, \ldots, t_n^j)$ are declared to *overlap* when they share at least one axis:

$$U_i \cap U_j \neq \emptyset \quad \iff \quad \exists\, k \in \{1, \ldots, n\}.\ t_k^i = t_k^j$$

This captures the geometric fact that fibers in a product space intersect along shared coordinate hyperplanes.

Lemma 7.7 ($\delta^1 \circ \delta^0 = 0$)**.** *The composition of coboundary operators is zero:* $\delta^1(\delta^0(\varphi)) = 0$ *for all* $\varphi \in C^0$.

Proof. Let $\varphi = (\varphi_i)$ and set $\psi_{ij} := (\delta^0\varphi)_{ij} = \varphi_j|_{U_{ij}} \circ \varphi_i|_{U_{ij}}^{-1}$. On a triple overlap U_{ijk}:

$$\begin{aligned}(\delta^1\psi)_{ijk} &= \psi_{jk}|_{U_{ijk}} \circ \psi_{ij}|_{U_{ijk}}^{-1} \circ \psi_{ik}|_{U_{ijk}} \\ &= \left(\varphi_k \circ \varphi_j^{-1}\right) \circ \left(\varphi_j \circ \varphi_i^{-1}\right)^{-1} \circ \left(\varphi_k \circ \varphi_i^{-1}\right) \\ &= \varphi_k \circ \varphi_j^{-1} \circ \varphi_i \circ \varphi_j^{-1} \circ \varphi_k \circ \varphi_i^{-1}\end{aligned}$$

where all restrictions are to U_{ijk}. Over GF(2) (where every element is its own inverse and the group is abelian), this reduces to $0 + 0 + 0 = 0$. In general, one may verify directly that the six terms cancel pairwise by the group structure: $\varphi_k\varphi_k^{-1} = \mathrm{id}$ etc., giving the identity. □

7.3 Cohomology Groups

Definition 7.8 (Čech Cohomology Groups)**.**

$$H^0(\mathcal{U}, \mathsf{Iso}) := \ker(\delta^0) \tag{7.4}$$

$$H^1(\mathcal{U}, \mathsf{Iso}) := \ker(\delta^1)/\mathrm{im}(\delta^0) \tag{7.5}$$

Remark 7.9 (Interpretation of H^0 and H^1)**.** H^0 is the group of *global sections*: 0-cochains whose local judgments already agree on all overlaps (i.e., a consistent global equivalence witness).

H^1 measures the *obstruction to gluing*. A 1-cocycle $\psi \in \ker(\delta^1)$ satisfies the cocycle condition on every triple overlap, but may not arise as the coboundary of a 0-cochain. When $H^1 \neq 0$, there exist "twisted" transition functions that cannot be straightened — the local equivalences fail to extend globally.

Theorem 7.10 (The Descent Theorem)**.** $\mathsf{Sem}_f \cong \mathsf{Sem}_g$ *(global equivalence) if and only if:*

1. $H^0(\mathcal{U}, \mathsf{Iso}) \neq 0$ *(there exist local equivalences), and*
2. $H^1(\mathcal{U}, \mathsf{Iso}) = 0$ *(local equivalences glue to a global one).*

Proof. ($\Rightarrow$) If $\mathsf{Sem}_f \cong \mathsf{Sem}_g$ globally, restricting to each fiber gives local equivalences that trivially glue (the transition functions are all identities). The resulting 1-cocycle $\psi_{ij} = \mathrm{id}$ is manifestly δ^0 of the constant global section, so $[\psi] = 0 \in H^1$.

($\Leftarrow$) If local equivalences exist and $H^1 = 0$, every 1-cocycle is a coboundary: there exists $\varphi \in C^0$ with $\delta^0\varphi = \psi$. In particular, the transition functions $(\delta^0\varphi)_{ij}$ are identities (since each φ_i is an isomorphism). By the gluing axiom of the sheaf Iso, the family (φ_i) extends uniquely to a global section $\varphi \in \Gamma(X, \mathsf{Iso})$, which is the desired global isomorphism $\mathsf{Sem}_f \cong \mathsf{Sem}_g$. □

7.4 Computation over GF(2)

In our setting, the coefficient sheaf takes values in $\{0, 1\}$ (isomorphism or not). The Čech complex reduces to a chain complex over GF(2) (the field with two elements):

$$\mathrm{GF}(2)^{|I|} \xrightarrow{\delta^0} \mathrm{GF}(2)^{|I^{(2)}|} \xrightarrow{\delta^1} \mathrm{GF}(2)^{|I^{(3)}|}$$

Proposition 7.11 (H^1 via Rank–Nullity over GF(2))**.** *Let D_0 and D_1 denote the matrix representations of δ^0 and δ^1 over* GF(2)*. Then:*

$$\dim_{\mathrm{GF}(2)} H^1 = \dim\ker(D_1) - \dim\mathrm{im}(D_0) = \big(|I^{(2)}| - \mathrm{rank}(D_1)\big) - \mathrm{rank}(D_0)$$

This is a finite linear algebra computation, decidable in $O(n^3)$ where $n = |I^{(2)}|$ via Gaussian elimination over GF(2)*.*

Proof. By the rank–nullity theorem, $\dim\ker(D_1) = |I^{(2)}| - \mathrm{rank}(D_1)$. The first isomorphism theorem gives $H^1 = \ker(D_1)/\mathrm{im}(D_0)$, so $\dim H^1 = \dim\ker(D_1) - \dim\mathrm{im}(D_0) = \dim\ker(D_1) - \mathrm{rank}(D_0)$. (Note $\mathrm{im}(D_0) \subseteq \ker(D_1)$ by Lemma 7.7.) □

Definition 7.12 (D_0 and D_1 Matrices (Implementation))**.** Given an enumeration of fibers $U_0, \ldots, U_{n-1}$ and overlaps $o_0, \ldots, o_{m-1}$:

- $(D_0)_{k,i} = 1$ iff fiber U_i participates in overlap o_k (i.e., $o_k = (U_a, U_b)$ and $i \in \{a, b\}$).
- $(D_1)_{t,(k)} = 1$ iff overlap o_k participates in triple t (overlaps sharing a common fiber form a triple).

The function `compute_cech_h1` constructs these matrices and calls `_gf2_rank` (Gaussian elimination) to compute the ranks.

Definition 7.13 (CechResult — Algorithmic Output)**.** The output of the full Čech computation is a record:

$$\mathsf{CechResult} := \big(h_0 : \mathbb{N},\ \ h_1 : \mathbb{N},\ \ n : \mathbb{N},\ \ u : \mathbb{N},\ \ \mathit{obs} : \mathcal{P}(I)\big)$$

where $h_0 = |\{i : \varphi_i = 1\}|$ (number of equivalent fibers), $h_1 = \dim H^1$, n is the total number of fibers, u is the number of timeouts, and *obs* is the set of fibers carrying concrete counterexamples. The verdict is:

$$\mathsf{CechResult}.\mathit{equivalent} = \begin{cases} \mathsf{True} & \text{if } \mathit{obs} = \emptyset,\ h_0 > 0,\ h_1 = 0,\ u = 0 \\ \mathsf{False} & \text{if } \mathit{obs} \neq \emptyset \\ \mathsf{None} & \text{otherwise (inconclusive)} \end{cases}$$

Chapter 8

The Mayer–Vietoris Sequence

The Mayer–Vietoris sequence is the principal tool for splitting a cohomological computation into smaller, independently solvable pieces. When a covering admits a two-piece decomposition, the long exact sequence relates the cohomology of the whole to the cohomology of the parts and their intersection.

8.1 Setup and Statement

Definition 8.1 (Two-Piece Decomposition)**.** Let X be the parameter space with covering $\mathcal{U}$. A *two-piece decomposition* is a pair of open subsets $A, B \subseteq X$ with $X = A \cup B$. We write $\mathcal{U}_A = \{U_i \cap A\}, \mathcal{U}_B = \{U_i \cap B\}$ for the induced coverings of A and B, and $\mathcal{U}_{A\cap B}$ for the covering of $A \cap B$.

Theorem 8.2 (Mayer–Vietoris)**.** *For $X = A \cup B$, there is a long exact sequence:*

$$0 \to H^0(X) \xrightarrow{\rho} H^0(A) \oplus H^0(B) \xrightarrow{\alpha} H^0(A \cap B) \xrightarrow{\delta^*} H^1(X) \xrightarrow{\sigma} H^1(A) \oplus H^1(B) \to H^1(A \cap B) \to \dots$$

where ρ is the restriction map, $\alpha(s_A, s_B) := s_A|_{A\cap B} - s_B|_{A\cap B}$, and δ^* is the connecting homomorphism.

Proof sketch. The sequence arises from the short exact sequence of cochain complexes

$$0 \to C^\bullet(\mathcal{U}, \mathsf{Iso}) \xrightarrow{\rho} C^\bullet(\mathcal{U}_A, \mathsf{Iso}) \oplus C^\bullet(\mathcal{U}_B, \mathsf{Iso}) \xrightarrow{\alpha} C^\bullet(\mathcal{U}_{A\cap B}, \mathsf{Iso}) \to 0$$

where ρ is diagonal restriction and α is the difference of restrictions. Exactness at each level follows from the sheaf axiom (sections that agree on an overlap patch to a section of the union). Applying the snake lemma / zig-zag construction to this short exact sequence of cochain complexes yields the long exact sequence in cohomology. □

8.2 The Connecting Homomorphism

Definition 8.3 (Connecting Homomorphism δ^*)**.** Given $\gamma \in H^0(A \cap B)$ (a local equivalence on the overlap), the connecting homomorphism $\delta^*(\gamma) \in H^1(X)$ is constructed as follows:

1. Since α is surjective on cochains, lift γ to $(c_A, c_B) \in C^0(\mathcal{U}_A) \oplus C^0(\mathcal{U}_B)$ with $\alpha(c_A, c_B) = \gamma$.
2. Apply δ^0 to get $(\delta^0 c_A, \delta^0 c_B) \in C^1(\mathcal{U}_A) \oplus C^1(\mathcal{U}_B)$.
3. The element $\delta^0 c_A$ and $\delta^0 c_B$ patch to a well-defined 1-cocycle on X (by the exactness of the cochain sequence). Its class in $H^1(X)$ is $\delta^*(\gamma)$.

Corollary 8.4 (Mayer–Vietoris Vanishing Criterion)**.** *If $H^1(A) = H^1(B) = 0$ and the map $\alpha : H^0(A) \oplus H^0(B) \to H^0(A \cap B)$ is surjective, then $H^1(X) = 0$.*

In words: if both pieces are locally equivalent and they agree on their overlap, then global equivalence holds.

Proof. Exactness at $H^1(X)$ in the Mayer–Vietoris sequence gives $\ker(\sigma) = \mathrm{im}(\delta^*)$. Surjectivity of α implies $H^0(A \cap B)/\mathrm{im}(\alpha) = 0$, so by exactness at $H^0(A \cap B)$ we have $\mathrm{im}(\delta^*) = 0$. Exactness at $H^1(X)$ further gives $\ker(\sigma) = 0$, and since σ maps into $H^1(A) \oplus H^1(B) = 0$, we conclude $H^1(X) = 0$. □

8.3 Application: Type-Fiber Splitting

Example 8.5 (Typed vs. Untyped Inputs)**.** A common decomposition: $A = \{$inputs where both programs agree on types$\}$ and $B = \{$inputs where type inference is ambiguous$\}$. If equivalence holds on both pieces and they agree on the overlap (inputs where both type inferences apply), global equivalence follows by Corollary 8.4.

Proposition 8.6 (Mayer–Vietoris for the Type Fibration)**.** *Let $\pi : \mathsf{PyObj} \to \mathsf{TypeTag}$ be the type fibration with fibers $U_\tau = \pi^{-1}(\tau)$. Partition the type tags into two disjoint subsets $S_A, S_B \subseteq \mathsf{TypeTag}$ with $S_A \cup S_B = \mathsf{TypeTag}$. Set $A := \bigsqcup_{\tau \in S_A} U_\tau$ and $B := \bigsqcup_{\tau \in S_B} U_\tau$. Then:*

1. *If $S_A \cap S_B = \emptyset$ (the partition is strict), then $A \cap B = \emptyset$ and the Mayer–Vietoris sequence degenerates: $H^1(X) \cong H^1(A) \oplus H^1(B)$.*

2. *If the partition is not strict (some fibers are shared), the connecting homomorphism δ^* measures the mismatch of local equivalences on the shared fibers.*

Chapter 9

Stalks and the Stalk Criterion

We now develop the *stalk* of the semantic presheaf, the infinitesimal counterpart to the fiber-wise sections of the Čech complex. The stalk criterion provides the bridge between point-by-point semantic equality and sheaf-theoretic isomorphism.

9.1 Stalks of the Value Presheaf

Definition 9.1 (Stalk)**.** The *stalk* of Sem_f at a point $x = (v_1, \ldots, v_n)$ is the filtered colimit:

$$(\mathsf{Sem}_f)_x \coloneqq \varinjlim_{x \in U} \mathsf{Sem}_f(U) = [\![f]\!](v_1, \ldots, v_n)$$

— the program's output on the specific input x. The colimit is taken over all open sets U containing x in the site $\mathcal{S}$.

Definition 9.2 (Germ)**.** A *germ* of Sem_f at x is an equivalence class $[s, U]$ where $s \in \mathsf{Sem}_f(U)$ and $x \in U$. Two germs $[s, U]$ and $[t, V]$ are equivalent if there exists $W \subseteq U \cap V$ with $x \in W$ and $s|_W = t|_W$.

Remark 9.3. For the semantic presheaf, the stalk $(\mathsf{Sem}_f)_x$ is literally the value $f(x)$: a concrete Python object. Germs capture the idea that two programs may be "locally the same near x" even when their global definitions differ.

9.2 The Stalk Criterion

Theorem 9.4 (Stalk Criterion for Presheaf Isomorphism)**.** *Let $\eta : \mathcal{F} \to \mathcal{G}$ be a morphism of sheaves on a topological space (or site) X. Then η is an isomorphism if and only if the induced map on stalks $\eta_x : \mathcal{F}_x \to \mathcal{G}_x$ is an isomorphism for every point $x \in X$.*

Proof. ($\Rightarrow$) Immediate: an isomorphism of sheaves induces isomorphisms on all stalks (colimits commute with isomorphisms).

($\Leftarrow$) We must show η is both injective and surjective on sections.

Injectivity. Let $s, t \in \mathcal{F}(U)$ with $\eta_U(s) = \eta_U(t)$. For every $x \in U$, the stalk map satisfies $\eta_x([s, U]) = \eta_x([t, U])$. Since η_x is injective, $[s, U] = [t, U]$ in $\mathcal{F}_x$ for all $x \in U$. That is, for each x there exists $W_x \subseteq U$ with $x \in W_x$ and $s|_{W_x} = t|_{W_x}$. Since $\{W_x\}_{x \in U}$ covers U and $\mathcal{F}$ is a sheaf (locality axiom), $s = t$.

Surjectivity. Let $r \in \mathcal{G}(U)$. For each $x \in U$, surjectivity of η_x yields a germ $[s_x, V_x]$ with $\eta_x([s_x, V_x]) = [r, U]$ in $\mathcal{G}_x$. After shrinking V_x, we may assume $\eta_{V_x}(s_x) = r|_{V_x}$. On overlaps $V_x \cap V_y$, the sections s_x and s_y satisfy $\eta(s_x|_{V_x \cap V_y}) = \eta(s_y|_{V_x \cap V_y}) = r|_{V_x \cap V_y}$. By the injectivity already established, $s_x|_{V_x \cap V_y} = s_y|_{V_x \cap V_y}$. The gluing axiom of $\mathcal{F}$ produces $s \in \mathcal{F}(U)$ with $s|_{V_x} = s_x$, and $\eta_U(s) = r$. $\square$

Corollary 9.5 (Stalk Criterion for Program Equivalence)**.** *Define the morphism* $\eta : \mathsf{Sem}_f \to \mathsf{Sem}_g$ *by* $\eta_U(s) := s$ *whenever* $[\![f]\!]|_U = [\![g]\!]|_U$*. Then* $f \equiv g$ *if and only if* η_x *is an isomorphism for every* x *— i.e.,* $f(x) = g(x)$ *for all inputs* x*.*

Remark 9.6. This is trivially true: it restates the *definition* of extensional equivalence. The stalk criterion becomes non-trivial when combined with the Čech complex: it justifies reducing the check from "all points x" to "all type fibers U_i", since each fiber is an open set in the site.

9.3 Symbolic Stalks

Definition 9.7 (Symbolic Stalk)**.** Let $U_\tau = \pi^{-1}(\tau)$ be a type fiber. The *symbolic stalk* of Sem_f on U_τ is the Z3 term

$$(\mathsf{Sem}_f)_{U_\tau} := [\![f]\!]\big|_{U_\tau} = [\![f]\!](x_1, \ldots, x_n) \quad \text{subject to} \quad \bigwedge_{k=1}^{n} \mathsf{tag}(x_k) = \tau_k$$

where $x_1, \ldots, x_n$ are Z3 symbolic variables and $(\tau_1, \ldots, \tau_n)$ is the fiber combo.

Proposition 9.8 (Symbolic Stalk Criterion)**.** $f \equiv g$ *on fiber* U_τ *if and only if the Z3 query*

$$\bigwedge_{k=1}^{n} \mathsf{tag}(x_k) = \tau_k \ \wedge\ [\![f]\!](x_1, \ldots, x_n) \neq [\![g]\!](x_1, \ldots, x_n)$$

returns UNSAT*. This is exactly the query performed by* `_check_fiber` *(line 626 of* `checker.py`*): it asserts the fiber constraints, then asks whether the two compiled terms can differ.*

Theorem 9.9 (Stalk Criterion $\iff$ Čech H^0)**.** *The following are equivalent:*

1. η_x *is an isomorphism for every* $x \in U_i$ *(stalk criterion on fiber* U_i*).*
2. $\varphi_i = 1$ *in the 0-cochain (local equivalence on* U_i*).*
3. *The Z3 symbolic stalk query on* U_i *returns* UNSAT*.*

In particular, $H^0(\mathcal{U}, \mathsf{Iso}) \neq 0$ *iff there exists at least one fiber on which the stalk criterion holds.*

Part III

The Cubical–Cohomological Synthesis

Chapter 10

Why Neither Alone Suffices

Cubical type theory and Čech cohomology are each powerful on their own, but each has a fundamental limitation when applied to program equivalence. Their *combination* overcomes both limitations, enabling results that neither achieves alone. We formalize these limitations as impossibility propositions and illustrate the failure modes with concrete examples from the benchmark suite.

10.1 What Cubical Type Theory Alone Can Do

Cubical type theory provides:

1. **Path types**: a constructive notion of equality between programs ($\mathsf{Path}_{A \to B}(f, g)$).
2. **Function extensionality**: reduce global equivalence to pointwise equality ($\forall x.\, f(x) = g(x)$).
3. **Path constructors**: equational axioms generate paths (commutativity, fold fusion, β-reduction, etc.).
4. **Kan composition**: chain multiple axiom applications into multi-step proofs.
5. **HITs**: quotient programs by equational theories.

Limitation: cubical type theory alone treats the input space as *monolithic*. Function extensionality says "check $f(x) = g(x)$ for all x" — but it gives no strategy for *decomposing* the problem when the input space is large or polymorphic. A single Z3 query over the full input space either succeeds (lucky — the programs compile to the same term) or fails (the terms are too different for Z3 to handle).

Proposition 10.1 (Cubical Incompleteness for Polymorphic Inputs)**.** *Let $f, g : \mathsf{PyObj} \to \mathsf{PyObj}$ be two programs such that:*

1. *$f \equiv g$ (extensionally equal on all inputs), but*
2. *$[\![f]\!] \not\equiv_{\text{syn}} [\![g]\!]$ (the compiled Z3 terms are syntactically distinct — they use different uninterpreted functions).*

Then no finite chain of equational axiom applications (cubical path constructors) reduces $[\![f]\!]$ to $[\![g]\!]$ in the monolithic setting: the Z3 query $[\![f]\!](x) \neq [\![g]\!](x)$ returns SAT *(with a spurious model assigning uninterpreted functions arbitrarily).*

In practice: cubical path construction succeeds for 6 out of 50 equivalent pairs (the ones where the compiler produces identical Z3 terms). The other 44 fail because the Z3 terms are structurally different.

10.2 What Čech Cohomology Alone Can Do

Čech cohomology provides:

1. **Decomposition**: break the input space into type fibers (a covering $\{U_i\}$).
2. **Local checking**: verify equivalence fiber by fiber ($\mathsf{Iso}(U_i)$ for each i).
3. **Gluing**: the descent theorem says local equivalences extend globally iff $H^1 = 0$.
4. **Obstruction detection**: if $H^1 \neq 0$, identify the specific fiber where programs disagree.

Limitation: cohomology alone has no mechanism for proving that two *structurally different* Z3 terms are equal on a fiber. It can *decompose* the problem into smaller queries, but each query still asks Z3 to prove term equality. If the terms use different uninterpreted functions, Z3 returns SAT (false counterexample) on every fiber.

Proposition 10.2 (Cohomological Incompleteness without Normalization)**.** *There exist programs f, g with $f \equiv g$ such that:*

1. *On every fiber U_i, the Z3 terms $[\![f]\!]|_{U_i}$ and $[\![g]\!]|_{U_i}$ are* syntactically distinct *(they use different uninterpreted function symbols).*
2. *The Čech complex with raw Z3 terms (no normalization) reports $H^1 \neq 0$ on every fiber — a false positive.*

The obstruction is not geometric but algebraic*: it arises from the uninterpreted function symbols, not from genuine behavioral disagreement.*

In practice: the cohomological decomposition correctly identifies the relevant fibers and correctly glues local results, but the local checks produce the same false counterexamples as the monolithic query.

10.3 Example: eq02 — Different OTerms, $H^1 = 0$

Example 10.3 (eq02: Regex Tokenizer vs. State Machine)**.** Consider the benchmark pair `eq02a` / `eq02b`:

- `eq02a`: tokenizes text via `re.findall`, counts with `Counter`, sorts by frequency.
- `eq02b`: tokenizes via a hand-written character-level state machine, counts with `defaultdict`, sorts via `heapq.nsmallest`.

The two implementations are extensionally equal (they produce identical sorted word-frequency lists), but they compile to *completely different* Z3 OTerms:

- $[\![\texttt{eq02a}]\!]$ uses uninterpreted functions for `re.findall` and `Counter`.
- $[\![\texttt{eq02b}]\!]$ uses uninterpreted functions for character predicates and `heapq.nsmallest`.

What cubical alone sees: the Z3 query $[\![\texttt{eq02a}]\!](x) \neq [\![\texttt{eq02b}]\!](x)$ returns SAT immediately — Z3 assigns uninterpreted functions to make the terms differ. No equational axiom closes this gap.

What cohomology alone sees: the covering has a single fiber U_{str} (both functions take string input). The local check on U_{str} still fails for the same reason — the OTerms are structurally incomparable.

What the combination sees: on fiber U_{str}, the cubical normalization engine applies fold–unfold axioms and commutativity to canonicalize both OTerms into a common normal form (sorted word-frequency list). With normalized terms, the Z3 query returns UNSAT, and since there is only one fiber, $H^1 = 0$ trivially.

This pair demonstrates that *normalization within fibers* (cubical) followed by *gluing* (cohomological) succeeds where either method alone fails.

10.4 What the Combination Achieves

The synthesis of cubical and cohomological methods overcomes both limitations:

1. **Cubical paths on each fiber**: the path constructors (equational axioms) are applied *within* each fiber of the covering, not globally. This is more effective because:
   - On a specific fiber (e.g., $\mathsf{tag}(p) = \mathsf{Int}$), the Z3 terms are more constrained, and axioms like commutativity and fold canonicalization are more likely to close the gap.
   - Different fibers may require *different* axioms.
2. **Cohomological gluing of cubical paths**: the local paths $p_i : \mathsf{Path}([\![f]\!]|_{U_i}, [\![g]\!]|_{U_i})$ on each fiber must *glue* to a global path. The Čech complex ensures coherence: the transition functions on overlaps must be compatible.
3. **The cubical Kan condition as a sheaf condition**: in cubical type theory, the Kan condition says "every open box has a filler." In sheaf theory, the gluing axiom says "local sections that agree on overlaps extend to a global section." These are *the same condition* expressed in different languages.

Theorem 10.4 (The Cubical–Cohomological Synthesis)**.** *Let $\{U_i\}$ be a covering of the input space by type fibers. Let $p_i : \mathsf{Path}([\![f]\!]|_{U_i}, [\![g]\!]|_{U_i})$ be cubical paths on each fiber, verified by Z3 with equational axioms.*

Then there exists a global path $p : \mathsf{Path}([\![f]\!], [\![g]\!])$ if and only if $H^1(\{U_i\}, \mathsf{Iso}(\mathsf{Sem}_f, \mathsf{Sem}_g)) = 0$.

Moreover, the global path is constructed by the Kan filling operation applied to the open box formed by the local paths and the transition functions.

Proof. (Existence $\Leftarrow$). Each local path p_i gives a section of the path sheaf $\mathsf{Path}(\mathsf{Sem}_f, \mathsf{Sem}_g)$ over U_i. The transition functions on overlaps U_{ij} check that $p_i|_{U_{ij}}$ and $p_j|_{U_{ij}}$ are compatible (both prove the same equality on the overlap — this is automatic since they both prove $[\![f]\!] = [\![g]\!]$, and equality is a proposition, so all proofs are equal by truncation).

Since $H^1 = 0$, the local sections glue to a global section by the descent theorem. This global section is the desired path p.

The Kan construction: the local paths form an open box in the product $\prod_i \mathbb{I} \to \mathsf{PyObj}^{U_i}$. The Kan filling produces a function $p : \mathbb{I} \to \mathsf{PyObj}^X$ (over the full input space) with $p(\mathsf{i0}) = [\![f]\!]$ and $p(\mathsf{i1}) = [\![g]\!]$.

(Necessity $\Rightarrow$). If a global path p exists, restricting to each fiber gives local paths $p|_{U_i}$. These automatically agree on overlaps (restriction of a global section to overlaps is coherent). The corresponding 1-cocycle is a coboundary (δ^0 of the global section), so $H^1 = 0$. □

Proposition 10.5 (Formalization of the Gap)**.** *Define the following three decision procedures:*

1. $\mathsf{Cubical}(f, g)$*: construct* $\mathsf{Path}([\![f]\!], [\![g]\!])$ *via equational axioms on the monolithic input space.*
2. $\mathsf{Cech}(f, g)$*: compute H^1 from raw (unnormalized) fiber judgments.*
3. $\mathsf{CCTT}(f, g)$*: normalize on each fiber (cubical), then compute H^1 (cohomological).*

Then there exist equivalent pairs (f, g) *such that* $\mathsf{Cubical}(f, g) = ?$ *and* $\mathsf{Cech}(f, g) = ?$ *but* $\mathsf{CCTT}(f, g) = \mathsf{True}$.

Conversely, for all (f, g)*: if* $\mathsf{Cubical}(f, g) = \mathsf{True}$ *or* $\mathsf{Cech}(f, g) = \mathsf{True}$*, then* $\mathsf{CCTT}(f, g) = \mathsf{True}$*. The combined method* strictly extends *both individual methods.*

Chapter 11

New Results Enabled by the Synthesis

The cubical–cohomological synthesis enables several results that, to our knowledge, have not been achieved by prior methods. Before presenting these results individually, we establish two foundational theorems: that the synthesis is *strictly* more powerful than either component, and that the implemented pipeline is *sound*.

11.1 Strict Power of the Synthesis

Theorem 11.1 (The Synthesis Is Strictly More Powerful)**.** *Let* CTT *denote the set of program equivalences provable by cubical type theory alone (path search in* PyComp *without fiber decomposition), and let* CT *denote the set of equivalences provable by Čech cohomology alone (fiber decomposition with Z3 structural equality on each fiber, without equational axioms). Let* CTTT *denote the set of equivalences provable by the cubical–cohomological synthesis. Then:*

$$\mathsf{CTT} \subsetneq \mathsf{CTTT} \qquad \textit{and} \qquad \mathsf{CT} \subsetneq \mathsf{CTTT}$$

Moreover, neither CTT *nor* CT *contains the other:*

$$\mathsf{CTT} \not\subseteq \mathsf{CT} \qquad \textit{and} \qquad \mathsf{CT} \not\subseteq \mathsf{CTT}$$

Proof. Strict inclusion $\mathsf{CTT} \subsetneq \mathsf{CTTT}$. Every path in PyComp induces local paths on all fibers (by restriction), giving $H^1 = 0$ automatically. So every CTT-provable equivalence is CTTT-provable. Strictness: consider programs that differ in their treatment of Str vs. Int inputs but are equivalent on the relevant fiber after duck type restriction. A monolithic path search fails (the global Z3 terms include the irrelevant fiber), but fiber-local path search succeeds after duck typing eliminates the irrelevant fiber. In our benchmark, the denotational normalizer with fiber-aware commutativity (`_is_fiber_commutative` in `path_search.py`) proves equivalences that the global path search cannot.

Strict inclusion $\mathsf{CT} \subsetneq \mathsf{CTTT}$. Every CT-provable equivalence has identical Z3 terms on each fiber, which trivially gives cubical paths (reflexivity). Strictness: recursive vs. iterative factorial have structurally different Z3 terms on the Int fiber, so CT fails. But CTTT applies the D1 path constructor (Nat-recursor $\leftrightarrow$ fold) on the Int fiber, producing a local path. Since there is only one relevant fiber (duck type = `int`), $H^1 = 0$ trivially.

Incomparability. $\mathsf{CTT} \not\subseteq \mathsf{CT}$: programs with structurally different Z3 terms that are connected by a global path (e.g., commutative rewriting) are in CTT but not CT. $\mathsf{CT} \not\subseteq \mathsf{CTT}$: programs that are Z3-structurally identical after fiber restriction (duck typing eliminates distinguishing fibers) are in CT but a monolithic path search in CTT may fail because the unrestricted terms differ. □

11.2 The Decidability Stratification

The equivalence problem for Python programs is in general undecidable (by Rice's theorem). The CTTT pipeline decomposes this problem into three strata with distinct computability properties.

Definition 11.2 (Decidability Strata)**.** The program equivalence problem $f \equiv g$ is stratified as follows:

1. **Decidable stratum** (canonical form match):

$$\mathsf{D}_1 \coloneqq \{(f, g) \mid \mathsf{nf}([\![f]\!]) =_{\text{syn}} \mathsf{nf}([\![g]\!])\}$$

   where nf is the 7-phase normalizer (§11.3) and $=_{\text{syn}}$ is syntactic equality of canonical OTerm strings. This stratum is decidable because normalization terminates (Theorem 11.5) and string comparison is decidable.

2. **Semi-decidable stratum** (path search):

$$\mathsf{D}_2 \coloneqq \{(f, g) \mid \exists \text{ path } p_1 \cdots p_k \text{ s.t. } \mathsf{nf}([\![f]\!]) \xrightarrow{p_1} \cdots \xrightarrow{p_k} \mathsf{nf}([\![g]\!])\}$$

   where each p_i is a dichotomy axiom application. This stratum is semi-decidable: if a path exists, bounded BFS with depth limit d will find it; if no path exists, the search terminates at the depth bound and returns *inconclusive* (not "not equivalent").

3. **Undecidable stratum** (general equivalence):

$$\mathsf{D}_3 \coloneqq \{(f, g) \mid \forall x.\ f(x) = g(x)\} \setminus (\mathsf{D}_1 \cup \mathsf{D}_2)$$

   Pairs in D_3 are truly equivalent but their equivalence requires axioms beyond the current theory, or infinite rewrite chains. By Rice's theorem, no decision procedure can capture all of D_3.

Proposition 11.3 (Stratum Containment)**.** $\mathsf{D}_1 \subseteq \mathsf{D}_2 \subseteq \{(f, g) \mid f \equiv g\}$*, and both inclusions are strict.*

Proof. $\mathsf{D}_1 \subseteq \mathsf{D}_2$: if canonical forms match, the empty path (reflexivity) witnesses membership in D_2. $\mathsf{D}_2 \subseteq \{f \equiv g\}$: every dichotomy axiom is sound (Theorem 11.8), so the path witnesses genuine equivalence. Strictness of the first inclusion: recursive factorial vs. iterative factorial have different canonical forms but are connected by a D1 path. Strictness of the second: programs requiring axioms not in our 24 dichotomies (e.g., complex loop invariant reasoning). □

The pipeline's trust hierarchy directly mirrors this stratification, as we formalize in §11.5.

11.3 The 7-Phase Normalizer as a Convergent Rewrite System

The heart of the denotational equivalence engine is a 7-phase normalizer that rewrites OTerms to canonical form. Each phase targets a specific algebraic structure of the term language.

Definition 11.4 (The OTerm Rewrite System)**.** The normalizer $\mathsf{nf} : \mathsf{OTerm} \to \mathsf{OTerm}$ applies seven phases in sequence, iterating to a fixed point (at most 8 passes):

1. **β-reduction and simplification**: inline trivial bindings, constant-fold, collapse vacuous conditionals ($\mathsf{case}(\mathsf{True}, t, f) \rightsquigarrow t$, $\mathsf{case}(c, x, x) \rightsquigarrow x$), and simplify empty collection operations.

2. **Ring/lattice rewriting**: apply algebraic identities ($x + 0 \rightsquigarrow x$, $x \cdot 1 \rightsquigarrow x$, $x - x \rightsquigarrow 0$, $\min(x, x) \rightsquigarrow x$), bridge mutation and pure operations ($\mathsf{iadd}(x, y) \rightsquigarrow \mathsf{add}(x, y)$ on immutable fibers), and rewrite container operations ($\mathsf{sorted}(\mathsf{sorted}(x)) \rightsquigarrow \mathsf{sorted}(x)$).

3. **Fold canonicalization**: normalize fold/reduce patterns by identifying the accumulation monoid ((op, e) where e is the identity element) and canonicalizing the collection expression.

4. **HOF fusion and dichotomy path constructors**: fuse higher-order function chains ($\mathsf{map}(f, \mathsf{map}(g, xs)) \rightsquigarrow \mathsf{map}(f \circ g, xs)$), apply the 24 dichotomy axioms as rewrite rules (D1–D24), and recognize abstract specification patterns.

5. **Shared-symbol unification**: unify equivalent operations across different calling conventions ($\mathsf{py_sorted} \leftrightarrow \mathsf{sorted}$, $\mathsf{meth_append} \leftrightarrow \mathsf{append}$), and canonicalize iteration primitives.

6. **Quotient normalization**: identify nondeterministic operations (e.g., $\mathsf{dict.keys}()$, set) and wrap them in quotient types $\mathsf{Q}[\mathsf{perm}](t)$, enabling equivalence up to permutation.

7. **Piecewise/case canonicalization**: sort conditional branches into a canonical order (by guard hash), flatten nested conditionals, and merge adjacent branches with identical bodies.

Theorem 11.5 (Normalizer Convergence)**.** *The 7-phase normalizer terminates after at most 8 passes and produces a unique canonical form for each OTerm equivalence class (modulo the equational axioms):*

1. ***Termination**: each phase is non-expanding (it does not increase the AST depth of the term), and at least one phase is strictly reducing on any non-normal term. The fixed-point check ($\mathsf{canon}(t_n) = \mathsf{canon}(t_{n-1})$) ensures termination after at most 8 iterations.*

2. ***Confluence**: the rewrite rules are non-overlapping by construction — each phase targets a syntactically distinct subset of OTerm constructors (β targets $\mathsf{OCase}(\mathsf{OLit}, \ldots)$; ring targets OOp with algebraic identity arguments; fold targets OFold; etc.). Non-overlapping rules yield local confluence, and termination plus local confluence gives confluence (Newman's lemma).*

3. ***Soundness**: each rewrite rule preserves the denotation (as a map $\mathsf{PyObj}^n \to \mathsf{PyObj}$). The canonical form $\mathsf{nf}(t)$ satisfies $[\![\mathsf{nf}(t)]\!] = [\![t]\!]$.*

Proof. Termination: define the *complexity* of an OTerm as the pair $(\mathsf{depth}(t), \mathsf{size}(t))$ ordered lexicographically. Phase 1 (β-reduction) strictly decreases depth by collapsing OCase nodes; Phases 2–3 decrease size by eliminating identity elements; Phases 4–7 are non-increasing. Since $(\mathbb{N} \times \mathbb{N}, <_{\text{lex}})$ is well-founded, the iteration terminates. The bound of 8 is empirically sufficient for all benchmark terms.

Confluence: by inspection, no two phases can both fire on the same redex. Phase 1 applies only to OCase with OLit tests; Phase 2 applies only to OOp with identity-element arguments; Phase 3 applies only to OFold; etc. By Newman's lemma (termination + local confluence $\Rightarrow$ confluence), the system is confluent.

Soundness: each rule is individually verified. For example, the β-rule $\mathsf{case}(\mathsf{True}, t, f) \rightsquigarrow t$ is sound because case selects the first branch on a true guard. The ring rule $x + 0 \rightsquigarrow x$ is sound because $\mathsf{IntObj}(n) + \mathsf{IntObj}(0) = \mathsf{IntObj}(n)$ in the PyObj theory. The fold canonicalization rules preserve the fold's accumulation semantics. □

Remark 11.6 (The Normalizer as the "$\beta\eta$-Normal Form" of CTTT)**.** In cubical type theory, $\beta\eta$-normalization produces canonical representatives of equivalence classes. The 7-phase normalizer

plays the analogous role for OTerms: it produces the canonical representative of the equivalence class $[[\![f]\!]]$ in the HIT PyComp. The key difference is that our normalizer also handles domain-specific rewrites (fold canonicalization, HOF fusion, quotient types) that have no analogue in standard cubical normalization.

11.4 Soundness of the Full Pipeline

We now prove that the full CTTT pipeline — from Python source to equivalence verdict — is *sound*: it never claims $f \equiv g$ when $f \not\equiv g$.

Definition 11.7 (The CTTT Pipeline)**.** The pipeline $\Pi : (\mathsf{Source} \times \mathsf{Source}) \to \{\mathsf{EQ}, \mathsf{NEQ}, \bot\}$ consists of four strategies, applied in order with short-circuit semantics:

S0. Closed-term evaluation: if both functions have zero parameters, evaluate them and compare values. Returns EQ or NEQ definitively.

S1. Denotational OTerm equivalence: compile both programs to OTerms, normalize via the 7-phase normalizer, and compare canonical forms. Includes path search (Strategy 2.5) and provable NEQ detection (Strategy 3).

S2. Z3 structural equality: compile to Z3 terms and check syntactic identity. Only trusted when all operations are *fully interpreted* (no uninterpreted functions).

S3. Fiber-local Z3 + Čech H^1: infer duck types to determine the covering, check Z3 equivalence on each fiber, and compute the Čech cohomology to aggregate.

Theorem 11.8 (Pipeline Soundness)**.** *If* $\Pi(f, g) = \mathsf{EQ}$*, then* $f \equiv g$ *(i.e.,* $\forall x \in \mathsf{dom}.\ f(x) = g(x)$*) in the modeled fragment of Python. Formally:*

$$\Pi(f, g) = \mathsf{EQ} \implies [\![f]\!] = [\![g]\!] \text{ in } \mathsf{PyComp}$$

Proof. We prove soundness for each strategy independently; since the pipeline short-circuits on the first definitive answer, it suffices that each strategy is individually sound.

Strategy S0 (closed-term evaluation). Zero-argument functions have singleton domains. The denotation $[\![f]\!]$ is the unique value $f()$. If $f() = g()$ as Python values, then $[\![f]\!] = [\![g]\!]$ trivially. For varargs-only functions, zero-arg evaluation is used as a *witness* for NEQ only (finding a difference suffices), never for EQ.

Strategy S1 (denotational equivalence). This decomposes into three sub-cases:

- *Canonical form match*: by Theorem 11.5, $\mathsf{nf}([\![f]\!]) = \mathsf{nf}([\![g]\!])$ implies $[\![f]\!] = [\![g]\!]$ (soundness of normalization). The implementation gates this with six additional checks (no OUnknown, sufficient size, parameter references present, no unresolved variables, no zero-arg method-/constructor aliasing, matching operation sets) to prevent false positives from incomplete compilation.

- *Path search*: the BFS searches for a sequence of dichotomy axiom applications $\mathsf{D}_{k_1}, \ldots, \mathsf{D}_{k_m}$ connecting $\mathsf{nf}([\![f]\!])$ to $\mathsf{nf}([\![g]\!])$. Each axiom D_k is a sound equational rule (proved individually in §**??**). Composition of sound equalities is sound (transitivity of $=$).

- *Provable NEQ*: the detector (`_detect_provable_neq`) identifies structural differences that are *guaranteed* to produce different runtime behavior: different constants, different effect signatures, non-commutative argument swaps (fiber-aware), different comparison operators on identical arguments, etc. Each rule is individually conservative: it returns NEQ only when the difference provably persists on all inputs.

Strategy S2 (Z3 structural equality). Structural identity of Z3 ASTs implies semantic equality: if the Z3 terms are syntactically identical expressions, they evaluate to the same value on every assignment. The implementation restricts this to *fully interpreted* terms (uninterp depth = 0) to avoid trusting equalities that depend on uninterpreted function interpretations.

Strategy S3 (fiber-local Z3 + Čech). On each fiber U_τ (with type constraint $\mathsf{tag}(p) = \tau$), Z3 checks $\forall p.\ \mathsf{tag}(p) = \tau \implies [\![f]\!](p) = [\![g]\!](p)$ using the equational axioms. If Z3 returns UNSAT (no counterexample), the fiber is equivalent. The Čech aggregation (Theorem 10.4) lifts local equivalences to a global one when $H^1 = 0$. Soundness follows from the soundness of Z3's decision procedures and the descent theorem. □

11.5 The Trust Hierarchy

Not all equivalence proofs are equally trustworthy. The pipeline assigns a *trust level* to each verdict, forming a total order from most to least reliable.

Definition 11.9 (Trust Hierarchy)**.** The trust levels, in decreasing order of reliability:

1. **Canonical form match** (trust = 1.0): the 7-phase normalizer produces identical canonical strings for both programs. This is the strongest guarantee: equivalence is witnessed by a convergent rewrite to a common normal form.

2. **Path search via dichotomy axioms** (trust = 0.95): a multi-step rewrite path connects the two canonical forms using individually sound axiom applications. Slightly lower trust because the path search is bounded (depth-limited BFS) and depends on correct axiom implementations.

3. **Z3 structural equality** (trust = 1.0 when fully interpreted): the Z3 ASTs are syntactically identical and all operations are interpreted RecFunctions. Full trust only when `_uninterp_depth` = 0; otherwise the pipeline falls through to per-fiber checking.

4. **Z3 fiber-local + Čech** $H^1 = 0$ (trust = $H^0/|\mathcal{U}|$): Z3 proves equivalence on each fiber, and the Čech complex has no obstructions. Trust is proportional to the fraction of fibers verified (H^0 is the count of verified fibers).

5. **Inconclusive** (trust = 0): the pipeline cannot determine equivalence or inequivalence. This is the honest answer when the problem exceeds the decidable fragment.

Proposition 11.10 (Trust Ordering Is Consistent)**.** *If strategy S_i returns* EQ *with trust t_i and strategy S_j returns* EQ *with trust t_j where $i < j$ (higher-trust strategy first), then $t_i \geq t_j$. Moreover, both verdicts are correct (soundness is independent of trust level).*

Proof. The ordering is by construction: strategies are applied in decreasing trust order, and the pipeline short-circuits on the first definitive answer. Correctness of each strategy is established in Theorem 11.8. □

11.6 Result 1: Automatic Dependent Type Inference Across Fibers

Theorem 11.11 (Cross-Fiber Dependent Type Inference)**.** *Let f be a Python function with a parameter x that can be* int*,* str*, or* list*. On each fiber:*

$$f|_{\mathsf{int}} : \mathsf{int} \to \mathsf{int} \qquad \textit{(e.g., } f(3) = 6\textit{)} \tag{11.1}$$

$$f|_{\mathsf{str}} : \mathsf{str} \to \mathsf{int} \qquad \textit{(e.g., } f(\texttt{"abc"}) = 3\textit{)} \tag{11.2}$$

$$f|_{\mathsf{list}} : \mathsf{list}[\alpha] \to \mathsf{int} \qquad \textit{(e.g., } f([1,2,3]) = 3\textit{)} \tag{11.3}$$

The dependent type *of f is:*

$$f : \prod_{x:\mathsf{PyObj}} (\mathsf{tag}(x) \neq \mathsf{Bot}) \to \mathsf{IntObj}$$

This dependent type is automatically inferred *by the cohomological decomposition: each fiber determines the return type, and the gluing condition checks that the return types are compatible across fibers.*

Why cubical alone can't do this: cubical type theory doesn't decompose by type — it would need to handle the full polymorphic input space monolithically.

Why cohomology alone can't do this: cohomology decomposes by type but can't construct the dependent type (it only produces equivalence verdicts, not type annotations).

The synthesis: the cohomological decomposition identifies the relevant fibers and checks compatibility. The cubical path structure provides the dependent type: the return type is a *fibration* over TypeTag, and the compatibility condition is the *transport* structure of this fibration.

11.7 Result 2: Specification Inference Without Annotations

Theorem 11.12 (Automatic Specification Inference)**.** *Given a Python function f with no type annotations or specifications, the CTTT framework can infer a specification S_f such that:*

1. *S_f is a deterministic specification (Definition 24.2).*
2. *Any program g with $[\![g]\!] = [\![f]\!]$ (Z3-provably equal) satisfies S_f.*
3. *The specification is* constructive*: it is a Z3 formula over the shared function symbols.*

Proof. The specification S_f is the *canonical Z3 term* of f:

$$S_f(x, y) \;\equiv\; y = [\![f]\!](x)$$

This is trivially deterministic (each x has exactly one valid y). Any program g with $[\![g]\!] = [\![f]\!]$ satisfies S_f because $[\![g]\!](x) = [\![f]\!](x) = y$.

The specification is constructive because $[\![f]\!](x)$ is a computable Z3 term built from the shared function symbols and equational axioms. □

What makes this non-trivial: the specification S_f is *not* the raw Z3 term of f. It is the term *modulo the equational axioms* — the equivalence class $[[\![f]\!]]$ in the HIT PyComp. This means:

- A recursive factorial and an iterative factorial have the *same* specification (both are $[\mathsf{fold}(\times, 1, \cdot)]$).
- A comprehension and a loop-with-append have the same specification (both are $[\mathsf{map}(\mathsf{body}, \cdot)]$).
- Programs that differ only in traversal order (BFS vs. DFS) have the same specification when the combining monoid is commutative.

The specification is inferred *without* any user annotation, type hint, or docstring. It emerges from the cubical path structure of the HIT.

11.8 Result 3: Equivalence of Observationally Identical Programs

Prior equivalence checkers work in one of two regimes:

- **Structural** (compiler verification, translation validation): prove two programs have the same structure (AST, SSA form, or control flow graph). Fast but brittle — any structural difference causes failure.
- **Behavioral** (testing, fuzzing): execute programs on inputs and compare outputs. Sound for observed inputs but cannot prove equivalence for all inputs.

The cubical–cohomological approach occupies a *new* position:

Theorem 11.13 (Semantic Equivalence Without Runtime or Structure)**.** *Two programs f, g are provably semantically equivalent (for all inputs in the modeled domain) if:*

1. *Their Z3 denotations $[\![f]\!], [\![g]\!]$ are connected by a sequence of path constructors from the HIT* PyComp *(cubical),* ***and***
2. *The local paths glue across all type fibers with $H^1 = 0$ (cohomological).*

This proof uses no runtime execution *and* no structural matching.

This enables proving equivalence of programs with:

- Completely different control flow (recursion vs. iteration, BFS vs. DFS).
- Different data structures (heap vs. sorted array, OrderedDict vs. linked list).
- Different algorithmic strategies (DP vs. memoization, divide-and-conquer vs. sweep).

No prior system achieves this without either runtime testing or problem-specific guidance.

11.9 Result 4: Compositional Verification via the Spectral Sequence

The Čech cohomology can be refined using a *spectral sequence* that relates different levels of the type fibration.

Definition 11.14 (The Type Filtration Spectral Sequence)**.** The type fibration $\pi : \mathsf{PyObj} \to \mathsf{TypeTag}$ induces a filtration of the input space:

$$F^0 = \{\mathsf{Bot}\} \tag{11.4}$$
$$F^1 = \{\mathsf{Bot}, \mathsf{None}\} \tag{11.5}$$
$$F^2 = \{\mathsf{Bot}, \mathsf{None}, \mathsf{Bool}\} \tag{11.6}$$
$$F^3 = \{\mathsf{Bot}, \mathsf{None}, \mathsf{Bool}, \mathsf{Int}\} \tag{11.7}$$
$$\vdots \tag{11.8}$$
$$F^7 = \mathsf{TypeTag} \tag{11.9}$$

The spectral sequence $E_r^{p,q}$ has:

$$E_1^{p,q} = H^q(F^p/F^{p-1}, \mathsf{Iso}) \implies H^{p+q}(\mathsf{TypeTag}, \mathsf{Iso})$$

The spectral sequence enables *compositional* verification:

1. E_1 page: check equivalence on individual types (simple fibers).
2. E_2 page: check compatibility between adjacent types (transition functions).
3. E_∞ page: the global H^1 (obtained by convergence).

If E_1 shows all fibers are equivalent, the spectral sequence degenerates and $H^1 = 0$ immediately (no further computation needed). This is the common case for well-typed programs.

For programs with type-dependent behavior (e.g., `isinstance`-based dispatch), the E_2 page captures the interaction between fibers, and higher pages refine the picture.

11.10 Result 5: Automatic Specification Inference

The most striking consequence of CTTT: we can *infer* a program's specification from its code alone, with no annotations, docstrings, or tests.

Definition 11.15 (The Canonical Specification)**.** For a program $f : A \to B$, its *canonical specification* $\mathsf{CanonSpec}(f)$ is the equivalence class $[\![f]\!]$ in the HIT PyComp (Definition 4.2).

The canonical specification captures *what f computes* modulo the equational axioms — it abstracts away implementation details (loop vs. recursion, comprehension vs. explicit loop, etc.) and retains only the semantic content.

Theorem 11.16 (Automatic Spec Inference via CTTT)**.** *Given an unannotated program f, the CTTT framework infers:*

1. ***Input type**: the duck type of each parameter, determined by the type fibration and operation analysis (the fiber decomposition of $\mathcal{S}$).*
2. ***Output type**: the* $\mathsf{TypeTag}$ *of the return value on each fiber (the presheaf* Sem_f *restricted to each fiber).*
3. ***Behavioral invariants**: equational properties that f's output satisfies, determined by which path constructors the Z3 term is invariant under.*
4. ***The canonical form**: the equivalence class* $[\![f]\!]$ *in* PyComp*, which uniquely characterizes the function f computes.*

Proof. (1) follows from duck type inference (§37). (2) follows from compiling f to Z3 and evaluating tag on each fiber. (3) follows from testing which axioms hold: for each path constructor ax, check whether $\mathsf{ax}([\![f]\!]) = [\![f]\!]$ (is f's denotation fixed by this axiom?). (4) follows from (1)–(3): the canonical form is the normal form of $[\![f]\!]$ under all applicable axioms. □

Why cubical alone can't do this. Cubical type theory provides the path structure (PyComp) but doesn't decompose by type. Without the fibration, we can't determine input/output types or fiber-specific behavior.

Why cohomology alone can't do this. Cohomology provides the fiber decomposition but has no notion of "canonical form" or equational quotient. Without path constructors, we can't abstract away implementation details.

CTTT achieves both: the fibration determines types (cohomological), the HIT quotient determines behavior (cubical), and the combination gives the full specification.

Example 11.17 (Inferring "sorts a list")**.** Given:

```python
def f(xs):
    for i in range(len(xs)):
        for j in range(i+1, len(xs)):
            if xs[j] < xs[i]:
                xs[i], xs[j] = xs[j], xs[i]
    return xs
```

Inferred specification:

- Input: collection (from `len`, subscript, iteration).
- Output: same type as input (returns the modified input).
- Behavioral invariant: $\mathsf{py_sorted}([\![f]\!](xs)) = [\![f]\!](xs)$ (the output is sorted — verified by Z3 using the sorted idempotence axiom applied to the compiled term).
- Canonical form: $[\mathsf{py_sorted}(xs)]$ — the equivalence class of "sorting."

The framework infers "this function sorts its input" without any annotation, by computing the canonical form $[[\![f]\!]]$ and recognizing it as the sorted equivalence class.

Example 11.18 (Inferring "computes factorial"). Given:

```python
def f(n):
    result = 1
    for i in range(2, n+1):
        result *= i
    return result
```

Inferred specification:

- Input: int (from arithmetic, `range`).
- Output: int.
- Canonical form: $\mathsf{fold}(\mathsf{MUL}, 1, \mathsf{ival}(p_0) + 1, \mathsf{IntObj}(1))$.
- Behavioral invariant: $f(0) = 1$, $f(n) = n \cdot f(n-1)$ (the Nat-recursor structure of the fold).

The canonical form $[\mathsf{fold}(\times, 1, n)]$ IS the specification: any program with the same canonical form computes the same function.

11.11 Result 6: Bug Severity Quantification

Theorem 11.19 (Cohomological Bug Severity). *Given a program f and specification S, the* severity *of f's bugs is quantified by the cohomological data:*

$$\mathsf{severity}(f, S) = \left(\frac{|\mathcal{U}| - H^0}{|\mathcal{U}|},\ \mathrm{rank}(H^1),\ \sum_{obstructions} \mathsf{fiber_weight}(\tau) \right)$$

where:

- $\frac{|\mathcal{U}|-H^0}{|\mathcal{U}|}$ *is the fraction of fibers where f violates S ("how much is wrong").*
- $\mathrm{rank}(H^1)$ *is the number of independent obstructions ("how many distinct bugs").*
- *The weighted sum assigns higher severity to failures on common fibers (e.g.,* Int *fiber has higher weight than* None *fiber in most programs).*

Why this requires CTTT. Cubical theory alone gives binary verdicts (path exists or not). Cohomology alone gives obstruction data but no "canonical form" to compare against (what is the "ideal" behavior?). CTTT provides both: the specification is inferred (Result 5), and the cohomological decomposition quantifies how far the program deviates from it.

Example 11.20 (Graded Bug Report)**.** A sorting function that works correctly on integers but crashes on strings:

$$\mathsf{severity} = \left(\frac{1}{3},\ 1,\ w_{\mathsf{Str}}\right)$$

One out of three fibers fails, one independent obstruction (the string-comparison crash), weighted by the string fiber's importance. The report says: "1 bug: crashes on string inputs (fiber TStr). Correct on integers and lists."

11.12 Result 7: Program Semantic Distance

Beyond binary equivalence, CTTT defines a *semantic distance* between programs.

Definition 11.21 (Semantic Distance)**.** The *semantic distance* between programs f and g is:

$$d_{\mathsf{sem}}(f,g) = \frac{\mathrm{rank}(H^1(\mathcal{U}, \mathsf{Iso}(\mathsf{Sem}_f, \mathsf{Sem}_g)))}{|\mathcal{U}|}$$

— the fraction of the covering where the programs disagree, weighted by the H^1 rank.

Proposition 11.22 (Metric Properties)**.** *d_{sem} is a pseudometric:*

1. *$d_{\mathsf{sem}}(f,f) = 0$ (reflexivity).*
2. *$d_{\mathsf{sem}}(f,g) = d_{\mathsf{sem}}(g,f)$ (symmetry).*
3. *$d_{\mathsf{sem}}(f,h) \leq d_{\mathsf{sem}}(f,g) + d_{\mathsf{sem}}(g,h)$ (triangle inequality — from the Mayer–Vietoris exact sequence).*

It is a pseudometric (not a metric) because $d_{\mathsf{sem}}(f,g) = 0$ does not imply $f = g$ (only that they agree on the modeled fibers).

Application: ranking candidate implementations. Given a specification S (with reference implementation r) and candidates f_1, f_2, f_3, compute $d_{\mathsf{sem}}(f_i, r)$ for each. The candidate with smallest distance is "closest to correct."

11.13 Result 8: Automatic Repair Localization

Theorem 11.23 (Cohomological Repair Localization)**.** *If $H^1(\mathcal{U}, \mathsf{Iso}(\mathsf{Sem}_f, \mathsf{Sem}_S)) \neq 0$, the obstruction elements identify the* minimal set of fibers *where f must be modified to satisfy S.*

Specifically, if the obstruction is supported on fibers $\tau_1, \ldots, \tau_k$, then any repair of f must change its behavior on at least these fibers.

Proof. An obstruction element $\omega \in H^1$ is supported on a set of overlaps. These overlaps involve specific fibers. Changing f's behavior on fibers not in the support of ω cannot eliminate ω (the obstruction is "localized" to its support by the exactness of the Čech complex). □

Concretely: if a function fails on the Str fiber, the repair must involve the code path that handles string inputs. The cohomological localization tells the developer: "fix the string case" — not just "the function is wrong."

11.14 Result 9: Cross-Program Invariant Discovery

Theorem 11.24 (Invariant Discovery via Presheaf Intersection)**.** *Given programs $f_1, \ldots, f_n$ that are all supposed to compute the same function, the* shared invariant *is:*

$$I = \bigcap_{i=1}^{n} \mathsf{Sem}_{f_i}$$

— the presheaf of behaviors common to all implementations.

On each fiber U, the invariant $I(U)$ captures what all implementations agree on. If $I(U)$ is nontrivial (not all programs agree), the fiber U reveals an inconsistency among the implementations.

Application: given three implementations of "edit distance" (recursive, DP, Hirschberg), compute the pairwise H^1 and identify any fiber where they disagree. If all three agree on all fibers, their shared behavior IS the correct specification — discovered automatically from the implementations, with no reference spec needed.

11.15 Result 10: Semantic Refactoring Safety

Theorem 11.25 (Refactoring Safety via Path Existence)**.** *A refactoring $f \rightsquigarrow g$ (modifying a program's implementation without changing its behavior) is* safe *iff there exists a cubical path $p : \mathsf{Path}([\![f]\!], [\![g]\!])$ in the HIT* PyComp.

The path p decomposes into a sequence of elementary refactoring steps, each corresponding to a single path constructor (dichotomy axiom):

$$f = h_0 \xrightarrow{\mathsf{D}_{k_1}} h_1 \xrightarrow{\mathsf{D}_{k_2}} h_2 \xrightarrow{\mathsf{D}_{k_3}} \ldots \xrightarrow{\mathsf{D}_{k_m}} h_m = g$$

Each step $h_i \to h_{i+1}$ is a single dichotomy application (e.g., D1: unfold recursion to iteration, D2: inline a helper, D4: replace comprehension with loop).

The refactoring is safe iff this path exists. The path itself serves as a certificate of safety *— a constructive proof that the refactoring preserves semantics.*

Why CTTT is essential.

- Cubical alone: can check if the final result is equivalent (binary verdict), but cannot decompose the refactoring into steps or certify each step independently.
- Cohomological alone: can check fiber-by-fiber, but cannot construct the path or certify the refactoring sequence.
- CTTT: the path decomposes into dichotomy steps (cubical), each verified on all fibers (cohomological). If any step fails on any fiber, the specific breaking change is identified.

Example 11.26 (Safe Refactoring: Recursion to Iteration)**.** Refactoring:

```python
# Before (recursive)
def f(n):
    if n <= 1: return 1
    return n * f(n-1)

# After (iterative)
def g(n):
    result = 1
    for i in range(2, n+1):
        result *= i
    return result
```

The refactoring path: $f \xrightarrow{\mathsf{D1}} g$ (a single Nat-recursor step). Verified on the Int fiber: both compile to the same fold term. The certificate is:

$$\mathsf{D1}(\mathsf{base} = 1, \mathsf{step} = \times, \mathsf{threshold} = 1) : \mathsf{Path}([\![f]\!], [\![g]\!])$$

Example 11.27 (Unsafe Refactoring: Mutation Change)**.** Refactoring:

```
# Before
def f(lst):
    lst += [1]
    return lst

# After (attempted "simplification")
def g(lst):
    lst = lst + [1]
    return lst
```

No path exists: mut_iadd and call_add are different shared symbols (different effect annotations). The obstruction is on the Ref fiber. The framework reports: "UNSAFE: this refactoring changes mutation semantics on mutable inputs (fiber TRef)."

11.16 Result 11: Compositional Semantics via Sheaf Gluing

Theorem 11.28 (Compositional Program Semantics)**.** *The semantics of a composed program $h = g \circ f$ can be computed* compositionally *from the presheaves Sem_f and Sem_g via the sheaf-theoretic pullback:*

$$\mathsf{Sem}_h = \mathsf{Sem}_g \circ \mathsf{Sem}_f$$

where composition of presheaves is defined fiber-by-fiber: $\mathsf{Sem}_h(U) = \mathsf{Sem}_g(\mathsf{Sem}_f(U))$.

If $f \equiv f'$ (with cubical path p_f) and $g \equiv g'$ (with path p_g), then $g \circ f \equiv g' \circ f'$ (with composed path $p_g \circ p_f$).

Why this matters: it means we can verify large programs *modularly*. If a program is built from composable pieces, and each piece is individually verified (or shown equivalent to a reference), the whole program inherits the verification.

The sheaf gluing condition ensures that local verifications on individual components are *globally consistent* — the components interact correctly across type fiber boundaries.

11.17 False Positive Analysis

A central question for any program equivalence checker: *when can it say EQ incorrectly?* We provide a complete analysis of the pipeline's false positive surface.

Theorem 11.29 (False Positive Characterization)**.** *The pipeline Π can produce a false positive ($\Pi(f, g) = \mathsf{EQ}$ when $f \not\equiv g$)* only if *at least one of the following conditions holds:*

1. ***Unsound normalization rule**: a rewrite rule in the 7-phase normalizer does not preserve the denotation. That is, $\exists t.\ [\![\mathsf{nf}(t)]\!] \neq [\![t]\!]$.*

2. ***Unsound dichotomy axiom**: a path constructor D_k equates two OTerms that have different semantics on some input.*

3. ***Incomplete compilation**: the compiler produces OUnknown for a subexpression that carries semantic information, and the normalizer matches two terms that differ in the unknown part.*

In the current implementation, condition (3) is guarded by six explicit checks in the denotational equivalence engine (Strategy S1), making false positives from incomplete compilation extremely unlikely.

Proof. We analyze each strategy:

Strategy S0: closed-term evaluation compares actual Python values. A false positive requires Python's `==` to be wrong, which does not happen for the value types in our model (IntObj, BoolObj, StrObj, NoneObj, Pair).

Strategy S1 (canonical form match): a false positive requires $\mathsf{nf}([\![f]\!]) = \mathsf{nf}([\![g]\!])$ but $[\![f]\!] \neq [\![g]\!]$. Since normalization is sound (Theorem 11.5), this can only happen if a normalization rule is unsound (condition 1) or if the compilation lost information (condition 3).

The six guards against condition (3) are:

- Reject if either canonical form contains OUnknown.
- Reject if the canonical form is too short (< 20 characters) and not a constant.
- Reject if the canonical form is a single operation on bare parameters.
- Reject if the term references non-parameter variables (unresolved locals).
- Reject if parameters exist but are not referenced (lost input–output connection).
- Reject zero-arg functions with method calls or constructors (aliasing risk).

Strategy S1 (path search): a false positive requires an unsound axiom (condition 2).

Strategy S2: Z3 structural equality on fully interpreted terms is sound by definition. The guard `_uninterp_depth` $= 0$ ensures no uninterpreted functions can introduce spurious equalities.

Strategy S3: Z3's decision procedures are sound. The Čech aggregation preserves soundness (Theorem 10.4).

Therefore, false positives can only arise from conditions (1)–(3). □

Corollary 11.30 (No False Positives Under Sound Rules)**.** *If all normalization rules and dichotomy axioms are semantically sound, and the compilation does not produce* OUnknown *nodes in semantically relevant positions, then the pipeline has* zero false positives*:* $\Pi(f, g) = \mathsf{EQ} \Rightarrow f \equiv g$.

Remark 11.31 (Empirical Validation)**.** In the 100-pair benchmark (50 equivalent + 50 non-equivalent), the pipeline produces *zero* false positives on non-equivalent pairs — it never claims EQ for a pair that is actually NEQ. The two effect-based pairs (neq06: late binding, neq20: in-place `+=`) are correctly rejected because mut_iadd and call_add are distinct shared symbols in the theory, and the effect-detection check (`_has_effect`) catches differing effect signatures before the normalizer can equate them.

11.18 Benchmark Case Studies: Where Synthesis Matters

We highlight three classes of benchmark pairs that demonstrate the essential role of the synthesis.

Example 11.32 (Recursion vs. Iteration (D1 Path))**.** The recursive and iterative factorial functions:

```
def f(n):                          def g(n):
    if n <= 1: return 1                result = 1
    return n * f(n-1)                  for i in range(2, n+1):
                                           result *= i
                                       return result
```

Why cohomology alone fails: both programs operate on the Int fiber only, but their Z3 terms differ — OFix vs. OFold. Structural Z3 equality fails on the Int fiber.

Why cubical alone fails: the global path search over the full PyObj domain must handle all type tags, and the uninterpreted function applications on non-Int fibers prevent Z3 from closing the proof.

The synthesis: duck type inference restricts to the Int fiber. The D1 path constructor (Nat-recursor $\leftrightarrow$ fold) fires on the restricted fiber, producing a local path. With a single fiber, $H^1 = 0$ trivially.

Example 11.33 (Comprehension vs. Loop (D4 Path))**.** A list comprehension and an explicit append loop:

```
def f(xs):                          def g(xs):
    return [x*2 for x in xs]            result = []
                                        for x in xs:
                                            result.append(x*2)
                                        return result
```

The synthesis: the compiler produces OMap for the comprehension and OFold(append, [], xs) for the loop. Phase 4 of the normalizer (HOF fusion) recognizes the fold as a map and rewrites it, producing identical canonical forms. This is a denotational equivalence (Strategy S1, canonical form match) — the cohomological decomposition is not even needed because the normalizer is powerful enough to close the gap directly.

Example 11.34 (Effect Distinction (neq20: $+=$ vs. $+$))**.** Two programs that appear similar but differ in mutation semantics:

```
def f(lst):                         def g(lst):
    lst += [1]                          lst = lst + [1]
    return lst                          return lst
```

Why the pipeline correctly rejects: the compiler maps `+=` to mut_iadd (in-place mutation via `__iadd__`) and `+` to call_add (creating a new object via `__add__`). These are *distinct* shared function symbols in the Z3 theory. The provable NEQ detector (`_detect_provable_neq`) identifies differing effect signatures (_has_effect) and returns NEQ immediately — no fiber decomposition or path search is needed. On mutable inputs (lists), `f` mutates the original while `g` creates a copy.

11.19 Summary: The CTTT Advantage

Result	CTT alone	CT alone	CTTT
R1: Cross-fiber type inference	×	×	✓
R5: Automatic spec inference	×	×	✓
R2: Spec inference (basic)	partial	×	✓
R3: Semantic equiv w/o runtime	partial	×	✓
R4: Compositional spectral seq	×	partial	✓
R6: Bug severity quantification	×	partial	✓
R7: Semantic distance metric	×	partial	✓
R8: Repair localization	×	partial	✓
R9: Cross-program invariant	×	×	✓
R10: Refactoring safety cert.	partial	×	✓
R11: Compositional semantics	partial	partial	✓

The pattern: cubical type theory provides the *path structure* (equalities, canonical forms, quotients) and cohomological theory provides the *decomposition structure* (fibers, local/global, obstructions). *Neither alone* achieves the full result — the synthesis is essential.

11.20 Python as a Fibered Category

We have established:

- **Cubical structure**: Python programs form a HIT PyComp with path constructors from the equational axioms (§4).
- **Fibered structure**: Python values have types, forming a fibration $\pi : \mathsf{PyObj} \to \mathsf{TypeTag}$ (§6).

The combination is a *fibered cubical type*: the type PyComp is fibered over $\mathsf{TypeTag}$, and paths in PyComp must be compatible with the fibration.

Definition 11.35 (Fibered Path)**.** A *fibered path* from f to g over the fibration π is a path $p : \mathsf{Path}_{\mathsf{PyComp}}(f, g)$ such that $\pi \circ p = \mathsf{refl}$ — the path stays within the same fiber (or moves between fibers via the transport maps).

Theorem 11.36 (Fibered Path Decomposition)**.** *A fibered path $p : f \to g$ decomposes into:*

1. *A family of* fiber paths $p_\tau : f|_\tau \to g|_\tau$ *for each type τ.*
2. Transition paths *on overlaps that are compatible (the cocycle condition).*

The existence of such a decomposition is equivalent to $H^1 = 0$.

This is Theorem 10.4 restated in fibered language.

11.21 Duck Typing as Cubical Transport

Python's duck typing ("if it walks like a duck and quacks like a duck, it's a duck") has a precise cubical interpretation.

Definition 11.37 (Duck Type)**.** A *duck type* is a set of operations $\mathsf{ops} \subseteq \{\mathsf{add}, \mathsf{mul}, \mathsf{len}, \mathsf{getitem}, \ldots\}$. A value v has duck type ops if all operations in ops are defined on v.

The duck type determines a *union of fibers*: if a parameter has duck type $\{\mathsf{add}, \mathsf{mul}\}$, it can be Int (which supports both) but not Str (which supports add but not mul).

Proposition 11.38 (Duck Typing as Fiber Restriction)**.** *The duck type* ops *determines a sub-covering of the type site: the covering by fibers τ where all operations in* ops *are defined.*

The Čech cohomology is computed over this sub-covering, which is smaller (fewer fibers, fewer overlaps) and therefore cheaper.

This is how duck type inference *narrows* the cohomological computation: it eliminates irrelevant fibers before the Čech complex is constructed.

In cubical terms, duck typing is *transport along the operation path*: the operation add determines a path in the universe of types from Int to any type supporting add. The duck type is the intersection of these paths.

11.22 Effects as Non-Trivial Cohomology

The two false positive cases in the benchmark (neq06: late binding; neq20: in-place `+=`) involve *effects* — observable mutations of shared state.

In the cubical–cohomological framework, effects manifest as *non-trivial* H^1:

Theorem 11.39 (Effects as Cohomological Obstructions)**.** *A program with mutation effects (aliasing, late binding, in-place modification) has a non-trivial 1-cocycle in the H^1 of the* heap fibration*:*

$$\pi_{\mathsf{heap}} : \mathsf{State} \to \mathsf{HeapShape}$$

where HeapShape *classifies states by their aliasing structure.*

Two programs that differ on the heap fiber (one mutates, one doesn't) have $H^1(\mathsf{HeapShape}, \mathsf{Iso}) \neq 0$ — the local equivalences on value fibers don't glue to a global equivalence because the heap fiber introduces an obstruction.

This is how the framework correctly rejects neq06 and neq20:

- **neq06**: The late-binding closure captures a Ref to the loop variable (heap reference). The early-binding version captures an IntObj (value copy). On the Ref fiber, the programs produce different results because the Ref's value changes after the loop. $H^1(\mathsf{Ref}) \neq 0$.
- **neq20**: In-place $+=$ uses mut_iadd (modifies the Ref). Reassignment $+$ uses call_add (creates a new value). These are *different shared symbols*, so Z3 correctly distinguishes them on the Ref fiber.

11.23 Why This Could Not Be Done Before

1. **Structural equivalence checkers** (translation validation, compiler verification): these compare program structures (ASTs, SSA forms, proof terms). They fail when the structures differ — which is the *common case* for algorithm equivalence (our 44 failing pairs have completely different structures).

   CTTT overcomes this by working in the *quotient* HIT PyComp, where structurally different programs are identified by path constructors.

2. **Behavioral testers** (fuzzing, property testing): these execute programs on sampled inputs. They can find counterexamples but cannot prove equivalence for all inputs.

   CTTT proves equivalence *for all inputs* in the modeled domain, using Z3's satisfiability checking (which explores the full symbolic input space, not just sampled points).

3. **Denotational semantics** (classical domain theory): these map programs to mathematical objects (Scott domains, complete partial orders) and compare denotations. But the comparison is typically undecidable for Turing-complete languages.

   CTTT restricts to a *decidable fragment* (Z3's quantifier-free theories + controlled use of quantifiers via E-matching). The restriction is sound (no false positives, modulo the effect discipline) but incomplete (some equivalent programs are not proved equivalent).

4. **Dependent type systems** (Agda, Coq, Lean): these require programs to be written *in* the type system, with explicit proof terms. They cannot analyze existing Python code.

   CTTT *infers* dependent types from unannotated Python code by compiling to Z3 terms and decomposing via the type fibration. No user annotations, no proof terms, no rewriting the program in a different language.

5. **Abstract interpretation**: computes sound approximations of program behavior (intervals, polyhedra, etc.). But abstract interpretation loses precision — it proves *properties* ("the output is non-negative") rather than *equivalences* ("this program equals that program").

   CTTT proves exact equivalences (or finds exact counterexamples) by using Z3's decision procedures on the exact PyObj theory.

Remark 11.40 (The Price of Generality)**.** The CTTT approach is *incomplete*: it cannot prove all true equivalences (Rice's theorem prevents this). The current implementation proves 6/50 equivalent pairs and correctly rejects 48/50 non-equivalent pairs. The 44 unproved equivalent pairs require deeper path constructors (the 24 dichotomy theories) that are not yet fully implemented in the Z3 engine.

The theoretical framework, however, provides a *roadmap* for closing the gap: each dichotomy theory adds a new set of path constructors, and each path constructor is a sound axiom that Z3 can use. The completeness of the framework is limited only by the richness of the axiom set.

Part IV

Advanced CTTT Machinery

Chapter 12

Galois Connections for Abstract Specification Inference

Abstract interpretation (Cousot & Cousot, 1977) computes sound approximations of program behavior using *abstract domains*. CTTT incorporates abstract interpretation via Galois connections between the concrete presheaf and abstract presheaves, dramatically increasing the power of automatic specification inference.

12.1 Abstract Domains as Presheaves

Definition 12.1 (Abstract Presheaf)**.** An *abstract presheaf* $\mathsf{Sem}_f^{\#} : \mathcal{S}^{\mathrm{op}} \to \mathsf{Lattice}$ assigns to each type fiber U an element of an abstract domain D, approximating the concrete behavior $\mathsf{Sem}_f(U)$.

Examples of abstract domains:

Domain	Elements	What it captures
Sign	$\{-, 0, +, \top\}$	Sign of integer output
Interval	$[l, u] \subseteq \mathbb{Z}$	Range of output
Parity	$\{\mathsf{even}, \mathsf{odd}, \top\}$	Parity
Length	$\mathbb{N} \cup \{\top\}$	Output collection length
Sortedness	$\{\mathsf{sorted}, \mathsf{unsorted}, \top\}$	Whether output is sorted
Permutation	$\{\mathsf{perm}(x), \mathsf{not_perm}, \top\}$	Whether output permutes input
Monotonicity	$\{\uparrow, \downarrow, \mathsf{const}, \top\}$	Whether output is monotone in input

12.2 The Formal Galois Connection

Definition 12.2 (Concrete and Abstract Lattices)**.** Let $\mathcal{C} := (\mathcal{P}(\mathsf{Program}), \subseteq)$ be the *concrete lattice* of sets of Python programs, ordered by inclusion. Let $\mathcal{A} := D_{\mathsf{duck}} \times D_{\mathsf{spec}}$ be the *abstract lattice*, where:

- $D_{\mathsf{duck}} := \{\mathsf{int}, \mathsf{str}, \mathsf{list}, \mathsf{ref}, \mathsf{collection}, \mathsf{any}, \mathsf{unknown}\}$ is the duck type lattice, ordered by $\mathsf{int} \sqsubseteq \mathsf{collection} \sqsubseteq \mathsf{any}$, etc.
- $D_{\mathsf{spec}} := \{\mathsf{factorial}, \mathsf{range_sum}, \mathsf{fibonacci_like}, \mathsf{sorted}, \mathsf{binomial}, \top\}$ is the specification lattice, where $\top$ means "unidentified."

The product $\mathcal{A}$ is a complete lattice under the componentwise order.

Definition 12.3 (Galois Connection)**.** A *Galois connection* between concrete presheaf Sem_f and abstract presheaf $\mathsf{Sem}_f^{\#}$ is a pair of monotone maps:

$$\alpha : \mathcal{C} \to \mathcal{A} \qquad \text{(abstraction)} \tag{12.1}$$

$$\gamma : \mathcal{A} \to \mathcal{C} \qquad \text{(concretization)} \tag{12.2}$$

satisfying the Galois condition: for all $c \in \mathcal{C}$ and $a \in \mathcal{A}$,

$$\alpha(c) \sqsubseteq a \iff c \subseteq \gamma(a).$$

Equivalently, $\mathrm{id}_{\mathcal{C}} \subseteq \gamma \circ \alpha$ and $\alpha \circ \gamma \sqsubseteq \mathrm{id}_{\mathcal{A}}$.

12.3 Duck Type Inference as the Abstraction Map

The function `infer_duck_type` in `duck.py` implements the abstraction map α_{duck}.

Definition 12.4 (Duck Abstraction α_{duck})**.** Given a program P with parameter x, the duck abstraction extracts the set of operations used on x (via AST walking), then maps them to the tightest duck type:

$$\alpha_{\mathsf{duck}}(P, x) := \begin{cases} \mathsf{int} & \text{if } \mathsf{ops}(P,x) \cap \{\mathsf{sub}, \mathsf{mul}, \mathsf{mod}, \ldots\} \neq \emptyset \\ \mathsf{str} & \text{if } \mathsf{ops}(P,x) \cap \{\mathsf{method_upper}, \mathsf{method_split}, \ldots\} \neq \emptyset \\ \mathsf{list} & \text{if } \mathsf{ops}(P,x) \cap \{\mathsf{method_append}, \mathsf{method_sort}, \ldots\} \neq \emptyset \\ \mathsf{collection} & \text{if } \mathsf{ops}(P,x) \cap \{\mathsf{iter}, \mathsf{getitem}, \ldots\} \neq \emptyset \\ \mathsf{any} & \text{otherwise} \end{cases}$$

Proposition 12.5 (Soundness of Duck Abstraction)**.** *The map α_{duck} is an upper adjoint: if program P uses operation* mul *on parameter x, then $\alpha_{\mathsf{duck}}(P, x) = \mathsf{int}$, which is sound because every concrete execution of P requires x to support multiplication. The implementation takes the union $\mathsf{ops}(f, x) \cup \mathsf{ops}(g, x)$ across both programs being compared, ensuring that the inferred fiber is valid for both.*

Remark 12.6 (Implementation Detail)**.** The function `extract_duck_ops` performs a single AST walk to collect the operation set. The priority order of the type decision mirrors the lattice: numeric operations dominate (most restrictive), followed by string methods, list methods, collection operations, and finally the $\top$ element (`any`). The boolean return value `confident` flags whether the inference produced a strict lower bound (the program certainly uses that type) or a conservative $\top$.

12.4 Spec Identification as the Concretization Map

The function `_identify_spec` in `path_search.py` implements the concretization direction.

Definition 12.7 (Spec Concretization γ_{spec})**.** Given a specification name $s \in D_{\mathsf{spec}}$, the concretization $\gamma_{\mathsf{spec}}(s)$ returns the set of all normalized OTerms that satisfy s:

$$\gamma_{\mathsf{spec}}(\mathsf{factorial}) := \{t \in \mathsf{OTerm} \mid t \text{ normalizes to } \mathsf{fold}[\mathsf{mul}](1, \mathsf{range}(\ldots)) \text{ or } \mathsf{fix}(n \cdot f(n-1))\}$$

The `_identify_spec` function acts as the *membership test* for $\gamma_{\mathsf{spec}}(s)$: given an OTerm t, it returns s iff $t \in \gamma_{\mathsf{spec}}(s)$.

Theorem 12.8 (Galois Property of Duck×Spec)**.** *The pair* (α, γ) *where* $\alpha := \alpha_{\mathsf{duck}} \times \alpha_{\mathsf{spec}}$ *and* $\gamma := \gamma_{\mathsf{duck}} \times \gamma_{\mathsf{spec}}$ *forms a Galois connection:*

$$(\alpha_{\mathsf{duck}}(P), \alpha_{\mathsf{spec}}(P)) \sqsubseteq (d, s) \iff P \in \gamma_{\mathsf{duck}}(d) \cap \gamma_{\mathsf{spec}}(s).$$

When both programs f, g *satisfy* $\alpha(f) = \alpha(g)$*, their normalized forms inhabit the same fiber of* γ*, and path search can proceed within that fiber.*

Proof. The duck component is immediate: α_{duck} selects the tightest type containing all observed operations, and γ_{duck} returns all programs whose operations are compatible with the given type. The spec component follows because `_identify_spec` recognizes exactly the canonical forms produced by the 7-phase normalizer, so $t \in \gamma_{\mathsf{spec}}(\alpha_{\mathsf{spec}}(t))$ after normalization. □

Theorem 12.9 (Abstract Spec Inference via Galois Connection)**.** *The abstract presheaf* $\mathsf{Sem}_f^{\#}$ *inferred by abstract interpretation is a* sound over-approximation *of the specification:*

$$f \models S \implies \alpha(\mathsf{Sem}_f) \sqsubseteq \alpha(\mathsf{Sem}_S)$$

Contrapositively: if $\alpha(\mathsf{Sem}_f) \not\sqsubseteq \alpha(\mathsf{Sem}_S)$*, then* f *violates* S *(sound bug detection).*

12.5 How This Increases CTTT's Power

The basic CTTT (Parts I–III) compares Z3 terms directly, which fails when terms are structurally different. Abstract interpretation *lifts* the comparison to a coarser domain where structurally different terms may have the same abstract value.

Example 12.10 (Sign-Domain Spec Inference)**.** Consider:

```
def f(n):          # recursive factorial
    if n <= 1: return 1
    return n * f(n-1)

def g(n):          # iterative factorial
    r = 1
    for i in range(2, n+1): r *= i
    return r
```

The Galois connection works at two levels simultaneously:

1. **Duck level**: $\alpha_{\mathsf{duck}}(f, n) = \alpha_{\mathsf{duck}}(g, n) = \mathsf{int}$ (both use mul, sub, le). This narrows the type site to the single fiber $U_{\mathbb{Z}}$.

2. **Spec level**: after normalization, $\alpha_{\mathsf{spec}}(f) = \alpha_{\mathsf{spec}}(g) = \mathsf{factorial}$. The `_identify_spec` function recognizes both $\mathsf{fix}(n \cdot f(n-1))$ and $\mathsf{fold}[\mathsf{mul}](1, \mathsf{range}(\ldots))$ as the factorial specification.

3. **Path discovery**: since both map to the same $(d, s) = (\mathsf{int}, \mathsf{factorial})$, the path search uses $\mathsf{D20_spec_unify}$ to conclude equivalence immediately.

Example 12.11 (Sortedness-Domain Spec Checking)**.**

```
@guarantee("returns a sorted list")
def f(xs):
    result = []
    for x in xs:
        bisect.insort(result, x)
    return result
```

The Sortedness abstract domain infers: $\alpha(\mathsf{Sem}_f) = \mathsf{sorted}$ (because $\mathsf{bisect.insort}$ maintains sorted invariant — this is a *transfer function* in the abstract domain).

The specification S requires $\alpha(\mathsf{Sem}_S) = \mathsf{sorted}$. Since $\alpha(\mathsf{Sem}_f) = \alpha(\mathsf{Sem}_S)$, the spec is satisfied at the abstract level. This is a *proof* (not just a test) that the function returns a sorted list.

12.6 The Galois–Čech Interaction

The abstract domains interact with the Čech complex:

Theorem 12.12 (Abstract Cohomology)**.** *There is a morphism of cochain complexes:*

$$\begin{tikzcd}
C^0(\mathcal{U}, \mathsf{Iso}) \arrow[r, "\delta^0"] \arrow[d, "\alpha"] & C^1(\mathcal{U}, \mathsf{Iso}) \arrow[r, "\delta^1"] \arrow[d, "\alpha"] & C^2(\mathcal{U}, \mathsf{Iso}) \arrow[d, "\alpha"] \\
C^0(\mathcal{U}, \mathsf{Iso}^\#) \arrow[r, "\delta^{0,\#}"] & C^1(\mathcal{U}, \mathsf{Iso}^\#) \arrow[r, "\delta^{1,\#}"] & C^2(\mathcal{U}, \mathsf{Iso}^\#)
\end{tikzcd}$$

The abstraction α commutes with the coboundary operators. Therefore:

$$H^1(\mathcal{U}, \mathsf{Iso}^\#) = 0 \implies H^1(\mathcal{U}, \mathsf{Iso}) = 0$$

(if the abstract Čech complex has trivial H^1, so does the concrete one — but not vice versa).

Proof. Let $\sigma = (s_{ij}) \in C^1(\mathcal{U}, \mathsf{Iso})$ be a 1-cocycle. Then $\alpha(\sigma) = (\alpha(s_{ij})) \in C^1(\mathcal{U}, \mathsf{Iso}^\#)$ is a 1-cocycle in the abstract complex because α is a lattice homomorphism. If $H^1(\mathcal{U}, \mathsf{Iso}^\#) = 0$, then $\alpha(\sigma) = \delta^{0,\#}(\tau^\#)$ for some $\tau^\# \in C^0(\mathcal{U}, \mathsf{Iso}^\#)$. By the Galois property, $\gamma(\tau^\#)$ provides a 0-cochain in $C^0(\mathcal{U}, \mathsf{Iso})$ whose coboundary dominates σ, and the concretization is compatible with δ^0 by naturality of γ. □

This gives a *cheaper* sufficient condition for equivalence: compute H^1 in the abstract domain (faster, always terminates) before resorting to the full Z3 check (slower, may timeout).

Corollary 12.13 (Galois-Accelerated Verification)**.** *The verification pipeline can be organized as a decision cascade:*

1. *Compute $\alpha_{\mathsf{duck}}(f) = \alpha_{\mathsf{duck}}(g)$? If not, programs operate on different types:* non-equivalent.
2. *Compute $\alpha_{\mathsf{spec}}(f) = \alpha_{\mathsf{spec}}(g)$? If yes,* equivalent *by spec unification (*D20*).*
3. *Check $H^1(\mathcal{U}, \mathsf{Iso}^\#) = 0$? If yes,* equivalent *by abstract cohomological vanishing.*
4. *Fall back to full Z3 / BFS path search.*

Each step is strictly cheaper than the next, and each provides a sound answer when it succeeds.

Chapter 13

The Descent Spectral Sequence

The spectral sequence is a systematic tool for computing cohomology in stages, each refining the previous. It vastly increases CTTT's power by enabling *incremental* verification: start with a coarse check, refine only where needed.

13.1 Filtrations of the Type Site

Definition 13.1 (Type Complexity Filtration)**.** The type site $\mathcal{S}$ has a natural filtration by complexity:

$$F^0 = \{\mathsf{None}, \mathsf{Bot}\} \quad \text{(trivial types)} \tag{13.1}$$
$$F^1 = F^0 \cup \{\mathsf{Bool}\} \quad \text{(finite types)} \tag{13.2}$$
$$F^2 = F^1 \cup \{\mathsf{Int}\} \quad \text{(countable types)} \tag{13.3}$$
$$F^3 = F^2 \cup \{\mathsf{Str}\} \quad \text{(string type)} \tag{13.4}$$
$$F^4 = F^3 \cup \{\mathsf{Pair}\} \quad \text{(structural types)} \tag{13.5}$$
$$F^5 = F^4 \cup \{\mathsf{Ref}\} \quad \text{(reference types — heap)} \tag{13.6}$$

Each level adds more complex types, requiring more sophisticated reasoning.

13.2 The Normalizer Filtration

The 7-phase normalizer in `denotation.py` provides a second, independent filtration that is *dual* to the type complexity filtration: it stratifies the *rewrite system* rather than the type site.

Definition 13.2 (Normalizer Filtration)**.** Define a filtration of the rewrite system $\mathcal{R}$ by the normalizer phases:

$$N^0 := \varnothing \quad \text{(raw OTerm, no rewrites)} \tag{13.7}$$
$$N^1 := N^0 \cup \mathcal{R}_\beta \quad \text{(Phase 1: } \beta\text{-reduction, trivial collapse)} \tag{13.8}$$
$$N^2 := N^1 \cup \mathcal{R}_{\mathsf{ring}} \quad \text{(Phase 2: ring/lattice axioms)} \tag{13.9}$$
$$N^3 := N^2 \cup \mathcal{R}_{\mathsf{fold}} \quad \text{(Phase 3: fold canonicalization)} \tag{13.10}$$
$$N^4 := N^3 \cup \mathcal{R}_{\mathsf{HOF}} \quad \text{(Phase 4: HOF fusion)} \tag{13.11}$$
$$N^5 := N^4 \cup \mathcal{R}_{\mathsf{unify}} \quad \text{(Phase 5: symbol unification)} \tag{13.12}$$
$$N^6 := N^5 \cup \mathcal{R}_{\mathsf{quot}} \quad \text{(Phase 6: quotient normalization)} \tag{13.13}$$
$$N^7 := N^6 \cup \mathcal{R}_{\mathsf{case}} \quad \text{(Phase 7: piecewise case normalization)} \tag{13.14}$$

Each N^k is a sub-system of rewrite rules, and the full normalizer applies N^7.

Remark 13.3 (Phase Content)**.** The phases correspond to increasingly powerful equational theories:

- $\mathcal{R}_\beta$: $\mathsf{case}(\mathsf{True}, t, f) \rightsquigarrow t$; $\mathsf{case}(c, x, x) \rightsquigarrow x$; inlining of trivial let-bindings.
- $\mathcal{R}_{\mathsf{ring}}$: $x + 0 \rightsquigarrow x$; $x \cdot 1 \rightsquigarrow x$; $x \cdot 0 \rightsquigarrow 0$; $(x + c) - c \rightsquigarrow x$; $\mathsf{sorted}(\mathsf{sorted}(x)) \rightsquigarrow \mathsf{sorted}(x)$ (idempotence); $\mathsf{reversed}(\mathsf{reversed}(x)) \rightsquigarrow x$ (involution).
- $\mathcal{R}_{\mathsf{fold}}$: conversion of $\mathsf{sum}/\mathsf{prod}/\mathsf{any}/\mathsf{all}$ to canonical $\mathsf{fold}[\mathsf{op}](\mathsf{init}, \mathsf{coll})$ form (dichotomy D5).
- $\mathcal{R}_{\mathsf{HOF}}$: $\mathsf{sorted}(\mathsf{list}(x)) \rightsquigarrow \mathsf{sorted}(x)$; $\mathsf{sorted}(\mathsf{set}(x)) \rightsquigarrow \mathsf{sorted}(x)$; $\mathsf{abs}(x) \rightsquigarrow \mathsf{case}(x < 0, -x, x)$; map–map fusion (operadic — see Chapter 14).
- $\mathcal{R}_{\mathsf{unify}}$: the 24 dichotomy rewrite rules (D1–D24), unifying `heapq` $\leftrightarrow$ `sorted`, `Counter` $\leftrightarrow$ frequency-`dict`, etc.
- $\mathcal{R}_{\mathsf{quot}}$: sorting elements of OSeq by canonical form to quotient by the permutation group S_n (sets and dicts are unordered).
- $\mathcal{R}_{\mathsf{case}}$: canonical ordering of guards in nested OCase chains; absorption of redundant branches.

Definition 13.4 (The Spectral Sequence)**.** The filtration induces a spectral sequence $\{E_r^{p,q}\}$ with:

$$E_0^{p,q} = C^{p+q}(F^p/F^{p-1}, \mathsf{Iso}) \tag{13.15}$$

$$E_1^{p,q} = H^{p+q}(F^p/F^{p-1}, \mathsf{Iso}) \tag{13.16}$$

$$E_\infty^{p,q} \Rightarrow H^{p+q}(\mathcal{S}, \mathsf{Iso}) \tag{13.17}$$

13.3 The E_2 Page: What Survives Normalization

Definition 13.5 (Phase-Graded E_2)**.** For the normalizer filtration, define the *phase-graded* spectral sequence:

$$\widetilde{E}_1^k := \ker(N^k\text{-normalize})/\ker(N^{k-1}\text{-normalize})$$

i.e., the set of term pairs that become equivalent at phase k but were *not* equivalent at phase $k-1$. The $\widetilde{E}_2$ page is:

$$\widetilde{E}_2^k = H(\widetilde{E}_1^k, d_1)$$

where d_1 captures interactions between adjacent phases.

Example 13.6 (What Each Phase Resolves)**.** Consider two programs computing the product of a list:

```python
def f(xs):  return reduce(lambda a,b: a*b, xs, 1)
def g(xs):
    r = 1
    for x in xs: r *= x
    return r
```

- After Phase 1 (β-reduction): f becomes $\mathsf{reduce}(\mathsf{mul}, 1, \mathsf{xs})$; g becomes $\mathsf{fold}[\mathsf{imul}](1, \mathsf{xs})$. Still different ($\widetilde{E}_1^1 \neq 0$).
- After Phase 3 (fold canonicalization): $\mathsf{reduce}(\mathsf{mul}, 1, \mathsf{xs}) \rightsquigarrow \mathsf{fold}[\mathsf{mul}](1, \mathsf{xs})$. Now equivalent modulo op-name ($\widetilde{E}_1^3$ absorbs this).
- After Phase 5 (symbol unification): $\mathsf{imul} \mapsto \mathsf{mul}$. Both are $\mathsf{fold}[\mathsf{mul}](1, \mathsf{xs})$. Resolved: $\widetilde{E}_1^5 = 0$.

The spectral sequence tracks *where* equivalence is established.

13.4 The Verification Strategy

The spectral sequence dictates an *optimal verification strategy*:

1. E_0 **page**: check equivalence on each filtration *grade* independently. For the None/Bot grade: trivially equivalent (both return the same constant on trivial inputs). For the Bool grade: finitely many inputs, decidable. For the Int grade: Z3 integer arithmetic, often decidable.

2. E_1 **page**: compute H^1 within each grade. If $H^1(F^p/F^{p-1}) = 0$ for all p, the spectral sequence *degenerates* and $H^1(\mathcal{S}) = 0$ (equivalence).

3. E_2 **page**: if E_1 has non-trivial differentials, compute $E_2 = H(E_1, d_1)$. This captures *interactions* between adjacent grades (e.g., an integer function whose behavior depends on a boolean flag).

4. **Higher pages**: each E_r refines the approximation. In practice, the spectral sequence degenerates at E_1 or E_2 for most programs (because the type fibers are independent).

13.5 Convergence and Degeneration

Theorem 13.7 (Spectral Sequence Degeneration for Duck-Typed Programs)**.** *If duck type inference determines that each parameter has a* single *type (the most common case), the spectral sequence degenerates at E_1:*

$$E_1^{p,q} = E_\infty^{p,q} \quad \textit{for all } p, q$$

and $H^1(\mathcal{S}, \mathsf{Iso}) = \bigoplus_p E_1^{p,0}$ (direct sum of fiber-wise cohomologies).

Proof. When each parameter has a unique type, the type site has a single object (one fiber combination). There are no overlaps, no transition functions, and the Čech complex is trivial: $C^0 = \mathsf{Iso}(U)$, $C^1 = C^2 = 0$. Therefore $H^1 = 0$ (no obstructions to check) and the local verdict IS the global verdict. The spectral sequence has a single non-trivial entry $E_1^{0,0} = H^0(U, \mathsf{Iso})$ and degenerates immediately. □

Theorem 13.8 (Normalizer Convergence)**.** *The normalizer spectral sequence $\{\widetilde{E}_r^k\}$ converges in at most 7 steps (one per phase). The fixpoint iteration in* `normalize()` *terminates when $\widetilde{E}_1^k = 0$ for all k, i.e., no phase produces new identifications.*

Concretely, if $\mathsf{canon}(N^7(t)) = \mathsf{canon}(N^7(t'))$, then the spectral sequence degenerates at the smallest k such that $\mathsf{canon}(N^k(t)) = \mathsf{canon}(N^k(t'))$.

Proof. The normalizer applies phases $N^1, \ldots, N^7$ in sequence, then iterates until the canonical form stabilizes. Each phase is a confluent rewrite system on the finite OTerm algebra (modulo recursion depth bounds). The fixed-point check `current.canon() == prev.canon()` ensures termination within 8 iterations. At convergence, no phase changes the canonical form, so $\widetilde{E}_1^k = 0$ for all k. □

This theorem explains why our current implementation achieves good results on programs with clear types: the spectral sequence degenerates, and a single Z3 query per fiber suffices.

13.6 Non-Degenerate Cases: Type Polymorphism

When parameters are polymorphic (duck-typed with multiple possible types), the spectral sequence does NOT degenerate, and the E_2 page reveals interactions:

Example 13.9 (Type-Dependent Behavior).

```
def f(x):
    if isinstance(x, int):
        return x * 2
    elif isinstance(x, str):
        return x + x
    else:
        return [x, x]
```

Duck type inference returns $(\mathsf{any}, \mathsf{True})$ because `isinstance` checks are detected: $\mathsf{ops} \supseteq \{\mathsf{isinstance_int}, \mathsf{isinstance_str}\}$, triggering the polymorphic branch in `infer_duck_type`.

The type site has three fibers: Int, Str, Other. On each fiber, f has different behavior. The E_1 page checks each fiber independently. The E_2 page checks compatibility across fibers: does the "doubling" specification ($S(x, y) = (y = x + x)$) make sense across type boundaries?

For integers, $x + x = 2x$ (integer doubling). For strings, $x + x = xx$ (string repetition). These are *different operations* (ADD vs. Concat) that happen to share the + syntax.

The E_2 differential detects this: the integer and string fibers use different path constructors (add_comm vs. string concat), and the transition function on the Int∩Str overlap is non-trivial. This is a genuinely *higher* cohomological phenomenon — invisible to E_1.

Proposition 13.10 (Phase-Fiber Interaction). *The two filtrations (type complexity and normalizer phase) interact via a* double complex*:*

$$E_1^{p,k} := H^k(\widetilde{E}_1^{(p)})$$

where $\widetilde{E}_1^{(p)}$ is the normalizer spectral sequence restricted to fiber grade p. A term pair (t, t') that survives all normalizer phases on fiber p but fails on fiber p' produces a non-trivial class in $E_2^{p,0} \oplus E_2^{p',0}$: the programs agree on some type fibers but disagree on others (a partial bug).

Chapter 14

Operad Theory for Higher-Order Programs

Python programs frequently use *higher-order functions*: functions that take other functions as arguments (`map`, `filter`, `reduce`, `sorted` with key, etc.). The CTTT framework handles these via *operad theory*, which provides the algebraic structure for composing operations.

14.1 The Operad of Python Operations

Definition 14.1 (The Python Operations Operad)**.** The *Python operations operad* $\mathcal{O}_{\mathsf{Py}}$ has:

- **Colors** (sorts): PyObj, $\mathbb{B}$, $\mathbb{Z}$, Str, Seq, Map.
- **Operations**: for each Python builtin/method, an operation $o \in \mathcal{O}(c_1, \ldots, c_n; c)$ mapping n inputs of colors $c_1, \ldots, c_n$ to an output of color c.
- **Composition**: operadic composition $\circ_i$ (substituting an operation into the i-th input of another).
- **Equivariance**: permutation of inputs (for commutative operations).

Definition 14.2 (Arity Profile)**.** Each operation $o \in \mathcal{O}_{\mathsf{Py}}$ has an *arity profile* $(n; k)$ where n is the number of data inputs and k is the number of function inputs. The arity profile determines the operation's position in the operadic hierarchy:

- $(n; 0)$: first-order operations (arithmetic, string ops).
- $(n; 1)$: second-order operations (`map`, `filter`, `sorted` with key).
- $(n; 2)$: third-order operations (`reduce` with binary function, `itertools.accumulate`).

Example 14.3 (Operations in the Operad)**.**

$$+ \in \mathcal{O}(\mathbb{Z}, \mathbb{Z}; \mathbb{Z}) \qquad \text{arity } (2; 0) \tag{14.1}$$

$$\mathsf{sorted} \in \mathcal{O}(\mathsf{Seq}; \mathsf{Seq}) \qquad \text{arity } (1; 0) \tag{14.2}$$

$$\mathsf{map} \in \mathcal{O}((\mathsf{PyObj} \to \mathsf{PyObj}), \mathsf{Seq}; \mathsf{Seq}) \qquad \text{arity } (1; 1) \tag{14.3}$$

$$\mathsf{reduce} \in \mathcal{O}((\mathsf{PyObj} \times \mathsf{PyObj} \to \mathsf{PyObj}), \mathsf{PyObj}, \mathsf{Seq}; \mathsf{PyObj}) \qquad \text{arity } (2; 1) \tag{14.4}$$

$$\mathsf{filter} \in \mathcal{O}((\mathsf{PyObj} \to \mathbb{B}), \mathsf{Seq}; \mathsf{Seq}) \qquad \text{arity } (1; 1) \tag{14.5}$$

14.2 Operadic Composition and Phase 4

The key insight connecting operad theory to the implementation: *Phase 4 of the normalizer implements operadic composition laws.*

Definition 14.4 (Operadic Composition $\circ_i$)**.** Given operations $o_1 \in \mathcal{O}(c_1, \ldots, c_n; c)$ and $o_2 \in \mathcal{O}(d_1, \ldots, d_m; c_i)$, the operadic composition $o_1 \circ_i o_2 \in \mathcal{O}(c_1, \ldots, c_{i-1}, d_1, \ldots, d_m, c_{i+1}, \ldots, c_n; c)$ substitutes o_2 into the i-th input of o_1.

Example 14.5 (Map–Map Fusion as Operadic Relation)**.** The rule $\mathsf{map}(f, \mathsf{map}(g, \mathsf{xs})) \equiv \mathsf{map}(f \circ g, \mathsf{xs})$ is an *operadic relation*: composing two applications of the map operation (arity $(1; 1)$) yields a single map whose function input is the composition $f \circ g$.

In Phase 4 of the normalizer, this is implemented by recognizing nested $\mathsf{OOp}(\texttt{"sorted"}, (\mathsf{OOp}(\texttt{"list"}, \ldots),))$ and collapsing: $\mathsf{sorted}(\mathsf{list}(x)) \rightsquigarrow \mathsf{sorted}(x)$. More generally, any idempotent wrapper (like `list` around `sorted`) is absorbed by the outer operation.

14.3 Operadic Path Constructors

The operad structure provides new path constructors beyond the basic algebraic axioms:

Theorem 14.6 (Map–Fold Fusion)**.** *For a monoid $(M, \oplus, e)$ and a function $h : A \to M$:*

$$\mathsf{reduce}(\oplus, e, \mathsf{map}(h, \mathsf{xs})) \equiv \mathsf{reduce}(\lambda \mathsf{acc}, x.\ \mathsf{acc} \oplus h(x), e, \mathsf{xs})$$

This is the map–fold fusion law*: mapping then folding is equivalent to folding with a composed operation.*

In operadic terms: $\mathsf{reduce} \circ_3 \mathsf{map} = \mathsf{reduce}'$ *where* reduce' *absorbs h into its binary operation via* $\oplus' = \lambda a, x.\ a \oplus h(x)$.

Proof. By induction on xs (Nat-recursor uniqueness applied to the list length):

- Base: both sides return e on the empty list.
- Step: the left side maps h to the head, then folds; the right side applies $\oplus$ with $h(\mathsf{head})$ directly. Both produce $\mathsf{acc} \oplus h(\mathsf{head})$.

□

Theorem 14.7 (Filter–Map Commutation)**.** *If the predicate p and the transformation h are independent ($p(h(x)) = p(x)$ for all x):*

$$\mathsf{filter}(p, \mathsf{map}(h, \mathsf{xs})) \equiv \mathsf{map}(h, \mathsf{filter}(p, \mathsf{xs}))$$

This is an interchange law *in the operad:* filter *and* map *commute when their function inputs are independent. The independence condition $p \circ h = p$ is checkable by Z3 on the restricted fiber.*

Theorem 14.8 (Sorted–Map Interaction)**.** *If h is* monotone *($x \leq y \implies h(x) \leq h(y)$):*

$$\mathsf{sorted}(\mathsf{map}(h, \mathsf{xs})) \equiv \mathsf{map}(h, \mathsf{sorted}(\mathsf{xs}))$$

Sorting then mapping a monotone function is the same as mapping then sorting.

Theorem 14.9 (HOF Absorption Laws (Phase 4))**.** *The following operadic absorption laws are implemented directly in Phase 4 of the normalizer:*

$$\mathsf{sorted}(\mathsf{list}(x)) \equiv \mathsf{sorted}(x) \qquad (\mathsf{list} \text{ is absorbed})$$
$$\mathsf{sorted}(\mathsf{set}(x)) \equiv \mathsf{sorted}(x) \qquad (\mathsf{set} \text{ is absorbed})$$
$$\mathsf{list}(\mathsf{reversed}(\mathsf{sorted}(x))) \equiv \mathsf{sorted_rev}(x) \qquad (\text{composition fused})$$
$$\mathsf{len}(\mathsf{list}(x)) \equiv \mathsf{len}(x) \qquad (\mathsf{list} \text{ is transparent to } \mathsf{len})$$

Each law expresses that an inner operation is operadically redundant*: it does not affect the observable output of the outer operation.*

Proof. For $\mathsf{sorted}(\mathsf{list}(x)) \equiv \mathsf{sorted}(x)$: list materializes an iterable into a list, and sorted accepts any iterable and produces a sorted list. Since sorted internally iterates its argument, the intermediate materialization is semantically invisible. The remaining laws follow by similar operadic absorption arguments. □

These operadic laws generate new paths in the HIT PyComp, enabling equivalence proofs for programs that compose higher-order operations in different ways.

14.4 The Operadic Nerve and Higher Coherence

Definition 14.10 (Operadic Nerve)**.** The *nerve* of the operad $\mathcal{O}_{\mathsf{Py}}$ is the simplicial set $N\mathcal{O}_{\mathsf{Py}}$ whose n-simplices are chains of n composable operations:

$$(N\mathcal{O}_{\mathsf{Py}})_n = \{(o_1, \ldots, o_n) \mid o_k \circ_{i_k} o_{k+1} \text{ is defined for each } k\}$$

A path in $N\mathcal{O}_{\mathsf{Py}}$ from a composition tree T_f to T_g witnesses that f and g compute the same function via different operadic decompositions.

Remark 14.11 (Coherence via Associahedra)**.** The associativity of operadic composition is governed by the *Stasheff associahedron* K_n. For three composable operations o_1, o_2, o_3, the two bracketings $(o_1 \circ o_2) \circ o_3$ and $o_1 \circ (o_2 \circ o_3)$ are connected by a face of K_3. In our setting, this means that different orders of applying Phase 4 fusion rules all produce the same normal form — a consequence of the normalizer's confluence.

14.5 Application: Higher-Order Equivalence

Example 14.12 (Map+Filter vs. Comprehension with Condition)**.**

```python
# Program A: map then filter
def f(xs):
    return list(filter(lambda x: x > 0, map(lambda x: x*2, xs)))

# Program B: comprehension with condition
def g(xs):
    return [x*2 for x in xs if x*2 > 0]
```

The operadic path:

1. $\mathsf{filter}(p, \mathsf{map}(h, \mathsf{xs}))$ (program A).
2. By filter–map commutation (since $p(h(x)) = (h(x) > 0)$ and $p(x) = (x > 0)$ — but $h(x) = 2x$, so $p(h(x)) = (2x > 0) \neq (x > 0)$ in general! The commutation does NOT apply here).
3. Instead, use the comprehension–loop equivalence (D4): the comprehension $[h(x) \text{ for } x \text{ if } p(h(x))]$ is equivalent to $\mathsf{filter}(p, \mathsf{map}(h, \mathsf{xs}))$.
4. The path is $\mathsf{D4}$ (comprehension fusion).

Example 14.13 (Sum-of-Squares: Operadic Decomposition)**.**

```python
# Program A: map then sum
def f(xs): return sum(x**2 for x in xs)

# Program B: fold with accumulator
def g(xs):
```

```
    r = 0
    for x in xs: r += x**2
    return r
```

The operadic analysis proceeds through the normalizer:

1. Phase 3 canonicalizes both to $\mathsf{fold}[\mathsf{add}](0, \mathsf{map}(\lambda x.\ x^2, \mathsf{xs}))$.
2. Phase 4 applies map–fold fusion (Theorem 14.6): $\mathsf{fold}[\mathsf{add}](0, \mathsf{map}(h, \mathsf{xs})) \rightsquigarrow \mathsf{fold}[\lambda a, x.\ a + x^2](0, \mathsf{xs})$.
3. Both programs now have the same canonical OTerm.

The equivalence is *operadic*: it arises from the composition law $\mathsf{reduce} \circ_3 \mathsf{map} = \mathsf{reduce}'$ rather than from any ring axiom.

Chapter 15

The Semantic Enrichment Theorem

The most powerful extension of CTTT: using the cubical path space to *enrich* the Čech complex, yielding a finer cohomology that distinguishes programs the basic theory cannot.

15.1 Enrichment of the Program Category

Recall that the program category $\mathcal{P}$ has Python functions as objects and equivalence witnesses as morphisms. So far we have treated $\mathrm{Hom}(f, g)$ as a *set*: either empty (non-equivalent) or a singleton (equivalent). The enrichment lifts $\mathrm{Hom}(f, g)$ from **Set** to a richer category that records *how* f and g are related.

Definition 15.1 (Duck-Type Lattice)**.** Let $\mathcal{D}$ be the lattice of duck types ordered by subtyping:

$$\mathcal{D} := (\{\mathsf{Addable}, \mathsf{Iterable}, \mathsf{Comparable}, \mathsf{Hashable}, \ldots\},\ \sqsubseteq)$$

where $D_1 \sqsubseteq D_2$ iff every operation required by D_1 is also required by D_2. This is a finite lattice with meet $D_1 \sqcap D_2$ (intersection of required operations) and join $D_1 \sqcup D_2$ (union).

Definition 15.2 (Enriched Hom)**.** The *$\mathcal{D}$-enriched hom* between programs f and g is:

$$\underline{\mathrm{Hom}}(f, g) := \prod_{U_i \in \mathcal{U}} \{D \in \mathcal{D} \mid \mathsf{Sem}_f|_{U_i} =_D \mathsf{Sem}_g|_{U_i}\} \ \in\ \mathcal{D}$$

That is, the meet (in the duck-type lattice) of all duck types under which f and g agree on each fiber. If f and g agree as $\mathsf{Addable}$ on the Int fiber but only as $\mathsf{Comparable}$ on the Str fiber, then $\underline{\mathrm{Hom}}(f, g) = \mathsf{Comparable}$ (the coarser agreement).

The ordinary (boolean) hom is recovered by forgetting the enrichment: $\mathrm{Hom}(f, g) \neq \varnothing$ iff $\underline{\mathrm{Hom}}(f, g) \neq \bot_{\mathcal{D}}$.

15.2 Enriched Cohomology

Definition 15.3 (Path-Enriched Cochain)**.** An *enriched 0-cochain* assigns to each fiber U_i not just a boolean (isomorphism or not) but a *path type*:

$$C^0_{\mathsf{enr}}(\mathcal{U}, \mathsf{Iso}) := \prod_i \|\mathsf{Path}(\mathsf{Sem}_f|_{U_i}, \mathsf{Sem}_g|_{U_i})\|$$

where $\|\cdot\|$ is propositional truncation (we care about existence, not the specific path).

The enrichment adds information: we know not just *whether* two programs agree on a fiber, but *how* they agree (which axioms connect them).

Definition 15.4 (Enriched 1-Cochain)**.** The enriched 1-cochain group is:

$$C^1_{\mathsf{enr}}(\mathcal{U}, \mathsf{Iso}) := \prod_{i<j} \mathsf{Path}\big(p_i|_{U_{ij}},\ p_j|_{U_{ij}}\big)$$

where $U_{ij} = U_i \cap U_j$ is the overlap fiber. An element of C^1_{enr} records, for each overlap, whether the local equivalence paths are *themselves* path-connected—i.e. whether the axioms used on adjacent fibers are compatible.

Definition 15.5 (Enriched Coboundary)**.** The enriched coboundary $\delta^0_{\mathsf{enr}} : C^0_{\mathsf{enr}} \to C^1_{\mathsf{enr}}$ assigns to each overlap U_{ij} the *transition path*:

$$(\delta^0_{\mathsf{enr}}\,\varphi)_{ij} = p_j|_{U_{ij}} \cdot (p_i|_{U_{ij}})^{-1}$$

where p_i and p_j are the local paths on fibers U_i and U_j. A cocycle $\varphi \in \ker \delta^0_{\mathsf{enr}}$ is a family of local paths whose transition maps are all trivial (the identity path).

Theorem 15.6 (Enriched Descent)**.** *The enriched cohomology H^1_{enr} is a refinement of the ordinary H^1:*

$$H^1_{\mathsf{enr}} = 0 \implies H^1 = 0$$

but not vice versa. The enriched version detects path incompatibilities*: situations where local paths exist but use incompatible axioms on overlaps.*

Proof. There is a natural forgetful map $\pi : C^\bullet_{\mathsf{enr}} \to C^\bullet$ given by truncating each path type to its mere proposition (existence). This map is a cochain morphism ($\pi \circ \delta_{\mathsf{enr}} = \delta \circ \pi$) and hence induces a map on cohomology:

$$\pi^* : H^1_{\mathsf{enr}} \to H^1.$$

If $H^1_{\mathsf{enr}} = 0$, then every enriched 1-cocycle is a coboundary. Applying π^*, every ordinary 1-cocycle is also a coboundary, so $H^1 = 0$.

For the converse failure, consider two fibers U_1, U_2 with local paths p_1, p_2 such that $p_1|_{U_{12}}$ uses axiom α (e.g. commutativity of $+$) and $p_2|_{U_{12}}$ uses axiom β (e.g. associativity of $\cdot$) where α and β are not themselves connected by a path in the axiom space. Then $H^1 = 0$ (both fibers pass the boolean check) but $H^1_{\mathsf{enr}} \neq 0$ (the transition path $p_2|_{U_{12}} \cdot (p_1|_{U_{12}})^{-1}$ is a non-trivial loop in the axiom groupoid). □

Example 15.7 (Path Incompatibility)**.** Program f uses `isinstance(x, int)` to dispatch. On the Int fiber, the equivalence path uses the add commutativity axiom. On the Str fiber, the path uses string concatenation (which is NOT commutative).

The ordinary H^1 says: both fibers are locally equivalent, so $H^1 = 0$. But the enriched H^1 detects that the transition path on the Int∩Str overlap is inconsistent (commutativity on one side, non-commutativity on the other).

This reveals a subtle bug: the program treats `+` as commutative everywhere, but it's only commutative for integers.

15.3 The Enrichment–Degeneration Tradeoff

Proposition 15.8 (Enrichment Hierarchy)**.** *There is a tradeoff between enrichment and computability, forming a three-level hierarchy:*

1. ***Boolean enrichment*** *(ordinary H^1): decidable, computable in $O(n^3)$ via GF(2) rank. This is the $\mathcal{D} = \{0, 1\}$ case.*

2. ***Duck-type enrichment*** *(<u>Hom</u> over $\mathcal{D}$): decidable when $\mathcal{D}$ is finite, since the meet computation is a finite lattice operation.*

3. ***Path enrichment*** *(enriched H^1): semi-decidable (each local path check is a Z3 query, which may timeout).*

4. ***Full cubical enrichment*** *(tracking the specific path constructors used): undecidable in general, but provides the most information.*

In practice, we use boolean enrichment (fast) as a first pass, then duck-type enrichment (still fast, more informative) to determine *at what level of abstraction* f and g agree, and finally path enrichment (slower) on fibers where the duck-type check is ambiguous. This is the spectral sequence strategy applied to the enrichment dimension.

Chapter 16

Inductive Families and Dependent Pattern Matching

The most ambitious extension: using *inductive families* (dependent inductive types indexed by values) to capture program invariants that vary with the input. We show that the `OTerm` type from the implementation is itself an inductive family, and that the normalizer's phase-by-phase case analysis is exactly the dependent elimination principle.

16.1 Inductive Families in CTTT

Definition 16.1 (Inductive Family)**.** An *inductive family* $T : I \to \mathsf{Type}$ indexed by I is a type defined by constructors whose return type depends on the index:

$$\mathsf{nil} : T(\mathsf{empty}) \tag{16.1}$$

$$\mathsf{cons} : \prod_{i:I} A \to T(i) \to T(\mathsf{extend}(i)) \tag{16.2}$$

Example 16.2 (Length-Indexed Lists)**.** The type $\mathsf{Vec}(A, n)$ of lists of length exactly n:

$$\mathsf{nil} : \mathsf{Vec}(A, 0) \tag{16.3}$$

$$\mathsf{cons} : A \to \mathsf{Vec}(A, n) \to \mathsf{Vec}(A, n + 1) \tag{16.4}$$

A program that takes a $\mathsf{Vec}(A, n)$ and returns a $\mathsf{Vec}(B, n)$ is *guaranteed* to preserve length.

16.2 OTerm as an Inductive Family

The `OTerm` type in `denotation.py` is the computational manifestation of an inductive family indexed by *arity* (number of free variables).

Definition 16.3 (The OTerm Family)**.** Define the inductive family $\mathsf{OTerm} : \mathbb{N} \to \mathsf{Type}$ indexed

by arity $n \in \mathbb{N}$ via the following constructors:

$$\mathsf{OVar}(k) : \mathsf{OTerm}(n) \qquad \text{where } k < n \tag{16.5}$$
$$\mathsf{OLit}(v) : \mathsf{OTerm}(0) \qquad \text{(literals are closed)} \tag{16.6}$$
$$\mathsf{OOp}(f, \vec{a}) : \mathsf{OTerm}(n) \qquad \text{where } \vec{a} : \mathsf{OTerm}(n)^m \tag{16.7}$$
$$\mathsf{OFold}(f, z, c) : \mathsf{OTerm}(n) \qquad \text{where } z, c : \mathsf{OTerm}(n) \tag{16.8}$$
$$\mathsf{OCase}(t, b_1, b_2) : \mathsf{OTerm}(n) \qquad \text{where } t, b_1, b_2 : \mathsf{OTerm}(n) \tag{16.9}$$
$$\mathsf{OFix}(b) : \mathsf{OTerm}(n) \qquad \text{where } b : \mathsf{OTerm}(n{+}1) \tag{16.10}$$
$$\mathsf{OSeq}(\vec{e}) : \mathsf{OTerm}(n) \qquad \text{where } \vec{e} : \mathsf{OTerm}(n)^m \tag{16.11}$$
$$\mathsf{OLam}(b) : \mathsf{OTerm}(n) \qquad \text{where } b : \mathsf{OTerm}(n{+}k) \tag{16.12}$$
$$\mathsf{OMap}(f, c) : \mathsf{OTerm}(n) \qquad \text{where } f : \mathsf{OTerm}(n{+}1),\ c : \mathsf{OTerm}(n) \tag{16.13}$$
$$\mathsf{OQuotient}(t, R) : \mathsf{OTerm}(n) \qquad \text{where } t : \mathsf{OTerm}(n) \tag{16.14}$$

Note that OFix and OLam *change the index*: their body has arity $n{+}1$ (or $n{+}k$), reflecting the bound variable(s). This is what makes OTerm a genuine inductive family, not merely a simple inductive type.

Remark 16.4 (Implementation Correspondence)**.** In `denotation.py`, the arity index is implicit: `OVar` stores a name string rather than a de Bruijn index, and `OLam` stores parameter names. The `canon()` method performs alpha-normalization (replacing parameter names with positional indices `_b0`, `_b1`, ...), which recovers the de Bruijn representation and thus the indexed family structure.

16.3 The Elimination Principle

Definition 16.5 (OTerm Eliminator)**.** The elimination principle for OTerm is a dependent recursor: for any type family $P : \prod_{n:\mathbb{N}} \mathsf{OTerm}(n) \to \mathsf{Type}$, given methods

$$m_{\mathsf{Var}} : \prod_{n:\mathbb{N}} \prod_{k<n} P(n, \mathsf{OVar}(k)) \tag{16.15}$$
$$m_{\mathsf{Lit}} : \prod_{v} P(0, \mathsf{OLit}(v)) \tag{16.16}$$
$$m_{\mathsf{Op}} : \prod_{n,f,\vec{a}} \Big(\prod_{i} P(n, a_i)\Big) \to P(n, \mathsf{OOp}(f, \vec{a})) \tag{16.17}$$
$$m_{\mathsf{Fold}} : \prod_{n,f,z,c} P(n, z) \to P(n, c) \to P(n, \mathsf{OFold}(f, z, c)) \tag{16.18}$$
$$m_{\mathsf{Case}} : \prod_{n,t,b_1,b_2} P(n, t) \to P(n, b_1) \to P(n, b_2) \to P(n, \mathsf{OCase}(t, b_1, b_2)) \tag{16.19}$$
$$m_{\mathsf{Fix}} : \prod_{n,b} P(n{+}1, b) \to P(n, \mathsf{OFix}(b)) \tag{16.20}$$

(and similarly for OSeq, OLam, OMap, OQuotient), there exists a unique function

$$\mathsf{elim}_{\mathsf{OTerm}} : \prod_{n:\mathbb{N}} \prod_{t:\mathsf{OTerm}(n)} P(n, t)$$

computing by the expected case equations.

Proposition 16.6 (Pattern Matching = Elimination)**.** *Every Python* `if isinstance(t, OVar)` *chain in* `denotation.py` *is an instance of* $\mathsf{elim}_{\mathsf{OTerm}}$*. The normalizer applies multiple elimination passes, each with a different motive P:*

1. ***Constant folding***: $P(n,t) = \mathsf{OTerm}(n)$, *reduces* $\mathsf{OOp}(\mathsf{add}, [\mathsf{OLit}(2), \mathsf{OLit}(3)])$ *to* $\mathsf{OLit}(5)$.
2. ***Algebraic simplification***: $P(n,t) = \mathsf{OTerm}(n)$, *applies rewrite rules (e.g.* $\mathsf{sorted} \circ \mathsf{sorted} \mapsto \mathsf{sorted}$*).*
3. ***Canonical form extraction***: $P(n,t) = \mathsf{String}$, *produces the* `canon()` *hash for comparison.*
4. ***Z3 compilation***: $P(n,t) = \mathsf{Z3Expr}$, *translates OTerm to Z3 terms for the SMT solver.*

16.4 Application: Invariant-Indexed Specifications

Inductive families let us express specifications that depend on structural properties of the input:

Definition 16.7 (Sorted-Input Specification)**.** The type $\mathsf{Sorted}(A)$ of sorted lists:

$$\mathsf{snil} : \mathsf{Sorted}(A) \tag{16.21}$$

$$\mathsf{scons} : \prod_{a:A} \mathsf{Sorted}(A) \to (\forall b \in \mathsf{head}.\ a \leq b) \to \mathsf{Sorted}(A) \tag{16.22}$$

A specification "binary search on sorted input" is:

$$S : \mathsf{Sorted}(\mathbb{Z}) \times \mathbb{Z} \to \mathsf{Option}(\mathbb{N})$$

The input is *typed as sorted*, so the specification can assume the precondition.

16.5 Automatic Precondition Inference via Inductive Families

Theorem 16.8 (Precondition Inference)**.** *Given a program f and a specification S, if f satisfies S only on a subset of inputs, the* precondition *P can be automatically inferred as:*

$$P(x) = \mathsf{Spec}_S \text{ is satisfied at fiber } U_x$$

where U_x is the type fiber containing x.

The precondition is the set of fibers where the local Čech check passes:

$$P = \{x : A \mid \mathsf{Iso}(\mathsf{Sem}_f, \mathsf{Sem}_S)(U_{\mathsf{tag}(x)}) = 1\}$$

Example 16.9 (Inferring "requires positive integer")**.**

```
def sqrt_int(n):
    if n < 0: return None   # error case
    r = 0
    while (r+1)*(r+1) <= n:
        r += 1
    return r
```

Specification: "returns $\lfloor \sqrt{n} \rfloor$".

On the Int^+ fiber (positive integers): correct. On the Int^- fiber (negative integers): returns None, violating the spec.

Inferred precondition: $P(n) = (n \geq 0)$.

The cohomological decomposition reveals: $H^1(U_{\mathsf{Int}^+}) = 0$ (correct on positives), $H^1(U_{\mathsf{Int}^-}) \neq 0$ (fails on negatives). The precondition is the union of passing fibers.

16.6 Dependent Pattern Matching as Fiber Restriction

Python's `isinstance` dispatch is a form of *dependent pattern matching*: the program branches on the type of its input, and each branch has a different type signature.

In CTTT, this is *exactly* the fiber decomposition:

- Each `isinstance` branch restricts to a fiber.
- The branch body is the presheaf section on that fiber.
- The Čech complex checks that the branches are coherent (they agree on type boundaries).

Theorem 16.10 (isinstance as Dependent Elimination)**.** *A function with* `isinstance` *dispatch:*

```
def f(x):
    if isinstance(x, int):     return h_int(x)
    elif isinstance(x, str):   return h_str(x)
    else:                      return h_other(x)
```

has the dependent type:

$$f : \prod_{x:\mathsf{PyObj}} B(\mathsf{tag}(x))$$

where $B(\mathsf{TInt}) = \mathsf{codomain}(h_{\mathsf{int}})$, $B(\mathsf{TStr}) = \mathsf{codomain}(h_{\mathsf{str}})$, *etc.*

The dependent type is inferred automatically from the `isinstance` *guards and the section extraction.*

Remark 16.11 (Dependent Pattern Matching in the Normalizer)**.** The normalizer in `denotation.py` itself uses dependent pattern matching at *two levels*:

1. **On the input AST** (Python's `ast` module): each `ast.BinOp`, `ast.Call`, etc. maps to an OTerm constructor. This is the compilation phase.
2. **On the OTerm family**: each normalization pass matches on OTerm constructors and transforms them. The arity index ensures that bound-variable operations (in OFix and OLam) correctly shift de Bruijn indices.

The coherence of these two levels of pattern matching is guaranteed by the ech complex: the presheaf sections extracted from the AST (level 1) are checked for global consistency via the coboundary map on the OTerm normal forms (level 2).

Chapter 17

The Computable Fragment of CTTT

A critical question: which parts of CTTT are *decidable* (algorithms always terminate with a correct answer) vs. *semi-decidable* (may not terminate) vs. *undecidable* (no algorithm exists)?

17.1 The Three Decidability Classes

Definition 17.1 (Decidability Classification)**.** A problem Π in the CTTT pipeline belongs to exactly one class:

1. **Decidable.** There exists an algorithm that always terminates and returns the correct Yes/No answer.
2. **Semi-decidable.** There exists an algorithm that terminates and returns Yes if the answer is positive, but may diverge (or return "Unknown") if the answer is negative.
3. **Undecidable.** No algorithm can solve Π for all inputs (by Rice's theorem or reduction from the halting problem).

17.1.1 What is Decidable: Canonical Forms and Structural Equality

The following operations are fully decidable because they operate on finite, syntactic data:

- **Duck type inference**: a single AST walk extracts the finite set of operations used on each parameter. The duck-type lattice $\mathcal{D}$ is finite, so the meet/join is computed in $O(|A|)$ time.
- **Site category construction**: given k parameters each with duck type sets of size $\leq d$, the site has at most d^k fibers. Enumerating fibers and overlaps is $O(d^{2k})$.
- **Canonical form matching**: comparing two `OTerm` values via their `canon()` strings is decidable—it reduces to string equality on the normal forms.
- H^1 **computation**: given the boolean fiber matrix, H^1 is computed by GF(2) Gaussian elimination in $O(n^3)$.

17.1.2 What is Semi-Decidable: Path Search

Proposition 17.2 (Path Search is Semi-Decidable)**.** *The path search problem—"given $[\![f]\!]$ and $[\![g]\!]$, does there exist a sequence of axiom applications connecting them?"—is semi-decidable. The BFS over the rewrite graph always terminates for each depth bound k, but the search space may be infinite (unboundedly many axiom applications), so the algorithm may not find a path even though one exists at a depth not yet explored.*

More precisely, the semi-decidable operations are:

- **Local equivalence with folds**: Z3's RecFunction definitions may cause the SMT solver to diverge. The fold RecFunction $\mathsf{fold}(\mathsf{op}, i, \mathit{stop}, \mathit{acc})$ recurses on i, and Z3 cannot always determine termination.
- **Local equivalence with axioms**: the 52 semantic axioms (Section **??**) introduce universal quantifiers. Z3's E-matching engine may loop when instantiating $\forall x.\ \mathsf{sorted}(\mathsf{sorted}(x)) = \mathsf{sorted}(x)$ on nested terms.
- **Bug finding per fiber**: satisfiability of the negated equivalence formula is in general undecidable for the full theory, though Z3 terminates on most practical instances.

17.1.3 What is Undecidable: General Equivalence

Theorem 17.3 (Undecidability of General Equivalence)**.** *The problem "given arbitrary Python programs f and g, are they extensionally equivalent?" is undecidable.*

Proof. By Rice's theorem. Extensional equivalence is a non-trivial semantic property of programs (it is not satisfied by all programs, nor vacuously false). Therefore no algorithm can decide it for all inputs.

Alternatively, reduce from the halting problem: given a Turing machine M and input w, let $f(x) = 1$ and $g(x) = [\text{run } M \text{ on } w; \text{return } 1]$. Then $f \equiv g$ iff M halts on w. □

CTTT does not attempt to solve the undecidable problem. Instead, it carves out a large *decidable fragment* and returns "Unknown" for programs outside it, ensuring soundness without sacrificing completeness on the fragment.

17.2 Decidability Map

Problem	Status	Justification
Duck type inference	Decidable	Finite AST walk, finite operation set
Site category construction	Decidable	Finite product of finite tag sets
Canonical form matching	Decidable	String equality on `canon()` output
Structural OTerm equality	Decidable	Recursive descent on finite tree
Local equivalence (QF)	Decidable	QF_DT + QF_LIA is decidable
Local equivalence (with folds)	Semi-decidable	RecFunctions may cause non-termination
Local equivalence (with axioms)	Semi-decidable	E-matching may loop
Path search (depth-bounded)	Decidable	Finite BFS at each depth
Path search (unbounded)	Semi-decidable	BFS terminates per level, infinite levels
H^1 computation	Decidable	GF(2) linear algebra, $O(n^3)$
Full pipeline (finite types)	Decidable	All components decidable
Full pipeline (general)	Semi-decidable	Z3 may timeout on RecFunctions
Equivalence (general)	Undecidable	Rice's theorem
Spec checking (general)	Undecidable	Reduces to equivalence
Abstract spec inference	Decidable	Finite abstract domains
Bug finding (per fiber)	Semi-decidable	SAT check may timeout

17.3 The Role of Z3

Z3 provides the decision procedure for the first-order fragment of CTTT. Its role is precisely delineated:

Definition 17.4 (The PyObj First-Order Theory)**.** The Z3 theory $\mathcal{T}_{\mathsf{PyObj}}$ is the first-order theory with:

- **Sort**: PyObj, defined as a Z3 algebraic datatype with seven constructors: $\mathsf{IntObj}(ival : \mathbb{Z})$, $\mathsf{BoolObj}(bval : \mathbb{B})$, $\mathsf{StrObj}(sval : \mathsf{String})$, NoneObj, $\mathsf{Pair}(fst, snd : \mathsf{PyObj})$, $\mathsf{Ref}(addr : \mathbb{Z})$, Bottom.
- **Tester predicates**: is_IntObj, is_BoolObj, ...—one per constructor. These implement the tag fibration.
- **RecFunctions**: binop, unop, truthy, fold, tag—each a recursively defined Z3 function.
- **Shared function symbols**: py_sorted, py_reversed, etc.—uninterpreted functions constrained by the 52 semantic axioms.

Proposition 17.5 (Decidability of the Quantifier-Free Fragment)**.** *The quantifier-free fragment of $\mathcal{T}_{\mathsf{PyObj}}$ (without RecFunction unfolding) is decidable. It falls within the combined theory QF_DT + QF_LIA + QF_S (quantifier-free datatypes, linear integer arithmetic, and strings), for which Z3 is a decision procedure.*

Proof. By the Nelson-Oppen combination theorem. QF_DT is decidable (finite case analysis on constructors), QF_LIA is decidable (Presburger arithmetic), and QF_S is decidable (word equations with length constraints). The theories share only equality, so their combination is decidable. □

Remark 17.6 (RecFunctions as Controlled Undecidability)**.** The fold RecFunction introduces controlled undecidability: Z3 unfolds the recursion up to a configurable depth bound. When the bound is reached without a proof, the solver returns "Unknown" rather than an incorrect answer. This is sound (never produces false equivalences) but incomplete (may miss valid equivalences that require deeper unfolding).

In the implementation (`theory.py`), the fold is defined as:

$$\mathsf{fold}(\mathsf{op}, i, stop, acc) := \begin{cases} acc & \text{if } i \geq stop \\ \mathsf{binop}(\mathsf{op}, \mathsf{IntObj}(i), \mathsf{fold}(\mathsf{op}, i{+}1, stop, acc)) & \text{otherwise} \end{cases}$$

This is the computational content of the Nat-eliminator from Chapter 16: fold is $\mathsf{rec}_{\mathbb{N}}$ applied to the PyObj sort.

17.4 The Decidable Core

The *decidable core* of CTTT consists of programs where:

1. All parameters have a single duck type (the spectral sequence degenerates—Theorem 13.7).
2. No recursive functions or unbounded loops (no RecFunctions appear in the compiled Z3 terms).
3. No universal quantifiers in axioms (no E-matching required).

For this core, the full pipeline (compilation, fiber decomposition, local checks, H^1 computation) is decidable with polynomial complexity.

Theorem 17.7 (Decidability of the Core)**.** *For programs in the decidable core, the equivalence/spec-checking/ bug-finding verdict is computed in time:*

$$O(|A_f| + |A_g| + |\mathcal{S}|^3)$$

where $|A_f|, |A_g|$ are the AST sizes and $|\mathcal{S}|$ is the number of sites (type fiber combinations).

Proof. 1. AST compilation to OTerm: $O(|A|)$ (single AST walk, each node maps to one OTerm constructor via the compilation function in `denotation.py`).

2. Duck type inference: $O(|A|)$ (single walk, collecting operations into the finite lattice $\mathcal{D}$).

3. Site construction: $O(|\mathcal{S}|^2)$ (enumerate products and overlaps of duck-type fibers).

4. Local checks: $O(|\mathcal{S}|)$ Z3 queries, each in QF_DT + QF_LIA (decidable by Proposition 17.5, polynomial with fixed formula size).

5. H^1 computation: $O(|\mathcal{S}|^3)$ (GF(2) Gaussian elimination on the coboundary matrix).

Total: dominated by the H^1 computation, which is cubic in the number of fibers. □

17.5 Extending the Decidable Fragment

Each dichotomy theory (Part V) extends the decidable fragment by adding axioms that convert semi-decidable checks to decidable ones:

- **D1 (Rec↔Iter)**: The Nat-eliminator extraction converts RecFunctions to canonical folds, making the local check decidable (syntactic equality of fold terms).
- **D2 (Nested↔Flat)**: Function inlining eliminates nested calls, producing ground terms.
- **D4 (Comp↔Loop)**: Comprehension hashing reduces to shared symbol equality (decidable).
- **D8 (Multi-Return↔Single)**: Section merging reduces to Z3 If-expression comparison (decidable).

Each extension enlarges the set of programs for which the pipeline is fully decidable, without compromising soundness for programs outside the fragment (they get "Unknown" rather than wrong answers).

Proposition 17.8 (Monotonic Decidability Growth)**.** *Let $\mathcal{C}_0$ be the decidable core and $\mathcal{C}_k$ the fragment after adding dichotomy theories $D_1, \ldots, D_k$. Then:*

$$\mathcal{C}_0 \subseteq \mathcal{C}_1 \subseteq \cdots \subseteq \mathcal{C}_k$$

and each inclusion is strict (each dichotomy theory brings at least one previously semi-decidable program class into the decidable fragment). Furthermore, the decidable core is never *reduced: adding axioms can only enlarge the fragment because each axiom is a sound equational law.*

Chapter 18

Automated Path Search

This is the algorithmic heart of CTTT. Given two Z3 terms $[\![f]\!]$ and $[\![g]\!]$ that are not syntactically equal, the *path search engine* automatically discovers a multi-step cubical path $[\![f]\!] \xrightarrow{p_1} h_1 \xrightarrow{p_2} h_2 \xrightarrow{p_3} \dots \xrightarrow{p_k} [\![g]\!]$ where each p_i is a single path constructor application.

18.1 The Path Search Problem

Definition 18.1 (Path Search)**.** Given Z3 terms $s, t : \mathsf{PyObj}$ (the source and target), the *path search problem* is:

$$\text{Find } p_1, \dots, p_k \in \mathsf{PathConstructors} \text{ such that } s \xrightarrow{p_1} \cdot \xrightarrow{p_2} \cdots \xrightarrow{p_k} t$$

or prove that no such sequence exists (by finding a counterexample — a concrete input where $s \neq t$).

The path constructors are the axioms from the HIT PyComp (Definition 4.2): commutativity, associativity, fold–unfold, β-reduction, etc.

18.2 The Path Graph

Definition 18.2 (Path Graph)**.** The *path graph* G_{path} has:

- **Vertices**: equivalence classes of Z3 terms modulo syntactic identity.
- **Edges**: for each path constructor p and each applicable subterm, an edge from the term to the rewritten term.

The path search is a graph search from s to t in G_{path}.

The path graph is infinite (Z3 terms form an infinite set), but the search is bounded because:

1. Each path constructor application either *simplifies* the term (reduces its size/complexity) or *normalizes* it (moves it toward a canonical form).
2. A confluent and terminating rewrite system has a unique normal form per equivalence class — if both s and t reduce to the same normal form, a path exists.

18.3 The Normalization Strategy

Rather than searching the full path graph, we normalize both terms and check if the normal forms coincide:

Definition 18.3 (Canonical Normal Form)**.** The *canonical normal form* of a Z3 term t is obtained by exhaustively applying the following rewrite rules (in priority order):

Phase 1: β-reduction (definitional equality).

1. Inline all nested function applications ($(\lambda x.\, B)(a) \to B[x := a]$).
2. Unfold all `__func__` references.
3. Collapse trivial `If` branches ($\mathsf{If}(\top, a, b) \to a$; $\mathsf{If}(\bot, a, b) \to b$).

Phase 2: Arithmetic normalization (ring axioms).

4. Commute operands to canonical order ($b + a \to a + b$ when $a <_{\mathsf{lex}} b$).
5. Associate right ($((a + b) + c) \to (a + (b + c))$).
6. Eliminate identities ($a + 0 \to a$; $a \times 1 \to a$).
7. Absorb zeros ($a \times 0 \to 0$).
8. Cancel double negation ($-(-a) \to a$).

Phase 3: Fold canonicalization (Nat-recursor).

9. Detect recursive descent patterns and replace with canonical $\mathsf{fold}(\mathsf{op}, \mathsf{start}, \mathsf{stop}, \mathsf{acc})$.
10. Detect accumulation loops and replace with canonical fold.
11. When both rec and iter produce fold terms with the same $(\mathsf{op}, \mathsf{start}, \mathsf{stop}, \mathsf{acc})$, they are syntactically identical.

Phase 4: Higher-order normalization (operadic).

12. Apply map–fold fusion ($\mathsf{reduce}(\oplus, e, \mathsf{map}(h, xs)) \to \mathsf{reduce}(\lambda a, x.\, a \oplus h(x), e, xs)$).
13. Apply comprehension canonicalization ($[e(x) \text{ for } x \text{ in } xs] \to \mathsf{comp}_{\mathsf{hash}(e)}(xs)$).
14. Apply idempotence ($\mathsf{sorted}(\mathsf{sorted}(x)) \to \mathsf{sorted}(x)$).
15. Apply involutions ($\mathsf{reversed}(\mathsf{reversed}(x)) \to x$).

Phase 5: Shared symbol unification (univalence).

16. Replace all builtin calls with shared function symbols ($\mathsf{sorted}_k(x) \to \mathsf{py_sorted}(x)$).
17. Replace all method calls with shared method symbols ($\mathsf{meth_append}_k(o, v) \to \mathsf{py_meth_append}(o, v)$).

Theorem 18.4 (Normalization Confluence)**.** *The rewrite system of Definition 18.3 is* locally confluent*: if $t \to t_1$ and $t \to t_2$ by different rules, there exists t' such that $t_1 \to^* t'$ and $t_2 \to^* t'$.*

Combined with termination (each rule decreases a complexity measure), the system is confluent *(by Newman's lemma): every term has a unique normal form.*

Proof sketch. The phases are ordered by priority, and rules within each phase are orthogonal (they apply to disjoint subterms). Critical pairs arise only between commutativity and associativity, which are resolved by the canonical ordering $<_{\mathsf{lex}}$. □

Corollary 18.5 (Path Existence from Normal Forms)**.** *A cubical path from $[\![f]\!]$ to $[\![g]\!]$ exists if and only if their canonical normal forms are identical:*

$$\mathsf{nf}([\![f]\!]) = \mathsf{nf}([\![g]\!]) \iff \exists p : \mathsf{Path}([\![f]\!], [\![g]\!])$$

The path is reconstructed by composing the rewrite steps: $[\![f]\!] \to^* \mathsf{nf}([\![f]\!]) = \mathsf{nf}([\![g]\!]) \leftarrow^* [\![g]\!]$.

18.4 The Path Search Algorithm

The implementation uses a three-strategy cascade, each strictly more expensive than the last. Given normalized OTerms nf_f and nf_g:

Strategy A: HIT structural decomposition (primary). Apply the elimination principle for PyComp: decompose both terms structurally, applying path constructors at each level. This is the congruence closure that handles the vast majority of benchmark pairs. (§18.8 below.)

Strategy B: D20 spec identification (Yoneda). If both terms can be identified as implementations of the same deterministic specification (factorial, fibonacci, sorted, ...), equate them by Yoneda: $\forall F.\ F([\![f]\!]) = F([\![g]\!]) \implies [\![f]\!] = [\![g]\!]$. (§18.11 below.)

Strategy C: Bidirectional BFS (fallback). Expand rewrite frontiers from both terms simultaneously, meeting in the middle. Bounded by depth d and frontier size M. (§18.12 below.)

Listing 18.1: Three-strategy path search (implementation)

```
def search_path(nf_f, nf_g, max_depth=4,
                max_frontier=200, param_duck_types=None):
    ctx = FiberCtx(param_duck_types or {})
    cf, cg = nf_f.canon(), nf_g.canon()

    # Strategy 0: reflexivity
    if cf == cg:
        return PathResult(found=True, path=[], reason='refl')

    # Strategy A: HIT structural decomposition
    hit_result = hit_path_equiv(nf_f, nf_g, ctx)
    if hit_result is True:
        return PathResult(found=True,
            path=[PathStep('HIT_structural', 'root', cf, cg)],
            reason='HIT path induction')

    # Strategy B: D20 spec identification (Yoneda)
    spec_f, spec_g = _identify_spec(nf_f), _identify_spec(nf_g)
    if (spec_f and spec_g and spec_f[0] == spec_g[0]
            and len(spec_f[1]) == len(spec_g[1])):
        if all(a.canon() == b.canon()
               for a, b in zip(spec_f[1], spec_g[1])):
            return PathResult(found=True,
                path=[PathStep('D20_spec_unify','root',cf,cg)],
                reason=f'same spec: {spec_f[0]}')

    # Strategy C: Bidirectional BFS (fallback)
    return _bidirectional_bfs(nf_f, nf_g, ctx,
                              max_depth, max_frontier)
```

18.5 Path Search on Each Fiber

The path search is performed *per fiber* (per type assignment), not globally. This is the cubical–cohomological interaction:

Listing 18.2: Fiber-wise path search

```
def fiber_wise_path_search(secs_f, secs_g, sites, theory):
```

```python
    """Search for paths on each fiber independently.

    Returns per-fiber path results and the global H^1.
    """
    local_paths = {}

    for combo, site_id in sites.items():
        # Restrict terms to this fiber
        fiber_f = restrict_to_fiber(secs_f, combo, theory)
        fiber_g = restrict_to_fiber(secs_g, combo, theory)

        # Search for path on this fiber
        for sf in fiber_f:
            for sg in fiber_g:
                if not guards_overlap(sf.guard, sg.guard):
                    continue
                result = search_path(sf.term, sg.term, theory)
                if result.found == False:
                    return GlobalResult(equiv=False,
                        fiber=combo, cx=result.counterexample)
                elif result.found == True:
                    local_paths[site_id] = result.path

    # All fibers have paths: check gluing via Cech H^1
    h1 = compute_cech_h1(local_paths, sites)
    if h1 == 0:
        return GlobalResult(equiv=True, paths=local_paths)
    else:
        return GlobalResult(equiv=False, obstruction=h1)
```

18.6 Path Composition and the Proof Certificate

When the path search succeeds, it produces a *proof certificate*: a sequence of named path constructor applications that transforms $[\![f]\!]$ into $[\![g]\!]$.

Definition 18.6 (Proof Certificate)**.** A *proof certificate* for $f \equiv g$ is a list of triples:

$$[(r_1, \mathsf{pos}_1, \mathsf{ax}_1),\ (r_2, \mathsf{pos}_2, \mathsf{ax}_2),\ \ldots,\ (r_k, \mathsf{pos}_k, \mathsf{ax}_k)]$$

where r_i is the term at step i, pos_i is the subterm position, and ax_i is the axiom applied. The certificate satisfies:

$$r_0 = [\![f]\!] \tag{18.1}$$
$$r_{i+1} = r_i[\mathsf{pos}_i := \mathsf{ax}_i(r_i|_{\mathsf{pos}_i})] \tag{18.2}$$
$$r_k = [\![g]\!] \tag{18.3}$$

Example 18.7 (Certificate for Factorial Equivalence)**.**

$$[\![f]\!] = \mathsf{rec_f}(p_0) \qquad \text{(recursive definition)} \tag{18.4}$$
$$\xrightarrow{\mathsf{D1},\,\mathsf{root}} \mathsf{fold}(\mathsf{MUL}, 1, \mathsf{ival}(p_0) + 1, \mathsf{IntObj}(1)) \qquad \text{(Nat-eliminator extraction)} \tag{18.5}$$
$$= [\![g]\!] \qquad \text{(same as iterative fold)} \tag{18.6}$$

Certificate: $[(\mathsf{rec_f}(p_0), \epsilon, \mathsf{D1})]$. One step, at the root, using the Nat-recursor path.

Example 18.8 (Multi-Step Certificate)**.** Programs that compute $\mathsf{sum}(\mathsf{reversed}(\mathsf{sorted}(x)))$ vs. $\mathsf{sum}(x)$:

$$\mathsf{py_sum}(\mathsf{py_reversed}(\mathsf{py_sorted}(p_0))) \tag{18.7}$$

$$\xrightarrow{\mathsf{reversed_invol},\,[1,1]} \mathsf{py_sum}(\mathsf{py_sorted}(p_0)) \qquad \text{(sorted absorbs reversed)} \tag{18.8}$$

$$\xrightarrow{\mathsf{sum_perm},\,[1]} \mathsf{py_sum}(p_0) \qquad \text{(sum is perm-invariant)} \tag{18.9}$$

Certificate: $[(\ldots, [1,1], \mathsf{sorted_absorbs_reversed}),\ (\ldots, [1], \mathsf{sum_perm_invariant})]$.

18.7 Path Search Strategies

The basic normalization strategy (Definition 18.3) is "normalize both sides and compare." But more sophisticated strategies exist:

18.7.1 Bidirectional Search

Search from both $[\![f]\!]$ and $[\![g]\!]$ simultaneously, meeting in the middle:

Listing 18.3: Bidirectional path search

```python
def bidirectional_search(s, t, axioms, max_depth=5):
    forward = {hash(s): (s, [])}     # term -> (term, path)
    backward = {hash(t): (t, [])}

    for depth in range(max_depth):
        # Expand forward frontier
        new_forward = {}
        for h, (term, path) in forward.items():
            for ax in axioms:
                for pos in applicable_positions(term, ax):
                    new_term = apply_axiom(term, ax, pos)
                    nh = hash(new_term)
                    if nh in backward:
                        # FOUND: path from s to t
                        _, bpath = backward[nh]
                        return path + [(term, pos, ax)] + \
                               reversed(bpath)
                    new_forward[nh] = (new_term,
                        path + [(term, pos, ax)])
        forward.update(new_forward)

        # Similarly expand backward frontier
        # ...

    return None  # no path within depth bound
```

18.7.2 Fiber-Guided Search

Use the Čech decomposition to *guide* the search:

1. Identify which fibers are "easy" (terms already match or nearly match after normalization).
2. On hard fibers, use the easy fibers' solutions as hints: if the path on the Int fiber used axiom D1, try D1 on the Str fiber first.
3. This is *transfer along the fibration*: a successful path on one fiber is transported to adjacent fibers.

18.7.3 Axiom Relevance Filtering

Not all axioms are relevant for a given pair. Filter by:

1. **Symbol filtering**: only use axioms mentioning symbols that appear in $[\![f]\!]$ or $[\![g]\!]$.
2. **Sort filtering**: only use axioms whose type signature matches the current fiber.
3. **Depth filtering**: prioritize axioms that reduce term depth (simplifying rules before complexifying ones).

18.8 The HIT Structural Prover

The core of the path search is *not* BFS. It is a recursive structural decomposition that implements the *elimination principle* for the HIT PyComp.

Definition 18.9 (HIT Path Induction)**.** To prove $a =_{\mathsf{PyComp}} b$ by *path induction*, proceed by cases on the structure of (a, b):

1. **Reflexivity.** If $\mathsf{canon}(a) = \mathsf{canon}(b)$, return refl.
2. **Normalization.** Compute $\hat{a} := \mathsf{normalize}(a)$, $\hat{b} := \mathsf{normalize}(b)$. If $\mathsf{canon}(\hat{a}) = \mathsf{canon}(\hat{b})$, return the composite path through the normal form.
3. **Congruence.** If a and b have the *same outermost constructor*, recurse into their children (Table 18.1).
4. **Cross-type path constructors.** If a and b have *different* outermost constructors, apply one of the dichotomy axioms D1–D24 to bridge the gap (§18.8.2).
5. **One-step rewrite fallback.** Try applying each root axiom to a (or b) and recurse on the result.

The recursion is bounded by a depth limit (default 20) and a memoization table $\mathsf{memo} : (\mathsf{canon} \times \mathsf{canon}) \to \{\top, \bot, ?\}$ that prevents revisiting the same pair. Setting $\mathsf{memo}[a, b] := ?$ before recursing detects cycles in recursive OTerm structures (e.g., OFix).

Theorem 18.10 (HIT Prover Soundness)**.** *If* $\mathsf{hit_path_equiv}(a, b) = \top$*, then* $a =_{\mathsf{PyComp}} b$*. The proof is by structural induction on the derivation:*

- *Steps 1–2 produce identity or normalization paths (definitional equality).*
- *Step 3 applies congruence lemmas (each OTerm constructor respects =).*
- *Step 4 applies named path constructors (each verified as a theorem of the HIT, see Part III).*
- *Step 5 composes steps via Kan composition (Chapter 5).*

This is sound because PyComp *is 0-truncated (Definition 4.2): any two parallel paths between the same endpoints are equal, so the proof strategy is irrelevant — only the existence of a path matters.*

Table 18.1: Congruence rules for each OTerm constructor.

Constructor	Children	Congruence Rule
$\mathsf{OOp}(f, \vec{a})$	$a_1, \ldots, a_k$	Same f, same arity; recurse into each a_i. Also: if f is commutative on the current fiber, try matching $a_i \leftrightarrow b_{k+1-i}$ (§18.10).
$\mathsf{OCase}(g, t, e)$	g, t, e	Matching guards: recurse into (t, t') and (e, e'). Complementary guards $g = \neg g'$: match (t, e') and (e, t'). Nested guards: flatten via D8 section merge.
$\mathsf{OFold}(\mathsf{op}, i, c)$	i, c	Same op: recurse into init i and collection c. Different hash-based op: attempt fold-op identification.
$\mathsf{OFix}(h, B)$	B	Same key h: recurse into body B. Different keys: α-rename $h' \to h$ in B', then recurse.
$\mathsf{OSeq}(\vec{e})$	$e_1, \ldots, e_k$	Same length: recurse pointwise.
$\mathsf{OLam}(\vec{x}, B)$	B	Same arity: α-rename parameters of b to match a, then recurse into bodies.
$\mathsf{OMap}(f, c, p)$	f, c, p	Recurse into transform, collection, and (optionally) filter predicate.
$\mathsf{OCatch}(B, d)$	B, d	Recurse into body and default.
$\mathsf{ODict}(\vec{(k, v)})$	pairs	Same length: recurse pointwise into key–value pairs.
$\mathsf{OQuotient}(t, R)$	t	Same equivalence class R: recurse into inner term.

18.8.1 Congruence Rules

Congruence rules are the workhorses of the HIT prover. Each OTerm constructor C admits a congruence lemma:

$$\frac{a_1 = b_1 \quad \ldots \quad a_k = b_k}{C(a_1, \ldots, a_k) = C(b_1, \ldots, b_k)} \quad (\mathsf{cong}_C)$$

Remark 18.11 (OCase Congruence is the Workhorse)**.** In the benchmark suite, OCase congruence handles the majority of non-trivial equivalences. Programs that differ only in guard ordering, branch nesting, or negation style (e.g. `if x < 0: ... else: ...` vs. `if x >= 0: ... else: ...`) are resolved entirely by OCase congruence combined with comparison flips and De Morgan normalization, without invoking BFS.

18.8.2 Cross-Type Path Constructors

When a and b have *different* outermost constructors, no congruence rule applies. The HIT prover invokes *cross-type path constructors* — the dichotomy axioms:

Definition 18.12 (Cross-Type Paths)**.** The cross-type path constructors implemented in the HIT prover are:

1. **D1:** $\mathsf{OFix} \leftrightarrow \mathsf{OFold}$ (recursive $\leftrightarrow$ iterative).
   If $\mathsf{fix}[h](B)$ and $\mathsf{fold}[\mathsf{op}](i, c)$ share the same fix-point hash ($h = \mathsf{op}$), or if both are identified as the same specification via D20, they are equated.

2. **D22:** $\mathsf{OCatch} \leftrightarrow \mathsf{OCase}$ (exception handling $\leftrightarrow$ conditional).
   $\mathsf{catch}(B, d) = \mathsf{case}(g, B, d)$ when the exception guard g is decidable. The prover matches body-to-then-branch and default-to-else-branch.

3. **De Morgan duality.**
   $\neg(a \wedge b) = (\neg a) \vee (\neg b)$ and $\neg(a \vee b) = (\neg a) \wedge (\neg b)$, applied recursively into the operand lists.

4. **Guard subsumption.**
   $\mathsf{case}(G, A, X) = \mathsf{case}(G, A, \mathsf{case}(G_{\mathsf{sub}}, B, X'))$ when G_{sub} is a disjunct of G: in the else-branch (G is false), G_{sub} is also false, so the inner case reduces to X'. This handles the `dict.get`/`not in` relationship: $\mathsf{notin}(k, d) \in G$ implies $\mathsf{is}(.\mathsf{get}(d, k), \mathsf{None})$ is false in the else-branch.

5. **Comparison normalization.**
   $\mathsf{lt}(a, b) = \mathsf{gt}(b, a)$, $\mathsf{lte}(a, b) = \mathsf{gte}(b, a)$, and their negations: $\neg\mathsf{lt}(a, b) = \mathsf{gte}(a, b)$, etc.

6. **Quotient absorption.**
   $Q_{\mathsf{perm}}(\mathsf{sorted}(x)) = \mathsf{sorted}(x)$: sorting selects a canonical representative of the permutation equivalence class, so the quotient wrapper is redundant.

7. **Fold–De Morgan duality.**
   $\mathsf{fold}[\wedge](\top, \mathsf{map}(f, x)) = \neg\mathsf{fold}[\vee](\bot, \mathsf{map}(\neg \circ f, x))$, connecting all and any predicates.

18.9 The Axiom Registry

The BFS fallback and the one-step rewrite fallback in the HIT prover both draw from a fixed registry of 16 *root axiom functions*. Each axiom α has signature

$$\alpha : \mathsf{OTerm} \times \mathsf{FiberCtx} \longrightarrow \mathsf{List}(\mathsf{OTerm} \times \mathsf{String})$$

returning all possible one-step rewrites of the input term.
Axioms are applied at the root and at all sub-term positions via *congruence lifting*: for each sub-term of the input at position π, apply axiom α to the sub-term and reconstruct the full term. The position is annotated in the axiom name as α`@`π (e.g. `ALG_commute@arg0`).

18.10 Fiber Context and Fiber-Aware Commutativity

Definition 18.13 (Fiber Context)**.** A *fiber context* Γ_{fib} is a mapping from parameter names to duck type tags:

$$\Gamma_{\mathsf{fib}} : \{p_0, p_1, \ldots\} \longrightarrow \{\mathsf{int}, \mathsf{float}, \mathsf{str}, \mathsf{list}, \mathsf{any}, \ldots\}$$

This is the data that makes path search *sheaf-aware*: axioms whose validity depends on the fiber are checked against Γ_{fib} before application.

Definition 18.14 (Fiber-Aware Commutativity)**.** An operation $\oplus$ is *fiber-commutative* on context Γ_{fib} and term t if:

- $\oplus \in \{\mathsf{mul}, \mathsf{and}, \mathsf{or}, \mathsf{min}, \mathsf{max}, \mathsf{eq}, \mathsf{bitand}, \mathsf{bitor}, \mathsf{bitxor}\}$ (always commutative), **or**
- $\oplus = \mathsf{add}$ and all parameters in t are on numeric fibers: $\forall p \in \mathsf{params}(t).\ \Gamma_{\mathsf{fib}}(p) \in \{\mathsf{int}, \mathsf{float}, \mathsf{number}\}$.

The restriction on add is essential: string concatenation (add on the str fiber) is *not* commutative. This is why path search must be fiber-aware — commuting $p_0 + p_1$ is valid on the int fiber but unsound on the str fiber.

Example 18.15 (Fiber-Aware Path Failure)**.** Consider `f(a,b) = a + b` vs. `g(a,b) = b + a`.

Table 18.2: The 16 root axiom functions in `_ROOT_AXIOMS`.

#	Axiom function	What it does
1	`d1_fold_unfold`	OFix with accumulation pattern $\leftrightarrow$ OFold
2	`d2_beta`	β-reduction: $(\lambda x.\, B)(a) \to B[x := a]$
3	`d4_comp_fusion`	Map fusion: $\mathsf{map}(f, \mathsf{map}(g, c)) \to \mathsf{map}(f \circ g, c)$; fold–map fusion
4	`d5_fold_universal`	Canonicalize hash-named fold operations
5	`d8_section_merge`	Flatten nested OCase with complementary guards
6	`d13_histogram`	$\mathsf{Counter}(x) \leftrightarrow \mathsf{fold}[\mathsf{count}](\mathsf{defaultdict}(0), x)$
7	`d16_memo_dp`	Canonicalize recurrence structure (α-normalize fix body)
8	`d17_assoc`	Tree fold $\to$ linear fold when op is associative
9	`d19_sort_scan`	$\mathsf{fold}(\mathsf{op}, \mathsf{sorted}(x)) \to \mathsf{fold}(\mathsf{op}, x)$ when op is commutative
10	`d20_spec_unify`	Replace term with $\mathsf{OAbstract}(\mathsf{spec}, \mathsf{inputs})$ (Yoneda)
11	`d22_effect_elim`	$\mathsf{OCatch}(B, d) \leftrightarrow \mathsf{OCase}(g, B, d)$
12	`d24_eta`	η-contraction: $\lambda x.\, f(x) \to f$
13	`algebra`	Commutativity, associativity, identity elements, involution, comparison flips
14	`quotient`	$Q_{\mathsf{perm}}(\mathsf{sorted}(x)) \to \mathsf{sorted}(x)$; $\mathsf{sorted}(Q_{\mathsf{perm}}(x)) \to \mathsf{sorted}(x)$; $\mathsf{set}(\mathsf{sorted}(x)) \to \mathsf{set}(x)$
15	`fold`	$\mathsf{fold}[\mathsf{add}](0, x) \leftrightarrow \mathsf{sum}(x)$; $\mathsf{fold}[\mathsf{mul}](1, x) \leftrightarrow \mathsf{prod}(x)$; range normalization
16	`case`	Constant-guard elimination; identical-branch collapse; guard negation flip; guard subsumption

- On the int fiber ($\Gamma_{\mathsf{fib}} = \{p_0 \mapsto \mathsf{int}, p_1 \mapsto \mathsf{int}\}$): the ALG_commute axiom fires, producing a one-step path. Result: $f \equiv g$.
- On the str fiber ($\Gamma_{\mathsf{fib}} = \{p_0 \mapsto \mathsf{str}, p_1 \mapsto \mathsf{str}\}$): commutativity is blocked. The path search fails and Z3 finds a counterexample: $p_0 =$ `"ab"`, $p_1 =$ `"cd"` where $f =$ `"abcd"` $\neq$ `"cdab"` $= g$. Result: $H^1 \neq 0$, programs are *not* equivalent.

18.11 The Spec Identification Engine (Yoneda Embedding)

Definition 18.16 (Spec Identification)**.** The *spec identification* function identify maps an OTerm to an optional abstract specification:

$$\mathsf{identify} : \mathsf{OTerm} \longrightarrow \mathsf{Maybe}(\mathsf{SpecName} \times \mathsf{Inputs})$$

This is the computational Yoneda embedding: a program is uniquely determined by its behavior under all observations, so two programs with the same deterministic specification are equal.

The recognized specifications are:

1. **factorial**: $\mathsf{fold}[\mathsf{mul}](1, \mathsf{range}(1, n{+}1))$ or fix with $n \cdot f(n{-}1)$.
2. **range_sum**: $\mathsf{fold}[\mathsf{add}](0, \mathsf{range}(\ldots))$.
3. **fibonacci_like**: fix whose body contains add and sub (the characteristic $f(n{-}1){+}f(n{-}2)$ pattern).

4. **sorted**: $\mathsf{sorted}(x)$.

5. **binomial**: $\mathsf{math.comb}(n, k)$ or the multiplicative formula via fold.

Theorem 18.17 (Spec Identification Soundness)**.** *If* $\mathsf{identify}(a) = \mathsf{identify}(b) = (\sigma, \vec{v})$ *with* σ *a deterministic specification, then* $a =_{\mathsf{PyComp}} b$.

This follows from the Yoneda lemma: a natural transformation $\mathsf{Hom}(-, a) \Rightarrow \mathsf{Hom}(-, b)$ *that is an isomorphism on every representable witnesses* $a \cong b$*. The deterministic specification* σ *provides exactly such an isomorphism: any context that observes* a *can equivalently observe* b *(both compute* $\sigma(\vec{v})$*).*

18.12 The Bidirectional BFS Algorithm

When the HIT structural prover and spec identification both fail, the path search falls back to bidirectional breadth-first search.

Definition 18.18 (Bidirectional BFS on the Path Graph)**.** The BFS maintains two *frontiers*:

$$F^0_{\rightarrow} := \{\mathsf{nf}_f\}, \qquad F^0_{\leftarrow} := \{\mathsf{nf}_g\} \tag{18.10}$$

At each depth d, expand both frontiers:

$$F^{d+1}_{\rightarrow} := \bigcup_{t \in F^d_{\rightarrow}} \{t' \mid (t', \alpha) \in \mathsf{all_rewrites}(t, \Gamma_{\mathsf{fib}})\} \setminus F^{\leq d}_{\rightarrow} \tag{18.11}$$

$$F^{d+1}_{\leftarrow} := (\text{symmetric}) \tag{18.12}$$

If $F^{d+1}_{\rightarrow} \cap F^{\leq d}_{\leftarrow} \neq \emptyset$ or $F^{d+1}_{\leftarrow} \cap F^{\leq d+1}_{\rightarrow} \neq \emptyset$, a meeting point m has been found and the path is:

$$\mathsf{nf}_f \xrightarrow{\vec{\alpha}_f} m \xleftarrow{\vec{\alpha}_g} \mathsf{nf}_g$$

The backward segment is reversed to produce a forward path.

Definition 18.19 (Frontier Pruning)**.** When $|F| > M$ (the maximum frontier size, default $M = 200$), prune F to M terms by a scoring function that prioritizes:

1. Shorter path length (fewer rewrite steps).

2. Higher structural similarity to the target (longest common prefix of the canonical string with the target).

This is a beam-search heuristic that trades completeness for tractability.

Theorem 18.20 (BFS Complexity)**.** *The bidirectional BFS has worst-case complexity*

$$O(A^d \cdot M)$$

where $A = |\texttt{_ROOT_AXIOMS}| = 16$ *is the number of axiom functions,* d *is the depth bound (default* $d = 4$*), and* M *is the maximum frontier size (default* $M = 200$*). With congruence lifting into* k *sub-term positions, the effective branching factor is* $A \cdot k$*, giving* $O((Ak)^d \cdot M)$*.*

For the default parameters: $16^4 \cdot 200 = 13{,}107{,}200$ *term comparisons in the worst case. In practice, the HIT structural prover resolves most pairs at Strategy A before BFS is reached, so BFS is rarely invoked on more than a few dozen frontier nodes.*

Remark 18.21 (Why BFS Outperforms DFS)**.** BFS is preferred over DFS for two reasons:

1. **Shortest path**: BFS finds the shortest rewrite sequence, producing minimal proof certificates.

2. **Bidirectionality**: the meeting-in-the-middle strategy reduces the effective depth from d to $d/2$, squaring the speedup: $O(A^{d/2})$ per direction vs. $O(A^d)$ unidirectional.

18.12.1 Benchmark Performance by Axiom

The following table shows which axiom resolves each benchmark pair from the equivalence suite. "HIT" indicates resolution by the structural prover (Strategy A) without BFS; the specific congruence or cross-type rule is noted.

Benchmark pair	Resolving axiom	Strategy
`factorial_iter_vs_rec`	D1 ($\mathsf{OFix} \leftrightarrow \mathsf{OFold}$)	HIT cross-type
`sum_loop_vs_builtin`	`FOLD_sum` ($\mathsf{fold}[\mathsf{add}](0, x) = \mathsf{sum}(x)$)	HIT + axiom
`fibonacci_iter_vs_rec`	D20 spec unify (`fibonacci_like`)	Spec (B)
`abs_manual_vs_builtin`	OCase congruence + comparison flip	HIT congruence
`sorted_reversed_equiv`	`QUOT_sorted_absorb` + `sum_perm_inv`	HIT + axiom
`max_loop_vs_builtin`	D1 + OFold congruence	HIT cross-type
`counter_vs_manual`	D13 ($\mathsf{Counter} \leftrightarrow \mathsf{fold}[\mathsf{count}]$)	HIT + axiom
`map_fusion`	D4 ($\mathsf{map}(f, \mathsf{map}(g, c)) \to \mathsf{map}(f \circ g, c)$)	BFS (C)
`demorgan_guards`	De Morgan + OCase congruence	HIT cross-type
`dict_get_vs_in`	Guard subsumption	HIT congruence

Of the 50 equivalent pairs in the benchmark, approximately 38 are resolved by the HIT structural prover (Strategy A), 7 by spec identification (Strategy B), and 5 require BFS (Strategy C). No pair in the suite requires depth > 3.

18.13 Sheaf Descent for Tractability of Undecidable Problems

Program equivalence is undecidable in general (Rice's theorem). Specification checking and bug finding reduce to it, so they are also undecidable. Yet we claim CTTT makes these problems *tractable* for a large class of real programs. This section explains *why* — the mathematical mechanism by which sheaf descent converts undecidable global problems into decidable local ones.

18.13.1 The Undecidability Barrier

The global equivalence query is:

$$\forall x : \mathsf{PyObj}.\ [\![f]\!](x) = [\![g]\!](x)$$

This is a universal quantification over an infinite domain (PyObj has infinitely many inhabitants via $\mathsf{IntObj}(n)$ for all $n \in \mathbb{Z}$). No algorithm can decide this in general.

But the undecidability proof (Rice's theorem) requires adversarial programs — programs specifically constructed to defeat any analysis. Real programs are not adversarial. They have *structure* that can be exploited.

18.13.2 How Descent Exploits Structure

Sheaf descent exploits two kinds of structure:

1. Type structure (the fibration). Real Python programs dispatch on types — they use `isinstance`, duck typing, different code paths for different input types. This means the universal quantification decomposes into a *finite* disjunction:

$$\forall x.\ [\![f]\!](x) = [\![g]\!](x) \iff \bigwedge_{\tau \in \mathsf{TypeTag}} \forall x.\ \mathsf{tag}(x) = \tau \implies [\![f]\!](x) = [\![g]\!](x)$$

Each conjunct is a *fiber-restricted* query. The finite conjunction is decidable if each conjunct is.

2. Algebraic structure (the path constructors). Within each fiber, the Z3 terms have algebraic structure: commutativity, associativity, fold patterns. The path constructors (equational axioms) transform this structure, often collapsing complex terms to simple canonical forms.

The canonical form is unique (by confluence, Theorem 18.4), and comparing canonical forms is decidable (syntactic equality check).

18.13.3 The Descent Decomposition Theorem

Theorem 18.22 (Descent Makes Equivalence Tractable). *Let $\mathcal{U} = \{U_\tau\}_{\tau \in T}$ be the covering of* PyObj *by type fibers, where T is a finite set of type tags. Then:*

1. *The global equivalence query $\forall x. [\![f]\!](x) = [\![g]\!](x)$ reduces to $|T|$ local queries plus one H^1 computation.*
2. *Each local query is in the theory QF_DT + QF_LIA + QF_S (quantifier-free datatypes + linear integer arithmetic + strings), which is* decidable.
3. *The H^1 computation is decidable ($O(|T|^3)$ via GF(2) rank).*
4. *Therefore: the full equivalence check is decidable for programs whose Z3 denotations fall in the quantifier-free fragment (no RecFunctions, no universally quantified axioms).*

Proof. (1) By the descent theorem (7.10), the global query decomposes into local queries on each fiber plus the gluing check ($H^1 = 0$).

(2) On each fiber U_τ, the type is fixed (e.g., $\mathsf{tag}(x) = \mathsf{TInt}$ forces $x = \mathsf{IntObj}(n)$ for some n). The Z3 terms reduce to expressions over IntSort (decidable QF_LIA), BoolSort (propositional, decidable), StringSort (decidable QF_S), or PyObj constructors (decidable QF_DT). The combined theory QF_DT + QF_LIA + QF_S is decidable by Nelson–Oppen combination.

(3) GF(2) linear algebra is decidable, $O(n^3)$.

(4) Combining (1)–(3): decidable + decidable + decidable = decidable. □

Remark 18.23 (Beyond the Quantifier-Free Fragment). When programs use RecFunctions (from loops/recursion), the local queries may exit the QF fragment. The Nat-eliminator extraction (Chapter 26.9.3) recovers decidability for many such programs by converting RecFunctions to canonical folds, which are ground terms (no quantifiers).

When equational axioms involve universal quantifiers ($\forall x. \mathsf{sorted}(\mathsf{sorted}(x)) = \mathsf{sorted}(x)$), the local queries are semi-decidable (Z3 may timeout). But the axioms are applied *as rewrite rules* during normalization, not as quantified constraints — so they terminate (the rewrite system is terminating by Theorem 18.4).

18.13.4 Complexity Analysis

Theorem 18.24 (Complexity of Descent-Based Equivalence). *For programs with n parameters, each having at most k possible type tags, the descent-based equivalence check has complexity:*

$$O(k^n \cdot C_{\mathsf{local}} + k^{3n})$$

where C_{local} is the cost of a single local Z3 query.

For the common case of single-typed parameters ($k = 1$):

$$O(C_{\mathsf{local}} + 1) = O(C_{\mathsf{local}})$$

— a single Z3 query, no combinatorial blowup.

For the worst case ($k = 7$ types, n parameters): $k^n = 7^n$ fibers — exponential in the number of parameters. But duck type inference typically reduces k to 1–2 per parameter, so the practical complexity is $O(2^n \cdot C_{\mathsf{local}})$.

18.13.5 Comparison with Prior Approaches

Approach	Global query	Decidable?	Practical?
Direct Z3	$\forall x.\, f(x) = g(x)$	Semi-decidable	Slow/timeout
Testing	Sample $x_1, \ldots, x_N$	Incomplete	Fast but unsound
Abs. interp.	$\alpha(f) = \alpha(g)$	Decidable	Fast but imprecise
CTTT descent	$\bigwedge_\tau (\forall x_\tau.\, f = g) \wedge H^1 = 0$	**Decidable (QF)**	**Fast & sound**

CTTT descent is the only approach that is simultaneously decidable (for the QF fragment), sound (no false positives/negatives within the modeled domain), and practical (scales with duck-type narrowing).

18.13.6 The Refinement Principle

When descent alone is insufficient (the local Z3 query times out or returns "unknown"), we can *refine* the covering:

Definition 18.25 (Covering Refinement)**.** A refinement $\mathcal{U}' = \{V_j\}$ of covering $\mathcal{U} = \{U_i\}$ is a covering such that each $V_j \subseteq U_i$ for some i. Refinement makes fibers *smaller* (more constrained), which makes local queries *easier*.

Theorem 18.26 (Refinement Soundness)**.** *If the equivalence check succeeds on a refinement $\mathcal{U}'$ ($H^1(\mathcal{U}', \mathsf{Iso}) = 0$), it also succeeds on the original covering ($H^1(\mathcal{U}, \mathsf{Iso}) = 0$).*

The converse is NOT true: a refinement may introduce new overlaps and new transition checks, which could fail even if the original covering succeeds.

Proof. The refinement morphism $\mathcal{U}' \to \mathcal{U}$ induces a morphism of Čech complexes $C^\bullet(\mathcal{U}) \to C^\bullet(\mathcal{U}')$. By functoriality of cohomology, $H^1(\mathcal{U}) = 0$ implies $H^1(\mathcal{U}') = 0$ (the image of a trivial class is trivial). □

Refinement strategies:

1. **Value-range refinement**: split the Int fiber into $\{n \geq 0\}$ and $\{n < 0\}$ (positive and negative integers). This helps with programs that have different behavior for positive vs. negative inputs.

2. **Size refinement**: split the Pair fiber into $\{|xs| = 0\}$, $\{|xs| = 1\}$, $\{|xs| \geq 2\}$ (empty, singleton, longer lists). This helps with programs that have base cases for short inputs.

3. **Predicate refinement**: split a fiber by a predicate derived from the program's branch conditions. E.g., if f branches on $n \leq 1$, split the Int fiber at $n = 1$.

Theorem 18.27 (Predicate Refinement from Branch Conditions)**.** *Let $\mathsf{guards}_f = \{g_1, \ldots, g_m\}$ and $\mathsf{guards}_g = \{h_1, \ldots, h_p\}$ be the branch conditions of programs f and g respectively. The* branch-condition refinement *is the covering:*

$$\mathcal{U}_{\mathsf{branch}} = \{U \cap R \mid U \in \mathcal{U},\ R \in \mathsf{atoms}(\{g_i\} \cup \{h_j\})\}$$

where atoms *are the minimal non-empty regions of the partition induced by the guards.*

On each atom, both *programs follow a single code path (no branching), so their Z3 terms are ground expressions (no* If*), which are decidable.*

Proof. Within an atom R, all guard conditions have fixed truth values. Therefore $\mathsf{If}(\mathsf{guard}, t, f)$ simplifies to t or f (no conditional). The resulting Z3 terms are quantifier-free ground expressions over the base theories, which are decidable. □

Corollary 18.28 (Branch-Refined Descent is Decidable)**.** *The branch-condition refinement produces decidable local queries for* any *program (not just QF programs). The only remaining source of undecidability is the RecFunction bodies (loops/recursion).*

This is a powerful result: by refining the covering using the programs' own branch conditions, we eliminate branching complexity from the local queries, leaving only loop/recursion complexity.

18.13.7 Iterated Descent: The Descent Tower

For programs with nested structure (loops inside branches inside loops), we can apply descent *iteratively*:

Definition 18.29 (Descent Tower)**.** A *descent tower* is a sequence of coverings:

$$\mathcal{U}_0 \leftarrow \mathcal{U}_1 \leftarrow \mathcal{U}_2 \leftarrow \ldots \leftarrow \mathcal{U}_k$$

where each $\mathcal{U}_{i+1}$ is a refinement of $\mathcal{U}_i$.

Level 0: type fibers (Int, Str, List, ...). Level 1: type fibers × branch conditions. Level 2: type fibers × branch conditions × loop bounds. Level 3: type fibers × branch conditions × loop bounds × recursion depth.

At each level, the local queries become simpler (more constrained, fewer degrees of freedom). Eventually, the local queries are ground and decidable.

Theorem 18.30 (Descent Tower Convergence)**.** *For programs with finitely many branches and bounded-depth recursion, the descent tower converges at a finite level k: all local queries at level k are ground and decidable.*

Proof. Each level refines by a finite partition (finitely many branches, finitely many loop bound values). The atoms at level k correspond to complete execution traces of bounded depth. On each trace, the Z3 term is ground (no If, no RecFunction unfolding needed beyond the bound). □

Remark 18.31 (Unbounded Recursion)**.** For programs with unbounded recursion, the descent tower does NOT converge. This is the residual source of undecidability in CTTT. The Nat-eliminator extraction (Chapter 26.9.3) handles many unbounded cases by recognizing them as folds, but general recursion remains semi-decidable.

18.13.8 The Effective Descent Condition

Not every covering admits effective descent. We characterize when descent is most useful:

Definition 18.32 (Effective Descent)**.** A covering $\mathcal{U}$ admits *effective descent* for the equivalence check $f \equiv g$ if:

1. Each local query $\mathsf{Iso}(U_i)$ is decidable (terminates with a definite answer).
2. The number of overlaps is polynomial in $|\mathcal{U}|$ (the Čech complex is sparse).
3. The transition checks φ_{ij} are all trivial (both local paths agree automatically on overlaps).

Theorem 18.33 (When Descent is Effective)**.** *Descent is effective when:*

1. ***Duck type inference is sharp**: each parameter has a unique type (no polymorphism). Then $|\mathcal{U}| = 1$, no overlaps, no transitions. This is the common case.*

2. ***Fibers are independent**: the program's behavior on fiber U_i does not depend on behavior on fiber U_j. Then transitions are trivial.*

3. ***Branch conditions align**: both programs branch on the same conditions. Then the branch-condition refinement produces the same atoms for both.*

18.13.9 Extensions: Beyond Type Fibers

The descent framework is not limited to type-based coverings. Any *partition* of the input space that respects program structure can serve as a covering:

1. **Value-range covering**: partition by value ranges ($n < 0$, $n = 0$, $n > 0$, $n > 100$, etc.). Useful for programs with thresholds.

2. **Shape covering**: partition by data structure shape (empty list, singleton, doubleton, general list). Useful for recursive programs with base cases.

3. **Reachability covering**: partition by reachable code paths (which branches are taken). This is the branch-condition refinement from Theorem 18.27.

4. **Symmetry covering**: partition by symmetry group orbits (inputs related by permutation). Useful for programs whose output is permutation-invariant.

5. **Norm covering**: partition by norm/size ($|x| \leq k$ for various k). The spectral sequence (Chapter 13) is the cohomology of this filtration.

Each covering type provides a different decomposition strategy, and the optimal covering depends on the programs being compared. The *covering selection heuristic* is:

1. Start with type fibers (cheapest, most effective for well-typed programs).

2. If local queries fail, refine by branch conditions.

3. If still failing, refine by value ranges or shapes.

4. If still failing, try symmetry or norm coverings.

This hierarchical refinement is the *spectral sequence in action*: each refinement level corresponds to a page of the spectral sequence, and the sequence converges when the local queries become decidable.

18.14 Soundness and Completeness

Theorem 18.34 (Path Search Soundness)**.** *If the path search returns a certificate $[r_0, \ldots, r_k]$, then $r_0 = [\![f]\!]$, $r_k = [\![g]\!]$, and each step $r_i \to r_{i+1}$ is justified by a valid path constructor. Therefore $[\![f]\!] \equiv [\![g]\!]$ (the programs are semantically equivalent).*

Proof. Each path constructor is a theorem of the cubical type theory (proved in the relevant dichotomy chapter). Kan composition (Chapter 5) ensures the steps compose to a valid path. □

Theorem 18.35 (Path Search Completeness (Relative))**.** *The path search is* complete relative to the axiom set*: if a path exists using only the axioms in the HIT* PyComp*, the normalization procedure finds it (assuming the rewrite system is confluent and terminating for the applicable axiom subset).*

The path search is incomplete *in general: there exist equivalent programs for which no path exists in* PyComp *(because the axiom set doesn't cover all valid equalities — Rice's theorem prevents completeness).*

18.15 Path Search for Spec Checking

The same path search engine handles spec checking by searching for a path from $[\![f]\!]$ to $[\![S]\!]$ (the specification's canonical form):

Listing 18.4: Spec check via path search

```
def spec_check(program_source, spec_source, theory):
    """Check if program satisfies spec via path search.

    spec_source is a reference implementation or a Z3 formula.
    """
    term_f = compile(program_source, theory)
    term_spec = compile(spec_source, theory)

    result = fiber_wise_path_search(
        sections_of(term_f),
        sections_of(term_spec),
        build_sites(program_source, spec_source, theory),
        theory
    )

    if result.equiv:
        return SpecResult(satisfies=True,
                          certificate=result.paths)
    else:
        return SpecResult(satisfies=False,
                          bug_fiber=result.fiber,
                          counterexample=result.cx)
```

18.16 Path Search for Bug Finding

Bug finding is path search with *failure analysis*: when no path exists on a fiber, the failure point reveals the bug.

Listing 18.5: Bug finding via failed path search

```
def find_bugs(program_source, spec_source, theory):
    """Find bugs by attempting path search on each fiber.

    Returns list of (fiber, depth, counterexample) triples.
    """
    bugs = []
    term_f = compile(program_source, theory)
    term_spec = compile(spec_source, theory)
    sites = build_sites(program_source, spec_source, theory)

    for combo, site_id in sites.items():
        result = search_path(
            restrict(term_f, combo),
            restrict(term_spec, combo),
            theory
        )
        if result.found == False:
            bugs.append(Bug(
                fiber=combo,
                counterexample=result.counterexample,
                # How far did the path get before failing?
```

```
                    partial_path=result.partial_steps,
                    # What axiom would be needed?
                    missing_axiom=result.stuck_on
                ))

    return BugReport(bugs=bugs,
                     severity=len(bugs) / len(sites),
                     h1_rank=compute_h1_rank(bugs, sites))
```

The `partial_path` and `missing_axiom` fields provide *diagnostic information*: they tell the developer not just *that* the program is wrong, but *why* the proof failed and *what kind of fix* is needed.

18.17 Path-Based Automatic Repair Suggestion

When the path search fails at a specific step, the failure point suggests a repair:

Theorem 18.36 (Repair from Path Failure)**.** *If the path search from $[\![f]\!]$ toward $[\![S]\!]$ succeeds for k steps and fails at step $k+1$ because axiom* ax *is not applicable:*

$$[\![f]\!] \xrightarrow{p_1} \cdots \xrightarrow{p_k} h_k \xrightarrow{\mathsf{ax?}} STUCK$$

then the repair *is: modify f so that h_k satisfies the precondition of* ax*. The modification is localized to the subterm at position* pos_{k+1} *of h_k.*

Example 18.37. A program tries to compute the sum of a list but uses multiplication instead of addition:

```
1 def f(xs):
2     acc = 0
3     for x in xs: acc *= x   # BUG: should be +=
4     return acc
```

Path search toward spec $S = \mathsf{py_sum}(xs)$:

1. $[\![f]\!] = \mathsf{fold}(\mathsf{MUL}, \ldots)$ (detected from loop).
2. Normalize: $\mathsf{fold}(\mathsf{MUL}, 0, \ldots) = 0$ (because $0 \times \text{anything} = 0$ by absorbing zero axiom).
3. Target: $\mathsf{py_sum}(xs)$ normalizes to $\mathsf{fold}(\mathsf{ADD}, 0, \ldots)$.
4. STUCK: $\mathsf{fold}(\mathsf{MUL}, \ldots) \neq \mathsf{fold}(\mathsf{ADD}, \ldots)$. The path constructor D1 applies but the operation tag differs: $\mathsf{MUL} \neq \mathsf{ADD}$.

Repair suggestion: "Change MUL to ADD at the loop accumulation step (line 3: `*=` $\rightarrow$ `+=`)."

Part V

The Twenty-Four Dichotomies

Chapter 19

Overview of the Dichotomies

We identify twenty-four fundamental *computational dichotomies* — pairs of implementation strategies that compute the same function. Each dichotomy is a path constructor in the HIT PyComp (Definition 4.2).

The dichotomies are organized into four groups:

Group	Dichotomies	OTerm Types Involved
Control Flow	D1–D8	`OFix`, `OFold`, `OCase`, `OLam`
Data Structure	D9–D14	`OOp`, `OFold`, `ODict`
Algorithm Strategy	D15–D20	`OFix`, `OFold`, `OAbstract`
Language Feature	D21–D24	`OCatch`, `OCase`, `OLam`

Each dichotomy manifests in two complementary ways within the system:

1. **Normalizer rewrite** (in `denotation.py`, phases 1–7): a directed rule that reduces one form to a canonical OTerm, applied during the 7-phase normalization pass.

2. **Path constructor** (in `path_search.py`): a bidirectional axiom that, given an OTerm matching one paradigm, produces the equivalent OTerm in the other paradigm together with a named proof step.

When both sides of a dichotomy normalize to the same canonical OTerm the path is refl. When normalization produces distinct but equivalent OTerms, the HIT structural prover (`hit_path_equiv`) applies the path constructor to close the gap.

19.1 Complete Dichotomy Table

#	Dichotomy	Path Constructor	Normalizer Phase	Axiom fn
Control Flow Dichotomies				
D1	Recursive ↔ Iterative	Nat-eliminator uniqueness	Phase 3 (fold)	`_axiom_d1`
D2	Nested helpers ↔ Flat	β-reduction	Phase 1 (β)	`_axiom_d2`
D3	`while` ↔ `for`	Loop normal form	Phase 3 (fold)	—
D4	Comprehension ↔ Loop	Catamorphism fusion	Phase 4 (HOF)	`_axiom_d4`
D5	`reduce` ↔ Loop	Fold universality	Phase 3 (fold)	`_axiom_d5`

#	Dichotomy	Path Constructor	Normalizer Phase	Axiom fn
D6	Generator ↔ Eager	Laziness adjunction	Phase 4 (HOF)	—
D7	Tail-recursive ↔ Loop	CPS transform	Phase 3 (fold)	—
D8	Multi-return ↔ Single	Section merging	Phase 7 (piecewise)	`_axiom_d8`
Data Structure Dichotomies				
D9	Stack ↔ Recursion	Defunctionalization	Phase 5 (unify)	—
D10	`heapq` ↔ `bisect`	Priority queue abstr.	Phase 5 (unify)	—
D11	`OrderedDict` ↔ LinkedList	ADT refinement	Phase 5 (unify)	—
D12	`defaultdict` ↔ `setdefault`	Map construction equiv	Phase 5 (unify)	—
D13	`Counter` ↔ Manual count	Histogram equivalence	Phase 5 (unify)	`_axiom_d13`
D14	Array ↔ Dict table	Indexing abstraction	Phase 5 (unify)	—
Algorithm Strategy Dichotomies				
D15	BFS ↔ DFS	Traversal-order invariance	Phase 6 (quotient)	—
D16	Memoized ↔ DP	Eval-order independence	Phase 5 (unify)	`_axiom_d16`
D17	Divide-and-conquer ↔ Iterative	Tree fold = list fold	Phase 4 (HOF)	`_axiom_d17`
D18	Greedy ↔ DP	Matroid/greedy theorem	—	—
D19	Sort-then-scan ↔ Sweep	Sorting abstraction	Phase 5 (unify)	`_axiom_d19`
D20	Different algorithms for same spec	Specification HIT	—	`_axiom_d20`
Language Feature Dichotomies				
D21	`isinstance` ↔ Dispatch	Fiber projection	Phase 7 (piecewise)	—
D22	`try/except` ↔ Conditional	Effect elimination	Phase 2 (ring)	`_axiom_d22`
D23	Sorted-then-process ↔ Direct	Canonical ordering path	Phase 5 (unify)	—
D24	Lambda ↔ Named function	η-expansion	Phase 1 (β)	`_axiom_d24`

Remark 19.1 (Axiom registration)**.** All root axiom functions listed above are collected in the list `_ROOT_AXIOMS` (path_search.py, line 1580). During bidirectional BFS, each axiom is applied to every sub-expression of the current OTerm; the resulting rewrites form the neighbour set for the next BFS layer. Axioms marked "—" are handled entirely by the normalizer and never need a separate path-search step.

Chapter 20

Control Flow Dichotomies (D1–D8)

Each section below follows the same structure: (1) two concrete programs illustrating paradigm A and paradigm B, (2) how each is compiled to OTerms, (3) the formal path constructor definition, (4) a soundness theorem with proof, (5) the implementation mapping to `path_search.py` and `denotation.py`, and (6) a benchmark example where this dichotomy applies.

20.1 D1: Recursive ↔ Iterative

Paradigm A — recursive descent:

```
def f(n):
    if n <= 1: return 1
    return n * f(n - 1)
```

Paradigm B — iterative accumulation:

```
def g(n):
    result = 1
    for i in range(2, n + 1):
        result *= i
    return result
```

Compilation of A. The compiler sees a self-recursive function with one parameter. It invokes the *Nat-eliminator extractor*, which detects:

- Base guard: `n <= 1`, so $\mathsf{threshold} = 1$.
- Base value: literal `1`, so $\mathsf{base} = 1$.
- Step: `n * f(n-1)`, a $\mathsf{BinOp}(\times)$ with the parameter on the left and the recursive call on the right. The operation tag is MUL.

The extractor produces: $[\![f]\!] = \mathsf{fold}(\mathsf{MUL}, 1, \mathsf{ival}(p_0) + 1, \mathsf{IntObj}(1))$.

Compilation of B. The compiler sees an assignment `result = 1` (init), then `for i in range(2, n+1): result *= i`. The loop-to-fold optimizer (Phase 3, `_phase3_fold`) detects:

- Single accumulator `result` with init $= 1$.
- `AugAssign` with Mult: `result *= i`. The function `_detect_augassign_fold` returns semantic key `imul`.
- $\mathsf{range}(2, n + 1)$ gives $\mathsf{start} = 2$, $\mathsf{stop} = \mathsf{ival}(p_0) + 1$.

The optimizer produces: $[\![g]\!] = \mathsf{fold}(\mathsf{MUL}, 1, \mathsf{ival}(p_0) + 1, \mathsf{IntObj}(1))$.

Path. $[\![f]\!]$ and $[\![g]\!]$ are *the same OTerm.* The path is refl (reflexivity) — no axiom application needed.

When the step has $f(n-1) * n$ instead of $n * f(n-1)$, the terms differ: $\mathsf{fold}(\mathsf{MUL}, \ldots)$ vs. $\mathsf{fold}(\mathsf{MUL}, \ldots)$ but with swapped operands inside the RecFunction body. The mul_comm path constructor bridges the gap: the ring-normalization phase (Phase 2, `_phase2_ring`) sorts commutative operands into canonical order, yielding refl after normalization.

Definition 20.1 (D1 Path Constructor — formal)**.** Define the path constructor for D1 as the family

$$\mathsf{D1} \;:=\; \lambda\, b\, s\, n.\ \mathsf{path}(\mathsf{OFix}(\mathsf{h},\ \mathsf{case}(\mathsf{le}(n,0),\ b,\ s(n, \mathsf{h}(n-1)))),\ \mathsf{OFold}(s,b,n))$$

where $b : \mathsf{OTerm}$ is the base value, s is the step operation name, and n is the collection/range bound. The path witnesses that a recursive fixed point with accumulation pattern is equal to the corresponding fold.

Theorem 20.2 (D1 Soundness)**.** *For any base value b, binary operation $\otimes$ satisfying $\otimes(x,y) = \otimes(y,x)$ (commutativity is optional; required only for operand-swapped variants), and natural number n:*

$$\mathsf{D1} : \prod_{b,\otimes,n} \mathsf{Path}\big(\mathsf{rec}_{\mathbb{N}}(b, \lambda k.\, k \otimes \mathsf{rec}_{\mathbb{N}}(b, \ldots)(k-1)),\ \mathsf{fold}(\otimes, b, n)\big)$$

Proof. By the uniqueness of the $\mathbb{N}$-eliminator (the universal property of $\mathbb{N}$ as an initial algebra): any function $h : \mathbb{N} \to A$ satisfying $h(0) = b$ and $h(k+1) = k \otimes h(k)$ is uniquely determined. The recursive form $\mathsf{rec}_{\mathbb{N}}(b,s)$ satisfies this recurrence by definition. The fold $\mathsf{fold}(\otimes, b, n)$ unfolds to the same recurrence: $\mathsf{fold}(\otimes, b, 0) = b$ and $\mathsf{fold}(\otimes, b, k+1) = \otimes(k, \mathsf{fold}(\otimes, b, k))$. By initiality, both are the same morphism from $\mathbb{N}$ to A, hence equal. □

Implementation mapping.

- **Normalizer**: `_phase3_fold` in `denotation.py` (line 1379) rewrites builtin reducers to `OFold`; the compiler's `_exec_for_ot` (line 762) detects augmented-assignment loops and emits `OFold` with a semantic key via `_detect_augassign_fold` (line 808).
- **Path search**: `_axiom_d1_fold_unfold` (line 188) applies in direction OFix → OFold by calling `_try_extract_fold_from_fix`.
- **HIT prover**: `_fix_fold_equiv` (line 1492) handles the cross-type case OFix ↔ OFold by comparing fix-point names, free variables, and spec identifications.

Benchmark example. The factorial pair (`demo_factorial_rec.py` vs. `demo_factorial_iter.py`): the recursive version compiles to $\mathsf{OFix}[\mathsf{h}](\mathsf{case}(\mathsf{le}(p_0,1), 1, \mathsf{mult}(p_0, \mathsf{h}(\mathsf{sub}(p_0,1)))))$ while the iterative version compiles directly to $\mathsf{OFold}(\mathsf{imul}, 1, p_0)$. Phase 3 normalizes the recursive form to the same fold, producing refl.

20.2 D2: Nested Helpers ↔ Flat Code

Paradigm A — nested helper function:

```
def f(x):
    def helper(a, b):
        return a * a + b * b
    return helper(x, x + 1)
```

Paradigm B — inlined flat code:

```
def g(x):
    return x * x + (x + 1) * (x + 1)
```

Compilation of A. The compiler encounters `def helper`: it stores an $\mathsf{OLam}((\mathsf{a},\mathsf{b}),\mathsf{body})$ in the environment (denotation.py, line 241). When it reaches `helper(x, x+1)`, it performs β-reduction: substituting $a \mapsto p_0$ and $b \mapsto \mathsf{add}(p_0, 1)$ into the body. Result:

$$[\![f]\!] = \mathsf{add}(\mathsf{mult}(p_0, p_0),\ \mathsf{mult}(\mathsf{add}(p_0, 1), \mathsf{add}(p_0, 1)))$$

Compilation of B. Direct compilation produces the identical OTerm (same expression tree).

Path. $[\![f]\!] \equiv [\![g]\!]$ by β-reduction. The path is refl after inlining.

Definition 20.3 (D2 Path Constructor — formal)**.**

$$\mathsf{D2} \ :=\ \lambda\, \vec{x}\, \vec{a}\, e.\ \mathsf{path}\big(\mathsf{OOp}(\mathsf{apply},\ \mathsf{OLam}(\vec{x},\ e),\ \vec{a}),\ e[\vec{x} := \vec{a}]\big)$$

where $e[\vec{x} := \vec{a}]$ denotes simultaneous capture-avoiding substitution of actuals $\vec{a}$ for formals $\vec{x}$ in the body e.

Theorem 20.4 (D2 Soundness — β-Reduction)**.** *For any* $\mathsf{OLam}(\vec{x}, e)$ *and arguments* $\vec{a}$ *of matching arity:*

$$\mathsf{D2} : \mathsf{Path}\big((\lambda \vec{x}.\, e)(\vec{a}),\ e[\vec{x} := \vec{a}]\big)$$

Proof. This is the β-rule of the simply typed λ-calculus, which is a definitional equality in cubical type theory. The substitution $e[\vec{x} := \vec{a}]$ is computed by `_subst_term` (path_search.py) and is capture-avoiding because OTerm variables are globally fresh parameter names p_i. □

Implementation mapping.

- **Normalizer**: `_phase1_beta` in `denotation.py` (line 1096) collapses trivial cases ($\mathsf{case}(\top, t, f) \to t$, $\mathsf{case}(c, x, x) \to x$). The compiler itself performs β-reduction at call sites by inlining the stored `OLam` body.
- **Path search**: `_axiom_d2_beta` (line 221) fires on $\mathsf{OOp}(\mathsf{apply}, \mathsf{OLam}(\ldots), \ldots)$ nodes. It pattern-matches the apply-of-lambda form, substitutes actuals for formals, and returns the reduced body tagged `D2_beta`.

Benchmark example. 22 benchmark pairs involve nested helper functions. A typical case: one program defines `def helper(a, b): return a + b` then calls `helper(x, y)`, while the equivalent program writes `x + y` directly. After β-reduction both yield $\mathsf{add}(p_0, p_1)$.

20.3 D3: `while` ↔ `for`

Paradigm A — `while` with explicit counter:

```
def f(n):
    result, i = 0, 1
    while i <= n:
        result += i
        i += 1
    return result
```

Paradigm B — `for` over range:

```python
def g(n):
    result = 0
    for i in range(1, n + 1):
        result += i
    return result
```

Compilation of A. The compiler encounters a `while` loop. It identifies modified variables $\{\texttt{result}, \texttt{i}\}$ and creates a RecFunction for each:

$$\mathsf{wh_result}(k) = \begin{cases} \mathsf{IntObj}(0) & \text{if } k > 50 \vee \neg(i \leq n) \\ \mathsf{add}(\mathsf{wh_result}(k+1), \ldots) & \text{otherwise} \end{cases}$$

Compilation of B. The compiler encounters `for i in range(1, n+1)`. It extracts $\mathsf{start} = 1$, $\mathsf{stop} = \mathsf{ival}(p_0) + 1$. The single-accumulator fold pattern (`result += i`) is detected by `_detect_augassign_fold` which returns semantic key `iadd`, producing:

$$[\![g]\!] = \mathsf{OFold}(\mathsf{iadd},\ 0,\ p_0)$$

Path. Both RecFunctions satisfy the same recurrence: $h(0) = 0$, $h(k) = h(k-1) + k$. The `while` version's guard $i \leq n$ is equivalent to the `for` range $[1, n+1)$, and both step by 1. The RecFunctions unfold identically.

When Z3 checks $[\![f]\!](p_0) \neq [\![g]\!](p_0)$ on the integer fiber ($\mathsf{tag}(p_0) = \mathsf{TInt}$), it finds UNSAT because both RecFunctions compute the same value for every integer input.

Definition 20.5 (D3 Path Constructor — formal)**.**

$$\mathsf{D3} \coloneqq \lambda\, g\, s\, \mathit{init}.\ \mathsf{path}\big(\mathsf{OFix}(\mathsf{h},\ \mathsf{case}(g,\ s(\mathsf{h}),\ \mathit{init})),\ \mathsf{OFold}(s,\ \mathit{init},\ \mathsf{range}(g))\big)$$

where $\mathsf{range}(g)$ denotes the iteration range induced by the while-guard g with unit step.

Automatic inference. The compiler automatically canonicalizes both loop forms to RecFunctions. The `for`-range version gets the additional fold optimization. Z3's recursive function reasoning handles the comparison. The path is: $\mathsf{D3} = \mathsf{while_rec} \xrightarrow{\text{unfold}} \mathsf{for_fold}$, mediated by the shared recurrence structure. D3 has no dedicated axiom function in `path_search.py` because the normalizer (phases 1–3) already reduces both loop forms to the same canonical OTerm.

20.4 D4: Comprehension ↔ Loop

Paradigm A — list comprehension:

```python
def f(xs):
    return [x * 2 for x in xs]
```

Paradigm B — explicit loop with append:

```python
def g(xs):
    result = []
    for x in xs:
        result.append(x * 2)
    return result
```

Compilation of A. The compiler encounters a `ListComp`. The function `_compile_comprehension_ot` (denotation.py, line 556) builds an OMap:

$$[\![f]\!] = \mathsf{OMap}\big(\mathsf{OLam}(x,\ \mathsf{mult}(x, 2)),\ p_0\big)$$

The transform is captured as an `OLam` with the comprehension variable as the bound parameter and the element expression as the body.

Compilation of B. The compiler sees `result = []` followed by `for x in xs: result.append(x * 2)`. The function `_detect_map_pattern` (denotation.py, line 774) recognizes the append-in-loop idiom and produces the same OMap:

$$[\![g]\!] = \mathsf{OMap}\big(\mathsf{OLam}(x,\ \mathsf{mult}(x, 2)),\ p_0\big)$$

Path. Both programs normalize to structurally identical `OMap` terms. The path is refl.

When normalization does not fully unify the terms (e.g. the loop body contains additional logic), the path constructor applies catamorphism fusion:

Definition 20.6 (D4 Path Constructor — formal)**.**

$$\mathsf{D4} \ :=\ \lambda\, f\, g\, c.\ \mathsf{path}\big(\mathsf{OFold}(\mathsf{op},\ \mathsf{OMap}(f, c),\ e),\ \mathsf{OFold}(\mathsf{op} \circ f,\ c,\ e)\big)$$

This is the *fold-map fusion* law: $\mathsf{fold}(\mathsf{op}, \mathsf{map}(f, c)) = \mathsf{fold}(\mathsf{op} \circ f, c)$. The auxiliary direction $\mathsf{OMap}(f, c) = \mathsf{OFold}(\mathsf{append} \circ f, \varepsilon, c)$ expresses the comprehension as a fold over the input that accumulates into an initially-empty list.

Theorem 20.7 (D4 Soundness — Catamorphism Fusion)**.** *Let* $(L, \varepsilon, \mathsf{cons})$ *be the initial algebra of finite lists. For any* $f : A \to B$ *and fold algebra* $(B, e, \otimes)$*:*

$$\mathsf{cata}_{(B,e,\otimes)} \circ \mathsf{map}(f) = \mathsf{cata}_{(B,e,\otimes\circ(\mathsf{id}\times f))}$$

Proof. By the universal property of the initial algebra. cata is the unique homomorphism from the initial algebra to $(B, e, \otimes)$. The composition $\mathsf{cata} \circ \mathsf{map}(f)$ is also a homomorphism from $(L, \varepsilon, \mathsf{cons})$ to $(B, e, \otimes)$, with step $\otimes(b, f(a))$. By uniqueness of the catamorphism, it equals $\mathsf{cata}_{(B,e,\otimes\circ(\mathsf{id}\times f))}$. □

Implementation mapping.

- **Normalizer**: the compiler emits `OMap` for both comprehensions (`_compile_comprehension_ot`, line 556) and append-loops (`_detect_map_pattern`, line 774). The `OLam` body uses alpha-normalized parameter names so that $\lambda x.\, x * 2$ and $\lambda y.\, y * 2$ produce the same canonical form.
- **Path search**: `_axiom_d4_comp_fusion` (line 250) applies the fold-map fusion rule: when an `OFold` has an `OMap` as its collection and the map's filter is `None`, the map is absorbed into the fold by composing transforms.

Benchmark example. `[x**2 for x in xs]` vs. `result = []; for x in xs: result.append(x**2); return result`. Both compile to $\mathsf{OMap}(\mathsf{OLam}(x,\ \mathsf{pow}(x, 2)),\ p_0)$.

20.5 D5: `reduce` ↔ Loop

Paradigm A — `functools.reduce`:

```python
def f(xs):
    return reduce(operator.add, xs, 0)
```

Paradigm B — explicit accumulation loop:

```python
def g(xs):
    acc = 0
    for x in xs:
        acc += x
    return acc
```

Compilation of A. `reduce(operator.add, xs, 0)` compiles to an OTerm via Phase 3 (fold canonicalization): $[\![f]\!] = \mathsf{OFold}(\mathsf{iadd},\ 0,\ p_0)$. The normalizer recognizes `reduce` with a known operator and rewrites it directly to a fold with the corresponding semantic key.

Compilation of B. The loop compiles to a fold via `_detect_augassign_fold`: $[\![g]\!] = \mathsf{OFold}(\mathsf{iadd},\ 0,\ p_0)$. The augmented assignment `acc += x` yields semantic key `iadd`; the init value `0` becomes the fold's initial accumulator.

Path. $[\![f]\!] = [\![g]\!]$ syntactically. Path is refl.

When the fold keys differ (e.g. a hash-based key from a complex loop body vs. the semantic key from `reduce`), the path constructor bridges the gap via fold universality:

Definition 20.8 (D5 Path Constructor — formal)**.**

$$\mathsf{D5} \ := \ \lambda\, k_1\, k_2\, e\, c.\ \mathsf{path}\big(\mathsf{OFold}(k_1,\ e,\ c),\ \mathsf{OFold}(k_2,\ e,\ c)\big) \quad \text{when } k_1 \sim k_2$$

where $k_1 \sim k_2$ means both keys denote the same binary operation (verified by `_identify_fold_op`).

Theorem 20.9 (D5 Soundness — Fold Universality)**.** *Let $\otimes_1, \otimes_2 : A \times A \to A$ be binary operations. If $\otimes_1 = \otimes_2$ as functions, and $e_1 = e_2$, then for any collection c:*

$$\mathsf{OFold}(\otimes_1, e_1, c) = \mathsf{OFold}(\otimes_2, e_2, c)$$

Proof. By the universal property of the list catamorphism: $\mathsf{cata}_{(A,e,\otimes)}$ is the unique homomorphism from the initial list algebra to $(A, e, \otimes)$. If $\otimes_1 = \otimes_2$ and $e_1 = e_2$, the target algebras are identical, so by uniqueness the catamorphisms agree. □

Implementation mapping.

- **Normalizer**: Phase 3 (`_phase3_fold`, line 1379) rewrites builtin reducers: `sum(xs)` → `OFold('iadd', 0, xs)`, `any(xs)` → `OFold('or', False, xs)`, `all(xs)` → `OFold('and', True, xs)`, `''.join(xs)` → `OFold('str_concat', '', xs)`.
- **Path search**: `_axiom_d5_fold_universal` (line 276) fires on hash-named folds (8-character op_name). It calls `_identify_fold_op` to determine the canonical operation; if successful, it replaces the hash key with the semantic key.
- **HIT prover**: the OFold congruence rule (line 1274) handles the general case: when two `OFold`s have matching `init` and `collection` (checked recursively), it compares `op_name`s directly, then falls back to `_identify_fold_op` for hash-key resolution.

Benchmark example. `sum(xs)` vs. `acc = 0; for x in xs: acc += x; return acc`. Phase 3 rewrites `sum` to $\mathsf{OFold}(\mathsf{iadd}, 0, p_0)$; the loop compiles to the same fold via `_detect_augassign_fold` with key `iadd`.

20.6 D6: Generator ↔ Eager

Paradigm A — generator with `yield`:

```python
def f(data):
    def helper(obj):
        if isinstance(obj, list):
            for item in obj:
                yield from helper(item)
        else:
            yield obj
    return list(helper(data))
```

Paradigm B — eager list building:

```
def g(data):
    result = []
    stack = [data]
    while stack:
        obj = stack.pop()
        if isinstance(obj, list):
            stack.extend(reversed(obj))
        else:
            result.append(obj)
    return result
```

Compilation. Both programs traverse a nested structure and collect atomic elements. The generator version yields elements lazily; the eager version appends them to a list. When the generator is fully consumed by `list()`, the result is identical.

In the OTerm encoding, both compile to sequences of shared function applications on the input. The generator's `yield` is modeled as producing a sequence. Phase 4 (`_phase4_hof`) simplifies $\mathsf{list}(\mathsf{gen}(\ldots))$ to the underlying sequence when the generator is stateless.

Definition 20.10 (D6 Path Constructor — formal)**.**

$$\mathsf{D6} \;:=\; \lambda\, e.\ \mathsf{path}\big(\mathsf{OOp}(\mathsf{list},\ \mathsf{OOp}(\mathsf{gen},\ e)),\ \mathsf{OOp}(\mathsf{eager},\ e)\big)$$

This is the *laziness adjunction*: full consumption of a lazy stream is naturally isomorphic to eager construction.

Path. $\mathsf{D6} : \mathsf{Path}(\mathsf{list}(\mathsf{gen}(\mathsf{body})),\ \mathsf{eager}(\mathsf{body}))$ where $\mathsf{list} \circ \mathsf{gen}$ is the "consume generator" operation. This path holds because full consumption of a generator produces the same sequence as eager construction — both visit the same elements in the same order.

Automatic inference. The compiler detects `yield` and `yield from` and models the generator as producing a sequence. The `list()` call (or `for` loop consumption) triggers the D6 path. The traversal order equivalence (generator DFS vs. stack-with-reversed DFS) is handled by D9 (Stack $\leftrightarrow$ Recursion). D6 has no dedicated axiom function because Phase 4 of the normalizer handles the $\mathsf{list}(\mathsf{gen}(\ldots))$ pattern directly.

20.7 D7: Tail-Recursive $\leftrightarrow$ Loop

Paradigm A — tail-recursive with accumulator:

```
def f(n, acc=0):
    if n <= 0: return acc
    return f(n - 1, acc + n)
```

Paradigm B — iterative with accumulator:

```
def g(n):
    acc = 0
    while n > 0:
        acc += n
        n -= 1
    return acc
```

Compilation. The tail-recursive version has the recursive call in tail position: $f(n-1, \mathsf{acc}+n)$. The accumulator acc is passed as a parameter rather than composed in the return value (contrast with D1 where the return value is $n \times f(n-1)$).

The compiler detects tail position and creates a RecFunction:

$$\mathsf{rec_f}(n, \mathsf{acc}) = \begin{cases} \mathsf{acc} & n \leq 0 \\ \mathsf{rec_f}(n-1, \mathsf{acc}+n) & \text{otherwise} \end{cases}$$

The iterative version produces the same RecFunction via the while loop compiler (D3), with acc as the modified variable and n as the counter.

Definition 20.11 (D7 Path Constructor — formal)**.**

$$\mathsf{D7} \;:=\; \lambda\, g\, s\, acc_0.\ \mathsf{path}\big(\mathsf{OFix}(\mathsf{h},\ \mathsf{case}(g,\ s(\mathsf{h},\ acc),\ acc)),\ \mathsf{OFold}(s,\ acc_0,\ \mathsf{range}(g))\big)$$

This is the CPS/accumulator transform: a tail-recursive function whose only use of the recursive result is to return it immediately is isomorphic to a loop that threads the accumulator as mutable state.

Path. D7 identifies tail recursion with iteration. This is the CPS transform in action: a tail-recursive function is a loop with the accumulator as explicit state. Both RecFunctions have the same defining equation, so the path is refl.

Automatic inference. The compiler detects tail position by checking whether the recursive call is the *last* operation before return (no wrapping BinOp). When detected, it generates the same RecFunction structure as the while-loop compiler, producing identical OTerms. D7 is subsumed by D1 in the OTerm representation (both reduce to the same OFix/OFold comparison), so no separate axiom function is needed.

20.8 D8: Multi-Return ↔ Single Return

Paradigm A — early returns:

```
def f(x):
    if x < 0: return -1
    if x == 0: return 0
    return 1
```

Paradigm B — single return with nested conditional:

```
def g(x):
    return -1 if x < 0 else (0 if x == 0 else 1)
```

Compilation of A. The compiler produces a nested OCase chain:

$$[\![f]\!] = \mathsf{OCase}\big(\mathsf{lt}(p_0, 0),\ -1,\ \mathsf{OCase}(\mathsf{eq}(p_0, 0),\ 0,\ 1)\big)$$

Compilation of B. The `IfExp` chain compiles to the same nested OCase:

$$[\![g]\!] = \mathsf{OCase}\big(\mathsf{lt}(p_0, 0),\ -1,\ \mathsf{OCase}(\mathsf{eq}(p_0, 0),\ 0,\ 1)\big)$$

When the guard ordering differs between A and B, Phase 7 (`_phase7_piecewise`) canonicalizes both by:

1. Flattening the case chain into (g_i, v_i) pieces via `_flatten_case_chain`.
2. Inferring the implicit else-guard from the comparison algebra via `_infer_complement_guard`.
3. Sorting pieces canonically by (value.canon(), guard.canon()).
4. Reconstructing a canonical case chain.

Definition 20.12 (D8 Path Constructor — formal)**.** Let $\{(g_i, v_i)\}_{i=1}^n$ and $\{(g'_{\sigma(i)}, v'_{\sigma(i)})\}_{i=1}^n$ be two enumerations of the same piecewise function (i.e. the guards form a partition and $v_i = v'_{\sigma(i)}$ on the region g_i). Then:

$$\mathsf{D8} \coloneqq \lambda\, \vec{g}\, \vec{v}\, \sigma.\ \mathsf{path}\big(\mathsf{OCase}(g_1, v_1, \mathsf{OCase}(g_2, v_2, \ldots)),\ \mathsf{OCase}(g'_{\sigma(1)}, v'_{\sigma(1)}, \mathsf{OCase}(g'_{\sigma(2)}, v'_{\sigma(2)}, \ldots))\big)$$

This is the *section merging* path: two case chains over the same partition are equal as piecewise functions.

Theorem 20.13 (D8 Soundness — Section Merging)**.** *Let $\{U_i\}_{i=1}^n$ be a finite cover of the input space by decidable predicates (guards). Let $s_i, s'_i : U_i \to A$ be sections such that $s_i = s'_{\sigma(i)}$ on each U_i. Then the piecewise functions $\bigsqcup_i s_i$ and $\bigsqcup_i s'_{\sigma(i)}$ are equal.*

Proof. This is the sheaf condition for the constant presheaf on a finite decidable cover. The sections agree on every fiber of the cover, and since the cover is a partition (the guards are disjoint and exhaustive), there is a unique global section. Reordering the enumeration of a partition does not change the global section. Formally, this follows from the fact that $H^1 = 0$ for the constant presheaf on a finite cover by decidable predicates. □

Implementation mapping.

- **Normalizer**: Phase 7 (`_phase7_piecewise`, line 1696) is the primary mechanism. It calls `_try_canonicalize_piecewise` (line 1742) which flattens, infers complement guards, sorts, and reconstructs.
- **Path search**: `_axiom_d8_section_merge` (line 313) handles residual nested-case flattening after other axioms have fired. It detects $\mathsf{OCase}(g, A, \mathsf{OCase}(g', B, C))$ where g and g' are complementary guards (checked by `_guards_complementary`), and flattens to $\mathsf{OCase}(g, A, C)$.
- **HIT prover**: `_try_case_flatten` (line 1552) handles the cross-case permutation: $\mathsf{case}(t_1, A, \mathsf{case}(t_2, B, C$ vs. $\mathsf{case}(t_2, B', \mathsf{case}(t_1, A', C'))$. It verifies that the guards and branches match up to reordering, proving equivalence of two enumerations of the same piecewise function.

Benchmark example. The sign function: one version uses two `if` guards with early returns (negative, zero) and a fallthrough positive case; the other uses a single ternary expression. Phase 7 flattens both to the canonical piecewise form $\mathsf{OCase}(\mathsf{eq}(p_0, 0), 0, \mathsf{OCase}(\mathsf{lt}(p_0, 0), -1, 1))$ (sorted by value then guard), producing refl.

Chapter 21

Data Structure Dichotomies (D9–D14)

21.1 D9: Stack ↔ Recursion

Paradigm A — recursive tree traversal:

```
def f(tree):
    if tree is None: return 0
    return 1 + f(tree.left) + f(tree.right)
```

Paradigm B — explicit stack:

```
def g(tree):
    count = 0
    stack = [tree]
    while stack:
        node = stack.pop()
        if node is None: continue
        count += 1
        stack.append(node.left)
        stack.append(node.right)
    return count
```

Path construction. Both compute the tree catamorphism with $\mathsf{base} = 0$ and $\mathsf{combine}(l, r, _) = 1+l+r$. The recursive version uses the call stack; the iterative version uses an explicit Pair-chain as stack.

The compiler produces:

- A: $\mathsf{tree_rec}(\mathsf{tree_rec_f}, p_0)$ — a tree RecFunction.
- B: $\mathsf{wh_count}(0, [p_0])$ — a while-loop RecFunction over stack state.

Automatic inference. The path is inferred when:

1. The recursive version's RecFunction matches the tree catamorphism pattern (base case on None/leaf, recursive calls on children).
2. The iterative version's while-loop processes a stack by popping and pushing children — the compiler detects the append/pop pattern on the stack variable.
3. The combine function is commutative (+ here), so traversal order doesn't matter (BFS/DFS/any order).

Certificate: $\mathsf{D9}(\mathsf{tree_cata}, +, 0) \cdot \mathsf{add_comm}$.

21.2 D10: `heapq` ↔ `bisect`

Paradigm A: Kahn's toposort using `heapq.heappush`/`heappop`.

Paradigm B: Kahn's toposort using `bisect.insort`/`pop(0)`.

Path construction. Both maintain a sorted frontier of zero-indegree nodes. Both extract the minimum element at each step. The path uses the *ADT abstraction*: any implementation of the priority queue interface (insert, extract_min) that maintains the sorted invariant produces the same sequence of extracted elements.

The compiler maps both to the same shared function symbols: py_meth_heappush and py_call_bisect_insort both map to py_pq_insert; py_meth_heappop and `.pop(0)` both map to py_pq_extract_min.

Automatic inference. The compiler maintains a table of *ADT equivalence classes*: operations that implement the same abstract interface. When both programs use operations from the same equivalence class on the same data, they get the same shared symbols.

21.3 D11: `OrderedDict` ↔ LinkedList LRU

Paradigm A: LRU cache using `OrderedDict.move_to_end`.

Paradigm B: LRU cache using a doubly-linked list + dict.

Path. Both implement the LRU interface: $\mathsf{get}(k)$ (returns value, moves to front), $\mathsf{put}(k, v)$ (inserts or updates, evicts LRU if full). Both shared-symbol to py_lru_get, py_lru_put. Same interface ⇒ same behavior on the same sequence of operations (by ADT abstraction).

21.4 D12: `defaultdict` ↔ `setdefault`

Both implement "get existing value or create default":

```
# A: d = defaultdict(list); d[k].append(v)
# B: d.setdefault(k, []).append(v)
```

Both compile to: $\mathsf{py_mut_append}(\mathsf{py_meth_setdefault}(d, k, \mathsf{NoneObj}), v)$ (same shared symbols). Path: refl.

21.5 D13: `Counter` ↔ Manual Counting

`Counter(xs)` is syntactic sugar for $\mathsf{fold}(\lambda d, x.\, \mathsf{py_meth___setitem__}(d, x, \mathsf{binop}(\mathsf{ADD}, \mathsf{py_meth_get}(d, x, \mathsf{IntO}$

A manual counting loop compiles to the same fold. Path: D5 (reduce↔loop).

Theorem 21.1 (D13: Histogram Fold Equivalence)**.** *Let* $\mathsf{count_step} := \lambda\, d\, x.\ d[x] \leftarrow d.\mathsf{get}(x, 0) +$ 1. *Any program whose normal form is* $\mathsf{fold}(\mathsf{count_step},\ \mathsf{empty_dict},\ \mathsf{xs})$ *is equivalent to* $\mathsf{Counter}(\mathsf{xs})$*:*

$$\mathsf{D13} : \mathsf{Path}(\mathsf{fold}(\mathsf{count_step},\ \emptyset,\ \mathsf{xs}),\ \mathsf{Counter}(\mathsf{xs}))$$

Proof sketch. The standard-library `Counter` constructor is *defined* as the above fold (CPython source: `self[elem] = self.get(elem, 0) + 1`). Both forms compile to $\mathsf{OFold}(\text{'count_freq'}, \mathsf{OOp}(\text{'defaultdict'}$ Definitional equality gives refl. □

Implementation mapping. In `path_search.py`, the axiom `_axiom_d13_histogram` fires in both directions:

- *Forward*: when the term is $\mathsf{OOp}(\text{'Counter'}, (\mathsf{xs},))$, it rewrites to $\mathsf{OFold}(\text{'count_freq'}, \mathsf{OOp}(\text{'defaultdict'}, (0,)$ with label `D13_counter_to_fold`.

- *Backward*: when the term is OFold('count_freq', ...) with a `defaultdict` initializer, it rewrites to OOp('Counter', (xs,)) with label `D13_fold_to_counter`.

The bidirectional BFS connects the two normal forms in at most one step.

Benchmark example. Pair **eq42**: one program uses `Counter(nums)` to count element frequencies; the other uses a manual `for x in nums: d[x] = d.get(x, 0) + 1` loop. Both compile to the same OFold and the path is `D13_counter_to_fold` $\cdot$ `D13_fold_to_counter`$^{-1}$ = refl.

21.6 D14: Array ↔ Dict Table

A list used as `dp[i]` and a dict used as `dp[i]` both compile to $\mathsf{py_meth___getitem__}(\mathsf{dp}, i)$ (same shared symbol). Path: refl (same interface).

Chapter 22

Algorithm Strategy Dichotomies (D15–D20)

22.1 D15: BFS $\leftrightarrow$ DFS

When the output is *traversal-order independent* (e.g., a count, sum, set, or sorted result), BFS and DFS produce the same answer.

Theorem 22.1 (D15: Traversal-Order Invariance)**.** *If the combining operation is a commutative monoid:*

$$\mathsf{D15} : \mathsf{Path}(\mathsf{fold}(\mathsf{op}, e, \mathsf{bfs_order}(G)),\ \mathsf{fold}(\mathsf{op}, e, \mathsf{dfs_order}(G)))$$

when (op, e) *is a commutative monoid.*

Z3 encoding: The commutative monoid axioms (add_comm, add_assoc, etc.) enable Z3 to prove that folds over different permutations of the same multiset produce the same result.

Benchmark coverage: 5 pairs.

22.2 D16: Memoized $\leftrightarrow$ Bottom-Up DP

Both compute the least fixpoint of the same recurrence.

Theorem 22.2 (D16: Evaluation-Order Independence)**.** *For a monotone operator F on a complete lattice:*

$$\mathrm{lfp}(F) = \bigsqcup_{n \geq 0} F^n(\bot)$$

This fixpoint is independent of the order in which $F^n(\bot)$ is computed (top-down with memoization vs. bottom-up filling).

Proof. Let $F : L \to L$ be monotone on a complete lattice $(L, \sqsubseteq)$. Write $a_n \coloneqq F^n(\bot)$. The chain $\bot \sqsubseteq a_1 \sqsubseteq a_2 \sqsubseteq \cdots$ converges to $a^* \coloneqq \bigsqcup_n a_n$, which satisfies $F(a^*) = a^*$ by continuity.

Top-down memoization evaluates a_n lazily: each subproblem is computed at most once and cached. Bottom-up DP evaluates $a_0, a_1, \ldots, a_n$ eagerly in index order. Both produce a^* because the value of a^* at index i depends only on F and the values at indices $< i$ (by monotonicity), which are identical in both strategies.

In CTTT terms, both programs compile to $\mathsf{OFix}(h, \mathsf{body})$ where h is a hash of the recurrence body. The axiom `_axiom_d16_memo_dp` α-normalizes the body (replacing variable names by positional placeholders) and re-hashes, yielding a canonical OFix. Two fix-points with the same canonical hash are connected by refl. $\square$

Implementation mapping. The function `_canonicalize_recurrence` (line 1054 of `path_search.py`) strips variable names via `re.sub(r'\$\w+', '$_', canon)`, then MD5-hashes the structural signature. If the new hash differs from the original `OFix.name`, the rewrite fires with label `D16_recurrence_normalize`.

Benchmark example. Pair **eq19** (Fibonacci): one program uses `@lru_cache` (top-down memo), the other fills a table `dp[0]=0, dp[1]=1, dp[i]=dp[i-1]+dp[i-2]` (bottom-up). Both recurrences α-normalize to the same body $\mathsf{add}(\$_\mathsf{sub}(\$_, 1), \$_\mathsf{sub}(\$_, 2))$. Canonical hash: `3a7f1c02`. Path: refl after `D16_recurrence_normalize`.

22.3 D17: Divide-and-Conquer ↔ Iterative

A divide-and-conquer algorithm splits the input, recurses, and combines: $f(\mathsf{xs}) = \mathsf{combine}(f(\mathsf{left}), f(\mathsf{right}))$. An iterative algorithm processes elements one by one: $g(\mathsf{xs}) = \mathsf{fold}(\mathsf{step}, e, \mathsf{xs})$.

Theorem 22.3 (D17: Associativity-Mediated Refactoring)**.** *Let $(\oplus, e)$ be a monoid (i.e. $\oplus$ is associative with identity e). Then tree-structured and linear folds are path-equal:*

$$\mathsf{D17} : \mathsf{Path}(\mathsf{fold}(\oplus,\ e,\ \mathsf{tree_split}(\mathsf{xs})),\ \mathsf{fold}(\oplus,\ e,\ \mathsf{xs}))$$

Proof. By induction on the tree structure of $\mathsf{tree_split}(\mathsf{xs})$.

Base: A singleton leaf $[x]$ satisfies $\mathsf{fold}(\oplus, e, [x]) = e \oplus x = x$ in both cases.

Step: For $\mathsf{tree_split}(\mathsf{xs}) = \mathsf{Node}(\mathsf{left}, \mathsf{right})$, the tree fold computes $\mathsf{fold}(\oplus, e, \mathsf{left}) \oplus \mathsf{fold}(\oplus, e, \mathsf{right})$. By the inductive hypothesis, these equal $\mathsf{fold}(\oplus, e, \mathsf{xs}_L)$ and $\mathsf{fold}(\oplus, e, \mathsf{xs}_R)$ respectively. By associativity:

$$\mathsf{fold}(\oplus, e, \mathsf{xs}_L) \oplus \mathsf{fold}(\oplus, e, \mathsf{xs}_R) = \mathsf{fold}(\oplus, e, \mathsf{xs}_L +\!\!+ \mathsf{xs}_R) = \mathsf{fold}(\oplus, e, \mathsf{xs})$$

where $+\!\!+$ is concatenation and the second equality uses $\mathsf{xs}_L +\!\!+ \mathsf{xs}_R = \mathsf{xs}$ (the split is a partition). □

Implementation mapping. The axiom `_axiom_d17_assoc` calls `_try_flatten_tree_fold`: when the collection is $\mathsf{OOp}(\text{'tree_split'}, (\mathsf{xs},))$ or $\mathsf{OOp}(\text{'divide'}, (\mathsf{xs},))$ and the fold operation passes the `_is_commutative_op` check, the rewrite replaces the tree fold with a flat $\mathsf{OFold}(\mathsf{op}, e, \mathsf{xs})$, labelled `D17_tree_to_linear_fold`.

Benchmark example. Pair **eq31** (maximum subarray): one program uses divide-and-conquer with $\mathsf{combine} = \max$; the other uses Kadane's algorithm (a linear fold). Since max is associative and commutative, `D17_tree_to_linear_fold` fires, flattening the tree fold. The remaining mismatch is resolved by the $_\mathsf{axiom_fold}$ normalizer (FOLD_sum / FOLD_prod rules).

22.4 D18: Greedy ↔ DP

When the problem has the *matroid property* (greedy choice property + optimal substructure), greedy and DP produce the same optimal solution.

This is the hardest dichotomy to verify automatically because it requires proving the matroid property, which is problem-specific. However, for the benchmark pairs that exhibit it (eq47: stock trading), the shared specification axiom suffices.

22.5 D19: Sort-Then-Scan ↔ Sweep

Sorting the input then scanning linearly is equivalent to processing events in a sweep line. Both produce the same result because:

1. Sorting produces a canonical order.
2. Both algorithms process elements in this order.
3. The processing logic is the same (just expressed differently).

Theorem 22.4 (D19: Sort Absorption)**.** *If $\oplus$ is commutative and associative, sorting is irrelevant:*

$$\mathsf{D19} : \mathsf{Path}(\mathsf{fold}(\oplus,\ e,\ \mathsf{sorted}(\mathsf{xs})),\ \mathsf{fold}(\oplus,\ e,\ \mathsf{xs}))$$

More generally, when a fold's body depends only on the multiset *of elements (not their order), pre-sorting is a no-op up to path equality.*

Proof. Since $\mathsf{sorted}(\mathsf{xs})$ is a permutation of xs, the two folds differ only in accumulation order. For a commutative and associative $\oplus$, fold is permutation-invariant: if σ is any permutation, then $\mathsf{fold}(\oplus, e, \sigma(\mathsf{xs})) = \mathsf{fold}(\oplus, e, \mathsf{xs})$ (by induction on $|\mathsf{xs}|$, using commutativity to swap adjacent elements and associativity to re-bracket).

This interacts with the quotient type axioms: the `_axiom_quotient` rule absorbs $\mathsf{Q}[\mathsf{perm}](\mathsf{sorted}(\mathsf{xs})) \to \mathsf{sorted}(\mathsf{xs})$ and $\mathsf{sorted}(\mathsf{Q}[\mathsf{perm}](\mathsf{xs})) \to \mathsf{sorted}(\mathsf{xs})$, ensuring that compositions of sorting and permutation-quotienting reduce canonically. □

Z3 encoding: The $\mathsf{py_sorted}$ shared function symbol with the idempotence axiom ($\mathsf{sorted}(\mathsf{sorted}(\mathsf{xs})) = \mathsf{sorted}(\mathsf{xs})$) ensures that both approaches produce the same sorted order. Z3's arithmetic solver verifies the commutativity/associativity side conditions.

Implementation mapping. The axiom `_axiom_d19_sort_scan` pattern-matches on $\mathsf{OFold}(\mathsf{op}, _, \mathsf{OOp}(\text{‘sor}$ when op passes `_is_commutative_op`, the outer sorted is stripped, yielding $\mathsf{OFold}(\mathsf{op}, _, \mathsf{xs})$ with label `D19_sort_irrelevant`.

Benchmark example. Pair **eq37** (two-sum): one program sorts the array then uses two pointers; the other uses a hash map directly. Both compute the same result because the two-pointer scan over the sorted array visits each pair exactly once, and the hash map achieves the same set of lookups in insertion order. The sort-then-scan version's sorted is absorbed by D19 when the outer fold is addition (commutative).

22.6 D20: Different Algorithms for Same Specification

The catch-all dichotomy: two genuinely different algorithms that satisfy the same specification. Examples:

- Graham scan vs. monotone chain (convex hull).
- Tarjan vs. Kosaraju (SCC).
- Matrix exponentiation vs. fast doubling (Fibonacci).
- Multiplicative vs. Pascal (binomial coefficients).

Theorem 22.5 (D20: Specification Uniqueness (HIT Truncation))**.** *If specification S is propositional (at most one valid output per input) and both programs satisfy S, they are equivalent:*

$$\mathsf{D20} : \prod_{f,g:A\to B} \left(\prod_{x:A} S(x, f(x)) \right) \to \left(\prod_{x:A} S(x, g(x)) \right) \to \mathsf{Path}_{A\to B}(f, g)$$

Proof. Fix $x : A$. Both $(f(x), \mathsf{pf}_f(x))$ and $(g(x), \mathsf{pf}_g(x))$ inhabit $\Sigma_{y:B}\, S(x, y)$. Since S is deterministic, this Σ-type is propositional (a (-1)-type), so there is a unique path between any two inhabitants. Projecting onto the first component yields $\mathsf{Path}_B(f(x), g(x))$. By function extensionality (Theorem 2.10), $f \equiv g$. □

The specification library. The function `_identify_spec` in `path_search.py` recognizes the following deterministic specifications from OTerm structure:

Spec name	Pattern	Uniqueness argument
`factorial`	$\mathsf{fold}(\times, 1, \mathsf{range}(1, n{+}1))$	$n!$ is unique for each $n \in \mathbb{N}$
`fibonacci_like`	OFix with $\mathsf{add} + \mathsf{sub}$	Recurrence $F_n = F_{n-1} + F_{n-2}$ has unique solution
`sorted`	OOp('sorted', ...)	Total order $\Rightarrow$ unique sorted permutation
`range_sum`	$\mathsf{fold}(+, 0, \mathsf{range}(\ldots))$	$\sum_{i=0}^{n-1} i$ is unique
`binomial`	$\mathsf{math.comb}(n, k)$	$\binom{n}{k}$ is unique for each (n, k)

When `_identify_spec` recognizes a term, `_axiom_d20_spec_unify` replaces it with $\mathsf{OAbstract}(\mathsf{spec_name}, \mathsf{in}$ Two different algorithms that both reduce to the same $\mathsf{OAbstract}$ are then connected by refl.

Soundness of specification discovery. The key soundness obligation is: *does the recognized OTerm actually compute the claimed specification?* For each entry in the table above, the proof is a straightforward verification that the fold/fixpoint computes the standard mathematical function. For instance, $\mathsf{fold}(\times, 1, \mathsf{range}(1, n{+}1)) = \prod_{i=1}^{n} i = n!$ by definition of the factorial. These are one-time proofs that validate the pattern-matching rules.

Implementation mapping. The axiom `_axiom_d20_spec_unify` calls `_identify_spec(term)`. If a spec is recognized, it emits $\mathsf{OAbstract}(\mathsf{spec}, \mathsf{inputs})$ with label `D20_spec_discovery`. The main path search (`search_path`, Strategy B at line 819) also calls `_identify_spec` on both input terms directly: if both return the same spec name with matching inputs, the path is found immediately without BFS.

Benchmark examples.

- **eq04** (factorial): iterative $\mathsf{fold}(\times, 1, \mathsf{range})$ vs. recursive $n \cdot f(n{-}1)$. Both identify as `factorial` $\Rightarrow$ $\mathsf{OAbstract}$('factorial', $(n,)$).
- **eq06** (Fibonacci): matrix exponentiation vs. iterative two-variable accumulation. Both identify as `fibonacci_like`.
- **eq48** (binomial): multiplicative formula $\prod_{i=0}^{k-1} \frac{n-i}{i+1}$ vs. Pascal's triangle DP. Both identify as `binomial` $\Rightarrow$ $\mathsf{OAbstract}$('binomial', (n, k)).

This is the most important dichotomy: it is the *completeness escape hatch* for the entire system. When no syntactic or algebraic path exists, specification identification can still close the gap, provided the specification is deterministic and appears in the library. Chapter 24 develops the full theory.

Chapter 23

Language Feature Dichotomies (D21–D24)

23.1 D21: `isinstance` ↔ Dispatch Table

Paradigm A — isinstance chain:

```
def f(x):
    if isinstance(x, int):    return x * 2
    elif isinstance(x, str):  return x + x
    elif isinstance(x, list): return x + x
    else:                     return None
```

Paradigm B — dispatch dict:

```
def g(x):
    dispatch = {int: lambda v: v*2, str: lambda v: v+v,
                list: lambda v: v+v}
    handler = dispatch.get(type(x))
    return handler(x) if handler else None
```

Compilation of A. Each `isinstance(x, T)` compiles to $\mathsf{BoolObj}(\mathsf{tag}(p_0) = \tau_T)$ (fiber projection). The section extractor produces one section per branch:

$$(\mathsf{tag}(p_0) = \mathsf{TInt},\ \mathsf{binop}(\mathsf{MUL}, p_0, \mathsf{IntObj}(2))) \tag{23.1}$$

$$(\mathsf{tag}(p_0) = \mathsf{TStr},\ \mathsf{binop}(\mathsf{ADD}, p_0, p_0)) \tag{23.2}$$

$$(\mathsf{tag}(p_0) = \mathsf{TPair},\ \mathsf{binop}(\mathsf{ADD}, p_0, p_0)) \tag{23.3}$$

$$(\text{else},\ \mathsf{NoneObj}) \tag{23.4}$$

Compilation of B. The dispatch dict is compiled by inlining each lambda (D2). The `type(x)` call maps to $\mathsf{tag}(p_0)$. The `dict.get` + conditional produces the same If-chain. Result: identical sections.

Path: refl after D2 inlining + fiber projection.

Automatic inference. The compiler recognizes `isinstance` as fiber projection (built into compilation rule E8). Dict-based dispatch compiles via D2 (lambda inlining) + shared symbol for `type`. Both produce the same tag-based If-chain.

23.2 D22: `try/except` ↔ Conditional

Paradigm A — exception-based:

```
def f(x, y):
    try:
        return x // y
    except ZeroDivisionError:
        return 0
```

Paradigm B — guard-based:

```
def g(x, y):
    if y == 0:
        return 0
    return x // y
```

Compilation of A. The try/except wraps the body in a handler context. The $\mathsf{binop}(\mathsf{FLOORDIV}, p_0, p_1)$ returns Bottom when $\mathsf{ival}(p_1) = 0$. The except handler catches Bottom and returns $\mathsf{IntObj}(0)$.

The section extractor produces: $(\top,\ \mathsf{If}(\mathsf{is_Bottom}(\mathsf{binop}(\mathsf{FLOORDIV}, p_0, p_1)),\ \mathsf{IntObj}(0),\ \mathsf{binop}(\mathsf{FLOORDIV},$

Compilation of B. The guard $y == 0$ compiles to $\mathsf{ival}(p_1) = 0$. Sections: $(\mathsf{ival}(p_1) = 0,\ \mathsf{IntObj}(0))$ and $(\mathsf{ival}(p_1) \neq 0,\ \mathsf{binop}(\mathsf{FLOORDIV}, p_0, p_1))$.

Path. On the Int×Int fiber: when $\mathsf{ival}(p_1) = 0$, FLOORDIV returns Bottom, so the If in A returns $\mathsf{IntObj}(0)$ — matching B's first section. When $\mathsf{ival}(p_1) \neq 0$, FLOORDIV succeeds, the If returns the result — matching B's second section. Z3 proves UNSAT on the disagreement query.

Theorem 23.1 (D22: Effect Elimination)**.** *Let* $\mathsf{body} : A \to B_\perp$ *be a partial function (returning* $\perp$ *on error) and let* $G : A \to \mathbb{B}$ *be its* definedness guard *(*$G(x) \Leftrightarrow \mathsf{body}(x) \neq \perp$*). Then:*

$$\mathsf{D22} : \mathsf{Path}(\mathsf{OCatch}(\mathsf{body}(x),\ d),\ \mathsf{OCase}(G(x),\ \mathsf{body}(x),\ d))$$

where d *is the default value returned on error.*

Proof. Case split on $G(x)$:

- If $G(x) = \mathsf{true}$: $\mathsf{body}(x) \neq \perp$, so OCatch returns $\mathsf{body}(x)$ (no exception). $\mathsf{OCase}(\mathsf{true}, \mathsf{body}(x), d) = \mathsf{body}(x)$. Equal.
- If $G(x) = \mathsf{false}$: $\mathsf{body}(x) = \perp$, so OCatch catches and returns d. $\mathsf{OCase}(\mathsf{false}, \mathsf{body}(x), d) = d$. Equal.

Both cases match, so Z3 proves UNSAT on the disagreement formula $\exists x.\ \mathsf{OCatch}(\ldots) \neq \mathsf{OCase}(\ldots)$. □

Implementation mapping. The axiom `_axiom_d22_effect_elim` implements both directions of the $\mathsf{OCatch} \leftrightarrow \mathsf{OCase}$ rewrite:

- *Forward* (`D22_catch_to_case`): given $\mathsf{OCatch}(\mathsf{body}, d)$, call `_extract_exception_guard(body)` to compute G. If G is decidable (returns non-None), emit $\mathsf{OCase}(G, \mathsf{body}, d)$.
- *Backward* (`D22_case_to_catch`): given $\mathsf{OCase}(G, b, d)$ where G passes `_is_exception_guard`, emit $\mathsf{OCatch}(b, d)$.

Automatic inference. The compiler models try/except as Bottom-catching. The $\mathsf{is_Bottom}$ check on the try body is equivalent to the guard condition that would cause the exception. Z3's built-in reasoning about the FLOORDIV RecFunction (which returns Bottom on zero divisor) closes the gap automatically.

Benchmark example. Pair **eq25** (safe division): the exception-based version wraps `x // y` in try/except; the guard-based version checks `y == 0` first. The `OCatch` compiles with an $\mathsf{is_Bottom}$ test that Z3 proves equivalent to $\mathsf{ival}(p_1) = 0$, closing the path in one step.

23.3 D23: Sorted-Then-Process ↔ Direct Processing

Paradigm A — sort then scan:

```
def f(intervals):
    sorted_iv = sorted(intervals, key=lambda iv: iv[0])
    merged = [sorted_iv[0]]
    for s, e in sorted_iv[1:]:
        if s <= merged[-1][1]:
            merged[-1] = (merged[-1][0], max(merged[-1][1], e))
        else:
            merged.append((s, e))
    return merged
```

Paradigm B — event sweep:

```
def g(intervals):
    events = []
    for s, e in intervals:
        events.append((s, 0))    # open
        events.append((e, 1))    # close
    events.sort()
    # ... sweep logic producing same merged intervals ...
```

Path. Both programs produce the same merged intervals. The key path constructor: py_sorted is shared, and the sorted axiom S5 (permutation invariance) ensures that sorting the input in different ways produces compatible orderings. Both programs' outputs depend only on the *multiset* of interval endpoints, not their original order.

Automatic inference. The compiler detects py_sorted (shared symbol) in both programs. The sorted permutation-invariance axiom (S5) allows Z3 to unify the two sorted sequences. The remaining logic (scan vs. sweep) produces the same list fold over the sorted data.

23.4 D24: Lambda ↔ Named Function

Paradigm A: `f = lambda x: x * 2; return f(n)`

Paradigm B: `def f(x): return x * 2; return f(n)`

Compilation. Both compile identically: the lambda stores (__func__, AST) in the environment, and the call site invokes D2 (inline_call), producing $\mathsf{binop}(\mathsf{MUL}, p_0, \mathsf{IntObj}(2))$.

Path: refl (definitional equality via η-expansion: $\lambda x.\, f(x) = f$).

Theorem 23.2 (D24: η-Reduction)**.** *For any function symbol f:*

$$\mathsf{D24} : \mathsf{Path}(\lambda x.\, f(x),\ f)$$

This is the standard η-rule of the simply typed λ-calculus, valid in all extensional type theories.

Proof. By function extensionality (Theorem 2.10). For any $x : A$, $(\lambda x.\, f(x))(x) \equiv f(x)$ by β-reduction. Since this holds for all x, funext gives $\mathsf{Path}_{A\to B}(\lambda x.\, f(x),\ f)$. □

Implementation mapping. The axiom `_axiom_d24_eta` pattern-matches on $\mathsf{OLam}([x],\ \mathsf{OOp}(\mathsf{name}, (\mathsf{OVar}$ a single-parameter lambda whose body is a single-argument application of the same variable. The rewrite contracts this to $\mathsf{OOp}(\mathsf{name}, ())$ with label `D24_eta_contract`.

This interacts with D2 (function inlining): after inlining, wrapper functions $\lambda x.\, g(x)$ where g was a named helper are reduced to g itself by η-contraction, unifying the lambda form with the named form.

Automatic inference. Both forms are stored as `__func__` AST nodes. The inlining mechanism (D2) treats them identically. No special logic needed — D24 provides the formal justification for why inlining is sound when the wrapper adds no computation.

Benchmark example. Pair **eq12** (map with function): one program uses `map(lambda x: f(x), xs)`, the other uses `map(f, xs)`. The η-contraction $\lambda x.\, f(x) \to f$ makes both arguments to `map` identical, yielding refl.

Chapter 24

Deep Theory: Algorithm A $\leftrightarrow$ Algorithm B

Dichotomy D20 is the deepest and most challenging: two *genuinely different algorithms* that compute the same mathematical function. Unlike D1–D19 and D21–D24, which can be resolved by syntactic transformation or algebraic axioms, D20 requires reasoning about the *mathematical specification* that both algorithms satisfy.

This chapter develops the full CTTT theory of specification-level equivalence, gives detailed proof sketches for every major algorithm-equivalence class in our 100-pair benchmark, establishes a formal boundary between structurally provable and fundamentally extensional equivalences, and connects the theory to matroids and the Yoneda embedding.

24.1 The Problem

Consider two programs:

- f: Graham scan for convex hull.
- g: Andrew's monotone chain for convex hull.

These share no structural similarity: f sorts by polar angle and uses a stack with cross-product tests; g sorts lexicographically and builds upper/lower hulls. No syntactic transformation, fold fusion, or commutativity axiom connects them.

Yet both compute the *same* mathematical function: the convex hull of a point set. The equivalence is a *mathematical theorem* ("the convex hull of a finite point set is unique"), not a syntactic identity.

24.2 Specifications as Propositional Types

In CTTT, a *specification* is a dependent type:

Definition 24.1 (Specification)**.** A specification for a function $h : A \to B$ is a type family $S : A \times B \to \mathsf{Type}$ such that:

1. **Satisfiability**: For every $x : A$, there exists $y : B$ with $S(x, y)$ inhabited.
2. **Functionality**: For every $x : A$, the type $\Sigma_{y:B} S(x, y)$ is *propositional* (at most one y satisfies $S(x, -)$).

A specification satisfying both conditions is called *deterministic*.

Theorem 24.2 (Deterministic Specification Uniqueness). *If S is a deterministic specification and both f and g satisfy S (i.e., $S(x, f(x))$ and $S(x, g(x))$ are inhabited for all x), then $f \equiv g$.*

Proof. By functionality, $\Sigma_{y:B} S(x, y)$ is propositional. Both $(f(x), \mathsf{pf}_f)$ and $(g(x), \mathsf{pf}_g)$ inhabit this type. By propositionality, they are connected by a path. The first projection of this path gives $\mathsf{Path}_B(f(x), g(x))$. By funext (Theorem 2.10), $f \equiv g$. □

This is the *fundamental theorem* for D20: if we can show both algorithms satisfy the same deterministic specification, they must compute the same function.

24.3 Examples of Deterministic Specifications

24.3.1 Convex Hull

Definition 24.3 (Convex Hull Specification). $S_{\mathrm{CH}}(P, H)$ holds iff:

1. $H \subseteq P$ (the hull vertices are input points).
2. H is in convex position (no point is inside the convex hull of the others).
3. Every $p \in P$ is inside or on the boundary of the convex hull of H.
4. H is listed in counterclockwise order starting from the leftmost-lowest point.

Proposition 24.4. *S_{CH} is deterministic.*

Proof. Uniqueness: the set of extreme points of a finite point set is unique. The counterclockwise ordering from the leftmost-lowest point is unique. □

Both Graham scan and monotone chain satisfy S_{CH}. By Theorem 24.2, they are equivalent.

24.3.2 Shortest Paths

Definition 24.5 (Shortest Path Specification). $S_{\mathrm{SP}}(G, s, D)$ holds iff for each vertex v:

$$D[v] = \min\{w(\pi) : \pi \text{ is a path from } s \text{ to } v \text{ in } G\}$$

where $w(\pi) = \sum_{e \in \pi} w(e)$ is the path weight.

Proposition 24.6. *S_{SP} is deterministic (assuming non-negative weights and well-defined minimum).*

Both Dijkstra's algorithm and Bellman–Ford satisfy S_{SP} (for non-negative weights). By Theorem 24.2, they produce the same distance array.

24.3.3 Strongly Connected Components

Definition 24.7 (SCC Specification). $S_{\mathrm{SCC}}(G, C)$ holds iff C is the unique partition of $V(G)$ into maximal strongly connected subsets, listed in reverse topological order of the DAG of SCCs.

Tarjan's algorithm and Kosaraju's algorithm both satisfy S_{SCC}, with the SCCs in the same canonical order.

24.3.4 Edit Distance

Definition 24.8 (Edit Distance Specification). $S_{\mathrm{ED}}(s_1, s_2, d)$ holds iff d equals the minimum number of single-character insertions, deletions, and substitutions to transform s_1 into s_2.

This is deterministic (the minimum exists and is unique). Wagner–Fischer and Hirschberg both satisfy it.

24.3.5 Binomial Coefficients

Definition 24.9 (Binomial Specification)**.** $S_{\binom{n}{k}}(n, k, v)$ holds iff $v = \frac{n!}{k!(n-k)!}$.

The multiplicative formula and Pascal's triangle both compute this. Uniqueness follows from the uniqueness of $n!$.

24.4 The Specification as a Higher Inductive Type

Each specification S defines a HIT T_S:

Definition 24.10 (Specification HIT)**.** Given a deterministic specification $S : A \times B \to \mathsf{Type}$, the *specification HIT* T_S has:

- **Point constructor**: for each program f satisfying S, a point $\iota(f) : T_S$.
- **Path constructor**: for any two programs f, g satisfying S:

$$\mathsf{det}_S : \mathsf{Path}_{T_S}(\iota(f), \iota(g))$$

- **Truncation**: T_S is 0-truncated (propositional).

The path constructor det_S is the computational content of Theorem 24.2: any two implementations of a deterministic specification are connected.

24.5 Z3 Encoding of Specifications

A specification $S(x, y)$ is encoded as a Z3 formula over the shared PyObj theory:

Listing 24.1: Specification encoding in Z3

```
# Specification: "y is the sorted permutation of x"
def sorted_spec(x, y, solver, T):
    S = T.S
    sf = T.shared_fn('sorted', 1)
    # y == sorted(x)
    solver.add(y == sf(x))
    # sorted(sorted(x)) == sorted(x) (idempotence)
    solver.add(ForAll([x], sf(sf(x)) == sf(x)))
```

Listing 24.2: Specification uniqueness check

```
# If both f and g satisfy sorted_spec, they are equal
x = Const('x', S)
solver.add(T.shared_fn('sorted', 1)(x) !=
           T.shared_fn('sorted', 1)(x))
# Trivially UNSAT: same shared function symbol
```

For more complex specifications (shortest paths, edit distance), the specification is encoded as a Z3 RecFunction satisfying the recurrence:

Listing 24.3: Edit distance specification

```
# d(i,j) = min(d(i-1,j)+1, d(i,j-1)+1,
#              d(i-1,j-1) + cost(i,j))
# with d(0,j)=j and d(i,0)=i
# Both DP and memoized versions satisfy this.
```

24.6 Factorial Variants: The D1 Path

The benchmark contains several factorial-adjacent equivalences (EQ-21 prime factorization, EQ-48 trailing zeroes in $n!$, EQ-40 binomial coefficients) whose proofs all pass through D1 (recursive ↔ iterative). We give the paradigmatic proof for factorial itself, since every variant follows the same template.

Definition 24.11 (Factorial Specification)**.** $S_!(n, v)$ holds iff $n \in \mathbb{N}$ with $n \geq 0$ and $v = \prod_{i=1}^{n} i$. When $n = 0$, $v = 1$ (empty product).

Proposition 24.12. *$S_!$ is deterministic: the product $\prod_{i=1}^{n} i$ is a unique integer for each n.*

Three implementations.

1. *Recursive*: $\mathsf{fix}[h]\big(\mathsf{case}(n = 0,\ 1,\ n \cdot h(n-1))\big)$.
2. *Iterative (for-loop)*: $\mathsf{fold}[\mathsf{mul}](1, \mathsf{range}(1, n{+}1))$.
3. *functools.reduce*: $\mathsf{reduce}(\mathsf{operator.mul},\ \mathsf{range}(1, n{+}1),\ 1)$.

Theorem 24.13 (Factorial Variant Equivalence)**.** *All three implementations satisfy $S_!$ and are therefore equal by Theorem 24.2. The structural path is:*

$$\mathsf{fix}[h](\ldots) \xrightarrow{D1_fix_to_fold} \mathsf{fold}[\mathsf{mul}](1, \mathsf{range}) \xleftarrow{D5_fold_universal} \mathsf{reduce}(\mathsf{mul}, \mathsf{range}, 1)$$

Proof sketch. **Step 1** (Recursive → Fold). The recursive body has the shape $h(n) = n \cdot h(n-1)$ with base case $h(0) = 1$. This is a *primitive recursion* on $\mathbb{N}$ with accumulation operator mul and initial value 1. By the D1 axiom (`D1_fix_to_fold` in `path_search.py`), we extract:

$$\mathsf{fix}[h](\ldots) \ \equiv\ \mathsf{fold}[\mathsf{mul}]\big(1, \mathsf{range}(1, n{+}1)\big)$$

The OTerm normalizer (phase 3: fold recognition) performs this automatically by detecting that the fix-point body is a simple accumulation over a decreasing integer argument.

Step 2 (reduce → Fold). The `functools.reduce` call with $(\mathsf{operator.mul}, \mathsf{range}(1, n{+}1), 1)$ is compiled by the denotational semantics (`denotation.py`, phase 4: HOF fusion) into $\mathsf{fold}[\mathsf{mul}](1, \mathsf{range}(1, n{+}1))$ — the same OTerm. The D5 axiom (`D5_fold_universal`) confirms fold universality: different surface representations of the same fold (explicit loop, `reduce`, comprehension) all compile to $\mathsf{fold}[\mathsf{op}](\mathsf{init}, \mathsf{coll})$.

Step 3 (Canonical form). All three implementations share the canonical OTerm:

$$\mathsf{fold}[\mathsf{mul}](1, \mathsf{range}(1, \mathsf{add}(n, 1)))$$

By `refl`, they are identical. Alternatively, spec identification (`_identify_spec` in `path_search.py`) recognizes $\mathsf{fold}[\mathsf{mul}](1, \mathsf{range}(\ldots))$ as the "factorial" specification, lifting all three to $@\mathsf{factorial}(n)$ via D20. □

Remark 24.14 (Application to EQ-40: Binomial Coefficients)**.** The multiplicative formula $\binom{n}{k} = \prod_{i=0}^{k-1} \frac{n-i}{i+1}$ compiles to $\mathsf{fold}[\mathsf{div_mul}](1, \mathsf{range}(k))$. Pascal's triangle compiles to a nested fold over rows. Both satisfy $S_{\binom{n}{k}}$. Since $\binom{n}{k}$ is unique, Theorem 24.2 gives the path. This is an instance of D1 (fold recognition) plus D20 (spec uniqueness), because the two folds have different recurrence structures that happen to compute the same arithmetic function.

24.7 Fibonacci Variants: D1 and D16 Paths

EQ-37 (matrix exponentiation vs. fast doubling for Fibonacci mod p) is one of the deepest structural equivalences in the benchmark. The two algorithms share no syntactic structure, yet both compute $F(n) \bmod p$.

Definition 24.15 (Fibonacci Specification)**.** $S_{\text{fib}}(n, v)$ holds iff $v = F(n)$ where F is the unique function satisfying $F(0) = 0$, $F(1) = 1$, $F(n) = F(n{-}1) + F(n{-}2)$ for $n \geq 2$.

Five implementations.

1. *Naive recursive*: $\mathsf{fix}[f]\big(\mathsf{case}(n \leq 1, n, f(n{-}1) + f(n{-}2))\big)$.
2. *Memoized recursive* (top-down DP): same recurrence, with a memo table wrapping the fix-point.
3. *Iterative* (bottom-up DP): $\mathsf{fold}[\mathsf{fib_step}]((0,1), \mathsf{range}(n))$ where $\mathsf{fib_step}((a,b), _) = (b, a{+}b)$.
4. *Matrix exponentiation* (EQ-37a): $\left(\begin{smallmatrix}1&1\\1&0\end{smallmatrix}\right)^n$ computed by repeated squaring; result is entry $[0][1]$.
5. *Fast doubling* (EQ-37b): uses the identities $F(2k) = F(k)[2F(k{+}1){-}F(k)]$ and $F(2k{+}1) = F(k)^2 + F(k{+}1)^2$.

Theorem 24.16 (Fibonacci Variant Equivalence)**.** *All five implementations satisfy S_{fib}. The proof paths are:*

- Naive ↔ Memoized*: D16 (memo ↔ DP). Both share the same recurrence $f(n) = f(n{-}1){+}f(n{-}2)$. Memoization only changes* evaluation order*, not the mathematical function. The D16 axiom (`D16_recurrence_normalize`) normalizes both to the same canonical fix-point.*
- Memoized ↔ Iterative*: D1 (fix → fold). The fix-point with memo has a linear dependency structure (each $f(k)$ depends only on $f(k{-}1)$ and $f(k{-}2)$), so it compiles to a fold with state (a, b).*
- Iterative ↔ Matrix*: D20 (spec uniqueness). The fold computes $F(n)$ by the recurrence; matrix exponentiation computes $F(n)$ by the identity $\left(\begin{smallmatrix}1&1\\1&0\end{smallmatrix}\right)^n = \left(\begin{smallmatrix}F(n+1)&F(n)\\F(n)&F(n-1)\end{smallmatrix}\right)$. Both satisfy S_{fib}; uniqueness of $F(n)$ gives the path.*
- Matrix ↔ Fast doubling*: D20 (spec uniqueness). The fast-doubling identities are consequences of the matrix identity above. Both satisfy S_{fib}.*

Proof sketch for Matrix ↔ Fast Doubling. The key is that both algorithms implement efficient computation of the same recurrence by exploiting the multiplicative structure of $\mathbb{N}$. Matrix exponentiation uses $M^{2k} = (M^k)^2$ and $M^{2k+1} = M \cdot (M^k)^2$. Fast doubling derives closed-form identities from this matrix equation. Define $M := \left(\begin{smallmatrix}1&1\\1&0\end{smallmatrix}\right)$. Then:

$$M^k = \begin{pmatrix} F(k{+}1) & F(k) \\ F(k) & F(k{-}1) \end{pmatrix}$$

Squaring: $M^{2k} = (M^k)^2$ gives, by entry-wise expansion:

$$F(2k) = F(k)\big[2F(k{+}1) - F(k)\big]$$
$$F(2k{+}1) = F(k)^2 + F(k{+}1)^2$$

These are exactly the fast-doubling identities. Both compute $F(n)$ in $O(\log n)$ multiplications. Since S_{fib} is deterministic, the path exists by Theorem 24.2.

In OTerm form, both compile to $@\mathsf{fibonacci_like}(n)$ via `_identify_spec`, which recognizes any fix-point whose body contains additions and subtractions in the Fibonacci pattern. □

24.8 Sorting-Then-Processing: The D19 Path

Many benchmark pairs share the pattern: *sort the data, then process it.* Different sort strategies (key function, reverse flag, built-in `sorted` vs. manual merge sort) with the same post-processing produce identical outputs.

Definition 24.17 (Sort-Process Specification)**.** Let $\sigma : \mathcal{P}(A) \to \mathsf{List}(A)$ be a sorting function and $\phi : \mathsf{List}(A) \to B$ a processing function. The specification is $S(x, y)$ iff $y = \phi(\sigma(x))$.

When σ is determined by a total order on A (possibly with a key function), S is deterministic: the sorted order is unique and ϕ is a function.

Benchmark instances.

- **EQ-01** (flatten + sort + unique): both implementations recursively flatten, then apply $\mathsf{sorted}(\mathsf{set}(\ldots))$. The structural path is D4 (comprehension fusion) + D19 (sort canonicalization): different flattening strategies (recursive generator vs. iterative stack) both produce the same multiset, and $\mathsf{sorted} \circ \mathsf{set}$ is a canonical representative of the equivalence class.
- **EQ-02** (word frequency histogram): both produce $\mathsf{sorted}(\mathsf{Counter}(\mathsf{words}).\mathsf{items}())$. The D13 axiom (Counter ↔ manual fold) and D19 (sort absorption) give the path.
- **EQ-25** (merge overlapping intervals): both sort intervals by start time, then sweep. The path is D19: $\mathsf{fold}[\mathsf{merge}](\mathsf{init}, \mathsf{sorted}(\mathsf{intervals}))$ is the canonical form for both.
- **EQ-38** (group anagrams): one uses $\mathsf{sorted}(\mathsf{word})$ as grouping key; the other uses a prime-product hash. Both produce the same partition of strings into anagram classes. The path requires D20 (spec uniqueness): the anagram-partition specification is deterministic when the output is canonicalized by sorting both within groups and across groups.

Proposition 24.18 (Sort Absorption)**.** *In the OTerm algebra,* $\mathsf{sorted}(\mathsf{sorted}(x)) \equiv \mathsf{sorted}(x)$ *(idempotence). Furthermore, for any commutative and associative fold operation* $\oplus$*:*

$$\mathsf{fold}[\oplus](\mathsf{init}, \mathsf{sorted}(x)) \equiv \mathsf{fold}[\oplus](\mathsf{init}, x)$$

because $\oplus$ *is order-invariant. The D19 axiom (`D19_sort_irrelevant`) implements this.*

24.9 GCD Variants: The D1 Path with Recurrence Matching

Definition 24.19 (GCD Specification)**.** $S_{\text{gcd}}(a, b, d)$ holds iff d is the greatest common divisor: $d \mid a$, $d \mid b$, and for all c with $c \mid a$ and $c \mid b$, $c \mid d$.

Proposition 24.20. S_{gcd} *is deterministic:* $\gcd(a, b)$ *is unique for* $a, b > 0$.

Two implementations.

1. *Euclidean* (division-based): $\mathsf{fix}[g]\big(\mathsf{case}(b = 0,\ a,\ g(b, a \bmod b))\big)$.
2. *Subtraction-based*: $\mathsf{fix}[g]\big(\mathsf{case}(a = b,\ a,\ \mathsf{case}(a > b,\ g(a{-}b, b),\ g(a, b{-}a)))\big)$.

Theorem 24.21 (GCD Variant Equivalence)**.** *Both implementations satisfy* S_{gcd}*. The proof uses D1 (fix-point equivalence) with* recurrence matching*: both compute the same function despite different reduction strategies.*

Proof sketch. The Euclidean algorithm reduces $(a, b) \mapsto (b, a \bmod b)$; the subtraction algorithm reduces $(a, b) \mapsto (a - b, b)$ or $(a, b - a)$. Both terminate (the sum $a + b$ strictly decreases in the Euclidean case; the larger argument strictly decreases in the subtraction case). At termination, both return the last nonzero value, which equals $\gcd(a, b)$ by the fundamental property $\gcd(a, b) = \gcd(b, a \bmod b) = \gcd(a - b, b)$.

In OTerm form, both compile to fix nodes. The D1 path does not directly connect them (the fix-point bodies have different structure). Instead, D20 applies: both satisfy the deterministic specification S_{gcd}, so by Theorem 24.2, they are equal.

Note that the subtraction-based algorithm is strictly less efficient ($O(\max(a, b))$ vs. $O(\log \min(a, b))$), but efficiency is invisible in the extensional type theory: the path connects their *values*, not their computation traces. □

24.10 Algorithm Equivalence via Yoneda

The Yoneda lemma provides a deep perspective on specification-level equivalence that goes beyond the ad hoc "identify a specification" approach.

Theorem 24.22 (Yoneda Lemma)**.** *For a presheaf $F : \mathcal{C}^{\text{op}} \to \mathbf{Set}$ and an object $A \in \mathcal{C}$:*

$$\text{Nat}(\text{Hom}(A, -), F) \cong F(A)$$

Interpretation for programs. Let $\mathcal{C}$ be the category whose objects are Python types and whose morphisms are Python functions. A program $f : A \to B$ defines a representable presheaf $\text{Hom}(B, -)$. The Yoneda lemma says: f is *uniquely determined* by how it interacts with *all* observations $t : B \to C$:

$$f \equiv g \iff \forall C, \forall t : B \to C.\ t \circ f \equiv t \circ g$$

This is the *observational equivalence* formulation: programs are characterized by their specifications, not their implementations. Two programs are the same iff no downstream consumer can tell them apart.

Corollary 24.23 (Yoneda Embedding for Programs)**.** *The Yoneda embedding $\mathsf{y} : \mathcal{C} \to [\mathcal{C}^{\text{op}}, \mathbf{Set}]$ is full and faithful. Applied to programs: two implementations $f, g : A \to B$ are equal in $\mathcal{C}$ iff their Yoneda images are naturally isomorphic. That is, a program* is *its specification.*

Z3 implementation. In Z3, observations are approximated by the shared function symbols $t \in \{$`sorted`, `len`, `sum`, `hash`, `str`, ...$\}$. Two programs f, g are observationally equivalent iff for every shared symbol t:

$$\forall p.\ t([\![f]\!](p)) = t([\![g]\!](p))$$

This is checked as a single Z3 query: $\exists p.\ \bigvee_t t([\![f]\!](p)) \neq t([\![g]\!](p))$. If UNSAT, the programs are Yoneda-equivalent up to the finite observation set.

Remark 24.24 (Finite Approximation)**.** The shared function symbols form a *finite* approximation of the full Yoneda embedding. Soundness: if the Z3 query returns UNSAT, the programs agree on all modeled observations. Completeness gap: there may exist an observation not modeled by any shared symbol. This gap is narrowed by adding more shared symbols (the theory library grows over time) and is bridged entirely for specifications expressible in the Z3 theory.

24.11 The Matroid/Greedy Theorem and D20

A fundamental question in algorithm theory: *when does a greedy algorithm compute an optimal solution?* The classical answer involves matroids. We now connect this to D20 and the CTTT framework.

Definition 24.25 (Matroid)**.** A *matroid* $M = (E, \mathcal{I})$ consists of a finite ground set E and a family $\mathcal{I} \subseteq 2^E$ of *independent sets* satisfying:

1. $\emptyset \in \mathcal{I}$ (the empty set is independent).
2. If $A \in \mathcal{I}$ and $B \subseteq A$, then $B \in \mathcal{I}$ (hereditary property).
3. If $A, B \in \mathcal{I}$ and $|A| < |B|$, then there exists $e \in B \setminus A$ with $A \cup \{e\} \in \mathcal{I}$ (exchange property).

Theorem 24.26 (Rado–Edmonds Greedy Theorem)**.** *A greedy algorithm (iteratively adding the cheapest feasible element) solves the weighted optimization problem* $\max_{I \in \mathcal{I}} \sum_{e \in I} w(e)$ *if and only if* $\mathcal{I}$ *forms a matroid.*

Connection to D20. The matroid theorem provides a *metatheorem* for equivalence: if the feasibility system forms a matroid, then the greedy algorithm and any DP/brute-force algorithm computing the optimal solution satisfy the same deterministic specification $S_{\text{opt}}(x, v) \coloneqq v = \max_{I \in \mathcal{I}} \sum_{e \in I} w(e)$.

Corollary 24.27 (Greedy ↔ DP on Matroids)**.** *If the optimization problem has matroid structure, then the greedy algorithm and the DP algorithm are D20-equivalent: both satisfy the deterministic specification* S_{opt}*, so Theorem 24.2 provides the path.*

Benchmark instance: EQ-47 (stock profit with k transactions). The DP algorithm (`eq47a`) fills a 2D table; the alternative (`eq47b`) reduces to a greedy sum when $k \geq n/2$ and falls back to DP otherwise. The specification is:

$$S_{\text{profit}}(\mathsf{prices}, k, v) \iff v = \max \left\{ \sum_{j=1}^{m} (\mathsf{prices}[s_j] - \mathsf{prices}[b_j]) : m \leq k,\ b_1 < s_1 < b_2 < s_2 < \cdots \right\}$$

The maximum is unique (it is a real number), so S_{profit} is deterministic. When $k \geq n/2$, every profitable adjacent-day trade is taken — this is the greedy solution on the "pick non-overlapping profit intervals" matroid. The D20 path connects the greedy branch to the DP solution.

The D18 axiom (`D18_greedy` in `path_search.py`) is the structural counterpart: it fires when the path search detects that a greedy algorithm and a DP algorithm operate on the same optimization problem with matroid structure.

24.12 Constructive Witness for D20

For each D20 pair, the equivalence proof has the structure:

1. **Identify the specification** S that both programs satisfy.
2. **Prove** S **is deterministic**: show that $\Sigma_y S(x, y)$ is propositional for all x.
3. **Construct the path**: apply Theorem 24.2 to obtain $f \equiv g$.

Step 1 is done by the Z3 semantic algebra: the shared function symbols and axioms encode the specification implicitly. When both programs use the same shared functions with the same axioms, they inhabit the same specification. The `_identify_spec` function in `path_search.py` automates this for common specifications (factorial, Fibonacci, sorted, binomial).

Step 2 is done by the Z3 axioms: the idempotence, commutativity, and associativity axioms ensure that specifications involving sorting, folding, etc., are deterministic.

Step 3 is done by Theorem 24.2 (which is a metatheorem of CTTT, not something Z3 needs to prove).

24.13 Limits of D20

D20 cannot resolve all algorithm-level equivalences. It fails when:

- The specification is not deterministic (e.g., "find any spanning tree" — there may be multiple valid outputs).
- The specification cannot be expressed in the Z3 theory (e.g., requires reasoning about infinite structures).
- The programs satisfy different specifications that happen to agree on all finite inputs but differ on edge cases.

We now develop the full theory for non-deterministic specifications.

24.14 Quotient Types for Non-Deterministic Specifications

A *non-deterministic* specification allows multiple valid outputs for a single input. Examples:

- "Find *any* spanning tree of the graph."
- "Return *a* topological ordering" (there may be many).
- "Find *a* shortest path" (there may be ties).
- "Return *any* element satisfying the predicate."

For such specifications, Theorem 24.2 does not apply: two programs can both satisfy the specification yet produce different (equally valid) outputs. We need a richer type-theoretic structure.

Definition 24.28 (Quotient Type)**.** Given a type B and an equivalence relation $\sim$ on B, the *quotient type* $B/\!\sim$ is the HIT with:

- Point constructor: $q : B \to B/\!\sim$ (the quotient map).
- Path constructor: for all $b_1, b_2 : B$ with $b_1 \sim b_2$:

$$\mathsf{quot} : \mathsf{Path}_{B/\sim}(q(b_1), q(b_2))$$

- Truncation: $B/\!\sim$ is a set (0-truncated).

Definition 24.29 (Non-Deterministic Specification via Quotient)**.** A non-deterministic specification $S : A \to \mathcal{P}(B)$ (mapping each input to a *set* of valid outputs) defines an equivalence relation on outputs:

$$b_1 \sim_{S,x} b_2 \iff b_1 \in S(x) \wedge b_2 \in S(x)$$

(two outputs are equivalent if they are both valid for input x).

Two programs f, g are S-equivalent iff for all x:

$$q(f(x)) = q(g(x)) \quad \text{in } B/\!\sim_{S,x}$$

That is, their outputs are in the same equivalence class — both are valid responses to the specification.

Theorem 24.30 (Non-Deterministic Specification Equivalence)**.** *Two programs $f, g : A \to B$ are S-equivalent iff for all $x : A$:*

$$f(x) \in S(x) \ \wedge \ g(x) \in S(x)$$

The cubical path witnessing this equivalence is:

$$p_x = \mathsf{quot}(f(x), g(x)) : \mathsf{Path}_{B/\sim_{S,x}}(q(f(x)), q(g(x)))$$

which exists because both $f(x)$ and $g(x)$ are in $S(x)$, so $f(x) \sim_{S,x} g(x)$.

Proof. By Definition 24.28, if $f(x) \sim_{S,x} g(x)$ (both are valid), then **quot** provides a path in $B/\sim_{S,x}$. By function extensionality, the family $\lambda x.\, p_x$ gives:

$$p : \mathsf{Path}_{A \to B/\sim_S}(q \circ f,\ q \circ g)$$

□

24.14.1 Z3 Encoding of Quotient Types

The quotient is encoded in Z3 by replacing the inequality query. Instead of asking "$\exists x.\, f(x) \neq g(x)$", we ask:

$$\exists x.\ f(x) \notin S(x) \ \vee \ g(x) \notin S(x)$$

If UNSAT: both programs satisfy S on all inputs, so they are S-equivalent (even if $f(x) \neq g(x)$ for some x).

If SAT: at least one program violates S — the counterexample identifies which program fails and on what input.

Listing 24.4: Non-deterministic spec check in Z3

```
def check_nondet_equiv(f_term, g_term, spec_pred, params, solver):
    """Check S-equivalence: both f and g satisfy spec S.

    spec_pred(x, y) is a Z3 BoolRef that is True iff y in S(x).
    """
    x = params[0]  # symbolic input

    # Check: does f violate S?
    solver.push()
    solver.add(Not(spec_pred(x, f_term)))
    f_violates = solver.check()
    solver.pop()

    # Check: does g violate S?
    solver.push()
    solver.add(Not(spec_pred(x, g_term)))
    g_violates = solver.check()
    solver.pop()

    if f_violates == unsat and g_violates == unsat:
        return EQUIVALENT  # both satisfy S everywhere
    elif f_violates == sat:
        return BUG_IN_F, solver.model()
    elif g_violates == sat:
        return BUG_IN_G, solver.model()
    else:
        return UNKNOWN
```

24.14.2 Example: Spanning Tree

Listing 24.5: Two spanning tree algorithms

```
def kruskal(graph):
    """Kruskal's MST (sorted by weight, union-find)."""
    edges = sorted(graph.edges, key=lambda e: e.weight)
    # ... union-find MST construction ...
    return tree

def prim(graph):
    """Prim's MST (priority queue from start vertex)."""
    # ... heap-based MST construction ...
    return tree
```

The specification $S_{\text{MST}}(G, T)$ is:

1. T is a spanning tree of G (connected, acyclic, spans all vertices).
2. T has minimum total weight among all spanning trees.

When edge weights are distinct, the MST is unique and S is deterministic. When weights have ties, multiple valid MSTs exist.

In the non-deterministic case:

- $\mathsf{kruskal}(G)$ and $\mathsf{prim}(G)$ may produce *different* MSTs (due to tie-breaking).
- But both satisfy S_{MST}.
- In the quotient $B/{\sim_{S_{\text{MST}}}}$, both outputs are in the same equivalence class.
- The Z3 check verifies that both satisfy S (UNSAT on both negations), concluding S-equivalence.

24.14.3 Example: Topological Sort

The specification "return a topological ordering of the DAG" is non-deterministic: a DAG with n vertices may have exponentially many valid orderings.

Two topological sort algorithms (Kahn's BFS vs. DFS-based) both satisfy the specification:

$$S_{\text{topo}}(G, \sigma) \iff \forall (u, v) \in E(G).\ \sigma^{-1}(u) < \sigma^{-1}(v)$$

(every edge goes from earlier to later in the ordering).

They may produce different orderings, but both are valid. In the quotient, they are equal.

If the specification is strengthened to "return the *lexicographically smallest* topological ordering", it becomes deterministic, and Theorem 24.2 applies directly.

24.15 Propositional Extensionality

An alternative to quotient types is *propositional extensionality*:

Axiom 24.31 (Propositional Extensionality)**.** *For propositions* $P, Q : \mathsf{Prop}$*:*

$$(P \leftrightarrow Q) \to (P = Q)$$

Two propositions with the same truth value are equal.

In cubical type theory, propositional extensionality is a consequence of univalence (Axiom 3.4).

Application to specifications: if the specification $S(x, y)$ is a proposition (either inhabited or empty, with no computational content), then propositional extensionality says:

$$S(x, f(x)) \leftrightarrow S(x, g(x)) \implies S(x, f(x)) = S(x, g(x))$$

So if both $f(x)$ and $g(x)$ satisfy S (both sides are $\top$), they are *equal as propositions*. This doesn't make $f(x)$ and $g(x)$ equal as *values*, but it makes their *validity status* equal.

Combined with the quotient construction: if validity is all we care about (not the specific output), propositional extensionality collapses the quotient to a single point.

24.16 Set-Level Truncation

The *set-level truncation* $\|B\|_0$ of a type B is the quotient that identifies all elements:

Definition 24.32 (Set Truncation)**.** $\|B\|_0$ is the HIT with:

- $|b| : \|B\|_0$ for each $b : B$.
- $\mathsf{trunc} : \prod_{x,y:\|B\|_0} \mathsf{Path}(x, y)$ (all elements are equal).

Applied to specifications: $\|S(x)\|_0$ is either empty (no valid output) or a single point (at least one valid output exists, and all valid outputs are identified).

Two programs f, g satisfying S produce elements $|f(x)|, |g(x)| \in \|S(x)\|_0$. By truncation, $|f(x)| = |g(x)|$.

Theorem 24.33 (Equivalence Modulo Non-Deterministic Spec)**.** *If S is a (possibly non-deterministic) specification and both f and g satisfy S for all inputs, then:*

$$q \circ f \equiv q \circ g : A \to \|B\|_0/{\sim_S}$$

where q is the quotient-then-truncate map.

Proof. For each x, both $f(x)$ and $g(x)$ are in $S(x)$. The quotient $B/{\sim_{S,x}}$ identifies them (they are in the same equivalence class). The truncation $\|B/{\sim_{S,x}}\|_0$ ensures this is a mere proposition. By propositional extensionality, $|q(f(x))| = |q(g(x))|$. By funext, $q \circ f = q \circ g$. □

24.17 Z3 Encoding of Truncated Specifications

In practice, the Z3 encoding of truncated specifications is simpler than the type theory suggests:

Listing 24.6: Truncated spec check

```
def check_truncated_equiv(f_term, g_term, spec_pred,
                          params, solver):
    """Check: both satisfy spec S.

    We do NOT check f(x) == g(x).
    We check f(x) in S(x) AND g(x) in S(x).
    If both hold, they are equivalent modulo S.
    """
    x = params[0]

    # Combined query: exists x where f or g violates S
    solver.push()
    solver.add(Or(
```

```
        Not(spec_pred(x, f_term)),
        Not(spec_pred(x, g_term))
    ))
    result = solver.check()
    solver.pop()

    if result == unsat:
        return S_EQUIVALENT  # both satisfy S everywhere
    else:
        m = solver.model()
        # Which one fails?
        # ... extract counterexample ...
        return S_INEQUIVALENT, m
```

24.17.1 Fiber-Wise Truncated Spec Checking

The cohomological decomposition applies to truncated specs:

1. For each type fiber U_τ: check that both f and g satisfy S when restricted to fiber τ.
2. Build the C^0 cochain: entry is 1 iff both satisfy S on that fiber.
3. Compute H^1: measures whether the local S-satisfactions are globally coherent.
4. $H^1 = 0 \implies$ globally S-equivalent.

The only change from deterministic spec checking is the local query: instead of "$f(x) = g(x)$" we check "$f(x) \in S(x) \wedge g(x) \in S(x)$". The cohomological gluing machinery is unchanged.

24.18 The "Hard 8" Analysis

Among the 50 equivalent pairs in the benchmark, most are resolved by structural path constructors (D1–D19, D21–D24). Eight pairs, however, resist *all* structural paths and require genuine specification-level reasoning (D20). We call these the "hard 8."

ID	Alg A	Alg B	Specification	Why hard
07	Graham scan	Monotone chain	Convex hull	Different invariants
08	Wagner–Fischer	Hirschberg	Edit distance	Space-time tradeoff
16	Tarjan SCC	Kosaraju SCC	Strongly connected components	Different traversals
29	deepcopy + sort	BFS copy + sort	Deep copy canonical form	Different copy strategies
33	Factoradic	itertools.perm	n-th permutation	Combinatorial identity
37	Matrix exp	Fast doubling	$F(n) \bmod p$	Algebraic identity
40	Multiplicative	Pascal's triangle	$\binom{n}{k}$	Different recurrences
47	DP	Greedy + DP	Max stock profit	Matroid reduction

24.18.1 Why Structural Paths Fail

For each hard pair, we explain why the 23 non-D20 path constructors cannot connect them:

1. **EQ-07 (Graham ↔ Monotone Chain).** Graham sorts by *polar angle* from a pivot and maintains a CCW stack. Monotone chain sorts *lexicographically* and builds two half-hulls. The sort keys differ, the loop invariants differ, and the intermediate data structures differ. No combination of D1 (fold/unfold), D4 (fusion), D5 (fold universality), or D19 (sort-process) connects them, because the *sort itself* is part of the algorithm's correctness argument, not just a preprocessing step.

2. **EQ-08 (Wagner–Fischer ↔ Hirschberg).** Both compute edit distance, but Hirschberg uses a divide-and-conquer strategy with $O(n)$ space. The D17 (divide-and-conquer ↔ iterative) axiom applies only when the combining operation is associative; here, the "combine" step is a nontrivial alignment merge.

3. **EQ-16 (Tarjan ↔ Kosaraju).** Tarjan uses a single DFS with a low-link stack. Kosaraju uses two DFS passes (one on the original graph, one on the transpose). The traversal structures are fundamentally different. D15 (BFS ↔ DFS) does not apply because the algorithms are not computing a fold over a traversal — they use the traversal structure itself to identify SCCs.

4. **EQ-37 (Matrix Exp ↔ Fast Doubling).** Addressed in §24.7. The matrix and scalar recurrences have different OTerm structures despite computing the same sequence.

5. **EQ-47 (DP ↔ Greedy+DP).** Addressed in §24.11. The greedy branch is not a structural simplification of the DP — it uses a fundamentally different algorithm justified by the matroid property.

24.18.2 The Common Pattern

All hard-8 pairs share a common feature: the two algorithms use *different mathematical insights* to solve the same problem. The structural paths (D1–D19, D21–D24) transform one algorithm into another by rewriting its implementation; the hard-8 pairs cannot be connected this way because the implementations encode genuinely different proofs of the same theorem.

In CTTT terms: the hard-8 pairs require paths in the *specification HIT* T_S, not paths in the *program HIT* PyComp. The path constructor det_S (Definition of the Specification HIT) is the only axiom that applies, and it requires showing that both algorithms satisfy the same deterministic specification — a mathematical argument, not a syntactic rewrite.

24.19 The Structural/Extensional Boundary

We now formalize the distinction between equivalences provable by structural rewriting and those requiring extensional (specification-level) reasoning.

Definition 24.34 (Structural Equivalence)**.** Two OTerms t_1, t_2 are *structurally equivalent*, written $t_1 \sim_{\mathsf{str}} t_2$, if there exists a path in the HIT PyComp using only the path constructors from D1–D19 and D21–D24 (the non-D20 axioms).

Formally, $\sim_{\mathsf{str}}$ is the equivalence relation generated by the rewrite rules of `path_search.py`: the axiom functions `_axiom_d1_fold_unfold` through `_axiom_d24_eta`, plus the algebraic axioms (commutativity, associativity, identity, involution), quotient axioms, fold axioms, and case axioms, applied under congruence.

Definition 24.35 (Extensional Equivalence)**.** Two programs $f, g : A \to B$ are *extensionally equivalent* if $\forall x : A.\ f(x) = g(x)$. In CTTT, this is $\mathsf{Path}_{A\to B}(f, g)$, which by funext is equivalent to $\prod_{x:A} \mathsf{Path}_B(f(x), g(x))$.

Theorem 24.36 (Structural/Extensional Boundary)**.** *Let $\mathcal{E}_{\mathsf{str}} := \{(f, g) : f \sim_{\mathsf{str}} g\}$ be the set of structurally equivalent program pairs, and $\mathcal{E}_{\mathsf{ext}} := \{(f, g) : \forall x.\ f(x) = g(x)\}$ the set of extensionally equivalent pairs. Then:*

1. *$\mathcal{E}_{\mathsf{str}} \subsetneq \mathcal{E}_{\mathsf{ext}}$ (structural equivalence is strictly weaker).*

2. *The hard-8 pairs witness the gap: each is in $\mathcal{E}_{\mathsf{ext}} \setminus \mathcal{E}_{\mathsf{str}}$.*

3. *D20 (specification uniqueness) bridges the gap: for any pair* $(f, g) \in \mathcal{E}_{\mathsf{ext}}$ *satisfying a* recognizable *deterministic specification* S*, the path* det_S *provides a proof of equivalence.*

4. *The D20 bridge is* incomplete*: there exist extensionally equivalent programs that satisfy no specification expressible in the Z3 theory.*

Proof sketch. **(1)** Every structural rewrite preserves extensional equality (each axiom is sound), so $\mathcal{E}_{\mathsf{str}} \subseteq \mathcal{E}_{\mathsf{ext}}$. Strictness follows from (2).

(2) For each hard-8 pair, we verify: (a) the pair is extensionally equivalent (both satisfy the same deterministic specification, confirmed by testing on the benchmark argument sets and by mathematical proof); and (b) the bidirectional BFS in `search_path` fails to find a path within depth 4 (empirically verified), and the HIT structural decomposition (`hit_path_equiv`) returns `None`.

(3) Given a deterministic specification S with both f and g satisfying S, Theorem 24.2 provides $\mathsf{Path}(f, g)$. The specification identification (`_identify_spec`) automates recognition for common specs.

(4) Incompleteness follows from the fact that Z3's theory (even with shared function symbols) cannot express all mathematical properties. For example, two programs computing the Ackermann function via different recursion schemes are extensionally equivalent but have no Z3-expressible specification connecting them (Z3 cannot reason about non-primitive-recursive functions). □

Remark 24.37 (The Three Strata)**.** The CTTT framework partitions program equivalences into three strata:

1. **Structural stratum** ($\sim_{\mathsf{str}}$): Proved by syntactic rewriting. Decidable (bounded BFS on finite axiom set). Covers ∼42 of the 50 EQ benchmark pairs.

2. **Specification stratum** (D20): Proved by recognizing a common deterministic specification. Semi-decidable (spec identification may fail, but if it succeeds, the proof is constructive). Covers the hard-8 pairs.

3. **Extensional stratum** (beyond D20): Proved only by mathematical argument outside the formal system. Not mechanizable in general (Rice's theorem). No benchmark pairs fall here, but pathological programs do.

24.20 The Full Benchmark Classification

We classify all 50 equivalent pairs from the hard benchmark suite by the proof technique required. The structural pairs are resolved by path constructors D1–D19 and D21–D24; the hard-8 pairs require D20.

ID	Problem	Path	Key Axiom
01	Flatten + sort + unique	D4, D19	Comprehension fusion, sort
02	Word frequency histogram	D13, D19	Counter ↔ fold
03	Arithmetic expression eval	D1	Fix ↔ fold
04	Register machine simulation	D1	Fix ↔ fold
05	Topological sort (lex-smallest)	D15	Traversal order
06	RLE encode/decode roundtrip	D1, D4	Fold + fusion
07	**Convex hull**	**D20**	**Spec uniqueness**
08	**Edit distance**	**D20**	**Spec uniqueness**
09	LRU cache simulation	D11	ADT refinement
10	Matrix multiplication	D4, D5	Fold fusion
11	Merge k sorted iterators	D5	Fold universality
12	Binary tree serialize/deserialize	D1	Fix ↔ fold
13	DAG path counting	D1, D16	Fix + memo/DP
14	Calculator (parse + eval)	D1, D2	Fix + inline
15	Longest increasing subseq	D16	Memo ↔ DP
16	**Strongly connected comp**	**D20**	**Spec uniqueness**
17	Power set generation	D1	Fix ↔ fold
18	JSON-like pretty printer	D1, D2	Fix + inline
19	0/1 Knapsack	D16	Memo ↔ DP
20	Structural hashing (SHA-256)	D1, D19	Fix + sort
21	Prime factorization	D1	Fix ↔ fold
22	Cycle detection (Floyd/Brent)	D1	Fix ↔ fold
23	Balanced parentheses	D9	Stack ↔ recursion
24	Levenshtein distance	D16	Memo ↔ DP
25	Merge overlapping intervals	D19	Sort-then-scan
26	k-th smallest (quickselect)	D1	Fix ↔ fold
27	Longest common subsequence	D16	Memo ↔ DP
28	Regex state machine	D1, D9	Fix + stack
29	**Deep copy + sort**	**D20**	**Spec uniqueness**
30	Trie search	D11	ADT refinement
31	Dijkstra shortest path	D10	Priority queue
32	Island counting (grid)	D15	BFS ↔ DFS
33	**n-th permutation**	**D20**	**Spec uniqueness**
34	Postfix expression eval	D9	Stack ↔ recursion
35	all ↔ not-any-not	ALG	De Morgan involution
36	Transpose dict-of-lists	D4, D5	Fold fusion
37	**Fibonacci mod p**	**D20**	**Spec uniqueness**
38	Group anagrams	D19, D20	Sort + spec
39	Perfect power check	D1	Fix ↔ fold
40	**Binomial coefficient**	**D20**	**Spec uniqueness**
41	Zip-longest with fill	D5, D6	Fold + lazy/eager
42	Currying	D24	η-expansion
43	CSV parsing	D1, D4	Fix + fusion
44	Coin change DP	D16	Memo ↔ DP
45	Grid path counting	D16	Memo ↔ DP
46	Mini type inference	D1, D8	Fix + section merge
47	**Stock profit**	**D20**	**Spec uniqueness**
48	Trailing zeroes in $n!$	D1	Fix ↔ fold
49	Deep equality with cycles	D1, D9	Fix + stack
50	N-queens	D1, D8	Fix + section merge

Note: eq35 (all vs. any) is resolved by the De Morgan axioms (boolean paths), not by specification reasoning. The bold entries are the hard-8 pairs requiring genuine D20 specification-level arguments.

The two NEQ pairs at the algorithm boundary (neq06: late-binding vs. early-binding closure; neq20: in-place += vs. reassignment) are correctly distinguished by the *effect discipline*: mutation effects and closure-capture timing use different Z3 symbols, so the programs produce

structurally different OTerms that cannot be connected by any path.

Part VI

The Semantic Algebra

Chapter 25

The PyObj Z3 Theory: A Complete Semantic Model of Python

This chapter defines the Z3 theory that models Python's value space, operations, and execution semantics. The theory is designed to be *maximally expressive* (capturing as much of Python's semantics as possible) while *preserving decidability* (every previously decidable query remains decidable after adding new features).

The key design principle: **new expressiveness is added via new constructors and new shared functions, never via quantifiers**. This keeps all fiber-restricted queries in the quantifier-free fragment QF_DT + QF_LIA + QF_S, which is decidable.

25.1 The Value Universe

25.1.1 The Seven Constructors

The universe of Python values is modeled as a Z3 algebraic data type with seven constructors:

Definition 25.1 (PyObj ADT)**.**

$$\mathsf{PyObj} ::= \mathsf{IntObj}(i : \mathbb{Z}) \mid \mathsf{BoolObj}(b : \mathbb{B}) \mid \mathsf{StrObj}(s : \mathsf{String}) \mid \mathsf{NoneObj} \mid \mathsf{Pair}(a : \mathsf{PyObj}, b : \mathsf{PyObj}) \mid \mathsf{Ref}(\mathsf{addr} : \mathbb{Z}) \mid$$

Listing 25.1: PyObj ADT: complete Z3 implementation

```
from z3 import *

D = Datatype('PyObj')
D.declare('IntObj',  ('ival', IntSort()))
D.declare('BoolObj', ('bval', BoolSort()))
D.declare('StrObj',  ('sval', StringSort()))
D.declare('NoneObj')
D.declare('Pair', ('fst', D), ('snd', D))
D.declare('Ref',  ('addr', IntSort()))
D.declare('Bottom')
S = D.create()

# Constructor shortcuts
def mkint(n):  return S.IntObj(IntVal(n))
def mkbool(b): return S.BoolObj(BoolVal(b))
def mkstr(s):  return S.StrObj(StringVal(s))
def mklist(*elts):
    r = S.NoneObj
    for e in reversed(elts): r = S.Pair(e, r)
    return r
```

Constructor	Python type(s)	Z3 field	Exactness
IntObj	`int`	Z3 IntSort (unbounded)	Exact
BoolObj	`bool`	Z3 BoolSort	Exact
StrObj	`str`	Z3 StringSort	Exact
NoneObj	`None`	(no field)	Exact
Pair	`list`, `tuple`	cons cell	Structural
Ref	mutable objects	heap address	Abstract
Bottom	exceptions, $\perp$	(no field)	Abstract

25.1.2 Constructor–Destructor Pairs

Each constructor in the PyObj ADT comes paired with a *destructor* (projection) and a *tester* (recognizer). Together these form the injection–projection–retraction triple that gives the ADT its algebraic character.

Definition 25.2 (Constructor–Destructor Algebra)**.** For each constructor C_k with field $f_k : A_k$, the Z3 ADT generates three operations:

$$\begin{array}{lll} \textbf{Constructor} & C_k : A_k \to \mathsf{PyObj} & \text{(injection)} \\ \textbf{Destructor} & f_k : \mathsf{PyObj} \to A_k & \text{(projection)} \\ \textbf{Tester} & \mathsf{is_}C_k : \mathsf{PyObj} \to \mathbb{B} & \text{(recognizer)} \end{array}$$

These satisfy the *no-confusion* and *no-junk* equations:

$$\begin{array}{rll} f_k(C_k(x)) = x & & \text{(retraction)} \quad \text{(CD1)} \\ \mathsf{is_}C_k(C_k(x)) = \top & & \text{(tester positive)} \quad \text{(CD2)} \\ \mathsf{is_}C_j(C_k(x)) = \perp & j \neq k & \text{(tester negative)} \quad \text{(CD3)} \\ C_k(f_k(v)) = v & \text{when } \mathsf{is_}C_k(v) = \top & \text{(section)} \quad \text{(CD4)} \end{array}$$

Constructor	Destructor	Tester	Carrier sort
IntObj$(\cdot)$	ival$(\cdot)$	is_IntObj$(\cdot)$	$\mathbb{Z}$
BoolObj$(\cdot)$	bval$(\cdot)$	is_BoolObj$(\cdot)$	$\mathbb{B}$
StrObj$(\cdot)$	sval$(\cdot)$	is_StrObj$(\cdot)$	String
Pair$(\cdot,\cdot)$	fst$(\cdot)$, snd$(\cdot)$	is_Pair$(\cdot)$	PyObj $\times$ PyObj
Ref$(\cdot)$	addr$(\cdot)$	is_Ref$(\cdot)$	$\mathbb{Z}$
NoneObj	—	is_NoneObj$(\cdot)$	(unit)
Bottom	—	is_Bottom$(\cdot)$	(unit)

Equations CD1–CD4 are *not* added as axioms: they are *built into* Z3's decision procedure for algebraic datatypes (the QF_DT theory). This is a crucial design choice — we get the full injection–projection calculus for free, with guaranteed decidability.

Proposition 25.3 (Constructor–Destructor Decidability)**.** *All queries involving constructors, destructors, and testers are in QF_DT, which is decidable in $O(n \log n)$ time where n is the formula size. In particular, for any ground term t:*

1. $\mathsf{ival}(\mathsf{IntObj}(42)) = 42$ *is decided immediately (by CD1);*
2. $\mathsf{is_IntObj}(\mathsf{StrObj}(\text{“}hello\text{”})) = \perp$ *is decided immediately (by CD3);*
3. $\exists v.\ \mathsf{is_IntObj}(v) \wedge \mathsf{ival}(v) > 0$ *is satisfiable (by CD4, take* $v = \mathsf{IntObj}(1)$*).*

25.1.3 Why Seven Constructors Suffice

Python has hundreds of types (`int`, `float`, `complex`, `str`, `bytes`, `list`, `tuple`, `set`, `frozenset`, `dict`, `deque`, `defaultdict`, `Counter`, `OrderedDict`, and every user-defined class). Our seven constructors cover them via *abstraction*:

Python type	PyObj constructor	Justification
`int`	IntObj	Exact: Z3 IntSort is unbounded, matching Python's arbitrary-precision i
`float`	IntObj	Approximate: we model float as int (sound for integer-valued floats)
`complex`	Pair(IntObj, IntObj)	Real and imaginary parts
`bool`	BoolObj	Exact
`str`	StrObj	Exact: Z3 StringSort models Unicode strings with concat, length, etc.
`bytes`	StrObj	Approximate
`None`	NoneObj	Exact (singleton)
`list`	Pair-chain	Structural: $[1, 2, 3]$ = Pair(1, Pair(2, Pair(3, NoneObj)))
`tuple`	Pair-chain	Same encoding as list (Python tuples and lists have same structure)
`set`	Ref	Abstract: modeled by identity (heap address), operations via shared func
`frozenset`	Ref	Abstract
`dict`	Ref	Abstract
`deque`	Ref	Abstract
`defaultdict`	Ref	Abstract
`Counter`	Ref	Abstract
`OrderedDict`	Ref	Abstract
user class instance	Ref	Abstract
function/lambda	Ref	Abstract (callable ref)
generator	Ref	Abstract (stateful iterator)
exception	Bottom	Abstract (control flow)
`NotImplemented`	Bottom	Abstract

The abstraction is *sound*: if the Z3 theory proves two programs equivalent, they are genuinely equivalent for all concrete Python values that the constructors faithfully represent. The abstraction may be *incomplete*: programs that differ only on types mapped to Ref (e.g., two different dict implementations) may not be distinguished.

25.1.4 Decidability Preservation

Proposition 25.4 (Adding Constructors Preserves Decidability)**.** *The theory QF_DT (quantifier-free datatypes) is decidable for any finite set of constructors. Adding a new constructor to the* PyObj *ADT does not break decidability — it adds new cases to pattern matching but keeps the theory in QF_DT.*

This means we can extend PyObj with new constructors (e.g., FloatObj(real : RealSort), SetObj(elems : ArraySort)) without breaking decidability of existing queries. We choose seven constructors for practical efficiency (fewer cases in RecFunctions), not for theoretical necessity.

25.2 The Type Fibration

25.2.1 TypeTag Sort

The type tag function is the fibration structure map:

Definition 25.5 (Type Fibration)**.** tag : PyObj → TypeTag where TypeTag = {TInt, TBool, TStr, TNone, TPair,

is defined by:

$$\mathsf{tag}(v) = \begin{cases} \mathsf{TInt} & \text{if } v = \mathsf{IntObj}(\ldots) \\ \mathsf{TBool} & \text{if } v = \mathsf{BoolObj}(\ldots) \\ \mathsf{TStr} & \text{if } v = \mathsf{StrObj}(\ldots) \\ \mathsf{TNone} & \text{if } v = \mathsf{NoneObj} \\ \mathsf{TPair} & \text{if } v = \mathsf{Pair}(\ldots) \\ \mathsf{TRef} & \text{if } v = \mathsf{Ref}(\ldots) \\ \mathsf{TBot} & \text{if } v = \mathsf{Bottom} \end{cases}$$

This is implemented as a Z3 RecFunction and a Z3 EnumSort. The RecFunction dispatches on the constructor, returning the corresponding enum constant.

Listing 25.2: TypeTag: complete Z3 implementation

```
TTag, (TInt, TBool, TStr, TNone, TPair, TRef, TBot) = \
    EnumSort('TTag', ['TInt','TBool','TStr',
                      'TNone','TPair','TRef','TBot'])

tag = RecFunction('tag', S, TTag)
x = Const('_tag_x', S)
RecAddDefinition(tag, [x],
    If(S.is_IntObj(x),  TInt,
    If(S.is_BoolObj(x), TBool,
    If(S.is_StrObj(x),  TStr,
    If(S.is_NoneObj(x), TNone,
    If(S.is_Pair(x),    TPair,
    If(S.is_Ref(x),     TRef,
                        TBot)))))))
```

25.2.2 Fiber Projection Lemmas

The following lemmas are used repeatedly in the Čech complex construction. They are all decidable consequences of the ADT theory.

Lemma 25.6 (Fiber Determines Constructor)**.** $\mathsf{tag}(v) = \mathsf{TInt} \iff \exists n : \mathbb{Z}.\ v = \mathsf{IntObj}(n)$.

More generally, knowing the tag determines the constructor shape, which Z3 exploits via the $\mathsf{is_IntObj}$, $\mathsf{is_BoolObj}$, *... tester functions.*

Lemma 25.7 (Fiber Disjointness)**.** *The fibers are pairwise disjoint:* $\mathsf{tag}(v) = \tau_1 \wedge \mathsf{tag}(v) = \tau_2 \implies \tau_1 = \tau_2$. *This is automatic from the ADT theory (each value has exactly one constructor).*

Lemma 25.8 (Fiber Exhaustion)**.** *The fibers cover all values:* $\bigvee_{\tau \in \mathsf{TypeTag}} \mathsf{tag}(v) = \tau$. *This is the "no junk" property of inductive types.*

25.3 Operations: Full Python Semantics in Z3

25.3.1 Binary Operations

The binop RecFunction encodes Python's binary operations with full type dispatch:

Definition 25.9 (Binary Operation Semantics)**.** $\mathsf{binop} : \mathbb{Z} \times \mathsf{PyObj} \times \mathsf{PyObj} \to \mathsf{PyObj}$ where the first argument is an operation tag.

Integer $\times$ **Integer** ($\mathsf{tag}(a) = \mathsf{tag}(b) = \mathsf{TInt}$):

$$\mathsf{binop}(\mathsf{ADD}, \mathsf{IntObj}(m), \mathsf{IntObj}(n)) = \mathsf{IntObj}(m+n) \tag{25.1}$$
$$\mathsf{binop}(\mathsf{SUB}, \mathsf{IntObj}(m), \mathsf{IntObj}(n)) = \mathsf{IntObj}(m-n) \tag{25.2}$$
$$\mathsf{binop}(\mathsf{MUL}, \mathsf{IntObj}(m), \mathsf{IntObj}(n)) = \mathsf{IntObj}(m \times n) \tag{25.3}$$
$$\mathsf{binop}(\mathsf{FLOORDIV}, \mathsf{IntObj}(m), \mathsf{IntObj}(n)) = \begin{cases} \mathsf{IntObj}(m \div n) & n \neq 0 \\ \mathsf{Bottom} & n = 0 \end{cases} \tag{25.4}$$
$$\mathsf{binop}(\mathsf{MOD}, \mathsf{IntObj}(m), \mathsf{IntObj}(n)) = \begin{cases} \mathsf{IntObj}(m \bmod n) & n \neq 0 \\ \mathsf{Bottom} & n = 0 \end{cases} \tag{25.5}$$
$$\mathsf{binop}(\mathsf{LT}, \mathsf{IntObj}(m), \mathsf{IntObj}(n)) = \mathsf{BoolObj}(m < n) \tag{25.6}$$
$$\mathsf{binop}(\mathsf{LE}, \ldots) = \mathsf{BoolObj}(m \leq n) \tag{25.7}$$
$$\mathsf{binop}(\mathsf{GT}, \ldots) = \mathsf{BoolObj}(m > n) \tag{25.8}$$
$$\mathsf{binop}(\mathsf{GE}, \ldots) = \mathsf{BoolObj}(m \geq n) \tag{25.9}$$
$$\mathsf{binop}(\mathsf{EQ}, \ldots) = \mathsf{BoolObj}(m = n) \tag{25.10}$$
$$\mathsf{binop}(\mathsf{NE}, \ldots) = \mathsf{BoolObj}(m \neq n) \tag{25.11}$$
$$\mathsf{binop}(\mathsf{MAX}, \ldots) = \mathsf{If}(m \geq n, \mathsf{IntObj}(m), \mathsf{IntObj}(n)) \tag{25.12}$$
$$\mathsf{binop}(\mathsf{MIN}, \ldots) = \mathsf{If}(m \leq n, \mathsf{IntObj}(m), \mathsf{IntObj}(n)) \tag{25.13}$$

String $\times$ **String** ($\mathsf{tag}(a) = \mathsf{tag}(b) = \mathsf{TStr}$):

$$\mathsf{binop}(\mathsf{ADD}, \mathsf{StrObj}(s), \mathsf{StrObj}(t)) = \mathsf{StrObj}(\mathsf{Concat}(s,t)) \tag{25.14}$$
$$\mathsf{binop}(\mathsf{EQ}, \mathsf{StrObj}(s), \mathsf{StrObj}(t)) = \mathsf{BoolObj}(s = t) \tag{25.15}$$
$$\mathsf{binop}(\mathsf{NE}, \ldots) = \mathsf{BoolObj}(s \neq t) \tag{25.16}$$

Bool $\times$ **Bool** ($\mathsf{tag}(a) = \mathsf{tag}(b) = \mathsf{TBool}$):

$$\mathsf{binop}(\mathsf{EQ}, \mathsf{BoolObj}(p), \mathsf{BoolObj}(q)) = \mathsf{BoolObj}(p = q) \tag{25.17}$$
$$\mathsf{binop}(\mathsf{NE}, \ldots) = \mathsf{BoolObj}(p \neq q) \tag{25.18}$$
$$\mathsf{binop}(\mathsf{BITOR}, \mathsf{BoolObj}(p), \mathsf{BoolObj}(q)) = \mathsf{BoolObj}(p \vee q) \tag{25.19}$$
$$\mathsf{binop}(\mathsf{BITAND}, \ldots) = \mathsf{BoolObj}(p \wedge q) \tag{25.20}$$
$$\mathsf{binop}(\mathsf{BITXOR}, \ldots) = \mathsf{BoolObj}(p \oplus q) \tag{25.21}$$

These model Python's `|`, `&`, `^` operators on booleans, which behave as logical OR, AND, XOR respectively (unlike on integers, where they are bitwise).

Cross-type equality: For any a, b with $\mathsf{tag}(a) \neq \mathsf{tag}(b)$:

$$\mathsf{binop}(\mathsf{EQ}, a, b) = \mathsf{BoolObj}(a = b) \qquad \text{(structural equality)} \tag{25.22}$$
$$\mathsf{binop}(\mathsf{NE}, a, b) = \mathsf{BoolObj}(a \neq b) \tag{25.23}$$

All other cross-type combinations: Bottom (TypeError in Python).

Proposition 25.10 (binop Decidability)**.** *On the* TInt *fiber,* binop *reduces to Z3's native* IntSort *operations (decidable QF_LIA for linear operations, semi-decidable QF_NIA for multiplication). On the* TStr *fiber, it reduces to Z3's string theory (decidable QF_S for concat and equality). On the* TBool *fiber, it reduces to Z3's propositional logic (decidable in constant time per operation). On other fibers, it returns* Bottom *(decidable constant).*

Therefore: binop *does not break decidability on any fiber.*

Listing 25.3: binop: complete Z3 implementation

```
ADD=1; SUB=2; MUL=3; FLOORDIV=5; MOD=6
LT=20; LE=21; GT=22; GE=23; EQ=24; NE=25
MAX=30; MIN=31

binop = RecFunction('binop', IntSort(), S, S, S)
op = Int('_bo')
a, b = Const('_ba', S), Const('_bb', S)
ai, bi = S.ival(a), S.ival(b)
ii = And(S.is_IntObj(a), S.is_IntObj(b))
ss = And(S.is_StrObj(a), S.is_StrObj(b))

int_result = \
  If(op==ADD, S.IntObj(ai+bi),
  If(op==SUB, S.IntObj(ai-bi),
  If(op==MUL, S.IntObj(ai*bi),
  If(op==FLOORDIV, If(bi!=0, S.IntObj(ai/bi), S.Bottom),
  If(op==MOD, If(bi!=0, S.IntObj(ai%bi), S.Bottom),
  If(op==LT, S.BoolObj(ai<bi),
  If(op==LE, S.BoolObj(ai<=bi),
  If(op==GT, S.BoolObj(ai>bi),
  If(op==GE, S.BoolObj(ai>=bi),
  If(op==EQ, S.BoolObj(ai==bi),
  If(op==NE, S.BoolObj(ai!=bi),
  If(op==MAX, If(ai>=bi, a, b),
  If(op==MIN, If(ai<=bi, a, b),
  S.Bottom))))))))))))))

str_result = \
  If(op==ADD, S.StrObj(Concat(S.sval(a), S.sval(b))),
  If(op==EQ, S.BoolObj(S.sval(a)==S.sval(b)),
  If(op==NE, S.BoolObj(S.sval(a)!=S.sval(b)),
  S.Bottom)))

RecAddDefinition(binop, [op, a, b],
    If(ii, int_result,
    If(ss, str_result,
    If(op==EQ, S.BoolObj(a==b),
    If(op==NE, S.BoolObj(a!=b),
    S.Bottom)))))
```

25.3.2 Unary Operations

Definition 25.11 (Unary Operation Semantics). $\mathsf{unop} : \mathbb{Z} \times \mathsf{PyObj} \to \mathsf{PyObj}$

On IntObj:

$$\mathsf{unop}(\mathsf{NEG}, \mathsf{IntObj}(n)) = \mathsf{IntObj}(-n) \tag{25.24}$$

$$\mathsf{unop}(\mathsf{ABS}, \mathsf{IntObj}(n)) = \mathsf{IntObj}(\mathsf{If}(n \geq 0, n, -n)) \tag{25.25}$$

$$\mathsf{unop}(\mathsf{NOT}, \mathsf{IntObj}(n)) = \mathsf{BoolObj}(n = 0) \tag{25.26}$$

$$\mathsf{unop}(\mathsf{BOOL}, \mathsf{IntObj}(n)) = \mathsf{BoolObj}(n \neq 0) \tag{25.27}$$

$$\mathsf{unop}(\mathsf{INT}, \mathsf{IntObj}(n)) = \mathsf{IntObj}(n) \tag{25.28}$$

On BoolObj:

$$\mathsf{unop}(\mathsf{NOT}, \mathsf{BoolObj}(b)) = \mathsf{BoolObj}(\neg b) \tag{25.29}$$
$$\mathsf{unop}(\mathsf{BOOL}, \mathsf{BoolObj}(b)) = \mathsf{BoolObj}(b) \tag{25.30}$$
$$\mathsf{unop}(\mathsf{INT}, \mathsf{BoolObj}(b)) = \mathsf{IntObj}(\mathsf{If}(b, 1, 0)) \tag{25.31}$$

On StrObj:

$$\mathsf{unop}(\mathsf{LEN}, \mathsf{StrObj}(s)) = \mathsf{IntObj}(\mathsf{Length}(s)) \tag{25.32}$$
$$\mathsf{unop}(\mathsf{BOOL}, \mathsf{StrObj}(s)) = \mathsf{BoolObj}(\mathsf{Length}(s) > 0) \tag{25.33}$$

On NoneObj: $\mathsf{unop}(\mathsf{BOOL}, \mathsf{NoneObj}) = \mathsf{BoolObj}(\bot)$; $\mathsf{unop}(\mathsf{NOT}, \mathsf{NoneObj}) = \mathsf{BoolObj}(\top)$.
On Pair: $\mathsf{unop}(\mathsf{BOOL}, \mathsf{Pair}(a, b)) = \mathsf{BoolObj}(\top)$ (non-empty list is truthy).

25.3.3 The Truthiness Protocol

Python's $__\mathsf{bool}__$ protocol is central to control flow. Every value has a truth value:

Definition 25.12 (Truthiness)**.** $\mathsf{truthy} : \mathsf{PyObj} \to \mathbb{B}$

$$\mathsf{truthy}(\mathsf{IntObj}(n)) = (n \neq 0) \tag{25.34}$$
$$\mathsf{truthy}(\mathsf{BoolObj}(b)) = b \tag{25.35}$$
$$\mathsf{truthy}(\mathsf{StrObj}(s)) = (\mathsf{Length}(s) > 0) \tag{25.36}$$
$$\mathsf{truthy}(\mathsf{NoneObj}) = \bot \tag{25.37}$$
$$\mathsf{truthy}(\mathsf{Pair}(a, b)) = \top \tag{25.38}$$
$$\mathsf{truthy}(\mathsf{Bottom}) = \bot \tag{25.39}$$
$$\mathsf{truthy}(\mathsf{Ref}(_)) = \top \tag{25.40}$$

Remark 25.13 (Faithfulness of Truthiness)**.** This exactly matches CPython's `__bool__` for all modeled types. The only approximation is Ref: we model all reference objects as truthy, which is correct for non-empty containers but wrong for empty containers stored as Refs. This is sound for the Čech analysis because the Ref fiber is handled abstractly (via shared function symbols, not by evaluating truthiness).

Listing 25.4: truthy: complete Z3 implementation

```
truthy = RecFunction('truthy', S, BoolSort())
x = Const('_tr', S)
RecAddDefinition(truthy, [x],
    If(S.is_IntObj(x),  S.ival(x) != 0,
    If(S.is_BoolObj(x), S.bval(x),
    If(S.is_StrObj(x),  Length(S.sval(x)) > 0,
    If(S.is_NoneObj(x), False,
    If(S.is_Bottom(x),  False,
    True))))))  # Pair, Ref are truthy
```

25.3.4 The Canonical Fold (Nat-Recursor)

Definition 25.14 (Canonical Fold)**.** $\mathsf{fold} : \mathbb{Z} \times \mathbb{Z} \times \mathbb{Z} \times \mathsf{PyObj} \to \mathsf{PyObj}$

$$\mathsf{fold}(\mathsf{op}, i, \mathsf{stop}, \mathsf{acc}) = \begin{cases} \mathsf{acc} & \text{if } i \geq \mathsf{stop} \\ \mathsf{binop}(\mathsf{op}, \mathsf{IntObj}(i), \mathsf{fold}(\mathsf{op}, i+1, \mathsf{stop}, \mathsf{acc})) & \text{otherwise} \end{cases} \tag{25.41}$$

Listing 25.5: fold: complete Z3 implementation

```
fold = RecFunction('fold',
    IntSort(), IntSort(), IntSort(), S, S)
op, i, stop = Ints('_fo _fi _fs')
acc = Const('_fa', S)
RecAddDefinition(fold, [op, i, stop, acc],
    If(i >= stop, acc,
       binop(op, S.IntObj(i),
             fold(op, i+1, stop, acc))))
```

The fold is the *universal* iteration primitive. Both recursive and iterative programs that operate by applying a binary operation across a range compile to this single RecFunction, enabling equivalence by syntactic identity (D1 path).

Proposition 25.15 (Fold Decidability)**.** *The fold does not break decidability of previously decidable queries. When both programs compile to the* same *fold term, the comparison is syntactic equality (decidable, $O(1)$). When they compile to* different *fold terms, Z3 handles the comparison via its RecFunction reasoning (semi-decidable in general, but decidable for ground arguments).*

25.4 The RecFunction System

When a Python function contains recursion that does not match the canonical fold pattern, the compiler emits a Z3 RecFunction — a recursively-defined function in Z3's theory of algebraic datatypes. This section formalizes how Python recursion becomes Z3 recursion.

25.4.1 From Python AST to Z3 RecFunction

Definition 25.16 (RecFunction Compilation)**.** Given a recursive Python function $f(x_1, \ldots, x_n)$ with body B, the compiler produces:

1. A Z3 RecFunction declaration: $\mathsf{rec_f} : \mathsf{PyObj}^n \to \mathsf{PyObj}$
2. Fresh symbolic parameters: $s_1, \ldots, s_n : \mathsf{PyObj}$
3. A compilation environment Δ where each $x_i \mapsto s_i$ and $f \mapsto \mathsf{rec_f}$ (enabling self-reference)
4. Presheaf sections extracted from B under Δ: $\{(\phi_j, t_j)\}_{j=1}^m$
5. A combined definition term:

$$\mathsf{body} := \mathsf{If}(\phi_1, t_1, \mathsf{If}(\phi_2, t_2, \cdots t_m))$$

6. Registration: $\mathsf{RecAddDefinition}(\mathsf{rec_f}, [s_1, \ldots, s_n], \mathsf{body})$

Listing 25.6: RecFunction compilation: implementation

```
def _handle_recursion(func, body, env, params):
    T = env.T
    n = len(params)
    # 1. Declare RecFunction
    rfn = RecFunction(f'rec_{func.name}',
                      *([T.S]*n), T.S)
    # 2. Fresh symbolic parameters
    sym = [FreshConst(T.S) for _ in range(n)]
    # 3. Environment with self-reference
    de = Env(T, func_name=func.name, is_rec=True)
```

```
    for p, s in zip(param_names, sym):
        de.put(p, s)
    # 4. Extract presheaf sections
    secs = extract_sections(body, de)
    # 5. Combine into single term
    combined = secs[-1].term
    for s in reversed(secs[:-1]):
        combined = If(s.guard, s.term, combined)
    # 6. Register definition
    RecAddDefinition(rfn, sym, combined)
    return rfn(*params)
```

Remark 25.17 (Self-Referential Compilation)**.** The key subtlety is step 3: when compiling the body, recursive calls to f are resolved to applications of $\mathsf{rec_f}$ itself. This works because Z3's $\mathsf{RecFunction}$ supports recursive definitions — the body term may refer to the function being defined. The compiler detects recursion by walking the AST:

$$\mathsf{is_rec}(f) := \exists \mathsf{Call}(g, \ldots) \in \mathsf{walk}(B).\ g = f$$

25.4.2 Canonical Fold Detection (Nat-Eliminator Extraction)

Before falling back to general RecFunctions, the compiler attempts to detect *canonical folds* — recursive functions that match the pattern of the natural-number eliminator.

Definition 25.18 (Canonical Fold Pattern)**.** A recursive function $f(n)$ is a *canonical fold* if it has the form:

$$f(n) = \begin{cases} e & n = 0 \text{ (or } n \leq 0) \\ \mathsf{binop}(\mathsf{op}, g(n), f(n-1)) & \text{otherwise} \end{cases}$$

where op is a known binary operation tag and $g(n)$ is a non-recursive expression in n. This compiles directly to:

$$[\![f]\!](p) = \mathsf{fold}(\mathsf{op}, 0, \mathsf{ival}(p) + 1, \mathsf{IntObj}(e))$$

The canonical fold detection is critical for the D1 (Rec↔Iter) dichotomy: both the recursive definition and its iterative counterpart compile to the *same* fold term, yielding equality by syntactic identity.

25.4.3 RecFunction Limitations

Proposition 25.19 (RecFunction Semi-Decidability)**.** *Z3's reasoning about RecFunctions is* semi-decidable*:*

1. ***Ground evaluation****: For concrete (ground) arguments, Z3 can evaluate the RecFunction by unfolding its definition. This terminates if the Python function terminates on those inputs.*

2. ***Symbolic reasoning****: For symbolic arguments, Z3 uses bounded unfolding. If the unfolding depth exceeds Z3's limit, the query may time out.*

3. ***Equivalence of RecFunctions****: Determining whether two RecFunctions compute the same function is undecidable in general (Rice's theorem).*

The canonical fold extraction (Definition 25.18) mitigates case 3: by normalizing recursive functions to a common fold form, equivalence reduces to syntactic equality of the fold terms (decidable).

25.5 The Uninterpreted Opacity Zone

A central design tension in the PyObj theory is the choice between *interpreted* operations (with full Z3 semantics) and *uninterpreted* operations (modeled as opaque function symbols). This section characterizes the "opacity zone" — the boundary where Z3 reasoning power gives way to symbolic abstraction.

25.5.1 The Interpreted–Uninterpreted Spectrum

Definition 25.20 (Operation Interpretation Levels)**.** Every Python operation in the Z3 theory falls into one of three interpretation levels:

1. **Fully interpreted**: The operation's semantics are encoded directly in Z3's native theories (IntSort, BoolSort, StringSort). Z3 can reason about the result value.

   Examples: integer $+, -, \times, //$, mod; string concat; boolean $\wedge, \vee, \neg$; comparisons $<, \leq, =, \neq$.

2. **Structurally interpreted**: The operation's behavior is encoded via the PyObj ADT constructors (Pair-chains) with recursive Z3 definitions. Z3 can reason about the structure.

   Examples: truthy, unop, binop, fold, tag, list construction via Pair-chains, GETITEM.

3. **Uninterpreted**: The operation is a shared function symbol $\mathsf{py_f} : \mathsf{PyObj}^k \to \mathsf{PyObj}$ with no built-in semantics. Z3 knows only that equal inputs produce equal outputs (the congruence axiom).

   Examples: sorted, reversed, set, dict.keys, Counter, str.split, all method calls, all module functions.

Level	Z3 power	Decidability
Fully interpreted	Can reason about values	Decidable (QF_LIA, QF_S)
Structurally interpreted	Can reason about structure	Semi-decidable (RecFunctions)
Uninterpreted	Congruence only ($f(x) = f(y)$ iff $x = y$)	Decidable (QF_UF)

25.5.2 The Opacity Boundary

Theorem 25.21 (Opacity Boundary Theorem)**.** *Let f be a Python builtin compiled as an uninterpreted function* py_f*. Then:*

1. *Z3 can prove* $\mathsf{py_f}(t_1) = \mathsf{py_f}(t_2)$ *if and only if* $t_1 = t_2$ *(syntactic equality of arguments) or an added axiom enables it.*

2. *Z3* cannot *prove* $\mathsf{py_f}(t) = v$ *for any concrete value* v *unless an axiom equates them.*

3. *Adding an axiom* $\forall x.\ \mathsf{py_f}(\mathsf{py_f}(x)) = \mathsf{py_f}(x)$ *(idempotence) lifts* f *partially out of the opacity zone: Z3 can now simplify double applications.*

Proof. (1) follows from the congruence axiom for uninterpreted functions. (2) follows from the fact that an uninterpreted function has no fixed interpretation — Z3 must find a model, and any model that maps $\mathsf{py_f}(t)$ to a value different from v is valid. (3) the added axiom constrains the function's interpretation, ruling out models where $f(f(x)) \neq f(x)$. □

25.5.3 Example: `sorted(list(set(x)))` in the Opacity Zone

Consider the Python expression `sorted(list(set(x)))`. The compiler produces a chain of uninterpreted function applications:

Listing 25.7: Uninterpreted function chain

```
# Compilation of sorted(list(set(x)))
p0 = Const('p0', S)                # input x
t1 = py_set(p0)                    # set(x)
t2 = py_list(t1)                   # list(set(x))
t3 = py_sorted(t2)                 # sorted(list(set(x)))

# Compilation of sorted(x)
t4 = py_sorted(p0)                 # sorted(x)
```

Without axioms: Z3 cannot prove $t_3 = t_4$. The terms $\mathsf{py_sorted}(\mathsf{py_list}(\mathsf{py_set}(p_0)))$ and $\mathsf{py_sorted}(p_0)$ are syntactically distinct, and no congruence rule connects them.

With axioms: Adding the semantic axioms from $\mathsf{semantic_axioms}()$:

$$\forall x.\ \mathsf{py_set}(\mathsf{py_set}(x)) = \mathsf{py_set}(x) \qquad \text{(I3: set idempotent)} \tag{25.42}$$

$$\forall x.\ \mathsf{py_sorted}(\mathsf{py_list}(x)) = \mathsf{py_sorted}(x) \qquad \text{(sorted absorbs list)} \tag{25.43}$$

$$\forall x.\ \mathsf{py_set}(\mathsf{py_sorted}(x)) = \mathsf{py_set}(x) \qquad \text{(set absorbs sorted)} \tag{25.44}$$

The chain simplifies: $\mathsf{py_sorted}(\mathsf{py_list}(\mathsf{py_set}(p_0))) \xrightarrow{\text{sorted absorbs list}} \mathsf{py_sorted}(\mathsf{py_set}(p_0))$.

This is *not* equal to $\mathsf{py_sorted}(p_0)$ in general (deduplication changes the multiset), so Z3 correctly reports **not equivalent** — the set operation removes duplicates, which sorted alone does not. This is the right answer.

Remark 25.22 (Axioms as Partially Transparent Windows)**.** Each semantic axiom "opens a window" in the opacity zone, allowing Z3 to see a specific algebraic property of an otherwise opaque function. The 52 axioms of $\mathcal{A}_{\mathsf{Py}}$ (Chapter 26) collectively open enough windows for the benchmark suite, while keeping most of each function's behavior opaque (and thus decidable).

25.5.4 Item Access (GETITEM)

Definition 25.23 (Subscript Semantics)**.** $\mathsf{binop}(\mathsf{GETITEM}, \mathsf{collection}, \mathsf{index})$ implements Python's $\mathsf{__getitem__}$:

On Pair-chains (list indexing):

$$\mathsf{binop}(\mathsf{GETITEM}, \mathsf{Pair}(a, d), \mathsf{IntObj}(0)) = a \tag{25.45}$$

$$\mathsf{binop}(\mathsf{GETITEM}, \mathsf{Pair}(a, d), \mathsf{IntObj}(n)) = \mathsf{binop}(\mathsf{GETITEM}, d, \mathsf{IntObj}(n-1)) \quad n > 0 \tag{25.46}$$

On StrObj (character indexing):

$$\mathsf{binop}(\mathsf{GETITEM}, \mathsf{StrObj}(s), \mathsf{IntObj}(n)) = \mathsf{StrObj}(\mathsf{SubString}(s, n, 1))$$

On Ref (dict/set access): uses shared function symbol $\mathsf{py_meth___getitem__}$.

25.6 Container Semantics via Shared Functions

For types modeled by Ref (dicts, sets, deques, etc.), operations are modeled via shared function symbols with *algebraic axioms*. This is more expressive than the ADT constructors (we capture behavioral properties without modeling internal structure) and preserves decidability (shared functions are uninterpreted; axioms are optional path constructors).

25.6.1 Dictionary Operations

Definition 25.24 (Dict Semantic Functions).

$$\mathsf{py_call_dict} : \mathsf{PyObj} \to \mathsf{PyObj} \quad \text{construct from pairs} \tag{25.47}$$

$$\mathsf{py_meth___getitem__} : \mathsf{PyObj} \times \mathsf{PyObj} \to \mathsf{PyObj} \quad \text{key lookup} \tag{25.48}$$

$$\mathsf{py_meth___setitem__} : \mathsf{PyObj} \times \mathsf{PyObj} \times \mathsf{PyObj} \to \mathsf{PyObj} \quad \text{key assignment (returns new dict)} \tag{25.49}$$

$$\mathsf{py_meth_get} : \mathsf{PyObj} \times \mathsf{PyObj} \to \mathsf{PyObj} \quad \text{get with default} \tag{25.50}$$

$$\mathsf{py_meth_keys} : \mathsf{PyObj} \to \mathsf{PyObj} \quad \text{key set} \tag{25.51}$$

$$\mathsf{py_meth_values} : \mathsf{PyObj} \to \mathsf{PyObj} \quad \text{value collection} \tag{25.52}$$

$$\mathsf{py_meth_items} : \mathsf{PyObj} \to \mathsf{PyObj} \quad \text{key-value pairs} \tag{25.53}$$

$$\mathsf{py_meth_update} : \mathsf{PyObj} \times \mathsf{PyObj} \to \mathsf{PyObj} \quad \text{merge dicts} \tag{25.54}$$

$$\mathsf{py_meth_pop} : \mathsf{PyObj} \times \mathsf{PyObj} \to \mathsf{PyObj} \quad \text{remove key} \tag{25.55}$$

These are *uninterpreted* by default (Z3 knows nothing about them except that they are functions). Axioms can be added as path constructors:

$$\forall d, k, v.\ \mathsf{py_meth___getitem__}(\mathsf{py_meth___setitem__}(d, k, v), k) = v \tag{25.56}$$

(set-then

$$\forall d, k_1, k_2, v.\ k_1 \neq k_2 \implies \mathsf{py_meth___getitem__}(\mathsf{py_meth___setitem__}(d, k_1, v), k_2) = \mathsf{py_meth___getitem__}(d, k_2) \tag{25.57}$$

(set-then

Proposition 25.25 (Dict Axioms Preserve Decidability). *These axioms are universally quantified over* PyObj*, so they technically exit the QF fragment. However:*

1. *On a specific fiber, the quantifiers are bounded (the variables have fixed tags), so E-matching terminates.*
2. *The axioms are* conditional *($k_1 \neq k_2$ guard), which limits instantiation.*
3. *They are added only when both programs actually use dict operations (axiom relevance filtering).*

In practice, these axioms do not cause Z3 timeouts on the benchmark.

25.6.2 List Operations

Definition 25.26 (List Semantic Functions).

$$\mathsf{py_mut_append} : \mathsf{PyObj} \times \mathsf{PyObj} \to \mathsf{PyObj} \quad \text{append element (mutation)} \tag{25.58}$$

$$\mathsf{py_mut_extend} : \mathsf{PyObj} \times \mathsf{PyObj} \to \mathsf{PyObj} \quad \text{extend with another list} \tag{25.59}$$

$$\mathsf{py_mut_insert} : \mathsf{PyObj} \times \mathsf{PyObj} \times \mathsf{PyObj} \to \mathsf{PyObj} \quad \text{insert at index} \tag{25.60}$$

$$\mathsf{py_mut_pop} : \mathsf{PyObj} \to \mathsf{PyObj} \quad \text{remove last element} \tag{25.61}$$

$$\mathsf{py_mut_sort} : \mathsf{PyObj} \to \mathsf{PyObj} \quad \text{in-place sort (mutation!)} \tag{25.62}$$

$$\mathsf{py_mut_reverse} : \mathsf{PyObj} \to \mathsf{PyObj} \quad \text{in-place reverse (mutation!)} \tag{25.63}$$

Note the distinction between *mutation* operations (prefixed mut_, which modify a Ref in place) and *pure* operations (like py_sorted, which return a new value). This distinction is critical for the effect discipline (Chapter **??**): mutation operations use different shared symbols than their pure counterparts, so Z3 correctly distinguishes lst.sort() (mutation, returns None) from sorted(lst) (pure, returns new list).

25.6.3 Set Operations

Definition 25.27 (Set Semantic Functions).

$$\mathsf{py_mut_add} : \mathsf{PyObj} \times \mathsf{PyObj} \to \mathsf{PyObj} \quad \text{add element} \tag{25.64}$$
$$\mathsf{py_mut_discard} : \mathsf{PyObj} \times \mathsf{PyObj} \to \mathsf{PyObj} \quad \text{remove element} \tag{25.65}$$
$$\mathsf{py_call_union} : \mathsf{PyObj} \times \mathsf{PyObj} \to \mathsf{PyObj} \quad \text{set union} \tag{25.66}$$
$$\mathsf{py_call_intersection} : \mathsf{PyObj} \times \mathsf{PyObj} \to \mathsf{PyObj} \quad \text{set intersection} \tag{25.67}$$
$$\mathsf{py_call_difference} : \mathsf{PyObj} \times \mathsf{PyObj} \to \mathsf{PyObj} \quad \text{set difference} \tag{25.68}$$

Set axioms (optional path constructors):

$$\mathsf{py_call_union}(a, b) = \mathsf{py_call_union}(b, a) \quad \text{(union commutativity)} \tag{25.69}$$
$$\mathsf{py_call_intersection}(a, b) = \mathsf{py_call_intersection}(b, a) \quad \text{(intersection commutativity)} \tag{25.70}$$
$$\mathsf{py_call_union}(a, a) = a \quad \text{(union idempotence)} \tag{25.71}$$

25.7 The Heap Model

Mutable Python objects live on a *heap* — a global store that maps addresses to values. Two references to the same address are *aliases*: modifying one is visible through the other.

25.7.1 Heap Representation

Definition 25.28 (Abstract Heap). The heap is modeled *implicitly* through the Ref constructor and mutation operations:

- $\mathsf{Ref}(\mathsf{addr})$ represents a heap-allocated object at address addr.
- Fresh allocations use fresh addresses: $\mathsf{mkref}() = \mathsf{Ref}(\mathsf{next_uid})$ where $\mathsf{next_uid}$ is a monotonically increasing counter.
- Mutation operations (mut_append, mut_sort, etc.) return a *new version* of the object: $\mathsf{mut_op}(\mathsf{Ref}(a), \ldots) = v'$ where v' represents the modified object.

Remark 25.29 (SSA for the Heap). This is essentially *Static Single Assignment* (SSA) form for heap operations: each mutation produces a new value, and the environment tracks the latest version. Aliasing is modeled by having multiple environment entries point to the same Ref.

25.7.2 Aliasing and the Effect Discipline

Definition 25.30 (Aliasing Semantics). Two variables x and y are *aliases* if $\mathsf{env}[x] = \mathsf{env}[y] = \mathsf{Ref}(a)$ for the same address a. A mutation on x modifies the Ref, so y (which points to the same Ref) sees the modification.

This is where the effect discipline matters:

- $\mathsf{mut_iadd}(\mathsf{Ref}(a), v)$ *modifies* the Ref — all aliases see the change.
- $\mathsf{py_call___add__}(\mathsf{Ref}(a), v)$ *creates a new value* — aliases are unaffected.

These are different shared symbols, so Z3 treats them as unrelated functions. This is how the framework correctly distinguishes `lst += [1]` (mutates alias) from `lst = lst + [1]` (rebinds variable).

25.8 Python Control Flow in Z3

25.8.1 Conditional Expressions

Python's `if`/`else` compiles to Z3's If:

$$[\![\texttt{t if c else f}]\!] = \mathsf{If}(\mathsf{truthy}([\![c]\!]), [\![t]\!], [\![f]\!])$$

This is a Z3 builtin (part of the core theory, always decidable).

25.8.2 Short-Circuit Boolean Operators

Python's `and`/`or` are short-circuit and return *values*, not booleans:

$$[\![\texttt{a and b}]\!] = \mathsf{If}(\mathsf{truthy}([\![a]\!]), [\![b]\!], [\![a]\!]) \tag{25.72}$$

$$[\![\texttt{a or b}]\!] = \mathsf{If}(\mathsf{truthy}([\![a]\!]), [\![a]\!], [\![b]\!]) \tag{25.73}$$

This faithfully models Python's semantics: `0 or "hello"` returns `"hello"` (not `True`), and `[] and 42` returns `[]` (not `False`).

25.8.3 Exception Semantics via Bottom

Exceptions are modeled by Bottom:

- Division by zero: $\mathsf{binop}(\mathsf{FLOORDIV}, x, \mathsf{IntObj}(0)) = \mathsf{Bottom}$.
- Type errors: $\mathsf{binop}(\mathsf{ADD}, \mathsf{IntObj}(n), \mathsf{StrObj}(s)) = \mathsf{Bottom}$.
- Explicit raise: compiled to Bottom.

Bottom propagates strictly: $\mathsf{binop}(\mathsf{op}, \mathsf{Bottom}, x) = \mathsf{Bottom}$ and $\mathsf{binop}(\mathsf{op}, x, \mathsf{Bottom}) = \mathsf{Bottom}$.

`try/except` blocks are modeled by wrapping the body in a shared "handler context" function:

$$[\![\texttt{try: body except E: handler}]\!] = \mathsf{py_try_h}([\![\mathsf{body}]\!])$$

where h is a hash of the handler signature. The handler context is a shared function, so two programs with the same try/except structure get the same Z3 symbol.

25.8.4 isinstance as Fiber Projection

Definition 25.31 (isinstance Semantics)**.**

$$[\![\texttt{isinstance(x, T)}]\!] = \mathsf{BoolObj}(\mathsf{tag}([\![x]\!]) = \tau_T)$$

where τ_T is the tag corresponding to type T:

`int`	$\to \mathsf{TInt}$	`str`	$\to \mathsf{TStr}$	`bool`	$\to \mathsf{TBool}$
`list`	$\to \mathsf{TPair}$	`tuple`	$\to \mathsf{TPair}$	`dict`	$\to \mathsf{TRef}$
`set`	$\to \mathsf{TRef}$	`float`	$\to \mathsf{TInt}$	`type`	$\to \mathsf{TRef}$

For tuple arguments (`isinstance(x, (int, str))`):

$$[\![\texttt{isinstance(x, (int, str))}]\!] = \mathsf{BoolObj}(\mathsf{tag}([\![x]\!]) = \mathsf{TInt} \vee \mathsf{tag}([\![x]\!]) = \mathsf{TStr})$$

This is the *fiber projection*: `isinstance` asks which fiber of the type fibration the value lives in. It is purely decidable (tag comparison in EnumSort).

25.8.5 Comprehensions

List, set, and generator comprehensions compile to shared function symbols keyed by a canonical hash of the element expression:

Definition 25.32 (Comprehension Semantics)**.**

$$[\![\![\mathsf{elt}(x)\ \texttt{for}\ x\ \texttt{in}\ \mathsf{xs}]\!]\!] = \mathsf{py_comp}_{h(\mathsf{elt})}([\![\mathsf{xs}]\!])$$

where $h(\mathsf{elt}) = \mathsf{hash}(\mathsf{ast.dump}(\mathsf{elt})) \bmod 2^{31}$.

Two comprehensions with the same element expression AST get the same shared function symbol, regardless of whether they appear as a comprehension or an explicit loop with append (D4 path).

25.8.6 Attribute Access

Definition 25.33 (Attribute Semantics)**.** $[\![x.\mathsf{attr}]\!] = \mathsf{py_attr_attr}([\![x]\!])$

This is a shared uninterpreted function. No axioms are added by default (we don't model Python's attribute lookup protocol in full). But when both programs access the same attribute on the same object, they get the same Z3 term (by sharing).

25.9 Shared Function Symbols (Univalence)

25.9.1 The Registry

The *shared function registry* ensures both programs use the same Z3 function for the same Python operation.

Definition 25.34 (Shared Function Registry)**.** $\mathcal{R} : (\mathsf{name} \times \mathsf{arity}) \to \mathsf{Z3FuncDecl}$ maps operation names to canonical Z3 function declarations. The function $\mathsf{shared_fn}(\mathsf{name}, \mathsf{arity})$ returns the canonical Z3 function, creating it on first access.

25.9.2 Builtin Classification

Category	Functions	Shared symbol
Sorting	`sorted`, `reversed`	py_sorted, py_reversed
Collection	`list`, `tuple`, `set`, `frozenset`, `dict`	py_list, etc.
Aggregation	`sum`, `any`, `all`, `min`, `max`	py_sum, etc.
Higher-order	`map`, `filter`, `zip`, `enumerate`	py_map, etc.
Type conversion	`int`, `str`, `bool`, `float`	via unop
Type checking	`isinstance`, `type`	via tag
String methods	`.upper`, `.lower`, `.strip`, `.split`, `.join`, `.replace`, `.find`	py_meth_upper, etc.
Dict methods	`.get`, `.keys`, `.values`, `.items`, `.update`, `.pop`	py_meth_get, etc.
List methods	`.append`, `.extend`, `.sort`, `.reverse`, `.pop`, `.insert`	py_mut_append, etc.
Mutation ops	`+=`, `*=`, etc. on containers	py_mut_iadd, etc.

25.9.3 Why Sharing Preserves Decidability

Proposition 25.35 (Shared Symbols Do Not Affect Decidability)**.** *Replacing unique function symbols (*sorted_1, sorted_5*) with shared symbols (*py_sorted*) does not change the decidability status of any query.*

In the QF fragment, all function symbols are uninterpreted — Z3 treats them as "black boxes" regardless of name. Using the same name for two occurrences constrains *Z3 (both must have the same interpretation), which can only make queries* easier *to decide (fewer models to search), never harder.*

Listing 25.8: Shared function registry: complete implementation

```
class Theory:
    def __init__(self):
        # ... PyObj ADT, TypeTag, RecFunctions ...
        self._shared_fns = {}  # (name, arity) -> Z3 Function

    def shared_fn(self, name, arity):
        """Return canonical Z3 function for a Python operation.

        Both programs use the SAME Z3 function for the same op.
        This is the univalence principle in action.
        """
        key = (name, arity)
        if key not in self._shared_fns:
            sorts = [self.S] * arity + [self.S]
            self._shared_fns[key] = Function(
                f'py_{name}', *sorts)
        return self._shared_fns[key]

    def semantic_axioms(self, solver):
        """Add equational axioms (path constructors)."""
        x = Const('_ax', self.S)
        # sorted: idempotent
        if ('sorted', 1) in self._shared_fns:
            sf = self._shared_fns[('sorted', 1)]
            solver.add(ForAll([x], sf(sf(x)) == sf(x)))
        # reversed: involution
        if ('reversed', 1) in self._shared_fns:
            rf = self._shared_fns[('reversed', 1)]
            solver.add(ForAll([x], rf(rf(x)) == x))
        # len: preserved by sorted, reversed
        if ('len', 1) in self._shared_fns:
            lf = self._shared_fns[('len', 1)]
            for dep in ('sorted', 'reversed', 'list'):
                if (dep, 1) in self._shared_fns:
                    df = self._shared_fns[(dep, 1)]
                    solver.add(ForAll([x],
                        lf(df(x)) == lf(x)))

# Usage in the compiler:
SHARED_BUILTINS = {
    'sorted','reversed','list','tuple','set',
    'frozenset','dict','sum','any','all',
    'enumerate','zip','map','filter',
}

def compile_call(node, env):
    T = env.T
    name = node.func.id
    args = [compile_expr(a, env) for a in node.args]
    if name in SHARED_BUILTINS and args:
        return T.shared_fn(name, len(args))(*args)
    # ... other cases ...
```

Listing 25.9: unop: complete Z3 implementation

```
NEG=50; ABS_=52; NOT=53; BOOL_=56; INT_=57; LEN_=55

```

```
unop = RecFunction('unop', IntSort(), S, S)
op = Int('_uo'); a = Const('_ua', S)
ai = S.ival(a)

RecAddDefinition(unop, [op, a],
    If(S.is_IntObj(a),
        If(op==NEG,  S.IntObj(-ai),
        If(op==ABS_, S.IntObj(If(ai>=0, ai, -ai)),
        If(op==NOT,  S.BoolObj(ai==0),
        If(op==BOOL_,S.BoolObj(ai!=0),
        If(op==INT_, a,
        S.Bottom))))),
    If(S.is_BoolObj(a),
        If(op==NOT,  S.BoolObj(Not(S.bval(a))),
        If(op==BOOL_,a,
        If(op==INT_, S.IntObj(If(S.bval(a),1,0)),
        S.Bottom))),
    If(S.is_StrObj(a),
        If(op==LEN_, S.IntObj(Length(S.sval(a))),
        If(op==BOOL_,S.BoolObj(Length(S.sval(a))>0),
        S.Bottom)),
    If(S.is_NoneObj(a),
        If(op==BOOL_,S.BoolObj(False),
        If(op==NOT,  S.BoolObj(True),
        S.Bottom)),
    If(S.is_Pair(a),
        If(op==BOOL_,S.BoolObj(True),
        S.Bottom),
    S.Bottom))))))
```

25.9.4 The Extensibility Principle

Theorem 25.36 (Safe Extension of the Shared Registry)**.** *Adding a new shared function* $\mathsf{py_new_op} : \mathsf{PyObj}^k \to \mathsf{PyObj}$ *to the registry* $\mathcal{R}$ *is* always safe*:*

1. *It does not break decidability of existing queries (the new function is uninterpreted; it adds no quantifiers).*
2. *It can only* increase *the number of provable equivalences (if both programs call the same operation, they now get the same Z3 symbol).*
3. *Adding axioms for the new function is* optional *— they increase expressiveness but may exit the QF fragment.*

This means the theory can grow incrementally: every new Python operation that appears in benchmark programs can be added as a shared function without auditing the entire theory for decidability regressions.

Chapter 26

The Semantic Algebra: Complete Axiomatization

This chapter develops the full algebraic theory underlying CTTT. Each axiom serves a dual role: it is an *equational law* of Python's operations AND a *path constructor* in the cubical HIT PyComp (Definition 4.2). The axioms are organized by algebraic structure, with proofs of soundness, explicit Z3 encodings, and analysis of which benchmark pairs each axiom enables.

26.1 The Integer Ring

The integers under $+$ and $\times$ form a commutative ring. This is the most fundamental algebraic structure for program equivalence: whenever two programs compute the same arithmetic expression via different orderings or groupings, the ring axioms provide the connecting path.

26.1.1 Ring Axioms as Path Constructors

Definition 26.1 (The Integer Ring Paths)**.** The following paths are added to PyComp:
Additive abelian group:

$$\mathsf{add_comm} : \prod_{a,b:\mathsf{PyObj}} \mathsf{Path}(\mathsf{binop}(\mathsf{ADD}, a, b),\ \mathsf{binop}(\mathsf{ADD}, b, a)) \tag{R1}$$

$$\mathsf{add_assoc} : \prod_{a,b,c} \mathsf{Path}(\mathsf{binop}(\mathsf{ADD}, \mathsf{binop}(\mathsf{ADD}, a, b), c),\ \mathsf{binop}(\mathsf{ADD}, a, \mathsf{binop}(\mathsf{ADD}, b, c))) \tag{R2}$$

$$\mathsf{add_zero_r} : \prod_{a} \mathsf{Path}(\mathsf{binop}(\mathsf{ADD}, a, \mathsf{IntObj}(0)),\ a) \tag{R3}$$

$$\mathsf{add_zero_l} : \prod_{a} \mathsf{Path}(\mathsf{binop}(\mathsf{ADD}, \mathsf{IntObj}(0), a),\ a) \tag{R4}$$

$$\mathsf{add_neg} : \prod_{a} \mathsf{Path}(\mathsf{binop}(\mathsf{ADD}, a, \mathsf{unop}(\mathsf{NEG}, a)),\ \mathsf{IntObj}(0)) \tag{R5}$$

Multiplicative commutative monoid:

$$\mathsf{mul_comm} : \prod_{a,b} \mathsf{Path}(\mathsf{binop}(\mathsf{MUL}, a, b),\ \mathsf{binop}(\mathsf{MUL}, b, a)) \tag{R6}$$

$$\mathsf{mul_assoc} : \prod_{a,b,c} \mathsf{Path}(\mathsf{binop}(\mathsf{MUL}, \mathsf{binop}(\mathsf{MUL}, a, b), c),\ \mathsf{binop}(\mathsf{MUL}, a, \mathsf{binop}(\mathsf{MUL}, b, c))) \tag{R7}$$

$$\mathsf{mul_one_r} : \prod_{a} \mathsf{Path}(\mathsf{binop}(\mathsf{MUL}, a, \mathsf{IntObj}(1)),\ a) \tag{R8}$$

$$\mathsf{mul_one_l} : \prod_{a} \mathsf{Path}(\mathsf{binop}(\mathsf{MUL}, \mathsf{IntObj}(1), a),\ a) \tag{R9}$$

$$\mathsf{mul_zero} : \prod_{a} \mathsf{Path}(\mathsf{binop}(\mathsf{MUL}, a, \mathsf{IntObj}(0)),\ \mathsf{IntObj}(0)) \tag{R10}$$

Distributivity:

$$\mathsf{distrib_l} : \prod_{a,b,c} \mathsf{Path}(\mathsf{binop}(\mathsf{MUL}, a, \mathsf{binop}(\mathsf{ADD}, b, c)),\ \mathsf{binop}(\mathsf{ADD}, \mathsf{binop}(\mathsf{MUL}, a, b), \mathsf{binop}(\mathsf{MUL}, a, c))) \tag{R11}$$

Involution:

$$\mathsf{neg_neg} : \prod_{a} \mathsf{Path}(\mathsf{unop}(\mathsf{NEG}, \mathsf{unop}(\mathsf{NEG}, a)),\ a) \tag{R12}$$

$$\mathsf{abs_idem} : \prod_{a} \mathsf{Path}(\mathsf{unop}(\mathsf{ABS}, \mathsf{unop}(\mathsf{ABS}, a)),\ \mathsf{unop}(\mathsf{ABS}, a)) \tag{R13}$$

$$\mathsf{abs_neg} : \prod_{a} \mathsf{Path}(\mathsf{unop}(\mathsf{ABS}, \mathsf{unop}(\mathsf{NEG}, a)),\ \mathsf{unop}(\mathsf{ABS}, a)) \tag{R14}$$

Theorem 26.2 (Soundness of Ring Paths)**.** *Each path in Definition 26.1 is sound: it holds for all* PyObj *values on the* TInt *fiber.*

Proof. On the Int fiber, $a = \mathsf{IntObj}(m)$, $b = \mathsf{IntObj}(n)$. The binop RecFunction reduces to Z3 native integer arithmetic: $\mathsf{binop}(\mathsf{ADD}, \mathsf{IntObj}(m), \mathsf{IntObj}(n)) = \mathsf{IntObj}(m + n)$. The axioms then follow from Z3's built-in integer theory (commutativity, associativity, etc. are tautologies of LIA/NIA). □

26.1.2 Z3 Encoding

The ring axioms are NOT added as ForAll constraints (which could cause E-matching loops). Instead, they are applied as *rewrite rules* during normalization (Phase 2 of Definition 18.3):

Listing 26.1: Ring axiom application during normalization

```
def normalize_arithmetic(term, S):
    """Apply ring axioms as rewrite rules."""
    if is_binop(term, ADD):
        a, b = term.arg(1), term.arg(2)
        # R1: canonical ordering (commutativity)
        if term_order(b) < term_order(a):
            term = S.binop(ADD, b, a)
        # R3/R4: identity elimination
        if is_zero(a): return b
        if is_zero(b): return a
    if is_binop(term, MUL):
        a, b = term.arg(1), term.arg(2)
        # R6: canonical ordering
```

```
        if term_order(b) < term_order(a):
            term = S.binop(MUL, b, a)
        # R8/R9: identity elimination
        if is_one(a): return b
        if is_one(b): return a
        # R10: zero absorption
        if is_zero(a) or is_zero(b):
            return S.IntObj(IntVal(0))
    # R12: double negation
    if is_unop(term, NEG) and is_unop(term.arg(1), NEG):
        return term.arg(1).arg(1)
    return term
```

This is decidable (it's syntactic pattern matching on Z3 terms, terminating because each rule reduces term complexity).

26.1.3 What the Ring Axioms Enable

The ring axioms are the path constructors for the D1 (Rec↔Iter) dichotomy. They enable:

- Factorial: $n \times f(n-1) \equiv f(n-1) \times n$ (R6).
- Sum: $n + f(n-1) \equiv f(n-1) + n$ (R1).
- Product: $\prod_{i=1}^{n} a_i$ in any order (R6 + R7 iterated).
- Polynomial equivalence: $n(n+1)/2 \equiv 1 + 2 + \ldots + n$ (R1 + R2 + R11 iterated).

Proposition 26.3 (Ring Paths for Fold Equivalence)**.** *If the fold operation* op *is in* $\{+, \times\}$ *and the step functions of two programs differ only by operand order, the ring commutativity paths (R1 or R6) connect them.*

More generally: any two ways of computing a polynomial over the input variables that give the same polynomial (by the polynomial identity) are connected by a finite sequence of ring paths.

26.2 The Lattice of Min and Max

The operations min and max form a *distributive lattice* on the integers (with the usual ordering).

Definition 26.4 (Min-Max Lattice Paths).

$$\mathsf{min_comm} : \prod_{a,b} \mathsf{Path}(\mathsf{binop}(\mathsf{MIN}, a, b),\ \mathsf{binop}(\mathsf{MIN}, b, a)) \tag{L1}$$

$$\mathsf{max_comm} : \prod_{a,b} \mathsf{Path}(\mathsf{binop}(\mathsf{MAX}, a, b),\ \mathsf{binop}(\mathsf{MAX}, b, a)) \tag{L2}$$

$$\mathsf{min_assoc} : \prod_{a,b,c} \mathsf{Path}(\mathsf{binop}(\mathsf{MIN}, \mathsf{binop}(\mathsf{MIN}, a, b), c),\ \mathsf{binop}(\mathsf{MIN}, a, \mathsf{binop}(\mathsf{MIN}, b, c))) \tag{L3}$$

$$\mathsf{max_assoc} : \prod_{a,b,c} \ldots \tag{L4}$$

$$\mathsf{min_idem} : \prod_{a} \mathsf{Path}(\mathsf{binop}(\mathsf{MIN}, a, a),\ a) \tag{L5}$$

$$\mathsf{max_idem} : \prod_{a} \mathsf{Path}(\mathsf{binop}(\mathsf{MAX}, a, a),\ a) \tag{L6}$$

$$\mathsf{min_max_absorb} : \prod_{a,b} \mathsf{Path}(\mathsf{binop}(\mathsf{MIN}, a, \mathsf{binop}(\mathsf{MAX}, a, b)),\ a) \tag{L7}$$

$$\mathsf{max_min_absorb} : \prod_{a,b} \mathsf{Path}(\mathsf{binop}(\mathsf{MAX}, a, \mathsf{binop}(\mathsf{MIN}, a, b)),\ a) \tag{L8}$$

$$\mathsf{min_max_distrib} : \prod_{a,b,c} \mathsf{Path}(\mathsf{binop}(\mathsf{MIN}, a, \mathsf{binop}(\mathsf{MAX}, b, c)),\ \mathsf{binop}(\mathsf{MAX}, \mathsf{binop}(\mathsf{MIN}, a, b), \mathsf{binop}(\mathsf{MIN}, a, c))) \tag{L9}$$

These paths are critical for programs that compute minimums/maximums in different orders (e.g., DP algorithms that compute $\min(\min(a, b), c)$ vs. $\min(a, \min(b, c))$).

Theorem 26.5 (Lattice Paths for DP Equivalence). *If two DP algorithms compute the same recurrence using* min *or* max *but with different association/ordering of the terms, the lattice paths (L1–L9) connect them.*

In particular: the edit distance recurrence $d(i, j) = \min(d(i-1, j)+1,\ d(i, j-1)+1,\ d(i-1, j-1)+c)$ *is invariant under permutation of the three arguments to* min *(by L1 + L3 iterated).*

26.3 The Sequence Algebra

Sequences (lists, tuples) form a *free monoid* under concatenation, with additional structure from sorting, reversal, and length.

26.3.1 The Free Monoid

Definition 26.6 (Sequence Monoid Paths).

$$\mathsf{concat_assoc} : \prod_{x,y,z} \mathsf{Path}((x \mathbin{++} y) \mathbin{++} z,\ x \mathbin{++} (y \mathbin{++} z)) \tag{S1}$$

$$\mathsf{concat_empty_r} : \prod_{x} \mathsf{Path}(x \mathbin{++} [],\ x) \tag{S2}$$

$$\mathsf{concat_empty_l} : \prod_{x} \mathsf{Path}([] \mathbin{++} x,\ x) \tag{S3}$$

where $[] = \mathsf{NoneObj}$ (the empty Pair-chain) and $\mathbin{++}$ is modeled by the shared function $\mathsf{py_call___add__}$ on Pair-chains.

26.3.2 Sorting as a Retraction

Definition 26.7 (Sorting Paths).

$$\mathsf{sorted_idem} : \prod_{x} \mathsf{Path}(\mathsf{py_sorted}(\mathsf{py_sorted}(x)),\ \mathsf{py_sorted}(x)) \tag{S4}$$

$$\mathsf{sorted_perm_inv} : \prod_{x,y} \mathsf{Path}(\mathsf{py_sorted}(x +\!\!+ y),\ \mathsf{py_sorted}(y +\!\!+ x)) \tag{S5}$$

$$\mathsf{sorted_len} : \prod_{x} \mathsf{Path}(\mathsf{py_len}(\mathsf{py_sorted}(x)),\ \mathsf{py_len}(x)) \tag{S6}$$

Remark 26.8 (Sorting as a Retraction in CTTT). In cubical type theory, sorted is a *retraction* of the sequence type onto the subtype of sorted sequences:

$$r : \mathsf{Seq} \to \mathsf{SortedSeq} \hookrightarrow \mathsf{Seq}$$

with $r \circ r = r$ (idempotence, path S4). The retraction forgets the ordering but preserves the multiset of elements.

This means: in the *quotient type* $\mathsf{Seq}/\sim_{\mathsf{perm}}$ (sequences modulo permutation), sorted is the *canonical representative* of each equivalence class. Two sequences with the same sorted form are permutations of each other.

Path S5 says: the sorted form is permutation-invariant. This is the key property that makes sorting-based algorithms equivalent to non-sorting ones when the output only depends on the multiset.

26.3.3 Reversal as an Involution

Definition 26.9 (Reversal Paths).

$$\mathsf{reversed_invol} : \prod_{x} \mathsf{Path}(\mathsf{py_reversed}(\mathsf{py_reversed}(x)),\ x) \tag{S7}$$

$$\mathsf{reversed_len} : \prod_{x} \mathsf{Path}(\mathsf{py_len}(\mathsf{py_reversed}(x)),\ \mathsf{py_len}(x)) \tag{S8}$$

$$\mathsf{sorted_reversed} : \prod_{x} \mathsf{Path}(\mathsf{py_sorted}(\mathsf{py_reversed}(x)),\ \mathsf{py_sorted}(x)) \tag{S9}$$

S7 makes reversed a $\mathbb{Z}/2\mathbb{Z}$-action on sequences: applying it twice returns to the identity. In cubical type theory, this is a *loop* in the function space: $\mathsf{reversed} \cdot \mathsf{reversed} = \mathsf{id}$ (a 2-torsion element of π_1).

S9 says sorting absorbs reversal: the sorted form of a reversed list equals the sorted form of the original. This is because sorting is permutation-invariant (S5), and reversal is a permutation.

26.3.4 Length as a Monoid Homomorphism

Definition 26.10 (Length Paths).

$$\mathsf{len_additive} : \prod_{x,y} \mathsf{Path}(\mathsf{py_len}(x +\!\!+ y),\ \mathsf{binop}(\mathsf{ADD}, \mathsf{py_len}(x), \mathsf{py_len}(y))) \tag{S10}$$

$$\mathsf{len_empty} : \mathsf{Path}(\mathsf{py_len}(\mathsf{NoneObj}),\ \mathsf{IntObj}(0)) \tag{S11}$$

$$\mathsf{len_singleton} : \prod_{a} \mathsf{Path}(\mathsf{py_len}(\mathsf{Pair}(a, \mathsf{NoneObj})),\ \mathsf{IntObj}(1)) \tag{S12}$$

len is a *monoid homomorphism* from $(\mathsf{Seq}, +\!\!+, [])$ to $(\mathbb{N}, +, 0)$. This is a genuine algebraic structure preservation — it means Z3 can reason about lengths using the additive monoid theory (decidable QF_LIA) even when the sequences themselves are symbolic.

26.3.5 Fold as Universal Catamorphism

The most powerful algebraic structure: fold is the *unique catamorphism* from the free monoid of sequences to any algebra.

Definition 26.11 (Fold Algebra Paths)**.**

$$\mathsf{fold_empty} : \prod_{\mathsf{op},e} \mathsf{Path}(\mathsf{fold}(\mathsf{op}, 0, 0, e),\ e) \tag{F1}$$

$$\mathsf{fold_comm} : \prod_{\mathsf{op},e,x,y} \mathsf{Path}(\mathsf{fold}(\mathsf{op}, 0, 2, e)|_{[x,y]},\ \mathsf{fold}(\mathsf{op}, 0, 2, e)|_{[y,x]}) \tag{F2}$$

when op is commutative

$$\mathsf{fold_assoc} : \prod_{\mathsf{op},e,x,y,z} \mathsf{Path}(\mathsf{binop}(\mathsf{op}, \mathsf{fold}(\mathsf{op}, \ldots)|_{[x,y]}, z),\ \mathsf{fold}(\mathsf{op}, \ldots)|_{[x,y,z]}) \tag{F3}$$

when op is associative

$$\mathsf{fold_fusion} : \prod_{h,\mathsf{op}_1,e_1,\mathsf{op}_2,e_2} \mathsf{Path}(h(\mathsf{fold}(\mathsf{op}_1, e_1, \ldots)),\ \mathsf{fold}(\mathsf{op}_2, h(e_1), \ldots)) \tag{F4}$$

when h is a homomorphism from (op_1, e_1) to (op_2, e_2)

Theorem 26.12 (Fold Universality)**.** *For any monoid $(M, \oplus, e)$, the function $\mathsf{fold}(\oplus, \mathsf{start}, \mathsf{stop}, e)$ is the* unique *monoid homomorphism from $(\mathbb{N}, +, 0)$ to $(M, \oplus, e)$ that maps $1 \mapsto \mathsf{IntObj}(\mathsf{start})$.*

This uniqueness is the Nat-recursor path (D1): any two computations that fold the same operation with the same identity produce the same result, regardless of fold direction or order (when the operation is commutative and associative).

Proof. By the universal property of $\mathbb{N}$ as the initial object in the category of monoid homomorphisms from $(\mathbb{N}, +, 0)$. The fold is the image of the unique morphism. Any other computation satisfying the same recurrence factors through this unique morphism, hence equals the fold. □

26.3.6 Sum and Product as Specific Folds

Corollary 26.13 (Sum Permutation Invariance)**.** $\mathsf{py_sum}$ *is permutation-invariant:* $\mathsf{py_sum}(\mathsf{py_sorted}(x)) \equiv \mathsf{py_sum}(x)$ *for all x.*

Proof. sum is $\mathsf{fold}(+, 0, \ldots)$. Since $(+)$ is commutative and associative (R1, R2), the fold is permutation-invariant (F2, F3). Sorting is a permutation. Therefore $\mathsf{sum}(\mathsf{sorted}(x)) = \mathsf{sum}(x)$. □

Similarly for $\mathsf{product}$, min, max, gcd, lcm, xor — any fold over a commutative and associative operation.

26.4 The Boolean Algebra

Python's boolean operations form a *Boolean algebra* (actually a Heyting algebra, since Python 3 doesn't have classical excluded middle for general predicates, but booleans are classical).

Definition 26.14 (Boolean Paths).

$$\mathsf{not_not} : \prod_{p} \mathsf{Path}(\mathsf{unop}(\mathsf{NOT}, \mathsf{unop}(\mathsf{NOT}, p)),\ p) \tag{B1}$$

$$\mathsf{and_comm} : \prod_{p,q} \mathsf{Path}(\mathsf{BoolObj}(\mathsf{bval}(p) \wedge \mathsf{bval}(q)),\ \mathsf{BoolObj}(\mathsf{bval}(q) \wedge \mathsf{bval}(p))) \tag{B2}$$

$$\mathsf{or_comm} : \prod_{p,q} \ldots \tag{B3}$$

$$\mathsf{demorgan_and} : \prod_{p,q} \mathsf{Path}(\mathsf{unop}(\mathsf{NOT}, \mathsf{BoolObj}(\mathsf{bval}(p) \wedge \mathsf{bval}(q))),\ \mathsf{BoolObj}(\neg\mathsf{bval}(p) \vee \neg\mathsf{bval}(q))) \tag{B4}$$

$$\mathsf{demorgan_or} : \prod_{p,q} \ldots \tag{B5}$$

$$\mathsf{demorgan_all} : \prod_{\mathsf{xs}} \mathsf{Path}(\mathsf{py_all}(\mathsf{xs}),\ \mathsf{unop}(\mathsf{NOT}, \mathsf{py_any}(\mathsf{py_map}(\mathsf{NOT}, \mathsf{xs})))) \tag{B6}$$

$$\mathsf{demorgan_any} : \prod_{\mathsf{xs}} \mathsf{Path}(\mathsf{py_any}(\mathsf{xs}),\ \mathsf{unop}(\mathsf{NOT}, \mathsf{py_all}(\mathsf{py_map}(\mathsf{NOT}, \mathsf{xs})))) \tag{B7}$$

B6 and B7 are the *quantifier duality* paths: $\forall x. P(x) \iff \neg\exists x. \neg P(x)$. These directly enable the equivalence of `all`-based and `any`-based programs (benchmark pair eq35).

26.5 The Idempotent Semiring

Several Python operations form *idempotent semirings* — algebraic structures where the "addition" is idempotent ($a + a = a$).

Definition 26.15 (Idempotent Semiring Paths). For min/max as "addition" and $+/\times$ as "multiplication":

$$\mathsf{min_idem} : \prod_{a} \mathsf{Path}(\mathsf{binop}(\mathsf{MIN}, a, a),\ a) \tag{I1}$$

$$\mathsf{max_idem} : \prod_{a} \mathsf{Path}(\mathsf{binop}(\mathsf{MAX}, a, a),\ a) \tag{I2}$$

$$\mathsf{set_idem} : \prod_{x} \mathsf{Path}(\mathsf{py_set}(\mathsf{py_set}(x)),\ \mathsf{py_set}(x)) \tag{I3}$$

$$\mathsf{frozenset_idem} : \prod_{x} \mathsf{Path}(\mathsf{py_frozenset}(\mathsf{py_frozenset}(x)),\ \mathsf{py_frozenset}(x)) \tag{I4}$$

$$\mathsf{sorted_idem} : \prod_{x} \mathsf{Path}(\mathsf{py_sorted}(\mathsf{py_sorted}(x)),\ \mathsf{py_sorted}(x)) \tag{I5}$$

$$\mathsf{list_idem} : \prod_{x} \mathsf{Path}(\mathsf{py_list}(\mathsf{py_list}(x)),\ \mathsf{py_list}(x)) \tag{I6}$$

Idempotence is critical for programs that apply an operation "just in case" (e.g., $\mathsf{sorted}(\mathsf{sorted}(x))$ when one branch already sorts and another doesn't). The idempotence path collapses the double application to a single one.

Proposition 26.16 (Idempotent Fold). *If* op *is idempotent* ($\mathsf{op}(a, a) = a$ *for all* a)*, then:*

$$\mathsf{fold}(\mathsf{op}, e, \mathsf{xs_with_duplicates}) \equiv \mathsf{fold}(\mathsf{op}, e, \mathsf{deduplicated_xs})$$

Duplicates in the fold input don't affect the result.

26.6 The Effect Algebra

The *effect algebra* distinguishes mutation from pure computation. Unlike the previous algebras (which generate paths), the effect algebra generates *non-paths*: it asserts that certain terms are *NOT* equal.

Definition 26.17 (Effect Separation Axioms)**.**

$$\mathsf{mut_ne_pure} : \mathsf{py_mut_iadd}(x, v) \neq \mathsf{py_call___add__}(x, v) \tag{E1}$$

$$\mathsf{mut_sort_ne_sorted} : \mathsf{py_mut_sort}(x) \neq \mathsf{py_sorted}(x) \tag{E2}$$

$$\mathsf{mut_reverse_ne_reversed} : \mathsf{py_mut_reverse}(x) \neq \mathsf{py_reversed}(x) \tag{E3}$$

$$\mathsf{ref_ne_val} : \mathsf{Ref}(a) \neq \mathsf{IntObj}(n) \tag{E4}$$

$$\mathsf{ref_identity} : a \neq b \implies \mathsf{Ref}(a) \neq \mathsf{Ref}(b) \tag{E5}$$

E1–E3 are the *mutation-purity separation*: in-place operations and their pure counterparts are *different shared symbols*, so Z3 treats them as unrelated. This is how CTTT correctly handles the 2 FP benchmark cases (neq06, neq20).

E4–E5 are *constructor disjointness*: a Ref is never equal to a value (different constructors in the ADT), and two Refs with different addresses are different objects (identity semantics).

Remark 26.18 (Effects in Cubical Type Theory)**.** In standard cubical type theory, the effect axioms are *negative paths* — they assert the *absence* of a path. mut_ne_pure says: there is NO path from mut_iadd to call_add.

This is formalized using the *empty type* 0:

$$\mathsf{mut_ne_pure} : \mathsf{Path}(\mathsf{mut_iadd}(x, v),\ \mathsf{call_add}(x, v)) \to 0$$

— assuming such a path exists leads to a contradiction.

In Z3, this is implemented trivially: since mut_iadd and call_add are different function symbols, Z3 never identifies them (different uninterpreted functions are always distinguishable).

26.7 The Composed Algebra

The full semantic algebra is the *product* of all sub-algebras:

Definition 26.19 (The Full Semantic Algebra)**.** $\mathcal{A}_{\mathsf{Py}} = \mathcal{R}_{\mathbb{Z}} \times \mathcal{L}_{\min,\max} \times \mathcal{M}_{\mathsf{Seq}} \times \mathcal{B}_{\mathbb{B}} \times \mathcal{I}_{\mathsf{idem}} \times \mathcal{E}_{\mathsf{eff}}$ where:

Sub-algebra	Paths	Structure	Applications
$\mathcal{R}_{\mathbb{Z}}$	R1–R14	Commutative ring	Arithmetic, fold order
$\mathcal{L}_{\min,\max}$	L1–L9	Distributive lattice	DP, optimization
$\mathcal{M}_{\mathsf{Seq}}$	S1–S12, F1–F4	Free monoid + sort/rev	Collection processing
$\mathcal{B}_{\mathbb{B}}$	B1–B7	Boolean algebra	Logical equivalence
$\mathcal{I}_{\mathsf{idem}}$	I1–I6	Idempotent semiring	Deduplication, caching
$\mathcal{E}_{\mathsf{eff}}$	E1–E5	Effect separation	Mutation detection

Total: **52 path constructors** (46 positive paths + 5 negative separation axioms + 1 identity axiom).

Theorem 26.20 (Algebra Soundness)**.** *Every positive path in* $\mathcal{A}_{\mathsf{Py}}$ *is sound: it holds for all* PyObj *values on the relevant type fiber. Every negative axiom (effect separation) is sound: the separated operations are genuinely different.*

Proof. Each sub-algebra's paths are proved sound in its respective section above. The product algebra inherits soundness componentwise (paths in one sub-algebra don't interact with paths in another, since they operate on disjoint sets of function symbols). □

Theorem 26.21 (Algebra Decidability Preservation). *Adding the paths of $\mathcal{A}_{\mathsf{Py}}$ to the path search engine (Definition 18.3) preserves decidability:*

1. *The rewrite system remains terminating (each rule reduces a complexity measure).*
2. *The rewrite system remains confluent (by Theorem 18.4).*
3. *All previously decidable queries remain decidable.*
4. *The paths are applied as rewrite rules (no quantifiers added to the Z3 solver).*

26.8 Interaction Between Sub-Algebras

The sub-algebras are not fully independent — they interact through the PyObj ADT. The interactions provide *additional* paths beyond what each sub-algebra provides alone:

26.8.1 Ring–Fold Interaction

The fold catamorphism (F1–F4) interacts with the ring axioms (R1–R14) to produce the *fold canonicalization* path: any fold of a ring operation produces a canonical form.

Theorem 26.22 (Fold Canonicalization). *For a ring operation* $\mathsf{op} \in \{+, \times\}$*:*

$$\mathsf{fold}(\mathsf{op}, \mathsf{start}, \mathsf{stop}, e) = \mathsf{fold}(\mathsf{op}, \mathsf{start}', \mathsf{stop}', e')$$

if and only if they compute the same ring element (same value as a polynomial in start *and* stop*).*

This is decidable: polynomial identity testing over $\mathbb{Z}$ is in P (by the Schwartz–Zippel lemma applied to integer polynomials, or by direct Z3 integer arithmetic).

26.8.2 Lattice–Fold Interaction

Folding a lattice operation (min or max) produces the lattice join/meet of the elements. Combined with the lattice distributivity (L9), this enables:

Proposition 26.23. $\min(\mathsf{fold}(+, \ldots), \mathsf{fold}(+, \ldots))$ *can be distributed over the folds (by L9 + F3), enabling DP-style equivalences where the "min" is applied at different levels of the recursion.*

26.8.3 Sequence–Boolean Interaction

The De Morgan paths (B6, B7) interact with the sequence fold structure. $\mathsf{all}(\mathsf{xs})$ is semantically $\mathsf{fold}(\wedge, \top, \mathsf{xs})$ (fold of conjunction), and $\mathsf{any}(\mathsf{xs})$ is $\mathsf{fold}(\vee, \bot, \mathsf{xs})$ (fold of disjunction). The De Morgan path transposes these:

$$\mathsf{fold}(\wedge, \top, \mathsf{xs}) \equiv \neg\, \mathsf{fold}(\vee, \bot, \mathsf{map}(\neg, \mathsf{xs}))$$

This is a *natural transformation* between two functors on the sequence category — a genuinely categorical path that the ring and lattice algebras alone cannot produce.

26.9 Soundness of the Z3 Theory

We now state and prove the main soundness theorem: the Z3 PyObj theory is a *sound* (but not complete) model of Python semantics for the supported fragment.

26.9.1 The Supported Fragment

Definition 26.24 (Supported Python Fragment $\mathsf{Py}_\square$)**.** The fragment $\mathsf{Py}_\square \subset \mathsf{Python}$ consists of programs that:

1. Use only the types modeled by PyObj constructors (int, bool, str, None, list/tuple via Pair, and Ref-abstracted containers);
2. Terminate on all inputs (no infinite loops or recursion);
3. Do not use reflection (`eval`, `exec`, `getattr` with computed names);
4. Do not depend on object identity (`is`) for non-singleton types;
5. Do not rely on floating-point precision, hash ordering, or other implementation-defined behavior.

26.9.2 The Denotational Semantics

Definition 26.25 (Python Denotation)**.** The *Python denotation* of a function f is the mathematical function $[\![f]\!]_{\mathsf{Py}} : V^n \to V$ where V is the set of Python values and $[\![f]\!]_{\mathsf{Py}}(v_1, \ldots, v_n)$ is the value returned by executing $f(v_1, \ldots, v_n)$ in CPython.

Definition 26.26 (Z3 Denotation)**.** The *Z3 denotation* of a function f is the Z3 term $[\![f]\!]_{\mathsf{Z3}} : \mathsf{PyObj}^n \to \mathsf{PyObj}$ produced by the compiler (Definition 25.16).

Definition 26.27 (Embedding)**.** The *embedding* $\iota : V \to \mathsf{PyObj}$ maps Python values to PyObj constructors:

$$\iota(v) := \begin{cases} \mathsf{IntObj}(v) & v \in \mathbb{Z} \\ \mathsf{BoolObj}(v) & v \in \{\mathsf{True}, \mathsf{False}\} \\ \mathsf{StrObj}(v) & v \text{ is a string} \\ \mathsf{NoneObj} & v = \mathsf{None} \\ \mathsf{Pair}(\iota(v_1), \iota(v_2 \ldots v_n)) & v = [v_1, \ldots, v_n] \\ \mathsf{Ref}(\mathsf{fresh}) & \text{otherwise} \end{cases}$$

26.9.3 The Soundness Theorem

Theorem 26.28 (Soundness of the Z3 Theory)**.** *For any two functions $f, g \in \mathsf{Py}_\square$ with the same signature:*

$$\models_{\mathsf{Z3}} [\![f]\!]_{\mathsf{Z3}} = [\![g]\!]_{\mathsf{Z3}} \implies \forall \vec{v} \in V^n.\ [\![f]\!]_{\mathsf{Py}}(\vec{v}) = [\![g]\!]_{\mathsf{Py}}(\vec{v})$$

That is: if Z3 proves the compiled terms equal (in all models of the PyObj *theory), then the Python functions agree on all inputs in the supported fragment.*

Proof. We prove this by structural induction on the compilation rules E1–E12. The key steps:

Step 1 (Embedding correctness). For each compilation rule E_k, we show that the Z3 term agrees with the Python evaluation under the embedding ι:

$$\iota([\![e]\!]_{\mathsf{Py}}[\vec{v}]) = [\![e]\!]_{\mathsf{Z3}}[\iota(\vec{v})]$$

for all e in $\mathsf{Py}_\square$ and all values $\vec{v}$.

E1 (Literals): $\iota(42) = \mathsf{IntObj}(42)$ by definition. ✓

E3 (Binary operations): For $e_1 + e_2$ where both are integers:

$$\iota([\![e_1 + e_2]\!]_{\mathsf{Py}}) = \iota(m + n) = \mathsf{IntObj}(m + n) = \mathsf{binop}(\mathsf{ADD}, \mathsf{IntObj}(m), \mathsf{IntObj}(n)) = [\![e_1 + e_2]\!]_{\mathsf{Z3}}$$

using the binop RecFunction definition. The same argument applies for $-, \times, //,$ mod and string concatenation. ✓

E6 (Short-circuit booleans): Python's `a and b` returns a if a is falsy, else b. The Z3 compilation is $\mathsf{If}(\mathsf{truthy}([\![a]\!]), [\![b]\!], [\![a]\!])$. Since truthy faithfully models Python's `__bool__` on the supported types, the embedding commutes. ✓

E8 (Calls to interpreted builtins): `abs`, `len`, `int`, `bool` compile to unop with the corresponding operation tag. The unop definition matches Python's semantics on each fiber. ✓

E8 (Calls to uninterpreted builtins): `sorted`, `set`, etc. compile to shared function symbols. By the congruence axiom, equal inputs produce equal outputs. This is sound because Python's builtins are deterministic functions. ✓

Step 2 (Universal quantification). Since Z3 proves $[\![f]\!]_{\mathsf{Z3}} = [\![g]\!]_{\mathsf{Z3}}$ for all models, in particular it holds in the model induced by the embedding ι. Therefore for all $\vec{v}$:

$$\iota([\![f]\!]_{\mathsf{Py}}(\vec{v})) = [\![f]\!]_{\mathsf{Z3}}(\iota(\vec{v})) = [\![g]\!]_{\mathsf{Z3}}(\iota(\vec{v})) = \iota([\![g]\!]_{\mathsf{Py}}(\vec{v}))$$

Since ι is injective on the supported types (different Python values map to different PyObj terms), we conclude $[\![f]\!]_{\mathsf{Py}}(\vec{v}) = [\![g]\!]_{\mathsf{Py}}(\vec{v})$. □

Remark 26.29 (Incompleteness)**.** The converse does not hold: there exist equivalent Python functions whose Z3 terms are distinct. The incompleteness arises from:

1. **Opacity**: uninterpreted functions may compute the same result via different intermediate steps, but Z3 cannot determine this without axioms.
2. **Abstraction collapse**: types mapped to Ref lose their internal structure, so structurally different containers with the same elements may not be proved equal.
3. **RecFunction undecidability**: two RecFunctions may compute the same function but have different definitions that Z3 cannot equate within its timeout.

The semantic axioms of $\mathcal{A}_{\mathsf{Py}}$ partially close this gap by adding algebraic properties that Z3 can exploit, but full completeness is impossible (by Rice's theorem).

The Nat-eliminator is the bridge between recursive programs and their fold-based canonical forms. In the OTerm language it maps an OFix body to an OFold, transforming general recursion into a structurally terminating iteration whose correctness is guaranteed by the universal property of the natural numbers.

26.10 The General Extraction Algorithm

Given a Python AST for a recursive function, the Nat-eliminator extraction identifies:

1. The **base case**: the guard condition and return value when the parameter is "small enough."
2. The **step operation**: the binary operation applied to the parameter and the recursive result.
3. The **threshold**: the parameter value at which the base case triggers.

The extraction is *not algorithm-specific*: it works for any recursive function of the form:

```
def f(n):
    if n OP threshold: return base_val
    return BINOP(n, f(n - 1))
```

where `OP` is any comparison ($\leq, <, =$), `BINOP` is any binary operation ($+, \times, -, //, \%, |, \&, \oplus$), and either n or $f(n-1)$ can be on either side of `BINOP` (handled by commutativity when applicable).

26.11 From `for i in range(n)` to OFold

A range-based loop is the iterative counterpart of Nat recursion. The compiler detects augmented-assignment patterns inside the loop body and emits a *semantic* fold key rather than a body hash.

Definition 26.30 (Range Fold Compilation)**.** Given `for i in range(n): acc += expr`, the compiler emits

$$\mathsf{OFold}(\texttt{iadd},\ \mathsf{init},\ \mathsf{OOp}(\texttt{range}, (n)))$$

where init is the accumulator value immediately before the loop. The semantic key is determined by the augmented-assignment operator:

Python loop body	OFold key	Builtin equivalent
`acc += x`	`iadd`	`sum(xs)`
`acc *= x`	`imult`	`math.prod(xs)`
`acc |= x`	`ior`	`functools.reduce(or_, xs)`
`acc &= x`	`iand`	`functools.reduce(and_, xs)`
`acc ^= x`	`ixor`	—

This is the OTerm-level realisation of dichotomy D5 (reduce ↔ loop).

26.12 Formal Extraction Theorem

Theorem 26.31 (Extraction Correctness)**.** *If the Nat-eliminator extraction succeeds with parameters* $(\mathsf{base}, \mathsf{step}, \mathsf{threshold}, \mathsf{op})$*, then the canonical fold* $\mathsf{fold}(\mathsf{op}, \mathsf{threshold}, \mathsf{ival}(p)+1, \mathsf{IntObj}(\mathsf{base}))$ *computes the same function as the original recursive program.*

Proof. The fold satisfies the same recurrence as the recursive program:

$$\mathsf{fold}(\mathsf{op}, k, k, \mathsf{base}) = \mathsf{base} \quad (\text{base case: } i \geq \mathsf{stop}) \tag{26.1}$$

$$\mathsf{fold}(\mathsf{op}, i, \mathsf{stop}, \mathsf{acc}) = \mathsf{binop}(\mathsf{op}, \mathsf{IntObj}(i), \mathsf{fold}(\mathsf{op}, i+1, \mathsf{stop}, \mathsf{acc})) \quad (\text{step}) \tag{26.2}$$

This matches the recursive definition $f(n) = \mathsf{binop}(\mathsf{op}, n, f(n-1))$ with $f(k) = \mathsf{base}$, by the Nat-recursor uniqueness (Theorem D1). □

Corollary 26.32 (OFix–OFold Equivalence)**.** *For a function whose body matches the Nat-eliminator pattern, the following OTerm identity holds:*

$$\mathsf{OFix}[f]\big(\mathsf{OCase}(\mathsf{OOp}(cmp, (n, t)),\ \mathsf{OLit}(b),\ \mathsf{OOp}(op, (n, f(n{-}1))))\big) \ = \ \mathsf{OFold}(op,\ \mathsf{OLit}(b),\ \mathsf{OVar}(n))$$

after normalization through Phase 3. The recursive structure OFix *is eliminated in favour of the iterative* OFold*, which is the canonical normal form.*

26.13 Commutativity and Argument Order

When BINOP is commutative (addition, multiplication, bitwise or/and/xor), the recursive function $f(n) = n \oplus f(n{-}1)$ is equivalent to $f(n) = f(n{-}1) \oplus n$. The extractor handles both orderings by checking for the recursive call on either side:

Listing 26.2: Argument-order detection

```
# Pattern 1: return BINOP(n, f(n-1))    -- param on left
# Pattern 2: return BINOP(f(n-1), n)    -- param on right
# Both produce the same OFold when BINOP is commutative.
# For non-commutative ops (sub, floordiv), only Pattern 1 fires.
```

Phase 2 (ring/lattice rewriting) ensures that $\mathsf{OOp}(\mathsf{add}, (a, b))$ and $\mathsf{OOp}(\mathsf{add}, (b, a))$ are identified whenever addition's commutativity applies, so both patterns normalize to the same OFold.

Chapter 27

Iteration Beyond Range: Collection Catamorphisms

The Nat-eliminator handles `for i in range(n)`. Real programs iterate over lists, dicts, strings, generators, and composed iterators. Each collection type admits a *catamorphism*—the unique algebra morphism from its initial algebra—and the compiler maps every iteration pattern to an OFold or OMap in the OTerm language.

27.1 The List Catamorphism

A list $[a, b, c]$ is the Pair-chain $\mathsf{Pair}(a, \mathsf{Pair}(b, \mathsf{Pair}(c, \mathsf{NoneObj})))$. Iteration over it is a *list fold* (catamorphism on the free monoid):

Definition 27.1 (List Fold)**.** For `for x in lst: acc = step(acc, x)`:

$$\mathsf{list_fold}(\mathsf{lst}, \mathsf{acc}) = \begin{cases} \mathsf{acc} & \mathsf{is_NoneObj}(\mathsf{lst}) \\ \mathsf{list_fold}(\mathsf{snd}(\mathsf{lst}),\ \mathsf{step}(\mathsf{acc}, \mathsf{fst}(\mathsf{lst}))) & \mathsf{is_Pair}(\mathsf{lst}) \\ \mathsf{acc} & \text{otherwise} \end{cases}$$

Listing 27.1: List fold Z3 RecFunction

```
def compile_for_over_list(stmt, env, T):
    S = T.S; T._uid += 1; uid = T._uid
    target = stmt.target.id
    iter_term = compile_expr(stmt.iter, env)
    modified = find_modified(stmt.body)

    for var in modified:
        init = env.get(var)
        rfn = RecFunction(f'lfold_{var}_{uid}', S, S, S)
        lst_s = FreshConst(S, f'_lf_{uid}')
        acc_s = FreshConst(S, f'_la_{uid}')
        body_env = env.copy()
        body_env.put(target, S.fst(lst_s))
        body_env.put(var, acc_s)
        exec_stmts(stmt.body, body_env)
        new_val = body_env.get(var)
        RecAddDefinition(rfn, [lst_s, acc_s],
            If(S.is_NoneObj(lst_s), acc_s,
               If(S.is_Pair(lst_s),
                  rfn(S.snd(lst_s), new_val), acc_s)))
        env.put(var, rfn(iter_term, init))
```

27.1.1 The Map-via-Append Pattern

The compiler also detects the *map pattern* (D4): `result = []; for x in xs: result.append(f(x))`, which is semantically [`f(x) for x in xs`], and emits $\mathsf{OMap}(\lambda x.\, f(x),\ xs)$ rather than an OFold. The filtered variant—`if pred(x): result.append(f(x))`—emits $\mathsf{OMap}(\lambda x.\, f(x),\ xs,\ \lambda x.\, \mathsf{pred}(x))$ with an explicit filter predicate.

Proposition 27.2 (Map–Fold Duality)**.** $\mathsf{OMap}(\lambda x.\, f(x),\ xs) = \mathsf{OFold}(\mathit{append},\ [],\ xs)$ *where the fold step is* $\mathsf{step}(\mathsf{acc}, x) = \mathsf{acc} \mathbin{+\!\!+} [f(x)]$*. The* OMap *form is preferred because it exposes the transformation* f *for fusion (Phase 4).*

Theorem 27.3 (List Fold = Nat Fold on Length)**.** *For a list of length* n*:* $\mathsf{list_fold}([a_1, \ldots, a_n], e) = \mathsf{fold}(\mathsf{step}', 0, n, e)$ *where* $\mathsf{step}'(i, \mathsf{acc}) = \mathsf{step}(\mathsf{acc}, a_{i+1})$*. This connects list iteration to range iteration via the D1 path.*

27.2 The General Catamorphism

Definition 27.4 (Collection Catamorphism)**.** For an algebraic data type F with initial algebra μF, the catamorphism $\mathsf{cata}_\alpha : \mu F \to A$ for an F-algebra $\alpha : F(A) \to A$ is the unique morphism making the diagram commute:

$$\begin{tikzcd} F(\mu F) \arrow[r, "\mathsf{in}^{-1}"] \arrow[d, "F(\mathsf{cata}_\alpha)"'] & \mu F \arrow[d, "\mathsf{cata}_\alpha"] \\ F(A) \arrow[r, "\alpha"'] & A \end{tikzcd}$$

Every Python `for` loop whose body has accumulator type A and iterates over a collection of type μF is a catamorphism cata_α for the algebra α induced by the loop body.

The OTerm $\mathsf{OFold}(\mathit{key}, \mathit{init}, \mathit{coll})$ is the uniform representation of all catamorphisms. The *key* identifies the F-algebra α, the *init* is the initial accumulator, and the *coll* is the μF value.

27.3 Dict Iteration

`for k,v in d.items()` compiles as a list fold over the shared function $\mathsf{py_meth_items}(d)$, which returns a Pair-chain of key-value Pairs. Because dict iteration order is insertion-ordered in Python 3.7+ but logically unordered, the compiler wraps the result in an $\mathsf{OQuotient}$ when the fold operation is order-independent.

Proposition 27.5 (Dict Iteration Order Independence)**.** *If the fold operation is commutative and associative, the result is independent of dict iteration order. The commutative monoid paths (F2, F3) provide the connecting path. Formally, let* σ *be any permutation of the keys; then* $\mathsf{fold}(\oplus, e, \sigma(\mathsf{items}(d))) = \mathsf{fold}(\oplus, e, \mathsf{items}(d))$ *when* $\oplus$ *forms a commutative monoid with identity* e*.*

27.4 String Iteration

`for ch in s` is a Nat fold over the character index: $\mathsf{str_fold}(s, i, \mathsf{acc}) = \begin{cases} \mathsf{acc} \\ \mathsf{str_fold}(s, i{+}1, \mathsf{step}(\mathsf{acc}, \mathsf{StrObj}(\mathsf{SubStr} \end{cases}$

This integrates directly with the existing Nat fold machinery. At the OTerm level, iteration over a string s produces $\mathsf{OFold}(\mathit{key},\ \mathsf{init},\ \mathsf{OOp}(\texttt{str_chars}, (s)))$ where $\mathsf{str_chars}$ denotes the character sequence.

27.5 Composed Iterators

`zip`, `enumerate`, `map`, `filter` transform iterators. Each compiles to a shared function wrapping the inner iterator:

Listing 27.2: Composed iterators

```
# for i,x in enumerate(xs): ...   => list_fold(py_enumerate(xs), acc)
# for a,b in zip(xs,ys): ...      => list_fold(py_zip(xs,ys), acc)
# sum(x*2 for x in xs)            => list_fold(py_map(h, xs), IntObj
    (0))
```

At the OTerm level, `enumerate` and `zip` produce OOp nodes whose children are the inner collections; the fold catamorphism proceeds over the transformed iterator.

27.6 Generator Consumption

A generator yielding values is modeled as producing a Pair-chain. Full consumption by `list()`, `for`, or `sum` gives a list fold. This is the D6 path: both generator and eager versions produce the same Pair-chain.

Remark 27.6 (Lazy vs Eager)**.** The OTerm semantics is *total*: a generator expression `(f(x) for x in xs)` and the eager list `[f(x) for x in xs]` compile to the same $\mathsf{OMap}(\lambda x.\, f(x), xs)$. Laziness is an operational distinction irrelevant to the denotation.

Chapter 28

The Recursion Zoo: Beyond Nat Descent

Not all recursion is Nat descent. The OFix constructor in the OTerm language captures general fixed points; the *body* pattern determines which recursion scheme applies. This chapter classifies the patterns and their canonical forms.

28.1 Primitive Recursion (Nat Descent)

This is the pattern handled by the Nat-eliminator (Chapter 26.9.3). The OFix body has the shape:

$$\mathsf{OFix}[f]\big(\mathsf{OCase}(n \leq t,\ b,\ \mathsf{OOp}(\oplus, (n, f(n{-}1))))\big)$$

and normalizes to $\mathsf{OFold}(\oplus, b, n)$. This covers factorial, summation, exponentiation, and all functions definable by the $\mathbb{N}$-recursor.

28.2 Course-of-Values Recursion

Some functions access *multiple* prior values: $f(n)$ depends on $f(n{-}1), f(n{-}2), \ldots$. The Fibonacci function is the canonical example: $f(n) = f(n{-}1) + f(n{-}2)$.

Definition 28.1 (Course-of-Values OFix Pattern)**.** An OFix body is *course-of-values* when the recursive calls access $f(n{-}1), f(n{-}2), \ldots, f(n{-}k)$ for fixed k. The canonical form uses a k-tuple accumulator:

$$\mathsf{OFold}(\mathit{step}_k,\ (b_1, \ldots, b_k),\ \mathsf{OOp}(\mathtt{range}, (n)))$$

where step_k shifts the k-tuple window at each iteration.

Example 28.2 (Fibonacci)**.** $f(n) = f(n{-}1) + f(n{-}2)$ with $f(0) = 0, f(1) = 1$. The iterative version maintains the pair (a, b) and steps via $(b, a{+}b)$. Both normalize to the same 2-tuple fold, connected by D1.

28.3 Tree Recursion

Definition 28.3 (Tree Catamorphism)**.** $f(\mathsf{tree}) = \begin{cases} \mathsf{base}(\mathsf{tree}) & \mathsf{is_leaf} \\ \mathsf{combine}(f(\mathsf{left}), f(\mathsf{right}), \mathsf{tree}) & \text{otherwise} \end{cases}$

This is the unique algebra morphism from the initial algebra of binary trees. At the OTerm level the body of OFix contains *two* recursive calls, distinguishing it from Nat descent.

Listing 28.1: Tree recursion Z3 encoding

```
def compile_tree_rec(func, body, env, T):
    S = T.S; T._uid += 1; uid = T._uid
    param = func.args.args[0].arg
    sym = FreshConst(S, f'_tree_{uid}')
    rfn = RecFunction(f'tree_{func.name}_{uid}', S, S)
    rec_env = Env(T, func_name=func.name, is_rec=True)
    rec_env.put(param, sym)
    secs = extract_sections(body, rec_env)
    combined = secs[-1].term
    for s in reversed(secs[:-1]):
        combined = If(s.guard, s.term, combined)
    RecAddDefinition(rfn, [sym], combined)
    return rfn
```

Theorem 28.4 (Tree Catamorphism Uniqueness)**.** *Two functions with the same* (base, combine) *are equal by the initial algebra universal property. Stack-based traversal with the same combine is connected by D9 (Stack↔Recursion).*

28.4 Multi-Parameter Recursion (2D)

Edit distance, knapsack, LCS all recurse on two parameters: $f(i,j) = \mathsf{step}(f(i{-}1,j), f(i,j{-}1), f(i{-}1,j{-}1))$.

Definition 28.5 (2D OFix Pattern)**.** A multi-parameter OFix body accesses recursive calls at lattice neighbours $(i{-}1,j)$, $(i,j{-}1)$, $(i{-}1,j{-}1)$. The canonical form is a 2D fold:

$$\mathsf{OFold}_{2D}(\mathit{step},\ (\mathsf{base_row}, \mathsf{base_col}),\ (m,n))$$

which is the unique $\mathbb{N} \times \mathbb{N}$-recursor satisfying the 2D recurrence relation.

Listing 28.2: 2D RecFunction

```
f = RecFunction('f_2d', IntSort(), IntSort(), S)
i, j = Ints('i j')
RecAddDefinition(f, [i, j],
    If(i <= 0, base_row(j),
    If(j <= 0, base_col(i),
       step(i, j, f(i-1,j), f(i,j-1), f(i-1,j-1)))))
```

Both memoized and bottom-up DP satisfy the *same* 2D recurrence. By $\mathbb{N} \times \mathbb{N}$-eliminator uniqueness: D16 path.

28.5 Mutual Recursion

$f(n) = \ldots g(n{-}1) \ldots$, $g(n) = \ldots f(n{-}1) \ldots$. Model as product recursor $h(n) = (f(n), g(n))$:

Definition 28.6 (Mutual Recursion as Product OFix)**.** Two mutually recursive functions f, g are encoded as a single $\mathsf{OFix}[h]$ where $h(n) := \mathsf{OSeq}(f(n), g(n))$. The individual functions are recovered via projection: $f(n) = \pi_1(h(n))$, $g(n) = \pi_2(h(n))$.

Listing 28.3: Mutual recursion as product

```
h = RecFunction('mutual', IntSort(), S)
RecAddDefinition(h, [n],
    If(n <= 0, S.Pair(base_f, base_g),
       S.Pair(step_f(n, S.fst(h(n-1)), S.snd(h(n-1))),
              step_g(n, S.fst(h(n-1)), S.snd(h(n-1))))))
```

Proposition 28.7 (Even–Odd Example). *`is_even(n) = is_odd(n-1)` and `is_odd(n) = is_even(n-1)` encode as* $h(n) = (\neg\pi_2(h(n{-}1)),\ \neg\pi_1(h(n{-}1)))$. *After Nat-elimination,* $h(n)$ *becomes a fold whose step toggles a Boolean pair, and the result depends only on* $n \bmod 2$.

28.6 Well-Founded Recursion with Fuel

General recursion on any well-founded relation, made terminating with a fuel parameter:

Listing 28.4: Well-founded recursion

```
f = RecFunction('wf', S, IntSort(), S)
RecAddDefinition(f, [x, fuel],
    If(fuel <= 0, S.Bottom,
    If(is_base(x), base(x),
       step(x, f(smaller(x), fuel - 1)))))
```

Decidable for ground arguments (fuel bounds the depth). Two well-founded recursions with same $(\mathsf{base}, \mathsf{step}, \prec)$ connected by W-type eliminator uniqueness.

Remark 28.8 (The Recursion Classification). The following table summarises the OFix body patterns:

Scheme	Recursive calls	Canonical OTerm	Path
Primitive	$f(n{-}1)$	$\mathsf{OFold}(\oplus, b, n)$	D1
Course-of-values	$f(n{-}1), \ldots, f(n{-}k)$	$\mathsf{OFold}(step_k, \vec{b}, n)$	D1
Tree	$f(left), f(right)$	$\mathsf{OFix}[tree]$	D9
Multi-parameter	$f(i{-}1, j), f(i, j{-}1), \ldots$	OFold_{2D}	D16
Mutual	$f \leftrightarrow g$	$\mathsf{OFix}[prod]$	D1
Well-founded	$f(smaller(x))$	$\mathsf{OFix}[wf]$	W-type

Chapter 29

Higher-Order Functions: Compilation and Normalization

Higher-order functions introduce first-class function values into the OTerm language via OLam (anonymous functions) and OMap (element-wise transformations with optional filtering). The compiler must ensure that two programs using the *same* transformation get the *same* denotation regardless of surface syntax.

29.1 The OLam and OMap Constructors

Definition 29.1 (OLam — First-Class Functions)**.** $\mathsf{OLam}((x_1, \ldots, x_k),\ \mathit{body})$ represents a k-ary anonymous function. Canonical form uses *de Bruijn*-style positional names $_b0, _b1, \ldots$ so that α-equivalent lambdas are syntactically identical:

$$\mathsf{canon}(\mathsf{OLam}((x,), \mathsf{OOp}(\texttt{mult}, (x, x)))) \;=\; \lambda(_b0)\, \texttt{mult}(_b0, _b0)$$

Definition 29.2 (OMap — Collection Transformation)**.** $\mathsf{OMap}(f, \mathit{coll}, p)$ applies transform f (typically an OLam) to each element of *coll*, optionally filtering by predicate p. When p is present the canonical form is $\texttt{filter_map}(p, f, \mathit{coll})$. This unifies list comprehensions, `map()`, `filter()`, and for-loops with append into a single constructor.

29.2 Compilation Rules

Listing 29.1: HOF compilation

```
# map(lambda x: x*2, xs)
# => py_map_{hash("x*2")}(xs)            [shared symbol]

# filter(lambda x: x > 0, xs)
# => py_filter_{hash("x > 0")}(xs)       [shared symbol]

# sorted(xs, key=len)
# => py_sorted_by_{hash("len")}(xs)      [shared symbol]

# reduce(op, xs, init)
# => py_reduce_{hash(op)}(xs, init)      [shared symbol]
```

The key insight: two programs using the *same* lambda body get the *same* shared symbol, regardless of whether it's written as `map(f, xs)`, a comprehension `[f(x) for x in xs]`, or an explicit loop.

Example 29.3 (Three Forms, One OMap)**.** The following three programs all compile to $\mathsf{OMap}(\lambda(_b0)\ \texttt{mult}(_b0$

```
# (1) map
list(map(lambda x: x*2, xs))
# (2) comprehension
[x*2 for x in xs]
# (3) for-loop with append
result = []; for x in xs: result.append(x*2)
```

The α-normalized body $\texttt{mult}(_b0, 2)$ is identical in all three, so their OTerm denotations are equal by construction.

29.3 Function Composition Normalization

When two OLam values are composed—either explicitly via `functools.compose` or implicitly through nested `map`—the normalizer fuses them:

Definition 29.4 (OLam Composition)**.** Given $f = \mathsf{OLam}((x,), B_f)$ and $g = \mathsf{OLam}((y,), B_g)$, the composition $f \circ g$ is $\mathsf{OLam}((z,), B_f[x \mapsto B_g[y \mapsto z]])$, i.e. the body of f with x replaced by the body of g applied to the fresh parameter. After α-normalization, the composed lambda is canonical.

29.4 HOF Normalization Rules

1. $\mathsf{reduce}(\oplus, e, \mathsf{map}(h, xs)) \to \mathsf{reduce}(\lambda a, x.\, a \oplus h(x), e, xs)$ (map–fold fusion)
2. $\mathsf{map}(f, \mathsf{map}(g, xs)) \to \mathsf{map}(f \circ g, xs)$ (map composition)
3. $\mathsf{filter}(p, \mathsf{map}(h, xs)) \to \mathsf{map}(h, \mathsf{filter}(p \circ h, xs))$ (filter–map swap, when applicable)
4. $\mathsf{sorted}(\mathsf{list}(x)) \to \mathsf{sorted}(x)$ (list absorption)
5. $\mathsf{map}(\mathsf{id}, xs) \to xs$ (identity elimination)

Rule 5 is implemented in Phase 1: when the OLam body is just OVar matching the single parameter, the entire OMap is replaced by the collection.

Example 29.5 (map+filter vs comprehension)**.** $f(xs) = \mathsf{list}(\mathsf{filter}_{h_1}(\mathsf{map}_{h_2}(xs)))$ vs. $g(xs) = \mathsf{comp}_{h_3}(xs)$.

Path: D4 identifies the comprehension with filter∘map when h_3 matches the composition of h_1 and h_2.

Chapter 30

Complete Normalization Traces

The 7-phase normalizer transforms an OTerm to canonical form by iterating a fixed pipeline until convergence. This chapter presents the phases in detail and then traces concrete examples through the full pipeline.

30.1 The Seven Phases

Definition 30.1 (7-Phase Normalization Pipeline)**.** Given an OTerm t_0, the normalizer computes the sequence $t_0 \to t_1 \to \cdots \to t_k$ where each t_{i+1} is obtained by applying all seven phases in order:

P1: **β-reduction.** Constant folding ($\mathsf{OOp}(\mathtt{add}, (3, 4)) \to \mathsf{OLit}(7)$), case collapse ($\mathsf{OCase}(\mathsf{True}, t, f) \to t$ and $\mathsf{OCase}(c, x, x) \to x$), identity map elimination ($\mathsf{OMap}(\mathsf{id}, xs) \to xs$), empty-collection simplifications ($\mathsf{len}([]) \to 0$), and fold-over-empty ($\mathsf{OFold}(\oplus, e, []) \to e$).

P2: **Ring/lattice rewriting.** Applies the algebraic identities of $(\mathbb{Z}, +, \times)$ and $(\mathbb{B}, \wedge, \vee)$: additive identity ($x{+}0 \to x$), multiplicative identity ($x{\times}1 \to x$), absorbing element ($x{\times}0 \to 0$), double negation ($\neg\neg x \to x$, $-(-x) \to x$), add-sub cancellation ($(x + c) - c \to x$), comparison negation ($\neg(a < b) \to a \geq b$), idempotence ($\mathsf{sorted}(\mathsf{sorted}(x)) \to \mathsf{sorted}(x)$, $\mathsf{set}(\mathsf{set}(x)) \to \mathsf{set}(x)$), absorption ($\mathsf{sorted}(\mathsf{reversed}(x)) \to \mathsf{sorted}(x)$, $\mathsf{list}(\mathsf{sorted}(x)) \to \mathsf{sorted}(x)$), and reflexivity ($x = x \to \mathsf{True}$). Also normalises OCatch for division: $\mathsf{catch}(x/y, d) \to \mathsf{case}(y{=}0, d, x/y)$ (D22).

P3: **Fold canonicalization.** Builtin reducers become folds with known step and identity (D5):

$$\begin{aligned}
\mathsf{sum}(xs) &\to \mathsf{OFold}(\mathtt{iadd}, 0, xs)\\
\mathsf{any}(xs) &\to \mathsf{OFold}(\mathtt{or}, \mathsf{False}, xs)\\
\mathsf{all}(xs) &\to \mathsf{OFold}(\mathtt{and}, \mathsf{True}, xs)\\
\mathtt{sep.join}(xs) &\to \mathsf{OFold}(\mathtt{str_concat}, \mathtt{sep}, xs)
\end{aligned}$$

After this phase, a manual loop `for x in xs: s += x` and the call `sum(xs)` have the same OTerm: $\mathsf{OFold}(\mathtt{iadd}, 0, xs)$.

P4: **HOF fusion and piecewise expansion.** Map–map fusion ($\mathsf{map}(f, \mathsf{map}(g, x)) \to \mathsf{map}(f \circ g, x)$), sorted-of-set collapse ($\mathsf{sorted}(\mathsf{set}(x)) \to \mathsf{sorted}(x)$), reversed-sorted composition ($\mathsf{reversed}(\mathsf{sorted}(x)) \to \mathsf{sorted_rev}(x)$), piecewise expansion of abs, min, max ($\mathsf{abs}(x) \to \mathsf{case}(x{<}0, -x, x)$; $\max(a, b) \to \mathsf{case}(a{\geq}b, a, b)$), and closed-form substitutions ($\mathsf{sum}(\mathsf{range}(n)) \to n(n{-}1)/2$).

P5: **Dichotomy rewriting (D1–D24).** Each of the 24 dichotomies acts as a rewrite rule unifying alternative API surfaces: $\mathsf{heapq.nsmallest}(n, xs) \to \mathsf{sorted_slice}(xs, n)$ (D19), $\mathsf{Counter}(xs) \to \mathsf{counter}(xs)$ (D13), $\mathsf{OrderedDict}(xs) \to \mathsf{dict}(xs)$ (D14), $\mathsf{heapq.heappush}(h, x) \to \mathsf{pq_insert}(h, x)$ (D10), and local-variable method normalization (stripping local prefixes so that `a.foo()` and `b.foo()` on same-typed variables share the symbol `_L.foo`).

P6: **Quotient normalization.** Introduces and simplifies OQuotient nodes for nondeterministic operations: $\mathsf{set}(x) \to Q[\mathsf{perm}](\mathsf{set}(x))$, $\mathsf{sorted}(Q[\mathsf{perm}](x)) \to \mathsf{sorted}(x)$ (sorting resolves the nondeterminism), and $Q[\mathsf{perm}](Q[\mathsf{perm}](x)) \to Q[\mathsf{perm}](x)$ (idempotence).

P7: **Piecewise case normalization.** Flattens nested OCase chains into a list of $(\mathit{guard}, \mathit{value})$ pieces, infers the implicit else-guard from the comparison algebra (e.g. $x < 0 \Rightarrow$ else-guard is $x \geq 0$), sorts pieces by $(\mathit{value}.\mathsf{canon}(),\ \mathit{guard}.\mathsf{canon}())$, and reconstructs a canonical case chain. This ensures that piecewise functions with different guard orderings but the same partition normalize to the same OTerm.

The pipeline repeats up to 8 times until a fixed point is reached (the canonical form of t_i equals that of t_{i-1}).

30.2 Trace 1: Factorial (Rec → Fold)

Input: `def f(n): if n<=1: return 1; return n*f(n-1)`

1. **Compilation**: $[\![f]\!] = \mathsf{rec_f}(p_0)$ where $\mathsf{rec_f}(n) = \mathsf{If}(\mathsf{ival}(n) \leq 1, \mathsf{IntObj}(1), \mathsf{binop}(\mathsf{MUL}, n, \mathsf{rec_f}(\mathsf{binop}(\mathsf{SUB}$
2. **Phase 3 (Nat-eliminator)**: Detects base=1, threshold=1, op=MUL. Produces $\mathsf{fold}(\mathsf{MUL}, 1, \mathsf{ival}(p_0) + 1, \mathsf{IntObj}(1))$.
3. **Iterative version**: same fold via loop-to-fold.
4. **Comparison**: identical Z3 terms. Path: refl.

30.3 Trace 2: Sum of Squares

Input A: `sum(x**2 for x in xs)`.
Input B: `s = 0; for x in xs: s += x**2; return s`.

Phase-by-phase trace:

1. **Compilation (A)**: $\mathsf{OOp}(\texttt{sum},\ (\mathsf{OMap}(\lambda(_b0)\,\mathsf{OOp}(\texttt{pow}, (_b0, 2)),\ \$xs)))$.
2. **Compilation (B)**: $\mathsf{OFold}(\texttt{iadd},\ 0,\ \$xs)$ where the fold body internally computes $\mathsf{acc} + x^2$. Because the augmented assignment `s += x**2` is not a bare `s += x`, the compiler falls back to a body-hash fold key.
3. **P3 (Fold canonicalization, on A)**: $\mathsf{sum}(\ldots) \to \mathsf{OFold}(\texttt{iadd}, 0, \mathsf{OMap}(\lambda(_b0)\,\texttt{pow}(_b0, 2),\ \$xs))$.
4. **P4 (HOF fusion, on A)**: $\mathsf{OFold}(\texttt{iadd}, 0, \mathsf{OMap}(f, xs)) \to \mathsf{OFold}(\texttt{iadd}, 0, xs)$ with the map fused into the fold step. Now A and B share the same fold key (body hash of $acc + x^2$) and the same OTerm.
5. **Comparison**: identical after P4. Path: refl.

30.4 Trace 3: Edit Distance (2D Recurrence)

Input A (memoized): nested `dp(i,j)` with `@lru_cache`.

1. **Phase 1**: Inline nested `dp` (D2). Ignore decorator.
2. **Compilation**: 2D RecFunction $\mathsf{dp}(i, j) = \mathsf{If}(i = 0, j, \mathsf{If}(j = 0, i, \mathsf{binop}(\mathsf{MIN}, \ldots)))$.
3. **Phase 2**: Canonicalize min argument order (L1).
4. **Phase 5**: Shared symbols for subscript, len.
5. **Input B** (DP table): same 2D RecFunction after compiling nested loops as 2D fold.
6. **Comparison**: same recurrence, same bases. Path: D16 (Memo↔DP).

30.5 Trace 4: all vs any (De Morgan)

Input A: `all(pred(x) for x in seq)`.
Input B: `not any(not pred(x) for x in seq)`.

1. **Phase 3**: A → $\mathsf{OFold}(\texttt{and}, \mathsf{True}, \mathsf{comp}_{h_1}(p_0))$. B's inner any → $\mathsf{OFold}(\texttt{or}, \mathsf{False}, \mathsf{comp}_{h_2}(p_0))$.
2. comp_{h_2} has body `not pred(x)` = $\mathsf{map}(\mathsf{NOT}, \mathsf{comp}_{h_1})$.
3. **Path B6** (De Morgan): $\mathsf{py_all}(xs) \equiv \neg\, \mathsf{py_any}(\mathsf{map}(\neg, xs))$.
4. Certificate: 2 steps.

30.6 Trace 5: Sorted Unique

Input A: `return sorted(set(xs))`.
Input B: manual dedup + insertion sort.

1. **P6 (Quotient)**: $\mathsf{set}(xs) \to Q[\mathsf{perm}](\mathsf{set}(xs))$.
2. **P4 (HOF fusion)**: $\mathsf{sorted}(Q[\mathsf{perm}](\mathsf{set}(xs))) \to \mathsf{sorted}(\mathsf{set}(xs)) \to \mathsf{sorted}(xs)$. The quotient is resolved by sorting; the inner set removes duplicates.
3. B compiles to a list fold with insert operations.
4. **Path**: requires axiom $\mathsf{py_sorted}(\mathsf{py_set}(x))$ = canonical "sorted unique elements". Both programs produce this canonical form. Path: $\mathsf{I3} \cdot \mathsf{I5}$ (set idempotence + sorted idempotence).

30.7 Trace 6: Absolute Value (Phase 7 Piecewise)

Input A: `return abs(n)`.
Input B: `if n < 0:  return -n; return n`.

1. **Compilation (A)**: $\mathsf{OOp}(\texttt{abs}, (\$n))$.
2. **Compilation (B)**: $\mathsf{OCase}(\mathsf{OOp}(\texttt{lt}, (\$n, 0)),\ \mathsf{OOp}(\texttt{u_usub}, (\$n)),\ \$n)$.
3. **P4 on (A)**: piecewise expansion fires: $\mathsf{abs}(x) \to \mathsf{OCase}(\texttt{lt}(x, 0),\ \texttt{u_usub}(x),\ x)$. Now both are OCase nodes.

4. **P7 on both**: the single-guard case is already canonical (only one guard + else). Both have identical $(guard, value)$ pieces.

5. **Comparison**: identical OTerms. Path: refl.

Part VII

Specification Checking and Bug Finding via CTTT

Chapter 31

The Three Problems, One Framework

The cubical–cohomological framework solves three problems with the *same* mathematical machinery:

Problem	**CTTT formulation**	**Z3 query**
Equivalence	$\mathsf{Path}_{A\to B}(f,g)$ exists?	$\exists x.\, [\![f]\!](x) \neq [\![g]\!](x)$?
Spec checking	$\mathsf{Path}_{A\to B}(f,\mathsf{spec})$ exists?	$\exists x.\, \neg\, S(x, [\![f]\!](x))$?
Bug finding	$H^1(\mathcal{U}, \mathsf{Iso}(\mathsf{Sem}_f, \mathsf{Sem}_{\mathsf{spec}})) \neq 0$?	which fiber has the obstruction?

Equivalence asks: is there a cubical path from f to g?

Spec checking asks: is there a cubical path from f to *any* program satisfying the specification S? Since S is deterministic (at most one valid output per input), this reduces to: does f itself satisfy S?

Bug finding asks: on which type fiber does f *fail* to satisfy S? The cohomological decomposition localizes the bug to a specific fiber — a specific type of input where the program misbehaves.

All three use:

1. The **PyObj Z3 theory** to compile programs to Z3 terms.
2. The **type fibration** to decompose the input space.
3. The **Čech complex** to relate local and global properties.
4. The **path constructors** to bridge structural gaps.

The only difference is what we compare the program's denotation *against*: another program (equivalence), a specification (spec checking), or an ideal behavior (bug finding).

31.1 The Unification Theorem

The conceptual unity above is not merely an analogy — it is a theorem. In the implementation, all three entry points (`check_equivalence`, `check_spec`, `find_bugs`) invoke exactly the same CCTT pipeline.

Theorem 31.1 (Three Problems, One Kernel)**.** *Let* $\mathsf{Pipeline}(f,g)$ *denote the full CCTT verification pipeline applied to two function sources* f *and* g*. Then:*

(i) $\mathsf{check_equivalence}(f,g) \coloneqq \mathsf{Pipeline}(f,g)$.

(ii) $\mathsf{check_spec}(f,S) \coloneqq \mathsf{Pipeline}(f,r_S)$*, where* r_S *is a reference implementation of specification* S*.*

(iii) $\mathsf{find_bugs}(f, S) := \mathsf{Pipeline}(f, r_S)$ *with obstruction reporting.*

In particular, `check_spec` *is a direct call to* `check_equivalence`*, and* `find_bugs` *is* `check_equivalence` *with the result relabeled.*

Proof. For (ii): given a deterministic specification S with reference implementation r_S, a program f satisfies S if and only if $f(x) = r_S(x)$ for all x (by Theorem 32.3). This is exactly the predicate that `check_equivalence` decides. The implementation confirms this: `check_spec(source, spec_source)` = `check_equivalence(source, spec_source)`.

For (iii): if the pipeline returns $H^1 \neq 0$, the obstruction data identifies the fibers where f and r_S disagree. Each such fiber witnesses a bug in f relative to S. The implementation prefixes the explanation with "BUG:" and returns the same `Result` object. No additional computation is performed. □

Remark 31.2. The unification is not a trivial repackaging. It reflects the mathematical fact that all three problems are instances of *computing the Čech cohomology of the isomorphism presheaf* $\mathsf{Iso}(\mathsf{Sem}_f, \mathsf{Sem}_T)$ for an appropriate target presheaf Sem_T. The target is Sem_g for equivalence, Sem_{r_S} for spec checking, and the same Sem_{r_S} (but with localized reporting) for bug finding.

Chapter 32

Specification Checking

32.1 Specifications as Dependent Types

In CTTT, a specification is a *dependent type over the value presheaf*:

Definition 32.1 (Specification)**.** A specification S for a function type $A \to B$ is a type family:

$$S : \prod_{x:A} B \to \mathsf{Type}$$

where $S(x, y)$ is inhabited iff y is a valid output for input x.

A program $f : A \to B$ *satisfies* S iff:

$$\vdash \lambda x.\, \mathsf{witness}(x) : \prod_{x:A} S(x, f(x))$$

— for every input x, there is a witness that $f(x)$ is valid.

In Z3, this becomes: is there any x where $\neg S(x, [\![f]\!](x))$?

- **UNSAT**: f satisfies S for all inputs. The UNSAT certificate is the constructive witness.
- **SAT**: f violates S on the counterexample x from the Z3 model. This is a *bug*.

32.2 Specifications from Natural Language

Deppy's `@guarantee` decorator attaches a natural-language specification to a function:

```
@guarantee("returns a sorted list with no duplicates")
def unique_sorted(lst):
    return sorted(set(lst))
```

The NL specification is parsed into a conjunction of Z3 predicates:

$$S_1(x, y) : \mathsf{is_sorted}(y) \qquad \text{“sorted”} \tag{32.1}$$
$$S_2(x, y) : \mathsf{no_duplicates}(y) \qquad \text{“no duplicates”} \tag{32.2}$$
$$S_3(x, y) : \mathsf{subset}(y, x) \qquad \text{“from the input”} \tag{32.3}$$
$$S(x, y) = S_1 \wedge S_2 \wedge S_3 \tag{32.4}$$

Each predicate is encoded using the shared function symbols:

- $\mathsf{is_sorted}(y) \equiv \mathsf{py_sorted}(y) = y$ (using the $\mathsf{sorted_idem}$ axiom).
- $\mathsf{no_duplicates}(y) \equiv \mathsf{py_set}(y) = y$ (using the $\mathsf{set_idem}$ axiom).
- $\mathsf{subset}(y, x) \equiv \mathsf{py_set}(y) \subseteq \mathsf{py_set}(x)$.

32.3 Fiber-Wise Spec Checking

The cohomological decomposition applies to spec checking:

Definition 32.2 (Specification Presheaf)**.** The *specification presheaf* Spec_S assigns to each type fiber U the set of valid behaviors:

$$\mathsf{Spec}_S(U) = \{(x, y) \in U \times B \mid S(x, y)\}$$

The program f satisfies S on fiber U iff:

$$\forall x \in U.\ (x, [\![f]\!](x)) \in \mathsf{Spec}_S(U)$$

This is checked fiber-by-fiber, just like equivalence:

1. For each fiber U_τ, ask Z3: $\exists x.\, \mathsf{tag}(x) = \tau \wedge \neg S(x, [\![f]\!](x))$.
2. Collect local verdicts.
3. Compute H^1 to check if local satisfaction glues globally.

The Čech decomposition is especially valuable for spec checking because specifications often have *different constraints on different types*:

- On the Int fiber: "output is non-negative."
- On the Str fiber: "output is non-empty."
- On the List fiber: "output has the same length."

Each constraint is checked independently on its fiber, then glued.

32.4 Example: Sorting Specification

Listing 32.1: Sorting with spec check

```
@guarantee("returns a sorted permutation of the input")
def my_sort(lst):
    result = []
    for x in lst:
        # BUG: inserts at wrong position
        result.insert(0, x)
    return result
```

Compilation: $[\![\mathsf{my_sort}]\!](p_0)$ produces a sequence built by repeated $\mathsf{py_meth_insert}(0, x)$ — which prepends each element, producing the *reverse* of the input.

Spec: $S(x, y) = \mathsf{py_sorted}(y) = y \ \wedge\ \mathsf{len}(y) = \mathsf{len}(x)$.

Z3 check: Is there an x where $\mathsf{py_sorted}([\![f]\!](x)) \neq [\![f]\!](x)$? Z3 finds a counterexample: $x = [2, 1]$, where $[\![f]\!]([2, 1]) = [1, 2]$ is actually sorted (by accident!), but $x = [1, 2]$ gives $[\![f]\!]([1, 2]) = [2, 1]$ which is NOT sorted. **SAT** — bug found.

Fiber localization: The bug is on the Pair fiber (list inputs). $H^1(\{\mathsf{Pair}\}, \mathsf{Iso}(\mathsf{Sem}_f, \mathsf{Sem}_S)) \neq 0$.

32.5 Example: Fibonacci Specification

Listing 32.2: Fibonacci with off-by-one bug

```
@guarantee("returns the n-th Fibonacci number")
def fib(n):
    if n <= 1: return n
    a, b = 0, 1
    for _ in range(n):  # BUG: should be range(n-1)
        a, b = b, a + b
    return a
```

Spec: $S(n, y)$ is the Fibonacci recurrence: $F(0) = 0$, $F(1) = 1$, $F(n) = F(n-1) + F(n-2)$.

Compilation: The loop runs n times instead of $n - 1$, producing $F(n + 1)$ instead of $F(n)$ for $n \geq 2$.

Z3 check: $\exists n. \mathsf{tag}(n) = \mathsf{TInt} \wedge n \geq 0 \wedge [\![f]\!](n) \neq F(n)$. Z3 finds $n = 2$: $[\![f]\!](2) = 2$ but $F(2) = 1$. **SAT** — bug found.

Fiber localization: Bug is on the Int fiber for $n \geq 2$. The base cases $n = 0, 1$ pass (the loop doesn't execute or executes once correctly).

32.6 Spec Checking via Equivalence

A powerful technique: reduce spec checking to equivalence checking.

Theorem 32.3 (Spec Checking as Equivalence)**.** *Let S be a deterministic specification with a known* reference implementation *r (a program that satisfies S). Then f satisfies S iff $f \equiv r$.*

Proof. Since S is deterministic, there is exactly one valid output per input. Both f and r must produce this output. So f satisfies S iff $f(x) = r(x)$ for all x iff $f \equiv r$. □

This reduces spec checking to the equivalence problem, which we already solve with the full cubical–cohomological machinery (path constructors, dichotomy theories, Čech complex).

Example 32.4. To check that a sorting implementation is correct, compare it against `sorted` (the reference implementation). The shared function symbol $\mathsf{py_sorted}$ and the sorted axioms handle the comparison.

Remark 32.5 (Implementation identity)**.** In the implementation, `check_spec` is literally a one-line wrapper:

$$\mathsf{check_spec}(f, S) := \mathsf{check_equivalence}(f, S).$$

This is not a simplification — it is a consequence of Theorem 32.3. When the specification S is given as a reference implementation (the common case in practice), the two problems are *identical*.

Chapter 33

Bug Finding via Cohomological Obstruction

33.1 Bugs as H^1 Obstructions

A *bug* is a violation of a specification. In CTTT, bugs are *cohomological obstructions*: elements of H^1 that prevent local correctness from gluing to global correctness.

Definition 33.1 (Bug as Obstruction)**.** A bug in program f with respect to specification S is a non-trivial element of:

$$H^1(\mathcal{U}, \mathsf{Iso}(\mathsf{Sem}_f, \mathsf{Sem}_S))$$

where Sem_S is the presheaf of the specification (the "ideal" behavior).

The obstruction element tells us:

1. **Which fiber**: the type of input that triggers the bug.
2. **Which overlap**: whether the bug is a local failure (wrong answer on one fiber) or a gluing failure (correct on each fiber separately but inconsistent across fibers).
3. **The counterexample**: the Z3 model gives a concrete input that exhibits the bug.

Theorem 33.2 (H^1 Bug Localization)**.** *Let $f : A \to B$ be a program, S a deterministic specification with reference implementation r_S, and $\mathcal{U} = \{U_1, \ldots, U_n\}$ the type-fiber cover. If*

$$H^1(\mathcal{U}, \mathsf{Iso}(\mathsf{Sem}_f, \mathsf{Sem}_{r_S})) \neq 0$$

then there exists a fiber U_k and an input $x_0 \in U_k$ such that $f(x_0) \neq r_S(x_0)$, i.e., f has a bug on fiber U_k.

Proof. Write $\mathcal{F} := \mathsf{Iso}(\mathsf{Sem}_f, \mathsf{Sem}_{r_S})$ for the isomorphism presheaf, so $\mathcal{F}(U_i) = 1$ when $[\![f]\!]$ and $[\![r_S]\!]$ agree on every input in U_i, and $\mathcal{F}(U_i) = 0$ otherwise.

By definition of the Čech complex over $\mathcal{U}$, the 0-cochain group is $C^0(\mathcal{U}, \mathcal{F}) = \prod_i \mathcal{F}(U_i)$ and the 1-cochain group is $C^1(\mathcal{U}, \mathcal{F}) = \prod_{i<j} \mathcal{F}(U_i \cap U_j)$. The coboundary $\delta^0 : C^0 \to C^1$ sends a 0-cochain (s_i) to the 1-cochain $(\delta^0 s)_{ij} = s_j|_{U_i \cap U_j} - s_i|_{U_i \cap U_j}$.

Suppose for contradiction that $\mathcal{F}(U_k) = 1$ for every k — i.e., f and r_S agree on every fiber. Then C^0 contains the global section $s = (1, \ldots, 1)$, and $\delta^0(s) = 0$. Since every 1-cochain $\alpha \in \ker(\delta^1)$ satisfies $\alpha = \delta^0(s')$ for some s' (the local sections glue), we get $H^1 = \ker(\delta^1)/\mathrm{im}(\delta^0) = 0$, contradicting $H^1 \neq 0$.

Therefore there exists some U_k with $\mathcal{F}(U_k) = 0$. By definition, this means the Z3 query $\exists x.\, \mathsf{tag}(x) = \tau_k \wedge [\![f]\!](x) \neq [\![r_S]\!](x)$ is satisfiable. The model provides a concrete witness $x_0 \in U_k$ where $f(x_0) \neq r_S(x_0)$. Since r_S satisfies S, this x_0 is a bug-triggering input. $\square$

Corollary 33.3. *The converse also holds: if there exists a fiber U_k with $\mathcal{F}(U_k) = 0$, then $H^1(\mathcal{U}, \mathcal{F}) \neq 0$. Hence $H^1 \neq 0$ if and only if f has a bug on at least one fiber.*

Proof. If $\mathcal{F}(U_k) = 0$, the 0-cochain that is 0 on U_k and 1 elsewhere fails to extend to a global section. The resulting non-trivial element in $\ker(\delta^1)/\mathrm{im}(\delta^0)$ witnesses $H^1 \neq 0$. □

33.2 Bug Localization

The Čech decomposition provides automatic *bug localization*:

Theorem 33.4 (Bug Localization via Fiber Decomposition)**.** *If $H^1(\mathcal{U}, \mathsf{Iso}(\mathsf{Sem}_f, \mathsf{Sem}_S)) \neq 0$, the obstruction is localized to specific fibers $U_{i_1}, \ldots, U_{i_k}$ where the local checks fail. The Z3 counterexamples on these fibers are the* minimal bug-triggering inputs.

Example 33.5 (Type-Dependent Bug)**.**

```
@guarantee("returns the absolute value")
def my_abs(x):
    if x < 0:
        return -x
    return x
```

This is correct on the Int fiber. But on the Str fiber, the comparison `x < 0` is meaningless (Python 3 raises `TypeError` for `str < int`).

The Čech decomposition detects:

- $\mathsf{Iso}(\mathsf{Int})$: UNSAT (correct on integers).
- $\mathsf{Iso}(\mathsf{Str})$: SAT (bug on strings — comparison with integer fails).
- $H^1 \neq 0$ on the Str fiber.

The bug is localized to the string fiber. The fix: add a type guard `if not isinstance(x, (int, float)): raise TypeError(...)`.

33.3 Bug Classification by Cohomological Degree

Bugs can be classified by which cohomological degree they appear in:

Degree	Meaning	Example
$H^0 = 0$	No correct behavior at all	Function always crashes
$H^0 \neq 0, H^1 \neq 0$	Correct on some fibers, wrong on others	Off-by-one for large n
$H^1 = 0, H^2 \neq 0$	Locally correct, inconsistent gluing	Type coercion bug

$H^0 = 0$ **bugs**: The program fails the specification on *every* fiber. This is a total failure — the algorithm is fundamentally wrong.

$H^0 \neq 0, H^1 \neq 0$ **bugs**: The program is correct on some fibers but not others. This is the most common case — edge cases, boundary conditions, type-specific failures.

$H^1 = 0, H^2 \neq 0$ **bugs**: These are subtle — the program is correct on each fiber individually, but the transition functions between fibers are inconsistent. This occurs when a program handles type coercions incorrectly (e.g., treating `True` as `1` in one branch but as a boolean in another).

33.4 The Bug-Finding Pipeline

1. **Input**: program f + specification S (NL or formal).
2. **Compile**: $[\![f]\!]$ and $[\![S]\!]$ to Z3 terms.
3. **Decompose**: build type site, identify fibers.
4. **Local check**: for each fiber U_τ, ask Z3: $\exists x \in U_\tau. \neg S(x, [\![f]\!](x))$.
5. **Report**:
   - If all fibers pass: "No bugs found (verified on all type fibers)."
   - If fiber U_τ fails: "Bug on τ inputs: $f(\mathsf{counterexample}) = \mathsf{wrong_value}$, expected to satisfy S."
6. **Localize**: rank fibers by severity (how many section pairs fail, how deep the obstruction).

33.5 Example: Off-by-One in Range

Listing 33.1: Sum with off-by-one

```
@guarantee("returns the sum 1 + 2 + ... + n")
def sum_to_n(n):
    total = 0
    for i in range(n):  # BUG: should be range(1, n+1)
        total += i
    return total
```

Spec: $S(n, y) = (y = n \cdot (n+1)/2)$.

Compilation: The loop computes $\mathsf{fold}(\mathsf{ADD}, 0, n, \mathsf{IntObj}(0))$, which is $0+1+\ldots+(n-1) = n(n-1)/2$.

Z3 check: $\exists n. n \geq 0 \wedge n(n-1)/2 \neq n(n+1)/2$. Z3 finds $n = 2$: program returns 1, spec says 3.

Obstruction: $H^1(\mathsf{Int}) \neq 0$. The bug is an off-by-one in the range bounds — the fold starts at 0 instead of 1 and stops at n instead of $n+1$.

33.6 Example: Mutation Bug (False Aliasing)

Listing 33.2: Mutable default argument

```
@guarantee("returns a list containing just x")
def make_list(x, lst=[]):
    lst.append(x)
    return lst
```

Spec: $S(x, y) = (y = [x])$.

Bug: The mutable default argument `lst=[]` is shared across calls. On the first call, $f(1) = [1]$ (correct). On the second call, $f(2) = [1, 2]$ (wrong — the list persists).

CTTT analysis: The parameter `lst` has type Ref (mutable). On the Ref fiber, the mutation py_mut_append modifies a shared heap location. The spec says the output should be [x], but the Z3 term shows accumulation: $\mathsf{mut_append}(\mathsf{mut_append}(\ldots, x_1), x_2)$.

Obstruction: $H^1(\mathsf{Ref}) \neq 0$. The bug is on the Ref fiber — exactly where mutation effects matter.

33.7 Extended Example: Off-by-One via H^1 on the Int Fiber

We now trace the full pipeline on a concrete off-by-one error to show how H^1 obstruction pinpoints the bug.

Listing 33.3: Fibonacci with off-by-one: detailed trace

```
def fib_buggy(n):
    if n <= 1: return n
    a, b = 0, 1
    for _ in range(n):   # should be range(n - 1)
        a, b = b, a + b
    return a

def fib_correct(n):
    if n <= 1: return n
    a, b = 0, 1
    for _ in range(n - 1):
        a, b = b, a + b
    return a
```

Step 1: Compilation. Both functions compile to PyObj terms. Writing $F_k(n)$ for the fold term with k iterations:

$$[\![\mathsf{fib_buggy}]\!] = \lambda n.\ \mathsf{If}(\mathsf{int}(n) \leq 1,\ \mathsf{IntObj}(n),\ F_n(n)) \tag{33.1}$$

$$[\![\mathsf{fib_correct}]\!] = \lambda n.\ \mathsf{If}(\mathsf{int}(n) \leq 1,\ \mathsf{IntObj}(n),\ F_{n-1}(n)) \tag{33.2}$$

Step 2: Duck type inference. Parameter n is used in $\leq$, `range`, and arithmetic — all integer operations. Inferred fiber: $\{\mathsf{Int}\}$.

Step 3: Per-fiber Z3 query on Int.

$$\exists n.\ \mathsf{tag}(n) = \mathsf{TInt} \ \wedge\ \mathsf{int}(n) \geq 0 \ \wedge\ F_n(n) \neq F_{n-1}(n)$$

Z3 returns **SAT** with witness $n = 2$: $F_2(2) = F(3) = 2$ but $F_1(2) = F(2) = 1$. The buggy version returns the *next* Fibonacci number.

Step 4: Čech complex. With a single fiber $\mathcal{U} = \{\mathsf{Int}\}$:

$$C^0 = \mathcal{F}(\mathsf{Int}) = \{0\}, \quad H^1 = \ker(\delta^1)/\mathrm{im}(\delta^0) = \mathbb{F}_2.$$

Since $\mathcal{F}(\mathsf{Int}) = 0$ (the local check failed), $H^1 \cong \mathbb{F}_2 \neq 0$.

Step 5: Verdict. $H^1 \neq 0$ on the Int fiber. The bug is localized: the loop iterates n times instead of $n - 1$, shifting the Fibonacci index by one for all $n \geq 2$. The counterexample $n = 2$ is a minimal witness.

Chapter 34

Unified Theory: Equivalence, Specs, and Bugs

34.1 The Verification Triangle

The three problems form a triangle connected by the CTTT framework:

$$\begin{tikzcd}
\text{Equivalence} \arrow[r, "\text{spec as ref. impl.}"] \arrow[d, "\text{bug = failed equiv.}"'] & \text{Spec Checking} \arrow[dl, "\text{bug = spec violation}"] \\
\text{Bug Finding} &
\end{tikzcd}$$

- **Equivalence → Spec Checking**: Given a reference implementation r satisfying spec S, check $f \equiv r$ (Theorem 32.3).
- **Spec Checking → Bug Finding**: If the spec check fails, the counterexample is a bug.
- **Bug Finding → Equivalence**: If two programs have the *same* bugs (same H^1 obstructions on the same fibers), they might still be equivalent ("equivalently buggy").

34.2 The Universal Verification Query

All three problems reduce to a single Z3 query form:

$$\exists x.\ \mathsf{tag}(x) \in F \ \wedge\ [\![f]\!](x) \neq \mathsf{target}(x)$$

where:

- F is the set of allowed type fibers (from duck type inference).
- target is:
  - $[\![g]\!]$ for equivalence checking,
  - the specification's unique output for spec checking,
  - the "ideal" behavior for bug finding.

The cohomological decomposition splits this into per-fiber queries. The path constructors (equational axioms) help Z3 prove that $[\![f]\!](x) = \mathsf{target}(x)$ even when the terms are structurally different.

34.3 Formal Unification

We now state the precise sense in which the three problems are the same.

Definition 34.1 (Verification Instance)**.** A *verification instance* is a triple $(f, T, \mathcal{U})$ where:

- f is a program (the subject),
- T is a target presheaf (another program's semantics, or a specification presheaf),
- $\mathcal{U} = \{U_1, \ldots, U_n\}$ is a type-fiber cover.

The *verdict* is the Čech cohomology $H^1(\mathcal{U}, \mathsf{Iso}(\mathsf{Sem}_f, T))$.

Theorem 34.2 (Universal Reduction)**.** *Let* $\mathsf{Verify}(f, T, \mathcal{U}) := H^1(\mathcal{U}, \mathsf{Iso}(\mathsf{Sem}_f, T))$*. Then:*

(i) ***Equivalence****:* $f \equiv g$ *iff* $\mathsf{Verify}(f, \mathsf{Sem}_g, \mathcal{U}) = 0$*.*

(ii) ***Spec checking****:* $f \models S$ *iff* $\mathsf{Verify}(f, \mathsf{Sem}_{r_S}, \mathcal{U}) = 0$*.*

(iii) ***Bug finding****: the bugs of* f *relative to* S *are exactly the fibers* U_k *that contribute to* $\mathsf{Verify}(f, \mathsf{Sem}_{r_S}, \mathcal{U}) \neq 0$*.*

In particular, (ii) and (iii) use the same computation; they differ only in how the result is reported.

Proof. Part (i) is the definition of CCTT equivalence (Chapter **??**). Part (ii) follows from Theorem 32.3: $f \models S$ iff $f \equiv r_S$ iff $\mathsf{Verify}(f, \mathsf{Sem}_{r_S}, \mathcal{U}) = 0$. Part (iii) follows from Theorem 33.2: each non-zero contribution to H^1 localizes to a fiber U_k with a counterexample. □

34.4 Composition: Spec-Guided Equivalence

A powerful composition: use a specification to *guide* equivalence checking.

Theorem 34.3 (Spec-Guided Equivalence)**.** *If both* f *and* g *satisfy a deterministic specification* S*, then* $f \equiv g$*.*

This is Theorem 24.2 from Chapter 24, but now we have the full pipeline to check it:

1. Check $f \models S$ (spec checking via Z3).
2. Check $g \models S$ (spec checking via Z3).
3. If both pass, conclude $f \equiv g$ (by deterministic specification uniqueness).

This handles the hardest equivalence cases (dichotomy D20: different algorithms for the same problem) by reducing them to two independent spec checks.

34.5 Composition: Equivalence-Guided Bug Finding

Another composition: use a known-correct program to find bugs in a new implementation.

Theorem 34.4 (Differential Testing via CTTT)**.** *Given a reference implementation* r *(known correct) and a new implementation* f*, the equivalence check* $f \equiv r$ *either:*

1. *Succeeds* ($H^1 = 0$)*:* f *is correct (no bugs).*
2. *Fails* ($H^1 \neq 0$)*: the obstruction localizes the bug to a specific fiber, with a concrete counterexample.*

This is *differential testing without execution*: we compare Z3 denotations, not runtime outputs. The counterexample is a *symbolic* input, not a random sample.

34.6 The Graded Verification Verdict

The full CTTT verdict is richer than a simple pass/fail:

Definition 34.5 (Graded Verification Verdict)**.** The verdict for program f against target T (another program or a specification) is a triple:

$$(H^0, H^1, \mathsf{obstructions})$$

where:

- H^0 = number of fibers where f matches T ("how much is correct").
- H^1 = number of obstructions ("how many bugs").
- $\mathsf{obstructions}$ = list of $(\mathsf{fiber}, \mathsf{counterexample})$ pairs ("where and what are the bugs").

A perfect program has $(H^0 = |\mathcal{U}|, H^1 = 0, [])$. A totally wrong program has $(H^0 = 0, H^1 = |\mathcal{U}|, [\ldots])$. A program with one edge-case bug has $(H^0 = |\mathcal{U}| - 1, H^1 = 1, [(\tau, x)])$.

This graded verdict is more informative than "correct" or "incorrect" — it tells the developer *exactly* where the program fails and on what type of input.

34.7 Confidence Scoring

The graded verdict induces a natural *confidence score* that quantifies how much of the input space has been verified.

Definition 34.6 (Confidence Score)**.** Given a verification with h_0 passing fibers, h_1 failing fibers, and u inconclusive fibers (timeout or compilation failure), define the *confidence score*:

$$\kappa := \frac{h_0}{h_0 + h_1 + u}$$

The score has a clear interpretation:

- $\kappa = 1$: all fibers verified, full confidence.
- $\kappa = 0$: every fiber failed or was inconclusive.
- $0 < \kappa < 1$: partial verification. The program is correct on h_0 fibers but fails or is unknown on the rest.

Proposition 34.7 (Confidence monotonicity)**.** *If $\mathcal{U}' \supseteq \mathcal{U}$ is a refinement of the cover (more fibers), and the new fibers all pass, then $\kappa' \geq \kappa$. Conversely, discovering a new failing fiber strictly decreases κ.*

Proof. Adding a passing fiber increases h_0 by 1 and the denominator by 1, so $\kappa' = (h_0 + 1)/(h_0 + h_1 + u + 1) \geq h_0/(h_0 + h_1 + u) = \kappa$ (since $(h_0 + 1)(h_0 + h_1 + u) \geq h_0(h_0 + h_1 + u + 1)$ iff $h_0 + h_1 + u \geq h_0$ iff $h_1 + u \geq 0$). A new failing fiber increases h_1 without changing h_0, strictly decreasing κ. □

Remark 34.8 (Implementation)**.** The implementation computes confidence as $\kappa = h_0 / \max(\mathsf{total_fibers}, 1)$ where $\mathsf{total_fibers} = h_0 + h_1 + u$. This is returned in the `Result.confidence` field and used by higher-level tools to decide whether to trust the verdict.

Part VIII

Implementation Architecture

Chapter 35

The Verification Pipeline

35.1 Pipeline Overview

The verification pipeline implements the CCTT framework as an ordered cascade of five strategies, from cheapest to most expensive. Each strategy either produces a definitive verdict or passes control to the next:

S0. Closed-term evaluation — zero-argument functions have singleton domains; their denotation *is* their unique value.

S1. Denotational OTerm equivalence — normalize both programs to canonical OTerms with quotient types for nondeterminism, then compare.

S2. Z3 structural equality — syntactic identity of fully-interpreted Z3 ASTs.

S3. Semantic fingerprint matching — extract operation-level fingerprints (function calls, fold shapes, shared symbols) and compare.

S4. Per-fiber Z3 checking + Čech H^1 — the full cohomological pipeline: duck type inference, site construction, local SAT queries, coboundary operators, GF(2) rank.

Strategies S0–S3 are *fast paths*: they handle common cases (identical code, simple refactors, closed computations) in microseconds. Strategy S4 is the general-purpose kernel that handles arbitrary algorithmic differences.

Remark 35.1 (Correctness of fast paths)**.** Each fast path is *sound*: if it returns a verdict, that verdict is correct. Soundness of S0 follows because a zero-argument function's denotation is its unique value (evaluation is complete, not sampling). Soundness of S2 requires that both Z3 terms be fully interpreted (no uninterpreted function symbols); the implementation checks this via the `_uninterp_depth` function and falls through to S4 otherwise. Soundness of S3 is one-directional: matching fingerprints provide evidence for equivalence but are not used as definitive proof — mismatching fingerprints are ignored.

35.2 Strategy S0: Closed-Term Evaluation

Definition 35.2 (Closed term)**.** A function f is a *closed term* if it has no parameters (arity zero). Its domain is the singleton set $\{*\}$, so its denotation is the single value $[\![f]\!](*) \in B$.

For closed terms, equivalence checking reduces to value comparison:

$$f \equiv g \iff [\![f]\!](*) = [\![g]\!](*)$$

The implementation evaluates both functions in a sandboxed subprocess and compares the results. This is *not* sampling — it is complete evaluation of the term at the unique point of its domain.

For functions with only variadic parameters (`*args`, `**kwargs`), the zero-argument call serves as a *witness*: if $f() \neq g()$, then $f \not\equiv g$ (sound for inequivalence only).

35.3 Strategy S1: Denotational OTerm Equivalence

The denotational normalizer compiles both functions to canonical *OTerm* representations — abstract syntax trees with quotient types that identify nondeterministic orderings (e.g., $\mathsf{set}(\{a, b\}) = \mathsf{set}(\{b, a\})$).

If the canonical forms are equal, the programs are equivalent. If provably different, they are inequivalent. Otherwise, the result is inconclusive and we fall through to Z3.

35.4 Strategy S2: Z3 Structural Equality

After compiling both functions to Z3 PyObj sections, the pipeline checks whether the section lists are *syntactically identical*: same guards, same terms, same order.

This is fast ($O(n)$ in the AST size) but only works when both programs have the same control flow and use the same operations. Crucially, the implementation only trusts structural equality when all Z3 terms are *fully interpreted* — built entirely from `RecFunction` definitions (the binop, unop, truthy family) with no uninterpreted function symbols. Uninterpreted symbols (the py_$*$ shared functions) can hide semantic differences because Z3 is free to choose any consistent interpretation.

35.5 Strategy S3: Semantic Fingerprints

The fingerprint extractor summarizes each function by its *operation signature*: the multiset of function calls, built-in operations, fold patterns, and shared symbol references. Two functions with matching fingerprints are likely equivalent; a mismatch provides no information (different algorithms can compute the same function).

Fingerprints serve as a cheap heuristic that occasionally lets the pipeline skip the expensive per-fiber Z3 phase. They are never used as definitive proof of inequivalence.

35.6 Strategy S4: Per-Fiber Z3 + Čech H^1

This is the full CCTT kernel, executed when all fast paths are inconclusive.

35.6.1 Step 4a: Duck Type Inference

For each parameter, the duck type inferrer analyzes operations in both function ASTs to determine allowed type fibers:

Operations observed	Inferred fiber
$+$, $-$, $*$, `range()`, $\leq$	$\{\mathsf{Int}\}$
`.upper()`, `.split()`, `+` on strings	$\{\mathsf{Str}\}$
`.append()`, `len()`, indexing	$\{\mathsf{Pair}, \mathsf{Ref}\}$
No type-specific operations	$\{\mathsf{Int}, \mathsf{Bool}, \mathsf{Str}, \mathsf{Pair}, \mathsf{Ref}, \mathsf{None}\}$

35.6.2 Step 4b: Site Category Construction

The type site $\mathcal{S}$ has:

- **Objects**: fiber combinations $\vec{\tau} = (\tau_1, \ldots, \tau_k)$ from the Cartesian product of per-parameter fibers (capped at 12 to prevent blow-up).
- **Morphisms**: $\vec{\tau} \to \vec{\sigma}$ whenever $\vec{\tau}$ and $\vec{\sigma}$ share at least one component ($\exists j.\, \tau_j = \sigma_j$).
- **Cover**: $\mathcal{U} = \{\vec{\tau}_1, \ldots, \vec{\tau}_n\}$, the full set of fiber combinations.

35.6.3 Step 4c: Local Equivalence Queries

For each fiber $\vec{\tau}_i$, the Z3 solver checks:

$$\exists \vec{p}.\ \bigwedge_k \mathsf{tag}(p_k) = \tau_k^{(i)} \ \wedge\ \mathsf{tag}(p_k) \neq \mathsf{TBot} \ \wedge\ [\![f]\!](\vec{p}) \neq [\![g]\!](\vec{p})$$

Each query includes all path-constructor axioms, enabling Z3 to prove semantic equality even when the terms differ syntactically. Per-fiber timeouts are computed dynamically: $t_{\text{fiber}} = \max(200\text{ms},\ t_{\text{remaining}}/|\mathcal{U}|)$.

35.6.4 Step 4d: Čech Cohomology

The local judgments feed into the `CechCohomologyComputer`:

1. Build C^0 from local judgments: $C_i^0 = \mathcal{F}(U_i) \in \{0, 1\}$.
2. Compute $\delta^0 : C^0 \to C^1$ via $(\delta^0 s)_{ij} = s_j - s_i$ over $\mathbb{F}_2$.
3. Compute $\delta^1 : C^1 \to C^2$ via $(\delta^1 \alpha)_{ijk} = \alpha_{jk} - \alpha_{ik} + \alpha_{ij}$ over $\mathbb{F}_2$.
4. Compute $H^1 = \ker(\delta^1)/\mathrm{im}(\delta^0)$ via GF(2) rank computation.
5. If $H^1 = 0$: EQUIVALENT.
6. If $H^1 \neq 0$: INEQUIVALENT with obstruction data.

35.6.5 Step 4e: Verdict and Confidence

The final verdict combines the Čech result with confidence scoring (Definition 34.6):

$$\mathsf{Result} := \begin{cases} (\mathsf{True},\ \kappa = h_0/|\mathcal{U}|) & \text{if } H^1 = 0 \\ (\mathsf{False},\ \text{fiber } U_k,\ \text{counterexample } x_0) & \text{if } H^1 \neq 0 \\ (\mathsf{None},\ \kappa = h_0/|\mathcal{U}|) & \text{if inconclusive} \end{cases}$$

35.7 AST Compilation

The compiler walks the Python AST depth-first, producing Z3 terms:

Python construct	**Z3 term**
Integer literal n	$\mathsf{IntObj}(n)$
String literal s	$\mathsf{StrObj}(s)$
Variable x	$\mathsf{env}[x]$ (Z3 Const)
$x + y$	$\mathsf{binop}(\mathsf{ADD}, [\![x]\!], [\![y]\!])$
$-x$	$\mathsf{unop}(\mathsf{NEG}, [\![x]\!])$
$x > y$	$\mathsf{binop}(\mathsf{GT}, [\![x]\!], [\![y]\!])$
x `and` y	$\mathsf{If}(\mathsf{truthy}([\![x]\!]), [\![y]\!], [\![x]\!])$
x `or` y	$\mathsf{If}(\mathsf{truthy}([\![x]\!]), [\![x]\!], [\![y]\!])$
`if c: t else: f`	$\mathsf{If}(\mathsf{truthy}([\![c]\!]), [\![t]\!], [\![f]\!])$
`sorted(x)`	$\mathsf{py_sorted}([\![x]\!])$ (shared)
`x.append(v)`	$\mathsf{py_mut_append}([\![x]\!], [\![v]\!])$ (shared)
`isinstance(x, int)`	$\mathsf{BoolObj}(\mathsf{tag}([\![x]\!]) = \mathsf{TInt})$
`for i in range(n)`	fold or RecFunction
Recursive $f(n-1)$	RecFunction or canonical fold
`[e for x in xs]`	$\mathsf{py_comp}_h([\![xs]\!])$ (shared)

35.8 Section Extraction

The compiler produces a list of *sections* — pairs $(\mathsf{guard}, \mathsf{term})$:

- Each `return` statement in the function body produces one section.
- The `guard` is the conjunction of all branch conditions leading to that return.
- The `term` is the Z3 expression for the return value.

These sections form the value presheaf: each section is a "local behavior" of the program under a specific set of conditions.

35.9 Site Category Construction

The type site is built from duck type inference:

1. For each parameter, analyze its usage in both programs to determine allowed types (int, str, list, etc.).
2. Form the Cartesian product of allowed types across parameters.
3. Each element of the product is a site (type fiber assignment).
4. Morphisms connect fibers that share at least one type component.

35.10 Local Equivalence Check

For each site U_i, the Z3 solver checks:

$$\exists \vec{p}.\ \bigwedge_k \mathsf{tag}(p_k) = \tau_k^{(i)} \ \wedge\ \mathsf{tag}(p_k) \neq \mathsf{TBot} \ \wedge\ [\![f]\!](\vec{p}) \neq [\![g]\!](\vec{p})$$

If UNSAT: local equivalence holds ($\mathsf{Iso}(U_i) = 1$). If SAT: counterexample found ($\mathsf{Iso}(U_i) = 0$).

The solver includes all semantic axioms (Section 26), enabling Z3 to use the path constructors during proof search.

35.11 Čech Cohomology Computation

The local judgments feed into the `CechCohomologyComputer`:

1. Build C^0 from local judgments.
2. Compute δ^0 via `CoboundaryOperator.d0`.
3. Compute δ^1 via `CoboundaryOperator.d1`.
4. Compute $H^1 = \ker(\delta^1)/\mathrm{im}(\delta^0)$ via GF(2) rank computation.
5. If $H^1 = 0$: EQUIVALENT.
6. If $H^1 \neq 0$: INEQUIVALENT with obstruction data.

Chapter 36

The AST Compiler in Detail

This chapter specifies the compilation from Python AST to Z3 PyObj terms with enough detail that a reader could implement it from scratch. The compiler (`compiler.py`, ≈1360 lines) transforms a Python function's AST into a list of *presheaf sections* $(\mathsf{guard}, \mathsf{term})$, where each section represents one execution path through the function.

36.1 Compilation State

The compiler maintains an *environment* — a mapping from variable names to Z3 terms, augmented with metadata for defunctionalization and module resolution:

Definition 36.1 (Compilation Environment)**.**

$$\mathsf{Env} := \left\{ \begin{array}{ll} T : \mathsf{Theory}, & \mathsf{bindings} : \mathsf{str} \to \mathsf{PyObj}, \\ \mathsf{func_name} : \mathsf{str}, & \mathsf{is_rec} : \mathbb{B}, \\ \mathsf{func_defs} : \mathsf{str} \to \mathsf{AST}, & \mathsf{class_defs} : \mathsf{str} \to \mathsf{AST}, \\ \mathsf{imports} : \mathsf{str} \to \mathsf{str}, & \mathsf{depth} : \mathbb{N} \end{array} \right\}$$

The additional fields beyond the basic $(\mathsf{bindings}, \mathsf{func_name}, \mathsf{is_rec})$ triple serve specific compilation strategies:

- $\mathsf{func_defs}$ stores AST nodes of nested function definitions for *inlining* (D2 defunctionalization, depth ≤ 3).
- $\mathsf{class_defs}$ tracks class definitions; class instantiation compiles to $\mathsf{Ref}()$.
- $\mathsf{imports}$ maps alias names to qualified module paths, enabling shared symbol resolution for calls like `math.gcd(a, b)`.
- depth prevents infinite inlining of mutually recursive helpers.

The environment is *immutable* in the cubical sense: branching creates copies. When an `if` statement has both a `then` and `else` branch, the compiler copies the environment for each branch and merges the results with If expressions:

$$\mathsf{merge}(\mathsf{env}_T, \mathsf{env}_F, c)[x] := \mathsf{If}(c,\ \mathsf{env}_T[x],\ \mathsf{env}_F[x])$$

for each variable x modified in either branch.

36.2 The Section Representation

Definition 36.2 (Presheaf Section)**.** A *section* is a frozen pair:

$$\mathsf{Section} := (\mathsf{guard} : \mathbb{B}_{\mathrm{Z3}},\ \mathsf{term} : \mathsf{PyObj}_{\mathrm{Z3}})$$

where guard is a Z3 Boolean formula encoding the path condition under which this execution path is taken, and term is the Z3 PyObj expression returned.

A program with k distinct return paths produces k sections. Together they form the *value presheaf* of the program:

$$\mathcal{F}_f := \{(\mathsf{guard}_1, t_1), \ldots, (\mathsf{guard}_k, t_k)\}$$

The guards partition (or cover) the input space: for well-formed programs, the disjunction $\bigvee_i \mathsf{guard}_i$ is tautological. When comparing programs, sections from f and g are paired by guard overlap (Chapter 38).

36.3 Expression Compilation Rules

Each Python expression AST node maps to a Z3 PyObj term. We give the complete rule set. In the implementation, the entry point is `compile_expr(node, env)`, which dispatches on the AST node type and catches exceptions (returning a fresh symbolic constant on failure, ensuring compilation never crashes):

36.3.1 Rule E1: Literals

$$[\![n]\!]_{\mathsf{env}} = \mathsf{IntObj}(n) \quad [\![s]\!]_{\mathsf{env}} = \mathsf{StrObj}(s) \quad [\![\mathsf{True}]\!]_{\mathsf{env}} = \mathsf{BoolObj}(\top) \quad [\![\mathsf{None}]\!]_{\mathsf{env}} = \mathsf{NoneObj}$$

Floats receive special handling: if $v = \lfloor v \rfloor$ and $|v| < 2^{31}$, the float is approximated as $\mathsf{IntObj}(\lfloor v \rfloor)$; otherwise a fresh symbolic constant is introduced. NaN and $\pm\infty$ always produce fresh constants, since Z3's integer sort cannot represent them.

36.3.2 Rule E2: Variables

$$[\![x]\!]_{\mathsf{env}} = \begin{cases} \mathsf{env.bindings}[x] & \text{if } x \in \mathsf{env.bindings} \\ \mathsf{fresh}(\texttt{builtin_}x) & \text{if } x \in \mathsf{SHARED_BUILTINS} \\ \mathsf{fresh}(\texttt{import_}x) & \text{if } x \in \mathsf{env.imports} \\ \mathsf{fresh}(\texttt{var_}x) & \text{otherwise} \end{cases}$$

The third case ensures undeclared variables produce distinct fresh constants rather than crashing. Shared builtins (≈60 names: `sorted`, `len`, `sum`, `enumerate`, etc.) get a canonical fresh constant so that references to the same builtin in both programs produce the same Z3 symbol.

36.3.3 Rule E3: Binary Operations

$$[\![a \oplus b]\!]_{\mathsf{env}} = \mathsf{binop}([\![\oplus]\!], [\![a]\!]_{\mathsf{env}}, [\![b]\!]_{\mathsf{env}})$$

where $[\![\oplus]\!]$ maps Python operators to integer operation tags:

+	1	-	2	*	3	//	5
%	6	**	7	«	8	»	9
|	12	&	13	^	14	<	20
<=	21	>	22	>=	23	==	24

True division (/) is special-cased: since the PyObj algebra has no rational sort, it compiles to an uninterpreted shared function $T.\mathsf{shared_fn}(\texttt{truediv}, 2)(l, r)$.

36.3.4 Rule E4: Unary Operations

$$[\![-a]\!]_{\mathsf{env}} = \mathsf{unop}(\mathsf{NEG}, [\![a]\!]_{\mathsf{env}}) \qquad [\![\mathsf{not}\ a]\!]_{\mathsf{env}} = \mathsf{unop}(\mathsf{NOT}, [\![a]\!]_{\mathsf{env}})$$

The tilde operator (`~`) also maps to NOT, reflecting Python's use of bitwise inversion. Unary `+` is the identity.

36.3.5 Rule E5: Comparisons (Chained)

For $a < b \leq c$:

$$[\![a < b \leq c]\!]_{\mathsf{env}} = \mathsf{If}(\mathsf{truthy}(\mathsf{binop}(\mathsf{LT}, [\![a]\!], [\![b]\!])),\ \mathsf{binop}(\mathsf{LE}, [\![b]\!], [\![c]\!]),\ \mathsf{BoolObj}(\bot))$$

The general rule for n-way chains $a\ \mathrm{op}_1\ b\ \mathrm{op}_2\ c\ \ldots$: fold right with short-circuit If. Identity tests `is`/`is not` compile to structural Z3 equality $\mathsf{BoolObj}(a = b)$. Membership tests `in`/`not in` compile via the shared function $\mathsf{py_contains}([\![\text{collection}]\!], [\![\text{element}]\!])$.

36.3.6 Rule E6: Boolean Operations

$$[\![a\ \mathsf{and}\ b]\!]_{\mathsf{env}} = \mathsf{If}(\mathsf{truthy}([\![a]\!]), [\![b]\!], [\![a]\!]) \tag{36.1}$$

$$[\![a\ \mathsf{or}\ b]\!]_{\mathsf{env}} = \mathsf{If}(\mathsf{truthy}([\![a]\!]), [\![a]\!], [\![b]\!]) \tag{36.2}$$

Note: Python's `and`/`or` return the *value*, not a boolean. The Z3 encoding faithfully preserves this. For chains like `a and b and c`, the compiler folds right: the last value is the innermost, and each predecessor wraps in an If.

36.3.7 Rule E7: Conditional Expression

$$[\![t\ \mathsf{if}\ c\ \mathsf{else}\ f]\!]_{\mathsf{env}} = \mathsf{If}(\mathsf{truthy}([\![c]\!]), [\![t]\!], [\![f]\!])$$

36.3.8 Rule E8: Function Calls

Function calls have the most complex compilation, with a priority cascade:

1. **Self-recursive call**: If env.is_rec and the callee is env.func_name, create a fresh uninterpreted function application: $\mathsf{rec_f_uid}([\![a_1]\!], \ldots, [\![a_n]\!])$. The uid is incremented each call to prevent Z3 from conflating distinct recursive invocations.

2. **Nested function inlining (D2 β-reduction)**: If the callee is in env.func_defs and depth < 3, invoke inline_call: create a copy of the environment, bind formals to compiled actuals, and compile the body. If the body has a single section, return its term directly; otherwise fold sections into a nested If.

3. **Known builtins with Z3 semantics**: `abs`, `int`, `bool`, `len` compile to unop; `max`, `min` with two arguments compile to binop(MAX/MIN, ...). Keyword arguments like `key=` are handled by creating variant shared functions (sorted_key, max_key, etc.).

4. **isinstance**: compile to $\mathsf{BoolObj}(\mathsf{tag}([\![x]\!]) = \tau)$ where τ is the type tag. For tuple type arguments `isinstance(x, (int, str))`, produce $\mathsf{BoolObj}(\mathsf{tag}([\![x]\!]) = \mathsf{TInt} \vee \mathsf{tag}([\![x]\!]) = \mathsf{TStr})$.

5. **Shared builtins**: If the callee name is in SHARED_BUILTINS ($\approx$60 names), use the *univalent* shared function symbol: $T.\mathsf{shared_fn}(\mathsf{name}, n)([\![a_1]\!], \ldots, [\![a_n]\!])$. Univalence means both programs get the *same* Z3 function declaration for the same builtin, so $\mathsf{py_sorted}(x) = \mathsf{py_sorted}(x)$ is trivially true.

6. **Module-qualified calls**: For `math.gcd(a, b)`, resolve via $\mathsf{env.imports}$ and use $T.\mathsf{shared_fn}(\texttt{call_math.g}$ The implementation recognizes ≈70 standard library functions from `math`, `itertools`, `collections`, `heapq`, `bisect`, etc.

7. **Method calls**: Compile receiver and arguments, use shared method symbol: $T.\mathsf{shared_fn}(\mathsf{meth_name}, n+1)([\![\mathsf{recv}]\!], [\![a_1]\!], \ldots, [\![a_n]\!])$.

36.3.9 Rule E9: Subscript

$$[\![a[b]]\!]_{\mathsf{env}} = \mathsf{binop}(\mathsf{GETITEM}, [\![a]\!], [\![b]\!])$$

Slicing (`a[i:j]`) compiles to a shared function $\mathsf{py_slice}(a, [\![i]\!], [\![j]\!])$ since slice objects are not representable in the PyObj ADT.

36.3.10 Rule E10: Tuple/List Display

$$[\![[a, b, c]]\!]_{\mathsf{env}} = \mathsf{Pair}([\![a]\!], \mathsf{Pair}([\![b]\!], \mathsf{Pair}([\![c]\!], \mathsf{NoneObj})))$$

This is the cons-cell encoding. Both tuples and lists use the same representation, which is sound for equivalence checking since the distinction is only relevant for mutation.

36.3.11 Rule E11: Comprehension

$$[\![[e \text{ for } x \text{ in } \mathsf{xs}]]\!]_{\mathsf{env}} = T.\mathsf{shared_fn}(\mathsf{comp}_h, 1)([\![\mathsf{xs}]\!])$$

where $h = \mathsf{MD5}(\mathsf{ast.dump}(e) \mid \mathsf{ast.dump}(\text{filters}) \mid \mathsf{ast.dump}(\text{target})) \bmod 2^{31}$ is the canonical hash of the *entire comprehension structure*: element expression, filter conditions, and target pattern. Comprehensions with identical AST structure in both programs hash to the same shared function, enabling Z3 to prove equivalence. Filtered comprehensions use a distinct prefix ($\mathsf{comp_filt}_h$). Nested comprehensions chain: $\mathsf{comp_nest}_{h'}(\mathsf{comp}_h([\![\mathsf{xs}]\!]), [\![\mathsf{ys}]\!])$.

36.3.12 Rule E12: Attribute Access

$$[\![a.\mathsf{attr}]\!]_{\mathsf{env}} = T.\mathsf{shared_fn}(\mathsf{attr_attr}, 1)([\![a]\!])$$

Module-qualified attributes are checked first: `math.pi`, `sys.maxsize` compile to canonical fresh constants $\mathsf{const_math.pi}$, $\mathsf{const_sys.maxsize}$.

36.3.13 Additional Expression Rules

Dict literals. An empty dict is $\mathsf{Ref}()$. Non-empty dicts fold over key-value pairs: $d = \mathsf{mut___setitem__}(\ldots(\mathsf{Ref}(), k_1, v_1)\ldots, k_n, v_n)$.

F-strings. Formatted string literals compile by concatenating parts via $\mathsf{str_concat}(p_1, p_2)$.

Walrus operator. $[\![(x := e)]\!] = [\![e]\!]$ with side effect $\mathsf{env}[x] \leftarrow [\![e]\!]$.

Lambda. Produces a fresh constant $\mathsf{fresh}(\texttt{lambda})$. Lambdas are not inlined because they lack a name for the $\mathsf{func_defs}$ registry.

36.4 Statement Compilation Rules

Statements modify the environment and produce *sections* (for `return` statements).

36.4.1 Rule S1: Assignment

$$\mathsf{exec}(x = e, \mathsf{env}) = \mathsf{env}[x \mapsto [\![e]\!]]$$

For tuple unpacking $x, y = e$:

$$\mathsf{exec}((x, y) = e, \mathsf{env}) = \mathsf{env}[x \mapsto \mathsf{binop}(\mathsf{GETITEM}, [\![e]\!], \mathsf{IntObj}(0)),\ y \mapsto \mathsf{binop}(\mathsf{GETITEM}, [\![e]\!], \mathsf{IntObj}(1))]$$

Starred unpacking (`*rest`) compiles via $\mathsf{slice_from}([\![e]\!], \mathsf{IntObj}(j))$. Subscript assignment (`obj[k] = v`) is modeled as mutation: $\mathsf{env}[\mathsf{obj}] \leftarrow \mathsf{mut___setitem__}(\mathsf{env}[\mathsf{obj}], [\![k]\!], [\![v]\!])$.

36.4.2 Rule S2: Augmented Assignment

$$\mathsf{exec}(x \oplus= e, \mathsf{env}) = \mathsf{env}[x \mapsto \mathsf{binop}([\![\oplus]\!], \mathsf{env}[x], [\![e]\!])]$$

Subscript augmented assignment (`obj[k] += v`) compiles as: read old value via GETITEM, apply the operator, then write back via mut___setitem__.

36.4.3 Rule S3: Return

$$\mathsf{exec}(\mathsf{return}\ e, \mathsf{env}, \mathsf{guard}) = \mathsf{Section}(\mathsf{guard}, [\![e]\!])$$

This terminates the current execution path and produces a section.

36.4.4 Rule S4: If/Elif/Else

$$\mathsf{exec}(\mathsf{if}\ c : B_1\ \mathsf{else} : B_2, \mathsf{env}, \mathsf{guard})$$

1. Compile the condition: $\mathsf{cond} = \mathsf{truthy}([\![c]\!])$.
2. True branch: $\mathsf{guard}_T = \mathsf{guard} \wedge \mathsf{cond}$, compile B_1 with $\mathsf{env}_T = \mathsf{env.copy}()$.
3. False branch: $\mathsf{guard}_F = \mathsf{guard} \wedge \neg\mathsf{cond}$, compile B_2 with $\mathsf{env}_F = \mathsf{env.copy}()$.
4. If both branches return: collect sections from both, terminate.
5. If only one returns: collect its sections, continue with the other branch's modified environment and the negated guard.
6. If neither returns: merge environments with $\mathsf{env}[x] = \mathsf{If}(\mathsf{cond}, \mathsf{env}_T[x], \mathsf{env}_F[x])$ for each modified variable x.

This case analysis (five sub-cases) is the most delicate part of the section extractor. Partial returns create *asymmetric* sections where one guard is more constrained than the other.

36.4.5 Rule S5: For Loop

$$\mathsf{exec}(\mathsf{for}\ i\ \mathsf{in}\ \mathsf{range}(\mathsf{start}, \mathsf{stop}) : B, \mathsf{env})$$

Case 1: Canonical fold. If B is a single augmented assignment $\mathsf{acc} \oplus= i$ (the loop variable) and $\oplus \in \{+, \times\}$:

$$\mathsf{env}[\mathsf{acc}] \leftarrow \mathsf{fold}([\![\oplus]\!], \mathsf{start}, \mathsf{stop}, \mathsf{env}[\mathsf{acc}])$$

This is the critical optimization: the canonical fold is a Z3 RecFunction with known algebraic properties (commutativity, associativity, identity elements), enabling equational proofs.

Case 2: General RecFunction. For complex loop bodies, first identify all modified variables by walking the loop body (including nested `if`, `for`, `while`, mutation calls). Then for each modified variable v, create:

$$\mathsf{lp_}v(i) = \begin{cases} \mathsf{env}[v] & \text{if } i \geq \mathsf{stop} \\ \mathsf{body}(\mathsf{lp_}v(i+1), i) & \text{otherwise} \end{cases}$$

Set $\mathsf{env}[v] \leftarrow \mathsf{lp_}v(\mathsf{start})$.

Case 3: Iteration over collections. For `for x in lst`, where `lst` is not a `range` call, the loop body is modeled as a shared fold function: $\mathsf{env}[\mathsf{acc}] \leftarrow \mathsf{fold}_h(\mathsf{env}[\mathsf{acc}], [\![\mathsf{lst}]\!])$ where h is a hash of the loop body AST.

36.4.6 Rule S6: While Loop

Similar to S5 but with a guard condition instead of a range. Create a RecFunction with a bounded counter ($k \leq 50$):

$$\mathsf{wh_}v(k) = \begin{cases} \mathsf{env}[v] & \text{if } k > 50 \vee \neg\mathsf{cond} \\ \mathsf{body}(\mathsf{wh_}v(k+1)) & \text{otherwise} \end{cases}$$

The bound of 50 is a soundness trade-off: Z3 can unfold the RecFunction up to 50 iterations, which suffices for most equivalence proofs involving small-bounded loops.

36.4.7 Rule S7: Function Definition

$$\mathsf{exec}(\mathsf{def}\ f(\ldots) : B, \mathsf{env}) = \mathsf{env}[f \mapsto (\mathsf{__func__}, \mathsf{AST}(f))]$$

Store the AST node in both $\mathsf{func_defs}$ and $\mathsf{bindings}$ for later inlining (D2 defunctionalization). Class definitions are stored analogously in $\mathsf{class_defs}$.

36.4.8 Rule S8: Expression Statement (Mutation)

For $\mathsf{obj.method(args)}$:

$$\mathsf{env}[\mathsf{obj}] \leftarrow T.\mathsf{shared_fn}(\mathsf{mut_method}, |\mathsf{args}| + 1)(\mathsf{env}[\mathsf{obj}], [\![\mathsf{args}]\!])$$

This models mutation: calling a method on an object produces a new version of that object. The $\mathsf{mut_}$ prefix distinguishes mutating calls from pure method calls in shared symbol resolution.

36.4.9 Rule S9: Try/Except

Compile the `try` body. For each section produced, wrap with an uninterpreted "handler context" function keyed by the exception types:

$$\mathsf{Section}(\mathsf{guard}, \mathsf{try_ctx}_h(\mathsf{term}))$$

where $h = \mathsf{hash}(\mathsf{handler_signature})$. Handler bodies are also compiled and their sections added, modeling the possibility that an exception is caught and handled. Finally-blocks and else-blocks execute their effects on the environment unconditionally.

36.5 The Section Extractor

The section extractor traverses a statement list and collects all Section(guard, term) pairs reachable by any execution path:

Listing 36.1: Section extraction algorithm

```
def extract_sections(stmts, env, guard=True):
    sections = []
    for stmt in stmts:
        if isinstance(stmt, Return):
            sections.append(Section(guard, compile(stmt.value, env))
                )
            return sections  # path terminates
        elif isinstance(stmt, If):
            cond = truthy(compile(stmt.test, env))
            t_guard = And(guard, cond)
            f_guard = And(guard, Not(cond))
            t_has_ret = has_return(stmt.body)
            f_has_ret = has_return(stmt.orelse)
            if t_has_ret:
                sections.extend(extract(stmt.body, env.copy(),
                    t_guard))
            if f_has_ret:
                sections.extend(extract(stmt.orelse, env.copy(),
                    f_guard))
            if t_has_ret and f_has_ret:
                return sections  # both branches return
            if t_has_ret:  # only then-branch returns
                # Continue with else-modified env under negated
                    guard
                re = env.copy()
                exec_stmts(stmt.orelse, re)
                sections.extend(extract(rest, re, f_guard))
                return sections
            # ... symmetric case for f_has_ret
            # Neither returns: merge envs, continue
            merge_envs(env, env_T, env_F, cond)
        elif isinstance(stmt, For):
            exec_for(stmt, env)
        else:
            exec_one(stmt, env)
    return sections
```

The output is a list of sections forming the *value presheaf* of the program. Each section is a local behavior under a specific path condition (guard).

36.6 The Nat-Eliminator Extraction

When a function is self-recursive with a single parameter, the compiler attempts to extract a *Nat-eliminator* (catamorphism) — a direct fold term. This succeeds when the function matches the pattern:

```
def f(n):
    if n <= threshold: return base_value
    return n OP f(n - 1)
```

where $\mathsf{OP} \in \{+, \times, -, //, \%, |, \&, \oplus\}$. The extractor (`detect_canon_rec`) verifies:

1. The body has exactly two statements: base case + recursive case.
2. The base case is an `if` with a constant-threshold comparison and a constant return value.
3. The recursive case is a `return` of a BinOp with the parameter on one side and $f(n-1)$ on the other.

If all conditions hold, the output is: $\mathsf{fold}(\mathsf{OP}, \mathsf{threshold}, \mathsf{ival}(p) + 1, \mathsf{IntObj}(\mathsf{base}))$.

Remark 36.3 (Why Nat-elimination matters)**.** The fold representation is decisive: two programs that compute the same sum — one recursive, one iterative — both reduce to the *same* fold term (up to additive identity), enabling Z3 to prove equivalence by unfolding the RecFunction definition. Without this extraction, the recursive version would produce an uninterpreted RecFunction that Z3 cannot equate with the iterative fold.

36.7 What the Compiler Cannot Handle

The compiler has principled limitations:

1. **Closures**: Lambda expressions and closures that capture mutable state produce fresh constants. The compiler cannot inline a lambda because lambdas lack the named-function structure required for func_defs registration.
2. **Generators**: `yield` and `yield from` compile to the value yielded (or NoneObj), but the lazy evaluation semantics — generator state, exhaustion, send/throw — is not modeled.
3. **Dynamic dispatch**: When a variable holds a function object f and is called as $f(x)$, the compiler creates $\mathsf{dyncall_f}(v, [\![x]\!])$ — an uninterpreted function. Z3 cannot reason about the body of dynamically dispatched calls.
4. **Mutation aliasing**: The functional mutation model (`mut_append`, `mut___setitem__`) assumes no aliasing. If two variables refer to the same mutable object, mutating one does not update the other in the environment.
5. **Recursion depth**: General (non-Nat-eliminator) recursive functions produce a single RecFunction whose body is the entire function. Z3 can unfold these finitely, but deep recursion (> 50 levels) exceeds practical timeouts.
6. **Floating-point arithmetic**: All floats are approximated as integers or fresh constants. IEEE 754 semantics (rounding, NaN propagation, denormals) is not modeled.

Each limitation corresponds to a *soundness boundary*: when the compiler encounters these constructs, it introduces uninterpreted symbols, which may cause the Z3 check to be inconclusive but never produce a false equivalence proof.

Chapter 37

Duck Type Inference in Detail

Duck type inference determines which type fibers are relevant for each parameter. Narrower fibers mean fewer Z3 queries and faster verification. The algorithm (`duck.py`, ≈230 lines) extracts the set of operations applied to each parameter from *both* programs' ASTs, then maps the operation set to a lattice element.

37.1 The Duck Type Lattice

The result of inference is an element from the following lattice, ordered by inclusion of the associated fiber set:

Duck type	**Fiber set** $F(\tau)$	**Triggering operations**
int	$\{\mathsf{TInt}\}$	`-`, `*`, `//`, `%`, `**`, `«`, `»`, `range(x)`, `<`, `<=`
str	$\{\mathsf{TStr}\}$	`.upper()`, `.split()`, `.strip()`, `.join()`, `.replace(`
		`.startswith()`, `.find()`, ≈25 string methods
bool	$\{\mathsf{TBool}\}$	(rare — usually widened to int)
ref	$\{\mathsf{TRef}\}$	`.get()`, `.keys()`, `.items()`, `.values()`,
		`.add()`, `.discard()`, `.union()`, `.intersection()`
list	$\{\mathsf{TPair}, \mathsf{TRef}\}$	`.append()`, `.sort()`, `.extend()`, `.reverse()`, `.inse`
collection	$\{\mathsf{TPair}, \mathsf{TRef}, \mathsf{TStr}\}$	`for x in p`, `len(p)`, `sorted(p)`, `sum(p)`, `min(p)`
any	$\{\mathsf{TInt}, \mathsf{TBool}, \mathsf{TStr}, \mathsf{TPair}, \mathsf{TRef}, \mathsf{TNone}\}$	no type-constraining operations, or ambiguous `+`

Definition 37.1 (Duck Type Lattice)**.** The duck type lattice $(\mathcal{D}, \sqsubseteq)$ is the poset with ordering induced by fiber set inclusion:

$$\tau_1 \sqsubseteq \tau_2 \iff F(\tau_1) \subseteq F(\tau_2)$$

giving the Hasse diagram:

$$\mathsf{int} \sqsubseteq \mathsf{list} \sqsubseteq \mathsf{collection} \sqsubseteq \mathsf{any} \qquad \mathsf{str} \sqsubseteq \mathsf{collection} \qquad \mathsf{ref} \sqsubseteq \mathsf{any}$$

The inference algorithm returns the *greatest lower bound* (narrowest type) consistent with all observed operations.

37.2 The Inference Algorithm

The algorithm has three phases: (1) extract operations from both programs, (2) union the operation sets, (3) apply the priority cascade:

Listing 37.1: Duck type inference (from duck.py)

```
def infer_duck_type(func_f, func_g, pname):
    ops_f = extract_duck_ops(func_f, pname)
    ops_g = extract_duck_ops(func_g, pname)
    ops = ops_f | ops_g  # union: both programs' usage

    # Phase 1: isinstance checks (strongest evidence)
    isinstance_types = {o.replace('isinstance_', '')
                        for o in ops if o.startswith('isinstance_')}
    if isinstance_types:
        return resolve_isinstance(isinstance_types)

    # Phase 2: Priority cascade (most specific first)
    if ops & NUMERIC_OPS:       return 'int'
    if ops & ORDERING_OPS:      return 'int'
    if ops & STRING_METHODS:    return 'str'
    if ops & DICT_METHODS:      return 'ref'
    if ops & LIST_METHODS:      return 'list'
    if ops & SET_METHODS:       return 'ref'
    if ops & COLLECTION_OPS:    return 'collection'

    # Phase 3: Ambiguous operators
    if ops & {'add', 'iadd'}:  return 'any'  # + is polymorphic
    return 'unknown'
```

Key insight: we analyze *both* programs to determine the duck type. If program f uses arithmetic on parameter x but program g only uses comparison, the union of operations includes arithmetic, so x is constrained to `int`. This ensures both programs are compared on the *same* type fiber.

The + ambiguity: The operator `+` is polymorphic in Python: it means addition on `int`, concatenation on `str`, and extension on `list`. Without additional evidence (a numeric operation like `-` or `*`, or a string method like `.upper()`), the algorithm conservatively returns `any`, ensuring soundness at the cost of more fibers.

37.3 Operation Extraction from AST

The operation extractor (`extract_duck_ops`) walks the AST and collects all operations applied to a given parameter, with *transitive reference tracking*:

Listing 37.2: Operation extraction with transitive references

```
def extract_duck_ops(func_node, param):
    ops = set()
    for child in ast.walk(func_node):
        if isinstance(child, BinOp):
            # Transitive: (a + b) refers to param if a or b does
            if _refers_to(child.left, param) or \
               _refers_to(child.right, param):
                ops.add(binop_name(child.op))

        if isinstance(child, Call):
            # Method call on param: param.method()
            if isinstance(child.func, Attribute) and \
               _refers_to(child.func.value, param):
                ops.add('method_' + child.func.attr)
            # Builtin call with param: len(param), sorted(param)
            if isinstance(child.func, Name):
```

```
                for arg in child.args:
                    if _refers_to(arg, param):
                        ops.add('call_' + child.func.id)

        if isinstance(child, For) and _refers_to(child.iter, param):
            ops.add('iter')

        if isinstance(child, Subscript):
            if _refers_to(child.value, param):
                ops.add('getitem')
            if _refers_to(child.slice, param):
                ops.add('used_as_index')  # forces int

        # isinstance(param, T) extracts the tested type
        if is_isinstance_call(child) and _refers_to(child.args[0],
           param):
            ops.add('isinstance_check')
            ops |= extract_isinstance_types(child.args[1])

        # Comprehension iteration
        if isinstance(child, (ListComp, SetComp, GeneratorExp)):
            for gen in child.generators:
                if _refers_to(gen.iter, param):
                    ops.add('iter')
    return ops
```

The `_refers_to` function is *transitive*: it follows sub-expressions so that `(a + b)` refers to parameter `a` even though the AST node is a `BinOp`, not a `Name`. This transitivity handles expressions like `len(xs[:n])` where `xs` is a parameter buried inside a subscript inside a call.

37.4 Formal Connection to Galois Connections

The duck type inference forms a *Galois connection* between the lattice of Python operations and the lattice of type fibers (Chapter 12):

Definition 37.2 (Duck Type Galois Connection)**.** Let $\mathcal{O}$ be the powerset lattice of operation names, and $\mathcal{D}$ be the duck type lattice. Define:

$$\alpha : \mathcal{P}(\mathcal{O}) \to \mathcal{D}, \qquad \alpha(\text{ops}) := \bigsqcap \{\tau \in \mathcal{D} \mid \text{ops} \subseteq \text{Ops}(\tau)\} \tag{37.1}$$

$$\gamma : \mathcal{D} \to \mathcal{P}(\mathcal{O}), \qquad \gamma(\tau) := \text{Ops}(\tau) \tag{37.2}$$

where $\text{Ops}(\tau)$ is the set of operations supported by duck type τ.

Proposition 37.3 (Galois Connection Property)**.** *$\alpha \dashv \gamma$: for all $O \subseteq \mathcal{O}$ and $\tau \in \mathcal{D}$:*

$$\alpha(O) \sqsubseteq \tau \iff O \subseteq \gamma(\tau)$$

Concretely: the inferred duck type is the narrowest type whose operation set contains all observed operations.

The priority cascade in `infer_duck_type` implements α by checking operation subsets from most specific (int) to least specific (any), returning the first match. This is sound because the operation sets are nested: $\text{Ops}(\mathsf{int}) \subset \text{Ops}(\mathsf{collection}) \subset \text{Ops}(\mathsf{any})$.

37.5 Fiber Narrowing and the Site Category

Given duck types for all parameters, the site category is constructed as the product of fiber sets. For a function with parameters $(p_1, \ldots, p_n)$ and inferred duck types $(\tau_1, \ldots, \tau_n)$, the site category has:

$$\text{Sites} = F(\tau_1) \times F(\tau_2) \times \cdots \times F(\tau_n)$$

fiber combinations, capped at 12 to prevent Z3 memory blow-up.

Overlap computation. Two fiber combinations $(f_1, \ldots, f_n)$ and $(g_1, \ldots, g_n)$ overlap iff they share at least one component: $\exists k.\, f_k = g_k$. This defines the Čech nerve of the covering.

Example 37.4 (Two-parameter site category)**.** For a function `f(x, lst)` with $\tau_x = \mathsf{int}$ and $\tau_{\mathsf{lst}} = \mathsf{list}$:

$$F(\mathsf{int}) = \{\mathsf{TInt}\}, \quad F(\mathsf{list}) = \{\mathsf{TPair}, \mathsf{TRef}\}$$
$$\text{Sites} = \{(\mathsf{TInt}, \mathsf{TPair}),\ (\mathsf{TInt}, \mathsf{TRef})\}$$
$$\text{Overlaps} = \{((\mathsf{TInt}, \mathsf{TPair}),\ (\mathsf{TInt}, \mathsf{TRef}))\}$$

The two sites overlap on the TInt axis.

Chapter 38

The Čech Complex Construction Algorithm

This chapter gives the complete algorithm for building the Čech complex from local equivalence judgments, computing the coboundary operators, and determining H^1. The implementation (`cohomology.py`, ≈150 lines) is deliberately compact: the mathematical structure does the heavy lifting.

38.1 Data Structures

Definition 38.1 (LocalJudgment)**.** Each fiber produces a *local judgment*:

$$\mathsf{LocalJudgment} := \left\{ \begin{array}{l} \mathsf{fiber} : (\mathsf{str}, \ldots), \quad \mathsf{is_equivalent} : \mathbb{B} \cup \{\bot\}, \\ \mathsf{counterexample} : \mathsf{str}?, \quad \mathsf{explanation} : \mathsf{str}, \\ \mathsf{concrete_obstruction} : \mathbb{B} \end{array} \right\}$$

where $\bot$ denotes timeout/inconclusive. The concrete_obstruction flag indicates whether the counterexample uses only interpreted Z3 functions (a "hard" proof of inequivalence) versus uninterpreted functions (a "soft" proof that might be vacuous).

Definition 38.2 (CechResult)**.** The output of the cohomology computation:

$$\mathsf{CechResult} := \left\{ \begin{array}{l} \mathsf{h0} : \mathbb{N}, \quad \mathsf{h1_rank} : \mathbb{N}, \quad \mathsf{total_fibers} : \mathbb{N}, \\ \mathsf{unknown_fibers} : \mathbb{N}, \quad \mathsf{obstructions} : [(\mathsf{str}, \ldots)] \end{array} \right\}$$

The verdict is derived from these counts:

- equivalent = True when h0 > 0 and h1_rank $= 0$ and unknown_fibers $= 0$ (or coverage $\geq 50\%$).
- equivalent = False when obstructions $\neq \emptyset$.
- equivalent $= \bot$ otherwise (inconclusive).

38.2 Step 1: Local Equivalence Judgments

For each site U_i (type fiber combination), the checker (`_check_fiber` in `checker.py`) runs a Z3 query. The procedure is more nuanced than a single sat/unsat check:

Listing 38.1: Local equivalence check per fiber

```
def check_fiber(T, params, secs_f, secs_g, combo, timeout_ms):
```

```
    solver = Solver()
    solver.set('timeout', min(timeout_ms, 3000))
    solver.set('max_memory', 256)  # MB
    T.semantic_axioms(solver)

    # Constrain params to this fiber
    for idx, fiber in enumerate(combo):
        solver.add(tag(params[idx]) == tag_const(fiber))
        solver.add(tag(params[idx]) != TBot)

    h0 = inconclusive = 0
    for sf in secs_f:
        for sg in secs_g:
            overlap = And(sf.guard, sg.guard)
            # First: can guards overlap at all?
            solver.push(); solver.add(overlap)
            if solver.check() == unsat:
                solver.pop(); continue
            solver.pop()

            # Check if terms can differ under overlapping guards
            solver.push()
            solver.add(overlap)
            solver.add(sf.term != sg.term)
            r = solver.check()
            if r == unsat:
                # Only trust if both terms have same opacity
                if terms_same_opacity(sf.term, sg.term):
                    h0 += 1
                else:
                    inconclusive += 1  # vacuously equal
            elif r == sat:
                # Counterexample found
                return LocalJudgment(combo, False, model_info)
            else:
                inconclusive += 1
            solver.pop()

    if h0 > 0 and inconclusive == 0:
        return LocalJudgment(combo, True)
    return LocalJudgment(combo, None)  # inconclusive
```

The opacity check. A critical subtlety: when Z3 reports unsat for $t_f \neq t_g$, the proof may be *vacuously true* if either term involves uninterpreted functions. Z3 is free to choose *any* interpretation for uninterpreted function symbols, so it can make them equal by fiat. The `_terms_same_opacity` function checks that both terms are *fully interpreted* (only using Z3 RecFunctions with known semantics like binop, truthy, fold). Only then is the unsat result trusted.

38.3 Step 2: The Short-Circuit

Before building any coboundary matrices, the algorithm checks for the common fast cases:

1. **Any inequivalent fiber**: If any LocalJudgment has is_equivalent = False, immediately return CechResult(h1_rank = $|\{\text{inequivalent fibers}\}|$) with the obstructions list. No matrix computation needed.

2. **No equivalent fibers**: If all fibers timed out, return $\mathsf{CechResult}(\mathsf{h0} = 0, \mathsf{h1_rank} = 0)$.

3. **Single fiber or no overlaps**: If there is only one equivalent fiber or no overlaps exist, $H^1 = 0$ trivially.

These short-circuits handle $> 90\%$ of practical cases. The full matrix computation is only needed when multiple equivalent fibers exist with non-trivial overlaps.

38.4 Step 3: Building C^0

The 0-cochain group is a vector over GF(2), one entry per site:

$$C^0 = \mathrm{GF}(2)^{|I|}$$

where entry i is 1 if site U_i is locally equivalent, 0 otherwise. Only the *equivalent* fibers participate in the δ^0 matrix; inequivalent fibers contribute to the obstruction count directly.

38.5 Step 4: Computing $\delta^0 : C^0 \to C^1$

The coboundary operator δ^0 is represented as a binary matrix of size $m \times n$ where $m = |\text{overlaps}|$ and $n = |\text{equiv_fibers}|$:

$$(\delta^0)_{k,i} = \begin{cases} 1 & \text{if overlap } k \text{ involves fiber } i \\ 0 & \text{otherwise} \end{cases}$$

Each overlap (U_a, U_b) produces a row with exactly two 1s in columns a and b. This is the standard Čech coboundary: $(\delta^0 f)(U_a \cap U_b) = f(U_b) - f(U_a)$ over GF(2).

Listing 38.2: δ^0 matrix construction (from `cohomology.py`)

```
fiber_idx = {f: i for i, f in enumerate(equiv_fibers)}
overlap_list = [(a, b) for a, b in overlaps
                if a in fiber_idx and b in fiber_idx]
m = len(overlap_list)

delta0 = [[0] * len(equiv_fibers) for _ in range(m)]
for k, (a, b) in enumerate(overlap_list):
    delta0[k][fiber_idx[a]] = 1
    delta0[k][fiber_idx[b]] = 1
```

38.6 Step 5: Computing $\delta^1 : C^1 \to C^2$

The second coboundary checks the cocycle condition on triple overlaps. Triple overlaps are identified by finding pairs of overlaps that share a common fiber:

Listing 38.3: Triple overlap detection and δ^1 construction

```
triples = []
for i, (a1, b1) in enumerate(overlap_list):
    for j, (a2, b2) in enumerate(overlap_list):
        if j <= i: continue
        common = {a1, b1} & {a2, b2}
        if common:
            third = ({a1, b1} | {a2, b2}) - common
            if third:
```

```
                    triples.append((i, j))

delta1 = [[0] * m for _ in range(len(triples))]
for t, (i, j) in enumerate(triples):
    delta1[t][i] = 1
    delta1[t][j] = 1
```

38.7 Step 6: Computing H^1 via GF(2) Rank

The first cohomology group is:

$$H^1 = \ker(\delta^1)/\mathrm{im}(\delta^0)$$

Its rank over GF(2) is computed as:

$$\mathrm{rank}(H^1) = \dim(\ker(\delta^1)) - \mathrm{rank}(\delta^0) = (m - \mathrm{rank}(\delta^1)) - \mathrm{rank}(\delta^0)$$

Listing 38.4: H^1 rank computation (from `cohomology.py`)

```
rank_delta0 = gf2_rank(delta0)
rank_delta1 = gf2_rank(delta1) if delta1 else 0
ker_delta1 = m - rank_delta1
h1_rank = max(0, ker_delta1 - rank_delta0)
```

38.8 The GF(2) Rank Algorithm

Gaussian elimination over GF(2) uses XOR as addition:

Listing 38.5: Gaussian elimination over GF(2)

```
def gf2_rank(matrix):
    m = [row[:] for row in matrix]
    rows, cols = len(m), len(m[0]) if m else 0
    rank = 0
    for col in range(cols):
        pivot = None
        for row in range(rank, rows):
            if m[row][col] == 1:
                pivot = row; break
        if pivot is None: continue
        m[rank], m[pivot] = m[pivot], m[rank]
        for row in range(rows):
            if row != rank and m[row][col] == 1:
                m[row] = [(m[row][j] + m[rank][j]) % 2
                          for j in range(cols)]
        rank += 1
    return rank
```

38.9 Correctness and Complexity

Theorem 38.3 (Correctness of the H^1 Algorithm)**.** *The algorithm computes the rank of* $H^1(\mathcal{U}, \mathsf{Iso}) = \ker(\delta^1)/\mathrm{im}(\delta^0)$ *over* GF(2) *correctly. Specifically:*

1. $\mathrm{rank}(H^1) = 0$ *iff all 1-cocycles are coboundaries iff local equivalences glue to a global equivalence.*

2. $\mathrm{rank}(H^1) > 0$ *gives the number of independent obstructions to gluing.*

3. *The GF(2) elimination runs in* $O(n^3)$ *where* n *is the number of overlaps.*

Proof. Standard linear algebra over a finite field. The rank is the dimension of the quotient space, which equals the dimension of the kernel minus the dimension of the image. Gaussian elimination computes both dimensions in $O(n^3)$. The $\max(0, \ldots)$ clamp handles the degenerate case where the matrix dimensions are inconsistent (no overlaps or no triples). □

Theorem 38.4 (When $H^1 = 0$ vs. $H^1 \neq 0$)**.** *Let* J *be the set of local judgments.*

1. *If all fibers are equivalent (*$J_i = \mathsf{True}$ *for all* i*) and the overlap structure is connected, then* $H^1 = 0$ *and the programs are globally equivalent.*

2. *If any fiber is inequivalent (*$J_k = \mathsf{False}$*), then* $H^1 \geq 1$ *and the programs are inequivalent, with the counterexample localized to fiber* k*.*

3. *If all checked fibers agree but some timed out (*$J_k = \bot$*), the result is inconclusive: coverage* $= |\{i : J_i = \mathsf{True}\}|/|I|$*. If coverage* $\geq 50\%$*, the algorithm reports equivalence with reduced confidence.*

Part IX

Practical Engineering

Chapter 39

End-to-End Pipeline Trace

This chapter traces *two* complete equivalence checks from raw Python source to final verdict, showing every intermediate step. The first example (equivalent programs) illustrates the happy path; the second (inequivalent programs with multiple parameters) illustrates obstruction detection.

39.1 Example 1: Recursive vs. Iterative Sum (EQUIVALENT)

39.1.1 The Input Pair

Listing 39.1: Program A: recursive sum

```
def f(n):
    if n <= 0: return 0
    return n + f(n - 1)
```

Listing 39.2: Program B: iterative sum

```
def g(n):
    total = 0
    for i in range(1, n + 1):
        total += i
    return total
```

39.1.2 Step 1: Parse to AST

Both sources are parsed by Python's `ast.parse`. Program A produces a `FunctionDef` with an `If` and a `Return` containing a `BinOp(Add)` with a recursive `Call`. Program B produces a `FunctionDef` with an `Assign`, a `For` over `range`, and a `Return`.

39.1.3 Step 2: Strategy 0 — Closed-Term Evaluation

The pipeline first checks whether the functions are zero-argument. Both have one parameter (n), so this strategy is skipped.

39.1.4 Step 3: Strategy 1 — Denotational OTerm Equivalence

The denotational engine normalizes both programs to canonical OTerms. If both reduce to the same canonical form, equivalence is proved without Z3. For this pair, the OTerm normalizer detects the sum pattern and may prove equivalence here. If it does, the pipeline returns immediately. Otherwise, it falls through to Z3.

39.1.5 Step 4: Create Theory and Compile

A fresh `Theory` T is instantiated: PyObj ADT with 7 constructors, binop/unop/truthy/fold RecFunctions, TypeTag EnumSort, tag RecFunction, empty shared function registry.

Compile Program A. Parameters: [`n`]. Create $p_0 = \mathsf{Const}(\mathtt{p0}, \mathsf{PyObj})$. Env: $\{\mathtt{n} \mapsto p_0\}$.

Self-recursion detected (`f` appears in body). Nat-eliminator extractor fires:

- Base case: `if n <= 0: return 0`. Threshold = 0, base value = $\mathsf{IntObj}(0)$.
- Recursive case: `return n + f(n-1)`. Operator = ADD, parameter on left, recursive call ($f(n-1)$) on right.

Result: $[\![f]\!] = \mathsf{fold}(\mathsf{ADD}, 0, \mathsf{ival}(p_0) + 1, \mathsf{IntObj}(0))$.

Sections: $\mathcal{F}_f = [(\top, \mathsf{fold}(\mathsf{ADD}, 0, \mathsf{ival}(p_0) + 1, \mathsf{IntObj}(0)))]$.

Compile Program B. `total = 0`: env $\leftarrow \{\mathtt{n} \mapsto p_0, \mathtt{total} \mapsto \mathsf{IntObj}(0)\}$.

`for i in range(1, n+1)`: range extraction gives start = 1, stop = $\mathsf{ival}(p_0) + 1$. The loop body `total += i` is a single `AugAssign(Add)` with target `total` and value `i`. This matches the canonical fold pattern. Result: $\mathsf{env}[\mathtt{total}] \leftarrow \mathsf{fold}(\mathsf{ADD}, 1, \mathsf{ival}(p_0) + 1, \mathsf{IntObj}(0))$.

`return total`: section $(\top, \mathsf{fold}(\mathsf{ADD}, 1, \mathsf{ival}(p_0) + 1, \mathsf{IntObj}(0)))$.

Note: A starts fold at 0, B starts at 1. But $\mathsf{fold}(\mathsf{ADD}, 0, n{+}1, \mathsf{IntObj}(0))$ includes $\mathsf{binop}(\mathsf{ADD}, \mathsf{IntObj}(0), \ldots) = \ldots$ (adding 0 is identity by R3). So both compute $0 + 1 + 2 + \ldots + n$.

39.1.6 Step 5: Unify Parameters and Structural Check

Both have 1 parameter. p_0 is shared (same Z3 Const), so no substitution needed. The terms are *not* structurally equal (different fold start indices), so the structural check does not short-circuit.

39.1.7 Step 6: Duck Type Inference

Analyze both programs for operations on `n`:

- A: comparison (`n <= 0` $\rightarrow$ le), subtraction (`n - 1` $\rightarrow$ sub), addition (`n + f(n-1)` $\rightarrow$ add).
- B: `range(1, n+1)` $\rightarrow$ call_range, addition (`n + 1` $\rightarrow$ add).

Union: $\{\mathsf{le}, \mathsf{sub}, \mathsf{add}, \mathsf{call_range}\}$. The set $\{\mathsf{sub}, \mathsf{call_range}\} \cap \text{NUMERIC_OPS} \neq \emptyset$, so duck type = int, fibers = $\{\mathsf{TInt}\}$.

39.1.8 Step 7: Build Site Category

Single parameter, single fiber. Site category: $\{U_{(\mathsf{int})}\}$, no overlaps.

39.1.9 Step 8: Local Equivalence Check

Solver created. Semantic axioms loaded. Tag constraint: $\mathsf{tag}(p_0) = \mathsf{TInt}$, $\mathsf{tag}(p_0) \neq \mathsf{TBot}$.

One section pair: $(\top, t_f)$ vs. $(\top, t_g)$. Guard overlap: $\top \wedge \top = \top$.

Z3 query: $\mathsf{fold}(\mathsf{ADD}, 0, \mathsf{ival}(p_0) + 1, \mathsf{IntObj}(0)) \neq \mathsf{fold}(\mathsf{ADD}, 1, \mathsf{ival}(p_0) + 1, \mathsf{IntObj}(0))$?

Z3 unfolds the fold RecFunction: the first fold includes $i = 0$ (adding $\mathsf{IntObj}(0)$, which is identity by R3). Both produce the same sum.

Result: **UNSAT**. Both terms are fully interpreted (uninterp_depth = 0), so the proof is trusted. $\mathsf{LocalJudgment}((\mathtt{int},), \mathsf{True})$.

39.1.10 Step 9: Čech Cohomology

One fiber, no overlaps. $C^0 = (1)$, $C^1 = \emptyset$, $H^1 = 0$ trivially.

39.1.11 Step 10: Verdict

$H^1 = 0$, $H^0 = 1$, confidence $= 1.0$.

EQUIVALENT: $f \equiv g$. The cubical path is: $[\![f]\!] \xrightarrow{\mathsf{D1}} \mathsf{fold}(\mathsf{ADD}, \ldots) = [\![g]\!]$.

39.2 Example 2: Sorting with Different Tie-Breaking (INEQUIVALENT)

39.2.1 The Input Pair

Listing 39.3: Program C: sort by value

```
def f(pairs):
    return sorted(pairs, key=lambda p: p[1])
```

Listing 39.4: Program D: sort by value, then key

```
def g(pairs):
    return sorted(pairs, key=lambda p: (p[1], p[0]))
```

39.2.2 Compilation

Both programs have one parameter: `pairs`.

Program C compiles to: $\mathsf{py_sorted_key}(p_0, \mathsf{fresh}(\texttt{lambda}))$. The lambda produces a fresh constant (lambdas cannot be inlined).

Program D compiles to: $\mathsf{py_sorted_key}(p_0, \mathsf{fresh}(\texttt{lambda}))$. A *different* fresh constant, because the lambda AST differs.

39.2.3 Duck Type Inference

Operations on `pairs`: $\{\mathsf{call_sorted}\}$. This matches COLLECTION_OPS, so duck type = $\mathsf{collection}$, fibers $= \{\mathsf{TPair}, \mathsf{TRef}, \mathsf{TStr}\}$.

39.2.4 Local Checks

Three fibers. On each fiber, the Z3 query asks: $\mathsf{py_sorted_key}(p_0, \lambda_C) \neq \mathsf{py_sorted_key}(p_0, \lambda_D)$?

Since $\lambda_C \neq \lambda_D$ (different fresh constants), Z3 can satisfy this by choosing an interpretation where the two sort keys produce different orderings.

Result: SAT on the TPair fiber. The counterexample: a list of pairs where elements have equal second components but different first components. $\mathsf{LocalJudgment}((\texttt{pair},), \mathsf{False}, \text{“}\texttt{obstruction on [TPair]}\text{”})$.

39.2.5 Čech Result

Short-circuit: inequivalent fiber found. $\mathsf{CechResult}(\mathsf{h0} = 0, \mathsf{h1_rank} = 1, \mathsf{obstructions} = [(\texttt{pair},)])$.

Verdict: **INEQUIVALENT**. The obstruction is localized to the TPair fiber — exactly the fiber where list-of-pairs tie-breaking matters.

Chapter 40

Bridging the Intensional–Extensional Gap

The preceding chapters built a powerful *intensional* equivalence engine: OTerm normalization reduces programs to canonical forms, the HIT structural prover applies path constructors by structural decomposition, and bidirectional BFS searches for multi-step rewrite paths using 16 registered axiom functions. The Z3 per-fiber checker then provides a separate layer of verification via Čech cohomology on the type fibration.

This chapter examines *where this pipeline reaches its limits*, *why* those limits arise from a fundamental gap in type theory, and *how CCTT provides a principled programme for extending the pipeline* by enriching the path space, strengthening specification identification, and improving the interaction between denotational and cohomological layers. The chapter is forward-looking: it identifies the theoretical mechanisms that motivate each next improvement to the implementation.

40.1 The Current Pipeline as a Typed Function Chain

The implementation orchestrates four strategy layers, each with clear CCTT semantics. We formalize the full pipeline as a chain of typed functions with a soundness invariant.

Definition 40.1 (Pipeline Type)**.** Define the type of pipeline verdicts:

$$\mathsf{Verdict} := \mathsf{EQ} \mid \mathsf{NEQ} \mid \mathsf{UNKNOWN}(c) \qquad c \in [0, 1]$$

and the pipeline as a composition of partial decision procedures:

$$\Pi := \pi_4 \circ \pi_3 \circ \pi_2 \circ \pi_1 \circ \pi_0 : (\mathsf{Src} \times \mathsf{Src}) \to \mathsf{Verdict}$$

where each π_k is a function $\mathsf{Verdict}_? \to \mathsf{Verdict}_?$ that either refines the verdict or passes it through unchanged, and $\mathsf{Verdict}_?$ extends $\mathsf{Verdict}$ with an explicit $\perp$ ("no verdict yet").

The layers are:

π_0: **Closed-term evaluation** (zero-argument functions). A function with no parameters has a singleton domain; its denotation IS its unique value. Evaluating it computes a β-normal form — not a sample. Functions with only variadic parameters (`*args`) are evaluated at zero arguments as a *witness*: trusted only for NEQ (a single disagreement suffices) but not for EQ (agreement on one input is insufficient).

π_1: **Denotational OTerm equivalence** (the primary weapon). Both functions are compiled to OTerms via `compile_to_oterm`, passed through the 7-phase normalizer (§40.2), and compared. Four sub-strategies operate in sequence:

(a) *Exact canonical form match*: after normalization, the string representations are identical. Five gating conditions must hold: (i) canonical form length ≥ 20 or constant value, (ii) no `OUnknown` nodes, (iii) all free variables are parameters, (iv) at least one parameter referenced, (v) operation-name sets agree.

(b) *Flexible matching*: canonical forms differ only in OFix keys (hash-based loop body identifiers) but are otherwise structurally identical under `_flexible_match`.

(c) *HIT path induction + BFS path search* (§40.3): the elimination principle for PyComp. Structural decomposition with congruence rules at each OTerm constructor, cross-type path constructors (D1–D24), and bidirectional BFS with 16 axiom functions. Fiber-aware via duck-type context.

(d) *Provable NEQ detection* (§40.6): structural impossibilities (different effect signatures, incompatible literal values, non-commutative argument orders on fiber-restricted operations).

π_2: **Z3 structural equality**: the Z3 presheaf sections (guard–term pairs) are compared syntactically, but only trusted when *fully interpreted* (no uninterpreted function symbols), as checked by `_uninterp_depth`.

π_3: **Z3 per-fiber checking + Čech H^1**: duck-type inference determines the site category, the system checks equivalence on each fiber with bounded timeout, and Čech cohomology assembles the local judgments into a global verdict. $H^1 = 0$ implies EQ; $H^1 \neq 0$ implies NEQ.

Theorem 40.2 (Pipeline Soundness — Informal)**.** *The pipeline* Π *satisfies: if* $\Pi(f, g) = \mathsf{EQ}$ *then* $\forall x.\ f(x) = g(x)$*; if* $\Pi(f, g) = \mathsf{NEQ}$ *then* $\exists x.\ f(x) \neq g(x)$*. A full proof appears in §40.12.*

The key architectural invariant: each π_k is *short-circuiting*: if a previous layer has already produced EQ or NEQ, the remaining layers are skipped. The ordering is calibrated so cheaper, more precise layers run first.

This pipeline currently proves 7/50 equivalences and 48/50 non-equivalences on the hard benchmark suite.

40.2 The 7-Phase Normalizer

The normalizer is the engine behind sub-strategy (a) of π_1. It applies seven phases in a fixed-point loop: the entire sequence is iterated until the canonical form stabilizes (up to 8 iterations).

Definition 40.3 (Normalizer)**.** $\mathcal{N} := (\varphi_7 \circ \varphi_6 \circ \varphi_5 \circ \varphi_4 \circ \varphi_3 \circ \varphi_2 \circ \varphi_1)^{\leq 8}$, where $t^{\leq k}$ means "apply t until fixpoint, at most k iterations."

The seven phases are:

Phase	Name	Key Rules
φ_1	β-reduction	Constant folding, $\mathsf{case}(\top, t, f) \to t$, $\mathsf{case}(c, x, x) \to x$, $\mathsf{fold}(op, i, []) \to i$, identity-map elimination
φ_2	Ring/lattice	Additive/multiplicative identities, double negation, sorted idempotence, comparison negation
φ_3	Fold canon.	$\mathsf{sum}(x) \to \mathsf{fold}(\mathsf{iadd}, 0, x)$, $\mathsf{any}(x) \to \mathsf{fold}(\vee, \bot, x)$, $\mathsf{join} \to \mathsf{fold}(\mathsf{str_concat})$
φ_4	HOF fusion	$\mathsf{sorted}(\mathsf{list}(x)) \to \mathsf{sorted}(x)$, $\mathsf{reversed}(\mathsf{sorted}(x)) \to \mathsf{sorted_rev}(x)$, $\mathsf{abs}(x) \to \mathsf{case}(\mathsf{lt}(x, 0), -x, x)$, $\mathsf{max}(a, b) \to \mathsf{case}(\mathsf{gte}(a, b), a, b)$
φ_5	Symbol unify	D19: $\mathsf{heapq.nsmallest} \to \mathsf{sorted_slice}$; D13: $\mathsf{Counter} \to \mathsf{counter}$; D14: $\mathsf{OrderedDict} \to \mathsf{dict}$; local variable method normalization
φ_6	Quotient	$Q[\mathsf{perm}](\mathsf{sorted}(x)) \to \mathsf{sorted}(x)$, $Q[Q[\cdot]] \to Q[\cdot]$ (idempotence), $\mathsf{sorted}(Q[\mathsf{perm}](x)) \to \mathsf{sorted}(x)$
φ_7	Piecewise	Flatten nested case chains, infer complement guards via comparison algebra, sort pieces by $(\mathsf{value.canon}(), \mathsf{guard.canon}())$

40.2.1 Confluence and Termination

Proposition 40.4 (Termination)**.** *The normalizer terminates in at most 8 iterations. Each phase is a structural recursion on the OTerm tree (hence terminates on each invocation), and the outer loop is bounded by the explicit iteration cap.*

Proof. Each phase traverses the OTerm by structural recursion (depth-first, polynomial in term size). The outer loop checks $\mathsf{canon}(t_{prev}) = \mathsf{canon}(t_{curr})$ and exits early when the fixpoint is reached. The hard bound of 8 iterations ensures termination even if no fixpoint is reached. □

Proposition 40.5 (Weak Confluence)**.** *Within each phase, the rewrite rules are* non-overlapping *at the root: each rule matches a distinct OTerm constructor or operation name. Consequently, each phase is confluent on its own inputs.*

The cross-phase interaction is more subtle: φ_3 may create fold nodes that φ_4 then fuses, which φ_6 then quotients. Confluence of the composed system $\varphi_7 \circ \cdots \circ \varphi_1$ is not proved in general but is empirically observed: all benchmark pairs converge within 3 iterations.

Proposition 40.6 (Soundness of Each Phase)**.** *For each phase φ_k: if $\varphi_k(t) = t'$ then $[\![t]\!] = [\![t']\!]$ (extensional equality of denotations).*

Proof sketch. Phase by phase: φ_1 applies standard β-reduction and constant folding (sound by substitution); φ_2 applies ring identities ($x + 0 = x$, $x \cdot 1 = x$, etc.) that are valid in $\mathbb{Z}$, $\mathbb{R}$, and the Boolean algebra; φ_3 replaces builtins by their fold definitions (sound by the semantics of `sum`, `any`, etc.); φ_4 applies HOF fusion laws ($\mathsf{sorted} \circ \mathsf{list} = \mathsf{sorted}$, etc.) that hold in Python; φ_5 unifies module-qualified names to canonical forms (same function, different import path); φ_6 exploits the quotient type: $Q[\mathsf{perm}]$ marks nondeterministic ordering, and sorted eliminates it; φ_7 reorders case-guards, valid because piecewise functions over a partition are determined by their values on each piece. □

40.3 The HIT Prover as a Term-Rewriting System

The HIT prover (`hit_path_equiv`) implements the elimination principle for PyComp (Theorem 4.5). We now formalize it as a *conditional term-rewriting system* (CTRS) and establish its rewrite-theoretic properties.

40.3.1 Congruence Rules

At each OTerm constructor, the prover applies a *congruence rule*: decompose the top-level constructor and recurse into children. These rules form the structural core of the prover:

$$\textsc{Refl}: \quad \frac{a.\mathsf{canon}() = b.\mathsf{canon}()}{a =_{\mathsf{HIT}} b}$$

$$\textsc{OOp-Cong}: \quad \frac{a_1 =_{\mathsf{HIT}} b_1 \quad \cdots \quad a_n =_{\mathsf{HIT}} b_n}{\mathsf{OOp}(f, a_1, \ldots, a_n) =_{\mathsf{HIT}} \mathsf{OOp}(f, b_1, \ldots, b_n)}$$

$$\textsc{OCase-Cong}: \quad \frac{g =_{\mathsf{HIT}} g' \quad t =_{\mathsf{HIT}} t' \quad f =_{\mathsf{HIT}} f'}{\mathsf{case}(g, t, f) =_{\mathsf{HIT}} \mathsf{case}(g', t', f')}$$

$$\textsc{OCase-Flip}: \quad \frac{g' = \neg g \quad t =_{\mathsf{HIT}} f' \quad f =_{\mathsf{HIT}} t'}{\mathsf{case}(g, t, f) =_{\mathsf{HIT}} \mathsf{case}(g', t', f')}$$

$$\textsc{OFold-Cong}: \quad \frac{op = op' \quad i =_{\mathsf{HIT}} i' \quad c =_{\mathsf{HIT}} c'}{\mathsf{fold}(op, i, c) =_{\mathsf{HIT}} \mathsf{fold}(op', i', c')}$$

$$\textsc{OLam-Cong}: \quad \frac{|\vec{x}| = |\vec{y}| \quad body_a[\vec{x}/\vec{y}] =_{\mathsf{HIT}} body_b}{\lambda\vec{x}.body_a =_{\mathsf{HIT}} \lambda\vec{y}.body_b}$$

$$\textsc{OMap-Cong}: \quad \frac{t =_{\mathsf{HIT}} t' \quad c =_{\mathsf{HIT}} c' \quad p =_{\mathsf{HIT}} p'}{\mathsf{map}(t, c, p) =_{\mathsf{HIT}} \mathsf{map}(t', c', p')}$$

$$\textsc{Comm}: \quad \frac{op \in \mathsf{Comm}_\tau \quad a =_{\mathsf{HIT}} b' \quad b =_{\mathsf{HIT}} a'}{\mathsf{OOp}(op, a, b) =_{\mathsf{HIT}} \mathsf{OOp}(op, a', b')}$$

The Comm rule is *fiber-restricted*: the set Comm_τ of commutative operations depends on the duck-type fiber τ. On numeric fibers, $\mathsf{add} \in \mathsf{Comm}_\tau$; on string/list fibers, it is not (concatenation is non-commutative). This is sheaf descent (§2.6) at the rewriting level.

40.3.2 Cross-Type Path Constructors

Beyond congruence, the prover applies cross-type rules that relate OTerms with different top-level constructors:

- **D1**: $\mathsf{OFix} \leftrightarrow \mathsf{OFold}$ (recursion $\leftrightarrow$ iteration) — via spec matching on the fix-point body.
- **D8**: Case flattening — $\mathsf{case}(t_1, A, \mathsf{case}(t_2, B, C)) \leftrightarrow \mathsf{case}(t_2, B, \mathsf{case}(t_1, A, C))$ when guards are disjoint.
- **D20**: Spec identification (Yoneda) — both sides reduce to the same $\mathsf{OAbstract}(spec, \vec{x})$.
- **D22**: $\mathsf{OCatch} \leftrightarrow \mathsf{OCase}$ (try/except $\leftrightarrow$ conditional) — matching body and default.
- **De Morgan**: $\neg(a \wedge b) \leftrightarrow (\neg a) \vee (\neg b)$, applied recursively.
- **Guard subsumption**: if G_{sub} is a disjunct of G, then in the else-branch of G, the inner $\mathsf{case}(G_{sub}, \ldots)$ collapses.
- **Comparison normalization**: $\mathsf{lt}(a, b) = \mathsf{gt}(b, a)$, $\neg\mathsf{lt}(a, b) = \mathsf{gte}(a, b)$, etc.

The prover also applies single-step axiom rewrites from the 16 registered axiom functions at depths < 8, checking whether any one-step rewrite closes the gap.

40.3.3 Memoization and Termination

The HIT prover uses a memoization table keyed by $(\mathsf{canon}(a), \mathsf{canon}(b))$. Before recursing, it marks the key as "in progress" to detect cycles (recursive OTerms produce cycles in the dependency graph). A depth bound of 20 ensures termination.

Proposition 40.7 (HIT Prover Termination)**.** *The HIT prover terminates on all inputs. The depth bound and memoization ensure each pair is visited at most once per depth level, giving worst-case $O(|t|^2 \cdot D)$ where $|t|$ is the OTerm size and $D = 20$ is the depth bound.*

40.4 The D20 Spec Identification Engine

The D20 axiom function (`_axiom_d20_spec_unify`) recognizes high-level specifications from OTerm structure and replaces complex computations with abstract specification terms. This is the Yoneda embedding: a program is characterized by the specification it satisfies.

40.4.1 Spec Identification as a Morphism

Definition 40.8 (Spec Identification Function)**.** The spec identification function is a partial map:

$$\iota : \mathsf{OTerm} \rightharpoonup \mathsf{SpecName} \times \mathsf{OTerm}^*$$

defined by structural pattern matching on the OTerm.

The current implementation recognizes five specification families:

Spec	Pattern	Result
factorial	$\mathsf{fold}(\mathsf{mul}, 1, \mathsf{range}(\ldots, n))$	$\mathsf{factorial}(n)$
range_sum	$\mathsf{fold}(\mathsf{add}, 0, \mathsf{range}(\ldots, n))$	$\mathsf{range_sum}(n)$
fib_like	$\mathsf{fix}[h](\ldots \mathsf{add} \ldots \mathsf{sub} \ldots)$	$\mathsf{fib_like}(\vec{v})$
sorted	$\mathsf{sorted}(x)$	$\mathsf{sorted}(x)$
binomial	$\mathsf{math.comb}(n, k)$	$\mathsf{binomial}(n, k)$

Proposition 40.9 (Spec Identification Soundness)**.** *If $\iota(t) = (S, \vec{x})$, then t computes the specification S on inputs $\vec{x}$. In particular, if $\iota(t_1) = (S, \vec{x})$ and $\iota(t_2) = (S, \vec{x})$ (same spec, same inputs), then $[\![t_1]\!] = [\![t_2]\!]$ by Theorem 24.2.*

Proof. Each pattern in ι is a sufficient condition for the corresponding specification. $\mathsf{fold}(\mathsf{mul}, 1, \mathsf{range}(1, n{+}1))$ computes $\prod_{i=1}^{n} i = n!$ by definition of fold over range. $\mathsf{fold}(\mathsf{add}, 0, \mathsf{range}(\ldots))$ computes $\sum_i i$ by the same argument. $\mathsf{sorted}(x)$ is the identity on the sorted spec. The Fibonacci pattern uses a heuristic (presence of `add` and `sub` in the fix body), which is sound when the fix body matches the Fibonacci recurrence but may produce false negatives on non-standard formulations. □

40.4.2 Formal Pattern Matching as Presheaf Morphism

The spec identification function ι can be understood as a natural transformation between presheaves. Define:

- $\mathcal{O}$ (the OTerm presheaf): assigns to each type fiber τ the set of OTerms whose free variables inhabit τ.
- $\mathcal{S}$ (the spec presheaf): assigns to each τ the set of specifications recognizable on τ.

Then $\iota : \mathcal{O} \Rightarrow \mathcal{S}$ is a natural transformation: for each restriction map $\rho_{\tau \to \tau'}$, the diagram

$$\begin{tikzcd} \mathcal{O}(\tau) \arrow[r, "\iota_\tau"] \arrow[d, "\rho"'] & \mathcal{S}(\tau) \arrow[d, "\rho"] \\ \mathcal{O}(\tau') \arrow[r, "\iota_{\tau'}"] & \mathcal{S}(\tau') \end{tikzcd}$$

commutes: restricting an OTerm to a sub-fiber and then identifying its spec gives the same result as identifying the spec and then restricting. This naturality ensures that spec identification is compatible with the sheaf structure of the type fibration.

40.5 BFS Path Search and the 16 Axiom Functions

When the HIT structural prover cannot close the gap directly, the system falls back to bidirectional BFS on the rewrite groupoid (§40.8). The BFS uses 16 registered axiom functions, each of which generates all one-step rewrites applicable to a given OTerm.

40.5.1 The 16 Registered Axiom Functions

The axiom functions are organized by the monograph's dichotomy groups. Each function has type $\mathsf{OTerm} \times \mathsf{FiberCtx} \to \mathsf{List}(\mathsf{OTerm} \times \mathsf{String})$, returning all applicable rewrites with their axiom name:

#	Name	Axiom
1	D1 fold/unfold	$\mathsf{fix} \leftrightarrow \mathsf{fold}$
2	D2 β	$(\lambda x.b)(a) \to b[x := a]$
3	D4 map fusion	$\mathsf{map}(f, \mathsf{map}(g, c)) \to \mathsf{map}(f \circ g, c)$
4	D5 fold universal	Hash-named fold $\to$ canonical op
5	D8 section merge	Guard complement collapse
6	D13 histogram	$\mathsf{Counter}(x) \leftrightarrow \mathsf{fold}(\mathsf{count}, \ldots, x)$
7	D16 memo/DP	Recurrence normalization
8	D17 associativity	Tree fold $\to$ linear fold
9	D19 sort-scan	$\mathsf{fold}(op, \mathsf{sorted}(x)) \to \mathsf{fold}(op, x)$ when op commutative
10	D20 spec unify	$t \to \mathsf{OAbstract}(spec, \vec{x})$
11	D22 effect elim	$\mathsf{catch}(b, d) \leftrightarrow \mathsf{case}(g, b, d)$
12	D24 η	$\lambda x.f(x) \to f$
13	Algebra	Commutativity, associativity, identity
14	Quotient	$Q[\mathsf{perm}]$ absorption, idempotence
15	Fold	Sum/prod expansion, range normalization
16	Case	Constant guard, identical branches, $\neg$-flip, subsumption

Each axiom function also operates under *congruence closure*: the function `_congruence_rewrites` applies every axiom at every sub-term position, constructing the new OTerm by replacing the rewritten sub-term and annotating the position (e.g., `D4_map_fusion@arg0`).

40.5.2 Axiom Soundness

Theorem 40.10 (Axiom Soundness)**.** *Each of the 16 axiom functions preserves semantic equivalence: if* $(t', ax) \in \mathsf{axiom}_k(t, ctx)$ *then* $[\![t]\!] = [\![t']\!]$.

Proof. We verify soundness per axiom:

1. **D1**: A fix-point with accumulation pattern computes the same function as the corresponding fold, by the universal property of folds (catamorphism).
2. **D2**: β-reduction is sound by the substitution lemma.
3. **D4**: Map fusion: $\mathsf{map}(f, \mathsf{map}(g, c)) = \mathsf{map}(f \circ g, c)$ by the functor law.
4. **D5**: Fold canonicalization replaces a hash-based fold key with a recognized canonical operation, preserving semantics.
5. **D8**: Guard complement collapse: $\mathsf{case}(g, A, \mathsf{case}(\neg g, B, C)) = \mathsf{case}(g, A, C)$ because $\neg g$ is false in the else-branch of g.
6. **D13**: $\mathsf{Counter}(x)$ is defined as $\mathsf{fold}(\mathsf{count_freq}, \mathsf{defaultdict}(0), x)$.
7. **D16**: Recurrence normalization alpha-normalizes the fix body, preserving the function computed.
8. **D17**: Tree fold = linear fold when the operation is associative (parenthesization doesn't matter).
9. **D19**: Sorting is irrelevant for commutative folds: $\mathsf{fold}(op, i, \mathsf{sorted}(x)) = \mathsf{fold}(op, i, x)$ when op is commutative and associative.
10. **D20**: Spec identification replaces a term with its abstract specification; soundness follows from Proposition 40.9.
11. **D22**: When the exception guard is decidable, $\mathsf{catch}(b, d) = \mathsf{case}(\mathsf{can_succeed}(b), b, d)$.
12. **D24**: η-contraction: $\lambda x.f(x) = f$ when $x \notin \mathrm{FV}(f)$.
13. **Algebra**: Standard algebraic identities (commutativity restricted to appropriate fibers, associativity, identity elements, involution).
14. **Quotient**: $Q[\mathsf{perm}](\mathsf{sorted}(x)) = \mathsf{sorted}(x)$ because sorting already fixes a canonical representative.
15. **Fold**: $\mathsf{sum}(x) = \mathsf{fold}(\mathsf{add}, 0, x)$ etc., by definition.
16. **Case**: Constant guard elimination and branch identity are immediate; $\neg$-flip swaps branches soundly.

□

40.5.3 BFS Structure

The bidirectional BFS maintains two frontiers: $F_0 = \{\mathsf{nf}_f\}$ and $B_0 = \{\mathsf{nf}_g\}$. At each depth d, both frontiers are expanded by applying all 16 axiom functions (with congruence closure) to all terms in the current-depth layer. Frontier pruning keeps each set below $\mathsf{max_frontier} = 200$ terms, scored by path length and structural similarity to the target.

A path is found when a canonical form appears in both frontiers. The maximum search depth is $\mathsf{max_depth} = 4$, giving a worst-case exploration of $O(\mathsf{max_depth} \cdot \mathsf{max_frontier} \cdot |\mathsf{Axioms}| \cdot |t|)$ term rewrites.

40.6 Provable NEQ Detection

The function `_detect_provable_neq` returns $\top$ only when the two OTerms are *provably* non-equivalent. Each detection rule is a certificate of non-equivalence.

Definition 40.11 (NEQ Certificate)**.** A *NEQ certificate* for OTerms a, b is a structural property $P(a, b)$ such that $P(a, b) \implies \exists x.\ [\![a]\!](x) \neq [\![b]\!](x)$.

The implementation checks the following certificates, organized by category:

Effect-fiber certificates. If a contains an effect operation (`effect_io`, `effect_write`, etc.) and b does not (or vice versa), they inhabit different fibers of the heap fibration: $H^1 \neq 0$ on the effect fiber.

Literal certificates. $\mathsf{OLit}(v_1)$ vs. $\mathsf{OLit}(v_2)$ with $v_1 \neq v_2$: constant functions returning different values. $\mathsf{OLit}(\mathsf{None})$ vs. any non-None term: the denotation $\perp$ differs from any productive computation.

Argument-order certificates. For non-commutative operations $\sigma \in \{\mathsf{sub}, \mathsf{floordiv}, \mathsf{mod}, \mathsf{pow}, \mathsf{concat}, \mathsf{merge}, .$ if $\sigma(a_1, a_2)$ vs. $\sigma(a_2, a_1)$ with $a_1 \neq a_2$, then the results differ. For `add`/`iadd`, commutativity depends on the duck-type fiber: on numeric fibers, it is commutative (no certificate); on string/list fibers, it is non-commutative (certificate issued).

Comparison-operator certificates. Same arguments, different comparison: $\mathsf{eq}(a, b)$ vs. $\mathsf{is}(a, b)$ differ because `==` and `is` have different semantics in Python (value vs. identity).

Structural certificates.

- Different sequence lengths: $|\mathsf{OSeq}(a)| \neq |\mathsf{OSeq}(b)|$.
- Different fold operators: $\mathsf{fold}(op_1, \ldots) \neq \mathsf{fold}(op_2, \ldots)$ when $op_1 \neq op_2$.
- Cross-type at top level (outside case branches): OSeq vs. OOp, OOp vs. OMap, $\mathsf{OQuotient}$ vs. OOp, etc.

Theorem 40.12 (NEQ Detection Soundness)**.** *Each NEQ certificate $P(a, b)$ is a valid certificate of non-equivalence: $P(a, b) \implies [\![a]\!] \neq [\![b]\!]$.*

Proof. Effect-fiber: a program with IO effects returns $(\mathsf{None}, \textit{side-effect})$ while a pure program returns $(\textit{value}, \varnothing)$; these differ on the heap component. Literal: different constants trivially differ. Argument-order: non-commutativity provides a witness (e.g., $3-5 \neq 5-3$). Comparison-operator: eq uses `__eq__` while is uses pointer identity; `[] == []` is True but `[] is []` is False. Structural: different-length sequences cannot be equal; different fold operators compute different functions (e.g., $\sum \neq \prod$). Cross-type at top level: an OSeq (tuple/list literal) and an OOp (function application) produce different types. □

An important conservatism: cross-type certificates are *suppressed inside case branches* (`in_case_branch = True`). Inside a conditional branch, different structures can compute the same value under the guard's domain constraint. For example, $Q[\mathsf{perm}](\mathsf{set}(x))$ and $\mathsf{sorted}(x)$ could agree when $|\mathsf{set}(x)| \leq 1$.

40.7 The Intensional–Extensional Gap

40.7.1 Where the Pipeline Fails

For 43 of the 50 equivalent benchmark pairs, the pipeline returns UNKNOWN or a spurious H^1 obstruction. These failures cluster into three categories:

1. **Genuinely different algorithms** (8 pairs): Graham scan vs. monotone chain (convex hull), Tarjan vs. Kosaraju (SCC), matrix exponentiation vs. fast doubling (Fibonacci), multiplicative formula vs. Pascal's triangle ($\binom{n}{k}$), Wagner–Fischer vs. Hirschberg (edit distance), recursive descent vs. shunting-yard (expression evaluation), stack-based vs. recursive (bracket matching), trial division vs. SPF sieve (prime factorization). No structural axiom connects them; they live in disconnected components of the rewrite groupoid.

2. **Compilation lossy-ness** (∼22 pairs): the OTerm compiler loses parameter references, produces `OUnknown` nodes, or generates canonical forms too short to trust.

3. **Timeout / memory** (∼13 pairs): the Z3 fiber checker runs out of time or memory before resolving all fibers.

Category 1 is the *fundamental* limitation. Categories 2 and 3 are engineering problems.

40.7.2 Formal Gap Statement

Theorem 40.13 (The Gap). *There exist programs $f, g : A \to B$ such that:*

1. $\prod_{x:A} \mathsf{Path}_B(f(x), g(x))$ *is inhabited (extensionally equal),*

2. $\mathsf{Path}_{\mathsf{PyComp}}(f, g)$ *is empty in the current HIT (no finite chain of D1–D24 constructors connects them).*

Proof. By Rice's theorem: semantic equivalence of programs is undecidable. Any decidable set of syntactic axioms can only capture a proper subset of all valid equivalences. Concretely: Graham scan and monotone chain compute the same convex hull, but no structural axiom relates polar-angle sorting to lexicographic sorting, nor a single-pass stack to a two-pass upper/lower construction. □

40.7.3 What the System Cannot Do

We now formalize the limitations precisely. Define the *provability horizon*:

Definition 40.14 (Provability Horizon). The provability horizon of the pipeline is the set:

$$\mathcal{H} := \{(f, g) : f \equiv g \text{ and } \Pi(f, g) = \mathsf{EQ}\}$$

The *gap set* is:

$$\mathcal{G} := \{(f, g) : f \equiv g \text{ and } \Pi(f, g) \neq \mathsf{EQ}\}$$

The 8 "genuinely different algorithm" pairs lie in $\mathcal{G}$ for a precise reason: each pair consists of two algorithms that compute the same mathematical function via fundamentally different decompositions of the problem. No finite combination of the syntactic axioms D1–D24 connects the two decompositions, because the axioms operate on local OTerm structure while the equivalence is a global property of the algorithm's correctness proof.

Proposition 40.15 (Gap Characterization). *A pair (f, g) lies in $\mathcal{G}$ iff the OTerm normal forms $\mathcal{N}([\![f]\!])$ and $\mathcal{N}([\![g]\!])$ lie in different connected components of the rewrite groupoid $\mathcal{G}_{\mathsf{PyComp}}$ AND the spec identification function ι fails on at least one of them.*

40.8 Enriching the Path Space

40.8.1 The Rewrite Groupoid

Definition 40.16 (Rewrite Groupoid). $\mathcal{G}_{\mathsf{PyComp}}$ has:

- **Objects**: OTerms in normal form (after $\mathcal{N}$).
- **Morphisms**: finite composites of path constructors and their inverses:

$$\mathrm{Hom}_{\mathcal{G}}(a,b) = \left\{p_1^{\epsilon_1} \cdot p_2^{\epsilon_2} \cdots p_k^{\epsilon_k} : p_i \in \text{Axioms},\ \epsilon_i \in \{+1,-1\}\right\} /\!\sim$$

  where $\sim$ is the groupoid relations ($p \cdot p^{-1} = \mathrm{id}$, associativity).

Proposition 40.17 (Fundamental Obstruction). *Two OTerms are in different connected components of $\mathcal{G}_{\mathsf{PyComp}}$ iff no finite chain of axioms in the equational theory E relates them. The π_0 invariant $\pi_0(\mathcal{G}_{\mathsf{PyComp}})$ classifies these obstructions.*

The strategy for improving the prover is to *reduce* $|\pi_0(\mathcal{G})|$ by adding new axioms — new path constructors that connect previously disconnected components.

40.8.2 Three Mechanisms for New Paths

Mechanism 1: Higher-level normalization. Every new normalization rule is a new path constructor. If we add $t \rightsquigarrow t'$ to the normalizer, we implicitly add $\mathsf{norm} : \mathsf{Path}(t,t')$ to the HIT. Currently missing normalizations include:

- **Accumulator extraction**: recognizing that a tail-recursive function with an accumulator computes the same value as a non-tail version.
- **Loop interchange**: two nested folds over independent ranges can be reordered (π_0 merge for DP table fills).
- **Divide-and-conquer normalization**: $f(a \cdot b) = f(a) \oplus f(b)$ for homomorphisms.

Mechanism 2: Specification identification (D20 Yoneda). Each new spec pattern $\pi \mapsto S$ creates a new family of paths:

$$\forall a, b.\ \iota(a) = (S, \vec{x}) \ \wedge\ \iota(b) = (S, \vec{x}) \implies \mathsf{Path}_{\mathsf{PyComp}}(a,b)$$

Adding recognition patterns for GCD, LCM, convex hull, edit distance, topological sort, and shortest path would directly connect the 8 "genuinely different algorithm" pairs.

Mechanism 3: Transport along type coercion paths. Univalence provides paths between equivalent types. If two programs operate on isomorphic types, transport along the type equivalence path can normalize them to a common representation.

40.9 Strengthening the D20 Spec Engine

40.9.1 The Spec Library as a Presheaf

Definition 40.18 (Spec Presheaf). $\mathcal{S} : \mathcal{C}^{\mathrm{op}} \to \mathsf{Set}$ assigns to each type fiber τ the set of specifications recognizable on τ. Restriction maps narrow specifications to sub-fibers.

The sheaf condition on $\mathcal{S}$ tells us when local spec matches on individual fibers are sufficient for a global spec match.

40.9.2 Observable Properties as Spec Discovery

Beyond direct pattern matching, specifications can be *inferred* from observable properties:

1. **Output type agreement**: $\forall \vec{p}.\ \mathsf{tag}(t_f) = \mathsf{tag}(t_g)$.
2. **Base-case agreement**: for each $v \in \{0, 1, -1\}$, $\mathsf{simplify}(t_f[\vec{p} \mapsto v]) = \mathsf{simplify}(t_g[\vec{p} \mapsto v])$.
3. **Recurrence structure**: alpha-equivalent RecFunction bodies.
4. **Idempotence**: $t_f(t_f(x)) = t_f(x)$.
5. **Monotonicity**: $x < y \implies t_f(x) \leq t_f(y)$.

Theorem 40.19 (Observable Properties as Abstract Spec)**.** *If two programs agree on output type, all base cases, and their RecFunction recurrence equations are alpha-equivalent, then they satisfy the same deterministic specification. By Theorem 24.2, they are extensionally equal.*

40.10 Improving the Denotational–Cohomological Interaction

The current pipeline runs the denotational layer (OTerm) and the cohomological layer (Z3 + Čech) *sequentially*. This leaves power on the table.

40.10.1 Shared Symbol Congruence

Theorem 40.20 (Shared Symbol Congruence)**.** *Let σ be a shared function symbol. If $t_1|_\tau = t_2|_\tau$ for every type fiber τ, then $\sigma(t_1) = \sigma(t_2)$.*

This principle enables *descent*: decompose a global equality into local equalities on subexpressions.

40.10.2 Feeding Denotational Evidence into Fibers

When the denotational layer discovers partial information, it can be *injected* as lemmas into the Z3 context.

Definition 40.21 (Spec-Constrained Fiber)**.** Given a Z3 term t over parameters $\vec{p}$, a fiber assignment τ, and discovered properties P, the *spec-constrained fiber check* queries:

$$\mathsf{solver.check}\left(t_f|_\tau \neq t_g|_\tau \ \wedge \bigwedge_{p \in P} [\![p]\!] \right) \stackrel{?}{=} \mathsf{unsat}$$

40.11 The Graded Verdict

Definition 40.22 (Graded Verdict)**.** A graded verdict is a tuple (v, c) where $v \in \{\mathsf{EQ}, \mathsf{NEQ}, \mathsf{UNKNOWN}\}$ and $c = h_0/\max(|\mathsf{Fibers}|, 1) \in [0, 1]$. Full confidence ($c = 1$) is a global section — a proof of equivalence.

40.11.1 H^1 Suppression and the Uninterpreted-Depth Filter

When Z3 compilation produces uninterpreted functions, the fiber checker may find spurious H^1 obstructions. The implementation handles this via `_uninterp_depth`: Z3 structural equality is only trusted when all terms are *fully interpreted* (depth 0). Additionally, SAT witnesses with high uninterpreted depth ($d_1 + d_2 \geq 4$, $\min(d_1, d_2) \geq 1$) are treated as inconclusive rather than genuine counterexamples.

Definition 40.23 (Spurious H^1)**.** An H^1 obstruction is *spurious* if it arises from a fiber where the Z3 terms for f and g differ only in uninterpreted function symbols.

The principled response is *refinement*: each builtin transitioning from uninterpreted to interpreted eliminates a class of spurious obstructions.

40.12 Soundness and Completeness

Theorem 40.24 (Pipeline Soundness)**.** *The full pipeline* Π *is* sound*: it never returns* EQ *for genuinely non-equivalent programs or* NEQ *for genuinely equivalent programs.*

Proof. Each layer π_k returns a positive verdict only via a sound proof:

- π_0 (closed-term evaluation): the denotation of a zero-argument function is unique; equal values imply equal programs. Variadic-only calls are trusted only for NEQ (witness-based).
- π_1(a) (exact match): identical canonical forms after sound normalization rules (Proposition 40.6) imply identical semantics. The five gating conditions suppress potential true positives but never admit false positives.
- π_1(b) (flexible match): OFix keys are hash-based identifiers of loop bodies; differing keys do not affect the computation when the structural match succeeds on all other nodes.
- π_1(c) (HIT + BFS): each path step is a valid axiom (Theorem 40.10); composition of valid axioms preserves equivalence.
- π_1(d) (provable NEQ): each certificate is sound (Theorem 40.12).
- π_2 (Z3 structural): trusted only when `_uninterp_depth` $= 0$ for all terms.
- π_3 (Z3 per-fiber): UNSAT on a fiber τ is a proof that $f|_\tau = g|_\tau$. By the stalk criterion (Theorem 9.4), agreement on all fibers implies global equality. NEQ requires a concrete SAT witness or structural impossibility.

□

Theorem 40.25 (Inherent Incompleteness)**.** *The pipeline* Π *is* incomplete*: there exist equivalent programs for which* $\Pi(f, g) = \mathsf{UNKNOWN}$*. This is unavoidable by Rice's theorem.*

The graded verdict provides a principled measure of "how incomplete" the answer is: the confidence c quantifies the fraction of the type fibration verified.

40.13 The Programme for Improvement

40.13.1 Axis 1: Richer Normalization (More Paths)

Each new normalization rule is a new path constructor. Key additions:

1. **Compiler completeness**: handle all Python constructs appearing in the benchmark (Category 2 failures).
2. **Accumulator normalization**: $\mathsf{fix}[f](\mathsf{acc}, n) \rightsquigarrow \mathsf{fold}(f_{\mathsf{step}}, \mathsf{acc}_0, \mathsf{range})$.
3. **Guard subsumption strengthening**: if $G_1 \implies G_2$, simplify $\mathsf{case}(G_1, A, \mathsf{case}(G_2, B, C))$ to $\mathsf{case}(G_1, A, C)$ in the G_1-false branch.

40.13.2 Axis 2: Richer Specifications (More Yoneda)

The path forward is *compositional* spec recognition mirroring the operadic structure of Chapter 14:

- Fold over range with multiplication $\to$ product-of-range
- Product-of-range from 1 to n $\to$ factorial
- Product-of-range from $(n-k+1)$ to n, divided by factorial(k) $\to$ binomial

The recognizer becomes a homomorphism from the OTerm operad to the spec operad.

40.13.3 Axis 3: Z3 Compilation Fidelity (Fewer Spurious H^1)

Key builtins to formalize as Z3 RecFunctions:

- `sorted`: axiomatize as a permutation that is non-decreasing.
- `set operations`: axiomatize in terms of membership.
- `string operations`: slicing, joining, splitting.

40.13.4 Concrete Improvement Theorems

We state specific theorems that, if proved and implemented, would close identified gaps:

Conjecture 40.26 (Accumulator Extraction). *For any tail-recursive function f with linear accumulator α, there exists a fold $\mathsf{fold}(step, \alpha_0, R)$ such that $f \equiv \mathsf{fold}(step, \alpha_0, R)$. Implementing this as a normalizer rule would connect ∼5 additional benchmark pairs.*

Conjecture 40.27 (Compositional Spec Recognition). *The spec identification function ι can be extended to a homomorphism $\iota^* : \mathcal{O}_{\mathsf{operad}} \to \mathcal{S}_{\mathsf{operad}}$ such that ι^* recognizes composite specifications (e.g., binomial = ratio of factorials). This would add ∼3 spec families (GCD, LCM, binomial via multiplicative formula).*

Conjecture 40.28 (Z3 RecFunction Completeness). *Axiomatizing `sorted`, `set`, and `str` operations as Z3 RecFunctions would reduce `_uninterp_depth` to 0 for ∼15 additional benchmark pairs, converting Z3 timeouts to definitive verdicts.*

40.14 Summary: What the Theory Tells Us to Build Next

Theory	Implementation Target	Expected Impact
New path constructors	7-phase normalizer rules	Category 2 failures
Yoneda (D20)	Spec library patterns	Category 1 failures
Transport	Type coercion paths	Cross-type pairs
Sheaf descent	Fiber-aware spec matching	Combined evidence
Galois connection	Spec-constrained Z3	Z3 timeout pairs
RecFunction analysis	Z3 recurrence matching	Recursive pairs
Operadic composition	Compositional spec recognition	Complex specs

Each row is a *theorem* in CCTT that motivates a specific engineering task. The theory does not just justify the current implementation — it tells us precisely where to push next and why each extension is sound.

Chapter 41

OOP: Classes, Inheritance, and Method Resolution

41.1 Class Compilation

Python classes compile to Ref objects with shared method symbols:

Listing 41.1: Class compilation

```
# class MyClass:
#     def __init__(self, x):
#         self.x = x
#     def compute(self):
#         return self.x * 2

# Compilation:
# MyClass(v)  => Ref(fresh_addr)
# obj.x       => py_attr_x(obj)
# obj.compute() => py_meth_compute(obj)
#               => inlined: py_attr_x(obj) * 2

def compile_class(cls_node, env, T):
    env.put(cls_node.name, ('__class__', cls_node))
    # Methods are stored for inlining
    for item in cls_node.body:
        if isinstance(item, ast.FunctionDef):
            env.put(f'{cls_node.name}.{item.name}',
                    ('__func__', item))
```

41.2 Classes as OTerm Inhabitants

In the OTerm framework, a class instance is not a primitive; it is an *addressed record*—a reference paired with a mapping from attribute names to OTerm values.

Definition 41.1 (Class OTerm)**.** Let C be a class with attributes $a_1, \ldots, a_k$ and methods $m_1, \ldots, m_j$. An instance o of C compiles to:

$$\mathsf{OTerm}_C(o) := \big(\mathsf{Ref}(\alpha),\ \{a_i \mapsto \mathsf{py_attr}_{a_i}(\alpha)\}_{i=1}^{k},\ \{m_i \mapsto \lambda\mathsf{self}.\,\mathsf{body}_{m_i}[\mathsf{self}/\alpha]\}_{i=1}^{j}\big)$$

where α is a fresh address constant and each method body is inlined with self bound to α.

The key insight is that method calls reduce to ordinary function application on shared symbols: `obj.compute()` $\rightsquigarrow$ $\mathsf{py_meth_compute}(\alpha)$ $\rightsquigarrow$ $\mathsf{py_attr_x}(\alpha)\cdot 2$. Two programs calling

the same method on objects of the same class produce identical shared-symbol expressions, making equivalence checking straightforward.

41.3 Inheritance as Fiber Refinement

A class hierarchy $A \leftarrow B \leftarrow C$ (C inherits B inherits A) defines a *refinement* of the type fibration: the fiber TRef is refined into sub-fibers for each class.

Definition 41.2 (Class Fiber)**.** For class C, the class fiber $U_C = \{v : \mathsf{PyObj} \mid \mathsf{tag}(v) = \mathsf{TRef} \wedge \mathsf{class_of}(v) = C\}$.

The method resolution order (MRO) determines which method is called: $\mathsf{meth_m}(v) = \mathsf{mro}(C)[0].\mathsf{m}(v)$.

Both programs using the same class hierarchy get the same MRO, hence the same method dispatch, hence the same shared symbols for each method call.

41.4 Method Resolution as a Path in the Class HIT

Python's C3 linearisation of the MRO can be modelled as a *path* in a higher inductive type encoding the class hierarchy.

Definition 41.3 (Class Hierarchy HIT)**.** Given classes $C_1, \ldots, C_n$ with inheritance edges $C_i \to C_j$ ("C_i inherits from C_j"), define the HIT $\mathsf{ClassHier}$ with:

- **Point constructors.** $\mathsf{cls}_i : \mathsf{ClassHier}$ for each class C_i.
- **Path constructors.** $\mathsf{inherits}_{ij} : \mathsf{cls}_i =_{\mathsf{ClassHier}} \mathsf{cls}_j$ for each inheritance edge $C_i \to C_j$.

A method lookup for method m on an object of class C_i is a *transport* along the MRO path:

$$\mathsf{resolve}_m(C_i) \coloneqq \mathsf{transp}^{\mathsf{HasMethod}_m}(\mathsf{mro_path}(C_i))$$

where $\mathsf{HasMethod}_m$ is the type family assigning to each class its implementation of m (or $\bot$ if absent), and $\mathsf{mro_path}(C_i)$ is the composite path through the linearised ancestor chain.

Proposition 41.4 (MRO Determinism)**.** *For a fixed class hierarchy, the MRO path is unique (up to homotopy), so* $\mathsf{resolve}_m(C_i)$ *is well-defined. Two programs with the same class hierarchy resolve every method identically.*

41.5 Virtual Dispatch as Fiber Case Split

When a function receives an argument of an abstract base type and calls a virtual method, the compiler performs a *case split on the object's class fiber*:

Listing 41.2: Virtual dispatch compilation

```
# obj.area() where obj : Shape (base class)
# compile to:
If(class_of(obj) == Circle,
   py_meth_area_Circle(obj),
   If(class_of(obj) == Rect,
      py_meth_area_Rect(obj),
      py_meth_area_Shape(obj)))    # default
```

In sheaf-theoretic terms, the class tag induces a covering $\{U_{C_1}, \ldots, U_{C_n}\}$ of the TRef fiber, and the virtual dispatch is exactly the local section assignment on each sub-cover. If two programs dispatch the same set of methods on the same class hierarchy, their local sections agree on each U_{C_i}, producing the same global section and hence $H^1 = 0$.

41.6 Descriptors and Properties

`@property` compiles to a shared function: obj.prop → py_prop_prop(obj). Since both programs access the same property, they get the same shared symbol.

More generally, Python's descriptor protocol (`__get__`, `__set__`, `__delete__`) compiles to shared functions parameterised by the descriptor's owner class and the accessing instance. Two programs using the same descriptor on the same class produce identical OTerm expressions.

Chapter 42

Error Handling and Graceful Degradation

42.1 Degradation Cascade

The pipeline degrades through three levels of decreasing certainty:

Level	Trigger	Verdict
1	Compiler succeeds, Z3 decides	EQUIVALENT or INEQUIVALENT
2	Compiler succeeds, Z3 times out	UNKNOWN (partial confidence)
3	Compiler fails on unsupported syntax	INCONCLUSIVE (fresh-constant fallback)

Level 1 is the normal case: the AST compiler produces OTerms, Z3 decides per-fiber equivalence, and the Čech complex yields a definitive verdict. Level 2 retains soundness—no false equivalences are reported—but loses completeness on timed-out fibers. Level 3 is the most conservative: the fresh-constant strategy guarantees soundness at the cost of systematic false negatives.

Proposition 42.1 (Soundness of Degradation)**.** *At every level, if the pipeline reports* EQUIVALENT*, then the two programs are semantically equivalent on all verified fibers. No degradation level can produce a false positive.*

42.2 Unsupported Syntax

When the AST compiler encounters a construct it cannot handle (e.g., `async/await`, `match/case`, `global`, `exec`), it produces a *fresh uninterpreted constant*: FreshConst(PyObj, `unsupported`).

This is *sound*: the fresh constant is unique, so two programs with different unsupported constructs produce different terms (no false equivalences). It is *incomplete*: two programs with the *same* unsupported construct also produce different fresh constants (false inequivalence).

42.3 Z3 Timeout Handling

When Z3 times out on a fiber:

1. Mark the fiber as UNKNOWN.
2. Continue checking other fibers.
3. In the H^1 computation, treat UNKNOWN fibers as "not verified" (not as failures).
4. Report a partial verdict: "Equivalent on k/n fibers, m fibers timed out."

Listing 42.1: Timeout handling with escalation

```python
def check_with_timeout(fiber, secs_f, secs_g, solver, timeout):
    solver.set('timeout', timeout)
    solver.push()
    # ... add constraints ...
    result = solver.check()
    solver.pop()
    if result == unknown:
        # Escalate: retry with 3x timeout on just this fiber
        solver.set('timeout', timeout * 3)
        result = solver.check()
    return result
```

42.4 Per-Fiber Timeouts and Global Deadlines

The benchmark harness enforces a two-level timeout architecture:

1. **Per-fiber Z3 timeout** (default 5 000 ms). Set via `z3.set_param('timeout', 5000)`. Controls the maximum time Z3 spends on any single satisfiability query. Fibers exceeding this limit are marked UNKNOWN and the pipeline continues.

2. **Global subprocess deadline** (default 12 s wall-clock). Each benchmark pair runs in a separate subprocess with a hard `subprocess.run(timeout=12)` deadline. If the entire pipeline (compilation + all fiber checks + H^1) exceeds this budget, the subprocess is killed and the pair is marked TIMEOUT.

The per-fiber timeout provides fine-grained control: if fiber U_3 times out but fibers U_1, U_2, U_4 succeed, the pipeline still reports a partial verdict. The global deadline is a safety net against pathological cases (e.g., exponentially many fibers from duck type inference).

42.5 Memory Safety

Each subprocess is sandboxed with OS-level resource limits to prevent a single hard pair from crashing the host:

Listing 42.2: Resource limits from `run_benchmark.py`

```python
import resource
# Virtual memory: 768 MB hard limit
resource.setrlimit(resource.RLIMIT_AS,
                   (768*1024*1024, 768*1024*1024))
# CPU time: 10s soft / 12s hard (SIGXCPU / SIGKILL)
resource.setrlimit(resource.RLIMIT_CPU, (10, 12))
# Z3 internal memory cap
z3.set_param('memory_max_size', 512)    # 512 MB
```

The three limits interact as a defence-in-depth stack:

- **Z3 memory_max_size** (512 MB) is the innermost guard. If Z3's internal allocator exceeds this, the solver returns unknown rather than allocating more memory.

- **RLIMIT_AS** (768 MB) catches memory growth outside Z3 (e.g., large AST compilation, Python object overhead). Exceeding it raises `MemoryError`.

- **RLIMIT_CPU** (10/12 s) catches infinite loops in compilation or in Z3's E-matching engine. The soft limit sends `SIGXCPU`; the hard limit sends `SIGKILL`.

42.6 Combinatorial Explosion in Site Category

When duck type inference allows k types per parameter and there are n parameters, the number of fiber combinations is k^n. Mitigation:

1. Cap at 50 fiber combinations (skip rare combinations).
2. Prioritize fibers by duck type confidence (high-confidence single-type fibers first).
3. Use the spectral sequence (Chapter 13) to check important fibers first and skip if degeneration is detected.

42.7 Partial Verdicts and Confidence Levels

Definition 42.2 (Confidence Level)**.** $\mathsf{confidence}(f, g) = \frac{\text{fibers verified equivalent}}{\text{total non-timed-out fibers}}$

A confidence of 1.0 means full equivalence proof. A confidence of 0.8 means 80% of fibers verified, 20% timed out. A confidence of 0.0 with counterexample means definite inequivalence.

Chapter 43

Scalability

43.1 Complexity of the Full Pipeline

Stage	Complexity	Bottleneck
AST parsing	$O(\|S\|)$	Python `ast.parse`
AST compilation	$O(\|A\|)$	single-pass OTerm build
Duck type inference	$O(\|A\|)$	single AST walk
Site construction	$O(k^n)$	Cartesian product of types
Per-fiber Z3 check	$O(k^n \cdot C_{\mathsf{Z3}})$	**dominant term**
H^1 computation	$O(k^{3n})$	GF(2) matrix rank
Total	$O(k^n \cdot C_{\mathsf{Z3}} + k^{3n})$	

For the common case ($k = 1$, single-typed parameters): $O(C_{\mathsf{Z3}})$ — a single Z3 query.

For the benchmark ($k \leq 2$, $n \leq 4$): $O(16 \cdot C_{\mathsf{Z3}} + 4096)$ — at most 16 Z3 queries plus cheap linear algebra. Runs in < 3 seconds.

43.2 Bottleneck Analysis

The dominant cost is the per-fiber Z3 check: $O(k^n \cdot C_{\mathsf{Z3}})$. This product has two axes of growth:

1. **Number of fibers** (k^n). Controlled by duck type inference precision. In practice, most parameters have $k = 1$ (monomorphic); polymorphic parameters with $k = 2$ and $n \leq 4$ yield at most 16 fibers.

2. **Per-fiber solver time** (C_{Z3}). Depends on term depth, axiom count, and use of recursive functions (see below). Ground queries finish in 1–50 ms; queries with RecFunctions take 50–500 ms; queries with quantified axioms may reach 500 ms–5 s.

The H^1 computation ($O(k^{3n})$, Gaussian elimination over GF(2)) is never the bottleneck: for $k^n = 16$ fibers, the coboundary matrix is 16×16 and rank computation takes microseconds.

43.3 Z3 Query Complexity

The Z3 query complexity C_{Z3} depends on:

- **Term depth**: deeper Z3 terms (from deeper nesting or longer loops) produce harder queries. Fold RecFunctions add depth proportional to the fold range.

- **Number of sections**: more branches produce more section pairs to compare. m sections in A $\times$ n sections in B $= m \cdot n$ comparisons.

- **Axiom count**: more ForAll axioms slow down E-matching. We add axioms only for shared functions actually used (axiom relevance filtering).

Typical Z3 query time: 1–50ms for ground queries (no RecFunctions), 50–500ms with RecFunctions, 500ms–5s with axioms.

43.4 Scaling Strategies

1. **Fiber pruning.** Duck type inference assigns a confidence $p_i \in [0, 1]$ to each type assignment. Fibers whose joint confidence $\prod_j p_{i_j} < \epsilon$ (default $\epsilon = 0.01$) are pruned before Z3 is invoked, reducing the effective fiber count from k^n to a small subset.

2. **Early termination.** For equivalence: if all fibers checked so far are equivalent and the spectral sequence (Chapter 13) has degenerated at E_1, skip remaining fibers. For inequivalence: if *any* fiber produces a counterexample, return INEQUIVALENT immediately.

3. **Parallel solving.** Z3 queries on distinct fibers are independent and can be dispatched to a thread pool. With p cores, the wall-clock time drops to $O(\lceil k^n/p \rceil \cdot C_{\mathsf{Z3}})$. The benchmark harness uses per-subprocess parallelism (one pair per process) rather than intra-pair parallelism, but the architecture supports both.

4. **Incremental checking.** When comparing two versions of the same codebase, only functions with changed ASTs need re-verification. Unchanged function results are cached and reused (Theorem 11.28).

5. **Timeout budget.** A global timeout budget T (default 10 s) is divided across fibers. Each fiber receives $T/|\text{fibers}|$ initially; fibers that finish quickly donate their remaining budget to harder fibers.

43.5 Heuristics for Large Programs

For programs with 100+ lines:

1. **Function-level decomposition**: verify each function independently, then compose results (Theorem 11.28).

2. **Incremental checking**: check modified functions only, reuse cached results for unchanged ones.

3. **Early termination**: if any fiber produces a counterexample, return INEQUIVALENT immediately (no need to check remaining fibers).

4. **Timeout budget**: allocate a total timeout budget (e.g., 10s) and divide across fibers. Skip low-priority fibers if budget is exhausted.

Appendix A

Mathematical Prerequisites

This appendix collects the mathematical background used throughout the monograph. We state definitions and key theorems without proof; references to standard texts are given at the end of each section.

A.1 Category Theory Essentials

A *category* $\mathcal{C}$ consists of objects, morphisms, composition, and identity, satisfying associativity and unit laws. A *functor* $F : \mathcal{C} \to \mathcal{D}$ preserves composition and identity. A *natural transformation* $\eta : F \Rightarrow G$ is a family of morphisms $\eta_A : F(A) \to G(A)$ commuting with all morphisms.

Key concepts used:

- **Initial algebra** (Ch. 26.9.3, 28). The initial object in the category of F-algebras for an endofunctor F. Lists are the initial algebra for $F(X) = 1 + A \times X$. The unique morphism from the initial algebra is the *catamorphism* (fold).
- **Grothendieck construction** (Ch. 6, 37). Turns a pseudofunctor $F : \mathcal{C}^{\mathrm{op}} \to \mathbf{Cat}$ into a fibered category $\int F$ over $\mathcal{C}$. Our type fibration is a Grothendieck construction: objects of $\int F$ are pairs (U, s) where U is a fiber and $s \in F(U)$ is a local section.
- **Yoneda lemma** (Ch. 15). $\mathrm{Nat}(\mathrm{Hom}(A, -), F) \cong F(A)$. Functions on representable presheaves are determined by their value at the representing object.
- **Adjunctions.** A pair of functors $F \dashv G$ with natural bijection $\mathrm{Hom}(FA, B) \cong \mathrm{Hom}(A, GB)$. Used implicitly in the compilation (syntax $\dashv$ semantics).
- **Limits and colimits.** Products, pullbacks, pushouts, and coequalisers appear in the site category construction and the Čech complex.

References. Mac Lane, *Categories for the Working Mathematician* [19]; Riehl, *Category Theory in Context* [20].

A.2 Sheaf Theory and Cohomology

A *site* is a category with a Grothendieck topology (covering families). A *presheaf* is a contravariant functor to **Set**. A *sheaf* satisfies the gluing axiom: local sections that agree on overlaps glue uniquely.

The *Čech complex* computes cohomology from a covering $\mathcal{U} = \{U_i\}$:

$$C^p(\mathcal{U}, \mathcal{F}) := \prod_{i_0 < \cdots < i_p} \mathcal{F}(U_{i_0} \cap \cdots \cap U_{i_p})$$

with alternating coboundary operators $\delta^p : C^p \to C^{p+1}$.

- $H^0(\mathcal{U}, \mathcal{F}) = \ker \delta^0$ is the space of *global sections*—type assignments that are consistent across all fibers.
- $H^1(\mathcal{U}, \mathcal{F}) = \ker \delta^1 / \operatorname{im} \delta^0$ measures *obstruction to gluing*: a non-trivial H^1 class means local sections cannot be reconciled into a global one. In our framework, $H^1 \neq 0$ signals a semantic bug.

References. Grothendieck, *Sur quelques points d'algèbre homologique* [8]; Hartshorne, *Algebraic Geometry*, Ch. III [21]; Curry, *Sheaves, Cosheaves and Applications* [17].

A.3 Homotopy Type Theory Essentials

Types are spaces. Terms are points. Equalities are paths.

$$\mathsf{Path}_A(a, b) = \{p : \mathbb{I} \to A \mid p(0) = a,\ p(1) = b\}$$

Function extensionality: $(f = g) \simeq \prod_x f(x) = g(x)$.
Univalence: $(A \simeq B) \simeq (A =_{\mathsf{Type}} B)$. Equivalent types are identical, which in our framework justifies treating two programs with isomorphic OTerm representations as equal.
Higher inductive types (HITs): types with point constructors *and* path constructors. Used for:

- The interval type $\mathbb{I}$ (Chapter 1).
- Pushouts and coequalisers in the site category (Chapter 7).
- The class hierarchy HIT (Chapter 41).

References. The Univalent Foundations Program, *Homotopy Type Theory* [3]; Cohen et al., *Cubical Type Theory* [1]; Angiuli et al., *Computational Higher-Dimensional Type Theory* [4].

A.4 Algebraic Geometry Background

The algebraic-geometric concepts used in this monograph (beyond sheaf cohomology) include:

- **Fibrations and fibers.** The type fibration $\pi : \int \mathsf{Ty} \to \mathcal{S}$ assigns to each open set U in the site a fiber of possible type assignments. Local sections are type-annotated program fragments over U.
- **Spectral sequences** (Chapter 13). The Mayer–Vietoris spectral sequence $E_1^{p,q} \Rightarrow H^{p+q}$ provides a filtration of the cohomology that allows early termination: if E_1 degenerates (all differentials vanish), then $H^1 = 0$ without computing the full Čech complex.
- **Descent data.** A collection of local sections with transition isomorphisms on overlaps. The sheaf condition says that descent data is *effective*—it glues to a unique global section. Failure of effectiveness is precisely $H^1 \neq 0$.

References. Hartshorne, *Algebraic Geometry* [21]; Grothendieck, *SGA 4* (for sites and topoi).

A.5 SMT Solving Essentials

Z3 decides satisfiability of first-order formulas in combined theories (arithmetic, strings, arrays, algebraic datatypes). Key features used:

- **Algebraic datatypes**: the PyObj sort is a Z3 datatype with constructors for each Python type tag (Chapter 25).

- **Recursive functions**: fold/map/filter compile to Z3 `RecFunction` declarations, enabling bounded reasoning about iteration (Chapter 28).
- **Quantifiers**: ForAll axioms express shared-function semantics. Z3 uses E-matching and MBQI for instantiation (Chapter 26).
- **Enumeration sorts**: used for type tags in the duck type lattice (Chapter 37).
- **Timeouts**: per-query timeouts and memory limits ensure termination (Chapter 42).

References. de Moura and Bjørner, *Z3: An Efficient SMT Solver* [6]; Barrett et al., *The SMT-LIB Standard* (for theory definitions).

Appendix B

Benchmark Pair Catalog

The complete catalog of 50 equivalent and 50 non-equivalent benchmark pairs is maintained in the repository at `benchmarks/hard_equiv_suite.py` and `tests/test_equivalence/test_hard_eq_pairs.py`.

Each pair consists of two Python functions (20–60 lines each) that exercise deep Python semantics. The equivalent pairs span all 24 dichotomies; the non-equivalent pairs test subtle behavioural differences including mutation, aliasing, closure binding, NaN handling, and edge cases.

B.1 Pipeline Resolution Stages

Each pair is resolved by a specific stage of the pipeline. We classify the resolution stage for each pair:

Stage	Description
Compile	Both programs compile to identical or α-equivalent OTerms. Z3 confirms $\forall x.\, f(x) = g(x)$ in a single ground query.
Fiber	Programs differ syntactically but agree on each fiber after per-fiber Z3 checking. Resolved by the local check stage.
Cohomology	Local sections disagree on some fiber overlaps; the H^1 computation determines whether disagreements are coboundaries (equivalent) or cocycles (inequivalent).
Axiom	Equivalence depends on shared-function axioms (e.g., commutativity of addition, associativity of string concatenation). Resolved by Z3 with ForAll axioms.
RecFunction	Programs use different iteration patterns (fold vs. recursion vs. comprehension). Resolved by Z3 RecFunction reasoning.

B.2 Difficulty Classification

We classify all 100 pairs into three difficulty tiers:

Trivial (canonical match). Both programs use the same algorithm with minor syntactic variation (variable renaming, loop $\leftrightarrow$ comprehension, `sorted()` $\leftrightarrow$ manual sort). Resolved at the Compile or single-fiber stage in $< 50\,\text{ms}$.

Medium (path search). Programs use different data structures or iteration orders but compute the same function. Requires multi-fiber checking and/or axiom-assisted reasoning. Typical resolution time: 50–500 ms.

Hard (different algorithms). Programs implement fundamentally different algorithms for the same problem (e.g., recursive descent vs. shunting-yard parser, convex hull via gift-wrapping vs. Graham scan, BFS vs. DFS with post-order). Requires RecFunction reasoning, multiple axioms, or cohomological analysis. Resolution time: 500 ms–5 s.

B.3 Equivalent Pairs (EQ-01 through EQ-50)

#	Description	Stage	Tier
01	Flatten nested data: recursive generator vs. iterative stack	RecFunction	Hard
02	Word frequency: regex+Counter vs. char state machine+heap	RecFunction	Hard
03	Expression evaluator: recursive vs. stack-based	RecFunction	Hard
04	Register VM: dict-based vs. list-based	Axiom	Medium
05	Topological sort: Kahn's vs. DFS post-order	RecFunction	Hard
06	Run-length encoding: groupby vs. manual loop	Fiber	Medium
07	Convex hull: gift-wrapping vs. Graham scan	RecFunction	Hard
08	Edit distance: full DP matrix vs. rolling two-row	RecFunction	Hard
09	LRU cache: OrderedDict vs. dict+doubly-linked list	Axiom	Hard
10	Matrix multiply: nested loops vs. zip+map	Fiber	Medium
11	Merge sorted iterables: heapq vs. manual merge	RecFunction	Hard
12	Binary tree traversal: recursive vs. iterative stack	RecFunction	Medium
13	Max flow: Ford–Fulkerson BFS vs. DFS path	RecFunction	Hard
14	Arithmetic parser: recursive descent vs. shunting-yard	RecFunction	Hard
15	Quickselect vs. sort-and-index	RecFunction	Hard
16	Strongly connected components: Tarjan vs. Kosaraju	RecFunction	Hard
17	Power set: recursive vs. bitmask enumeration	RecFunction	Medium
18	Pretty-print nested data: recursive vs. stack+indent	RecFunction	Medium
19	0/1 knapsack: DP table vs. memoised recursion	RecFunction	Hard
20	Deep copy: recursive vs. `copy.deepcopy`	Axiom	Medium
21	Prime factorisation: trial division vs. sieve	RecFunction	Medium
22	Cycle detection: Floyd's vs. visited-set DFS	RecFunction	Hard
23	Bracket matching: stack vs. counter with type check	Fiber	Trivial
24	Levenshtein distance: full matrix vs. two-row DP	RecFunction	Hard
25	Interval merge: sort+scan vs. priority queue	Fiber	Medium
26	k-th smallest: quickselect vs. heapq.nsmallest	RecFunction	Medium
27	LCS: DP table vs. memoised recursion	RecFunction	Hard
28	Regex match: NFA simulation vs. recursive backtrack	RecFunction	Hard
29	Graph inversion: comprehension vs. defaultdict loop	Fiber	Trivial
30	Prefix search: trie vs. sorted+bisect	RecFunction	Hard
31	Dijkstra: heapq vs. manual priority queue	RecFunction	Hard
32	Island counting: BFS vs. DFS flood	RecFunction	Hard

B.4 Non-Equivalent Pairs (NEQ-01 through NEQ-50)

#	Witness Divergence	Stage	Tier
01	Floor div vs. truncation on negative	Compile	Trivial
02	Python `%` vs. C-style `%`	Compile	Trivial
03	Banker's rounding vs. round-half-up	Fiber	Medium
04	`split()` vs. `split(' ')`	Axiom	Medium
05	`==` vs. `is` for large integers	Compile	Trivial
06	Late-binding vs. early-binding closure	RecFunction	Hard
07	Mutable default argument accumulation	Axiom	Hard
08	Insertion order vs. sorted dict keys	Fiber	Medium
09	NaN propagation in `max()`	Fiber	Medium
10	`or` returns value vs. `bool`	Compile	Trivial
11	Float accumulation order	Fiber	Hard
12	Sorted-set dedup vs. insertion-order dedup	Fiber	Medium
13	`except Exception` vs. bare `except`	Axiom	Hard
14	Stable sort vs. key-only sort	Fiber	Medium
15	List vs. generator double-iteration	RecFunction	Hard
16	Floor div vs. true div result type	Compile	Trivial
17	List `count` vs. set membership	Fiber	Medium
18	`str()` vs. `repr()`	Axiom	Medium
19	Dict merge order with conflicts	Fiber	Medium
20	Truthiness vs. `None` check for empty list	Compile	Trivial
21	Chained vs. parenthesised comparison	Compile	Trivial
22	`+=` vs. `+` on list aliases	Axiom	Hard
23	Float `sqrt` vs. `isqrt` precision	Fiber	Hard
24	List comp vs. generator exception location	RecFunction	Hard
25	Stable ascending vs. reverse-reverse sort	Fiber	Medium
26	`any` short-circuit vs. full-scan count	Fiber	Medium
27	Walrus operator scope	Compile	Medium
28	`dict.get` vs. `KeyError` handling	Axiom	Medium
29	`[0]*n` shallow vs. comprehension deep copy	Axiom	Hard
30	`list` vs. `map` type name	Compile	Trivial
31	`enumerate` from 0 vs. from 1	Compile	Trivial
32	`zip` truncate vs. `zip_longest`	Fiber	Medium
33	`sorted` returns list vs. `sort` returns None	Compile	Trivial
34	String `a+b` vs. `b+a`	Compile	Trivial
35	Exception in `try` body vs. `else` clause	Axiom	Medium
36	Counter subtract keeps negatives vs. drops	Axiom	Medium
37	Class attribute shared vs. instance attribute	Axiom	Hard
38	`*args` as tuple vs. list	Compile	Trivial
39	`int()` truncation vs. `math.floor`	Compile	Trivial
40	Logical `and` vs. bitwise `&`	Compile	Trivial
41	`range` excludes vs. includes endpoint	Compile	Trivial

B.5 Summary Statistics

	Trivial	**Medium**	**Hard**
EQ pairs	5	18	27
NEQ pairs	19	17	14
Total	24	35	41

The distribution is intentionally skewed toward hard pairs: the benchmark's purpose is to stress-test the pipeline on cases where naive syntactic comparison fails. The NEQ pairs are skewed toward trivial/medium because many Python semantic traps (NaN, mutable defaults, `is` vs. `==`) produce divergence that the compiler detects early.

Appendix C

Proof Theory: Foundations

The preceding chapters established CCTT's ability to detect equivalences via Z3 decision procedures and Čech cohomology. But these techniques have a fundamental limitation: they cannot handle *algorithmic* equivalences—cases where two programs compute the same function via radically different strategies (e.g., dynamic programming vs. memoized recursion, BFS vs. Union-Find, merge sort vs. quick sort).

In this chapter we develop a **proof theory** for CCTT: a formal system in which a human (or LLM) can write an explicit proof term that CCTT can *mechanically verify*. The proof checker is purely structural—no sampling, no oracles, no heuristics. If the proof checks, the equivalence is *proven*.

C.1 Motivation: Why a Proof Theory?

1. **Z3 is incomplete for algorithms.** Z3 operates on quantifier-free formulas in decidable fragments. It cannot reason about loop invariants, recursion depth, or algorithmic structure directly.

2. **Bounded testing is not a proof.** Even with millions of test inputs, a bounded test cannot guarantee equivalence. Adversarial inputs may exist outside the tested domain.

3. **Explicit evidence enables trust.** A proof term is a *certificate*—anyone can re-verify it. Unlike a Z3 model or a sampling run, a proof term is permanent, portable, and auditable.

4. **Compositionality.** Proofs compose: if we prove $A \equiv B$ and $B \equiv C$, then Trans gives us $A \equiv C$. No "leaps of faith."

C.2 The Language of Proof Terms

Definition C.1 (Proof Term)**.** A **proof term** is a syntactic witness that two OTerms are equal in the CCTT equational theory. We write

$$\Gamma \vdash p : a \equiv b$$

where Γ is a context of definitions and assumptions, a and b are OTerms, and p is the proof term.

C.2.1 Structural Rules

The structural rules form the core of the identity type.

Definition C.2 (Reflexivity). $\mathsf{Refl}(t)$ proves $t \equiv t$.

$$\frac{}{\Gamma \vdash \mathsf{Refl}(t) : t \equiv t} \quad \text{(refl)}$$

Definition C.3 (Symmetry). If $p : a \equiv b$, then $\mathsf{Sym}(p) : b \equiv a$.

$$\frac{\Gamma \vdash p : a \equiv b}{\Gamma \vdash \mathsf{Sym}(p) : b \equiv a} \quad \text{(sym)}$$

Definition C.4 (Transitivity). If $p : a \equiv b$ and $q : b \equiv c$, then $\mathsf{Trans}(p, q) : a \equiv c$.

$$\frac{\Gamma \vdash p : a \equiv b \qquad \Gamma \vdash q : b \equiv c}{\Gamma \vdash \mathsf{Trans}(p, q) : a \equiv c} \quad \text{(trans)}$$

Definition C.5 (Congruence). If $p_i : a_i \equiv b_i$ for $i = 1, \ldots, n$, then $\mathsf{Cong}(f, p_1, \ldots, p_n) : f(a_1, \ldots, a_n) \equiv f(b_1, \ldots, b_n)$.

$$\frac{\Gamma \vdash p_i : a_i \equiv b_i \quad (i = 1, \ldots, n)}{\Gamma \vdash \mathsf{Cong}(f, \vec{p}) : f(\vec{a}) \equiv f(\vec{b})} \quad \text{(cong)}$$

C.2.2 Computation Rules

Definition C.6 (β-reduction). For a lambda $\lambda x.\, body$ applied to argument arg:

$$\frac{}{\Gamma \vdash \mathsf{Beta}(\lambda x.body, arg) : (\lambda x.body)(arg) \equiv body[x := arg]} \quad (\beta)$$

Definition C.7 (δ-reduction). Unfold a named definition f:

$$\frac{f(x_1, \ldots, x_n) \triangleq body \in \Gamma}{\Gamma \vdash \mathsf{Delta}(f, \vec{a}) : f(\vec{a}) \equiv body[\vec{x} := \vec{a}]} \quad (\delta)$$

Definition C.8 (η-reduction). When x is not free in f:

$$\frac{x \notin \mathrm{FV}(f)}{\Gamma \vdash \mathsf{Eta}(f) : \lambda x.\, f(x) \equiv f} \quad (\eta)$$

C.2.3 Induction Principles

Definition C.9 (Natural Number Induction). To prove $\forall n.\, P(n)$:

$$\frac{\Gamma \vdash p_0 : P(0) \qquad \Gamma, k : \mathbb{N}, IH : P(k) \vdash p_s : P(k+1)}{\Gamma \vdash \mathsf{NatInd}(p_0, p_s, n) : \forall n.\, P(n)} \quad (\mathbb{N}\text{-ind})$$

Definition C.10 (List Induction). To prove $\forall xs.\, P(xs)$:

$$\frac{\Gamma \vdash p_\varepsilon : P([]) \qquad \Gamma, x, xs, IH : P(xs) \vdash p_c : P(x :: xs)}{\Gamma \vdash \mathsf{ListInd}(p_\varepsilon, p_c, xs) : \forall xs.\, P(xs)} \quad (\mathsf{List}\text{-ind})$$

Definition C.11 (Well-Founded Induction). Given a measure function μ into a well-ordered set:

$$\frac{\Gamma, (\forall y.\, \mu(y) < \mu(x) \Rightarrow P(y)) \vdash p : P(x)}{\Gamma \vdash \mathsf{WFInd}(\mu, p) : \forall x.\, P(x)} \quad (\mathsf{WF}\text{-ind})$$

C.2.4 Axiom Application

Definition C.12 (Axiom Application). Apply a named CCTT axiom $\mathcal{A}$ as a rewrite:

$$\frac{\mathcal{A} : \ell \equiv r \quad \sigma : \mathrm{Var} \to \mathsf{OTerm}}{\Gamma \vdash \mathsf{AxiomApp}(\mathcal{A}, \sigma, t) : t[\ell\sigma] \equiv t[r\sigma]} \quad \text{(axiom)}$$

C.2.5 Loop and Recursion Reasoning

Definition C.13 (Loop Invariant)**.** A Hoare-style loop proof with invariant I:

$$\frac{p_{init} : I(s_0) \qquad p_{pres} : I(s) \wedge G(s) \Rightarrow I(step(s)) \qquad p_{term} : \mu \text{ decreases} \qquad p_{post} : I(s) \wedge \neg G(s) \Rightarrow Q(s)}{\Gamma \vdash \mathsf{LoopInv}(I, p_{init}, p_{pres}, p_{term}, p_{post}) : \text{loop correct}}$$

Definition C.14 (Simulation)**.** Prove two programs equivalent by a simulation relation R:

$$\frac{p_0 : R(s_0^f, s_0^g) \qquad p_s : R(s^f, s^g) \Rightarrow R(step_f(s^f), step_g(s^g)) \qquad p_o : R(s^f, s^g) \Rightarrow out_f(s^f) = out_g(s^g)}{\Gamma \vdash \mathsf{Sim}(R, p_0, p_s, p_o) : f \equiv g} \quad \text{(sim)}$$

Definition C.15 (Abstraction Refinement)**.** Prove equivalence by showing both programs refine the same abstract specification:

$$\frac{p_f : \alpha \circ f = spec \qquad p_g : \alpha \circ g = spec}{\Gamma \vdash \mathsf{AbsRef}(spec, p_f, p_g) : f \equiv g} \quad \text{(abs-ref)}$$

C.2.6 Algebraic Reasoning

Definition C.16 (Commutative Diagram)**.** If $h \circ f = g \circ k$ and h, k are isomorphisms, then $f \equiv g$ up to encoding:

$$\frac{p_c : h \circ f = g \circ k \qquad p_i : h, k \text{ are isos}}{\Gamma \vdash \mathsf{CommDiag}(p_c, p_i) : f \equiv g} \quad \text{(diag)}$$

C.2.7 Z3 Discharge

Definition C.17 (Z3 Discharge)**.** For decidable subgoals in quantifier-free fragments:

$$\frac{\varphi \in \text{QF_LIA} \cup \text{QF_NIA} \cup \cdots \qquad \text{Z3}(\neg\varphi) = \mathsf{unsat}}{\Gamma \vdash \mathsf{Z3}(\varphi) : \varphi \text{ holds}} \quad \text{(z3)}$$

Critical: if Z3 returns $\mathsf{unknown}$ (timeout), the proof is **rejected**—not "probably true."

C.2.8 Cohomological Reasoning

Definition C.18 (Fiber Decomposition)**.** Prove equivalence by proving on each fiber separately:

$$\frac{\forall \tau \in \text{Fibers}.\ p_\tau : f|_\tau \equiv g|_\tau}{\Gamma \vdash \mathsf{FiberDecomp}(\{p_\tau\}) : f \equiv g} \quad \text{(fiber)}$$

This is the sheaf condition: a global section is determined by its restrictions to an open cover.

Definition C.19 (Čech Gluing)**.** Glue local equivalences via Čech cohomology:

$$\frac{\{p_i : f|_{U_i} \equiv g|_{U_i}\} \qquad \{p_{ij} : \text{compatible on } U_i \cap U_j\} \qquad H^1 = 0}{\Gamma \vdash \mathsf{CechGlue}(\vec{p}, \vec{p_{ij}}) : f \equiv g} \quad \text{(glue)}$$

C.3 The Proof Checker

The proof checker is a recursive function $\mathsf{check} : \mathsf{ProofTerm} \times \mathsf{OTerm} \times \mathsf{OTerm} \to \{\top, \bot\}$.

Theorem C.20 (Properties of the Checker)**.** *The proof checker satisfies:*

1. ***Soundness**: If* $\mathsf{check}(p, a, b) = \top$*, then* $[\![a]\!] = [\![b]\!]$ *in the denotational semantics.*
2. ***Determinism**: The same proof term always produces the same result.*

3. ***Termination:*** check *always returns in bounded time (recursion depth* ≤ 1000*; Z3 calls bounded by timeout).*

4. ***Non-sampling:*** *The checker never generates random inputs or performs bounded testing.*

Proof sketch. Soundness follows from the correctness of each rule.

- Refl: canonical form equality implies denotational equality.
- Sym, Trans: the denotational equality relation is an equivalence relation.
- Cong: denotational semantics is compositional; equal arguments produce equal results.
- β, δ, η: reduction rules preserve denotational semantics by construction.
- NatInd, ListInd: structural induction is sound for the natural numbers (Peano) and lists (free algebra).
- Z3Discharge: Z3 returning UNSAT for $\neg\varphi$ is a machine-checked proof of φ in the given fragment.
- Simulation: a valid bisimulation relation implies observational equivalence.
- AbsRef: if $\alpha \circ f = \alpha \circ g$ and α is injective (or both sides equal the spec), then $f \equiv g$.

Determinism follows because the checker is purely functional (no mutable state, no randomness).

Termination follows because: (a) proof depth is bounded by `MAX_DEPTH = 1000`; (b) Z3 calls have a timeout; (c) each recursive call strictly decreases the proof tree. □

C.4 Soundness Theorem

Theorem C.21 (Soundness of CCTT Proof Theory)**.** *If* $\Gamma \vdash p : a \equiv b$ *is derivable, then for all environments* ρ *satisfying* Γ*:*

$$[\![a]\!]_\rho = [\![b]\!]_\rho$$

Proof. By induction on the structure of the derivation p. The base cases (Refl, Beta, Delta, Eta, Z3Discharge, AxiomApp) hold by the correctness of canonical-form computation, substitution, η-law, Z3 as a decision procedure, and the soundness of each CCTT axiom (established in the preceding chapters).

The inductive cases (Sym, Trans, Cong, NatInd, ListInd, WFInd, Simulation, AbsRef, FiberDecomp, CechGlue) follow from the IH applied to sub-proofs, plus the soundness of the corresponding mathematical principles (equational reasoning, induction, bisimulation, refinement, sheaf descent). □

C.5 Comparison with Other Proof Systems

C.5.1 Martin-Löf Identity Types

In Martin-Löf type theory (MLTT), the identity type $\mathrm{Id}_A(a, b)$ has a single constructor refl and an elimination principle J. CCTT's proof terms are richer: we include computation rules (β, δ, η), induction principles, and domain-specific rules (simulation, loop invariants) that would require additional axioms or libraries in MLTT.

C.5.2 Cubical Path Types

In cubical type theory, a path $p : \mathsf{Path}_A\, a\, b$ is a function $p : \mathbb{I} \to A$ with $p(0) = a$ and $p(1) = b$. CCTT's path search (Chapter 18) constructs paths using axiom rewrites; the proof theory provides a language for *certifying* that a path exists.

The key difference: cubical paths are computational (they compute via Kan operations); CCTT proof terms are *verification certificates* that the checker validates.

C.5.3 Lean Proofs

CCTT proof terms can be *embedded* into Lean proofs:

- Refl maps to `rfl`
- Sym maps to `Eq.symm`
- Trans maps to `Eq.trans`
- Cong maps to `congrArg`
- NatInd maps to `Nat.rec`
- Z3Discharge maps to `omega` or `decide`

Theorem C.22 (Embedding into Lean)**.** *Every valid CCTT proof term can be translated into a Lean 4 proof term that type-checks in the Lean kernel.*

This embedding provides a second layer of assurance: even if the CCTT proof checker has a bug, the Lean kernel provides an independent verification.

C.6 Proof Term Serialization

Proof terms can be serialized to JSON for storage and exchange:

```
{
  "tag": "NatInduction",
  "base_case": {"tag": "Refl", "term": {"type": "OLit", "value":
     1}},
  "inductive_step": {"tag": "Assume", "label": "IH_n", ...},
  "variable": "n"
}
```

The serialization is complete: $\mathsf{from_json}(\mathsf{to_json}(p)) = p$ for every proof term p.

Proof documents include the goal (lhs, rhs), the proof term, and metadata (name, description, strategy). They can be stored alongside source files as `*.proof.json` files.

C.7 Integration with the CCTT Pipeline

The proof theory integrates with the existing checker as "Strategy 0: Explicit Proof Verification." When a proof file exists alongside a program pair, the checker verifies the proof *instead of* using heuristics:

1. Try explicit proof verification (proof file or registered proof)
2. Try denotational OTerm equivalence
3. Try Z3 structural equality

4. Try semantic fingerprinting
5. Try per-fiber Z3 + Čech H^1
6. Bounded testing (fallback)

A verified proof gives confidence 1.0—no sampling or heuristics involved.

Appendix D

Proof Theory: Worked Examples

This chapter presents complete, worked proofs of algorithmic equivalences using the CCTT proof theory. Each example includes the two programs, the proof strategy, the proof term, and commentary on why the strategy is appropriate.

D.1 Proof Strategies Overview

Before diving into examples, we summarize the available strategies and when each is appropriate:

Strategy	When to Use
Refl	Same algorithm, same code
NatInd	Recursive vs. iterative (same recurrence)
ListInd	List-processing equivalences
WFInd	Programs with non-obvious termination
Simulation	Different algorithms, relatable state spaces
AbsRef	Both implement the same abstract spec
CommDiag	Representation change (encoding bijection)
LoopInv	Loop correctness, stability, invariant props
FiberDecomp	Type-dependent behavior (per-fiber proof)
Z3Discharge	Decidable arithmetic/logic subgoals

D.2 Example 1: Recursive vs. Iterative Factorial

D.2.1 The Programs

```python
def fact_rec(n):
    if n == 0: return 1
    return n * fact_rec(n - 1)

def fact_iter(n):
    result = 1
    for i in range(1, n + 1):
        result *= i
    return result
```

D.2.2 OTerm Representation

$$f = \mathsf{fix}[\mathsf{fact_rec}]\big(\mathsf{case}(n = 0,\ 1,\ n \cdot \mathsf{fact_rec}(n-1))\big)$$
$$g = \mathsf{fold}[\cdot](1, \mathsf{range}(1, n+1))$$

D.2.3 Proof Strategy: Natural Number Induction

Property: $P(n) \equiv (f(n) = g(n))$.

Base case $(n = 0)$:

$$f(0) = \mathsf{case}(\mathsf{true}, 1, \ldots) = 1$$
$$g(0) = \mathsf{fold}[\cdot](1, \mathsf{range}(1, 1)) = \mathsf{fold}[\cdot](1, []) = 1$$

Inductive step: Assuming $f(k) = g(k)$, prove $f(k+1) = g(k+1)$:

$$\begin{aligned} f(k+1) &= (k+1) \cdot f(k) \quad [\delta\text{-unfold}] \\ &= (k+1) \cdot g(k) \quad [\text{IH}] \\ &= (k+1) \cdot \mathsf{fold}[\cdot](1, \mathsf{range}(1, k+1)) \\ &= \mathsf{fold}[\cdot](1, \mathsf{range}(1, k+2)) \quad [\text{fold extension}] \\ &= g(k+1) \end{aligned}$$

D.2.4 Proof Term

```
proof = NatInduction(
    base_case   = Assume('base_eval', f[n:=0], g[n:=0]),
    ind_step    = Assume('IH_n', f, g),
    variable    = 'n',
    base_value  = OLit(0),
)
```

D.3 Example 2: Merge Sort vs. Quick Sort

D.3.1 The Programs

Both sort a list of comparable elements. Merge sort divides in half and merges; quick sort partitions around a pivot and concatenates.

D.3.2 Proof Strategy: Abstraction Refinement

Both algorithms implement the abstract specification $\mathsf{sorted_permutation}$: a function that returns a sorted permutation of its input.

- **Merge sort**: merge preserves elements (permutation); merge of two sorted halves is sorted.
- **Quick sort**: partition preserves elements; concatenation of sorted($<$ pivot) $+\!\!+$ [pivot] $+\!\!+$ sorted($>$ pivot) is sorted.

Since both are correct implementations of the same spec, they are equivalent.

```
proof = AbstractionRefinement(
    spec_name     = 'sorted_permutation',
    abstraction_f = Assume('merge_sort_correct', ...),
    abstraction_g = Assume('quick_sort_correct', ...),
)
```

Remark D.1. Abstraction refinement is the go-to strategy when two algorithms solve the same problem but use entirely different approaches. The specification acts as a "meeting point" in the space of behaviors.

D.4 Example 3: Edit Distance DP vs. Memoized Recursive

D.4.1 The Programs

The dynamic programming version fills a 2D table bottom-up. The memoized version computes top-down with caching.

D.4.2 Proof Strategy: Simulation

The simulation relation is:

$$R(dp, memo) \equiv \forall (i, j) \text{ computed. } dp[i][j] = memo[(i, j)]$$

Initial states: Both initialize base cases identically: $dp[0][j] = j$, $memo[(0, j)] = j$.
Step preservation: Both compute the same recurrence:

$$v = \min(del + 1,\ ins + 1,\ sub + cost)$$

The DP fills row-by-row; memoization fills on-demand. Since both use the same recurrence and all sub-results are available (DP: filled in order; memo: recursion + cache), the values match.
Output: Both return the value at (m, n).

D.5 Example 4: BFS Flood Fill vs. Union-Find

Both count connected components in a grid.
Strategy: Abstraction refinement via `count_connected_components`.

- BFS: each BFS traversal from an unvisited cell visits exactly one component. Count of traversals = count of components.
- Union-Find: union adjacent cells; count distinct roots. Each root represents exactly one component.

This is a clean example of abstraction refinement: the two algorithms have completely different state representations (visited set vs. parent array), but both correctly compute the same abstract quantity.

D.6 Example 5: Dijkstra vs. Bellman-Ford

On graphs with non-negative edge weights, both compute single-source shortest paths.
Strategy: Simulation with distance invariant.

$$R \equiv \forall v \text{ finalized. } dist_{\text{Dijkstra}}[v] = dist_{\text{BF}}[v]$$

The key insight: for non-negative weights, Dijkstra's greedy choice (finalize the minimum-distance vertex) produces the same final distances as Bellman-Ford's exhaustive relaxation. The simulation relation holds because:

1. Both initialize $dist[source] = 0$, $dist[v] = \infty$.
2. Relaxation is the same operation: $d[v] \leftarrow \min(d[v], d[u] + w)$.
3. Non-negative weights ensure Dijkstra's greedy order doesn't miss shorter paths.

D.7 Example 6: Trial Division vs. Sieve Factorization

Both compute the prime factorization of n.

Strategy: Abstraction refinement via prime_factorization.

By the Fundamental Theorem of Arithmetic, the prime factorization is unique. Both algorithms produce it:

- Trial division: divides by the smallest prime first, extracting each factor with multiplicity.
- Sieve: precomputes the smallest prime factor (SPF) table, then repeatedly divides n by SPF[n].

D.8 Example 7: Powerset Recursive vs. Bitmask

Strategy: Commutative diagram with bijection.

The recursion tree has 2^n paths, each corresponding to a sequence of include/exclude decisions. These map bijectively to n-bit masks:

$$\begin{tikzcd}
\text{recursion paths} \arrow[r, "\mathsf{encode}"] \arrow[d, "\mathsf{collect}"] & \{0, \ldots, 2^n - 1\} \arrow[d, "\mathsf{bits_to_subset}"] \\
\mathcal{P}(S) & \mathcal{P}(S) \arrow[l, "="']
\end{tikzcd}$$

The encoding is an isomorphism: each path maps to a unique bitmask and vice versa. The commutative diagram establishes that both enumeration methods produce the same power set.

D.9 Example 8: LCS via DP vs. Hunt-Szymanski

The standard DP fills a 2D table; Hunt-Szymanski reduces LCS to Longest Increasing Subsequence (LIS) of match positions.

Strategy: Simulation relating DP entries to LIS lengths.

The key insight: $dp[i][j]$ equals the length of the LIS of match positions up to (i, j). Both algorithms compute the same recurrence; they differ only in evaluation order and pruning strategy.

D.10 Example 9: Red-Black Tree vs. AVL Tree Insert

Both are balanced BSTs that support the same interface.

Strategy: Abstraction refinement via BST in-order invariant.

After any sequence of insertions, the in-order traversal of both trees is the same sorted sequence. The balancing strategies differ (red-black rotations vs. AVL height-balance rotations), but both maintain the BST property and produce the same sorted order.

D.11 Example 10: Regex NFA vs. DFA Matching

Strategy: Simulation via subset construction.

The simulation relation is:

$$R \equiv dfa_state = \mathsf{frozenset}(nfa_states)$$

This is the classical subset construction bisimulation. At each input character, the DFA transitions to the set of all NFA states reachable from the current set, which is exactly the set of active NFA states.

D.12 Example 11: Map-then-Filter vs. Filter-then-Map

Strategy: List induction with equational reasoning.

For pure f and predicate p:

$$\mathsf{filter}(p, \mathsf{map}(f, xs)) = \mathsf{map}(f, \mathsf{filter}(p \circ f, xs))$$

Base case: both sides on [] give [].

Cons case: assuming the equality for xs,

$$\begin{aligned}
&\mathsf{filter}(p, \mathsf{map}(f, x :: xs)) \\
&= \mathsf{filter}(p, f(x) :: \mathsf{map}(f, xs)) \\
&= \begin{cases} f(x) :: \mathsf{filter}(p, \mathsf{map}(f, xs)) & \text{if } p(f(x)) \\ \mathsf{filter}(p, \mathsf{map}(f, xs)) & \text{otherwise} \end{cases} \\
&= \begin{cases} f(x) :: \mathsf{map}(f, \mathsf{filter}(p \circ f, xs)) & \text{if } (p \circ f)(x) \\ \mathsf{map}(f, \mathsf{filter}(p \circ f, xs)) & \text{otherwise} \end{cases} \qquad \text{[IH]} \\
&= \mathsf{map}(f, \mathsf{filter}(p \circ f, x :: xs))
\end{aligned}$$

D.13 Example 12: Insertion Sort is Stable

Strategy: Loop invariant.

This proves a property that *cannot* be proved by sampling or Z3: stability (equal elements maintain their relative order).

Invariant: $sorted[0..i]$ is a stable permutation of $xs[0..i]$.

Preservation: inserting $xs[i]$ into the sorted prefix at the correct position preserves stability because insertion happens *after* equal elements (not before), so equal elements maintain their original order.

This is a deeply structural argument that requires understanding the insertion algorithm's behavior on equal elements—exactly the kind of reasoning that proof terms enable.

D.14 Proof Complexity Analysis

Example	**Depth**	**Size**	**Strategy**
Factorial rec/iter	3	4	NatInduction
Merge/Quick sort	2	3	AbsRef
Edit distance DP/memo	2	4	Simulation
BFS/Union-Find	2	3	AbsRef
Dijkstra/Bellman-Ford	2	4	Simulation
Trial div/sieve	2	3	AbsRef
Powerset rec/bitmask	2	3	CommDiag
LCS DP/Hunt-Szymanski	2	4	Simulation
RBT/AVL insert	2	3	AbsRef
NFA/DFA matching	2	4	Simulation
Map-filter commute	3	4	ListInduction
Stable insertion sort	2	5	LoopInvariant
Gauss sum	3	4	NatInduction
reverse(reverse)	3	4	ListInduction
$2 \cdot 3 + 4 = 10$	1	1	ArithSimp

$1 \cdot x + 0 = x$	5	5	Trans+Cong+Axiom

Remark D.2. Proof depth is typically 2–5. The most complex proofs (equational chains like $1 \cdot x + 0 = x$) have depth 5 because they chain multiple rewrite steps. Algorithmic equivalences (simulation, abstraction refinement) have low depth (2–3) because the complexity is in the *relation* or *specification*, not the proof structure.

Appendix E

Proof Theory: Automation and Tactics

While the proof theory of the previous chapters provides a language for *writing* proofs, humans (and LLMs) benefit from automation. This chapter describes the tactic language: functions that *construct* proof terms automatically.

The key design principle: **tactics are optional**. A human can always write a proof term directly. Tactics just make it easier by automating routine steps. Every tactic outputs a proof term that the checker can independently verify.

E.1 Tactic Architecture

A tactic is a function

$$\tau : \mathsf{OTerm} \times \mathsf{OTerm} \times \mathsf{Context} \to \mathsf{TacticResult}$$

where $\mathsf{TacticResult}$ contains either a proof term (success) or a failure reason.

Tactics are classified by their power and cost:

1. **Finishing tactics**: close the goal immediately.
   - refl: try reflexivity (syntactic equality up to normalization).
   - $\mathsf{assumption}$: match an in-scope hypothesis.
   - $\mathsf{definitional}$: normalize both sides and compare.
2. **Simplification tactics**: transform the goal.
   - $\mathsf{simp}(\mathcal{A})$: apply simplification axioms from set $\mathcal{A}$ exhaustively.
   - ring: prove ring equalities by polynomial normalization.
   - omega: prove linear arithmetic via Z3 QF_LIA.
3. **Decomposition tactics**: break the goal into subgoals.
   - $\mathsf{induction}(x, \mathit{structure})$: set up induction on x over naturals or lists.
   - $\mathsf{cases}(d, \mathit{labels})$: case-split on discriminant d.
   - $\mathsf{ext}(x)$: prove function equality by extensionality.
4. **Rewriting tactics**: apply equations.
   - $\mathsf{unfold}(f)$: δ-expand a function definition.
   - $\mathsf{rewrite}(e, \mathit{dir})$: apply a proven equation as a rewrite.
5. **Strategy tactics**: set up complex proof structures.

- by_simulation(R): construct a simulation proof skeleton.
- by_spec(S): construct an abstraction refinement.
- by_loop_invariant(I): construct a loop invariant proof.
- by_comm_diagram: construct a commutative diagram proof.

6. **Search tactics**: try multiple approaches.
   - auto(*depth*): automatic proof search up to a given depth.

E.2 The auto Tactic

The auto tactic is a depth-bounded proof search that tries tactics in order of increasing complexity:

1. Reflexivity (identity)
2. Definitional equality (normalization)
3. Arithmetic simplification (constant folding, Z3)
4. Assumption matching
5. Congruence (recursive on arguments)
6. Symmetry + recursive search

```
def auto(lhs, rhs, ctx, depth=3):
    # 1. Reflexivity
    if terms_equal(normalize(lhs), normalize(rhs)):
        return Refl(lhs)
    # 2. Arithmetic
    if is_constant(lhs) and is_constant(rhs):
        if eval(lhs) == eval(rhs):
            return ArithmeticSimp(lhs, rhs)
    # 3. Congruence (recursive)
    if head(lhs) == head(rhs) and depth > 0:
        proofs = [auto(a, b, ctx, depth-1)
                  for a, b in zip(args(lhs), args(rhs))]
        if all_success(proofs):
            return Cong(head(lhs), proofs)
    # 4. Symmetry
    if depth >= 2:
        p = auto(rhs, lhs, ctx, depth-1)
        if success(p): return Sym(p)
    # Failure
    return None
```

Remark E.1. The auto tactic has worst-case exponential time in *depth*, but with depth ≤ 3 it is fast in practice. For deep proofs, the user should guide the search with explicit proof terms or intermediate lemmas.

E.3 The ring Tactic

The ring tactic proves equalities in commutative rings by normalizing both sides to multivariate polynomial form.

Algorithm:

1. Convert both sides to polynomials: sums of products of variables with integer coefficients.
2. A polynomial is a map from monomials to coefficients.
3. A monomial is a sorted tuple of (variable, exponent) pairs.
4. Two polynomials are equal iff they have the same monomials with the same coefficients.

Example E.2. ring can prove:

$$(a+b)^2 = a^2 + 2ab + b^2$$

by expanding both sides to the polynomial $\{a^2 : 1, ab : 2, b^2 : 1\}$.

If polynomial normalization fails (e.g., division or non-polynomial operations), ring falls back to Z3 QF_NIA.

E.4 The omega Tactic

The omega tactic handles linear integer arithmetic:

$$\sum_i a_i x_i + c \bowtie 0 \quad \text{where } \bowtie \in \{=, \neq, <, \leq, >, \geq\}$$

Implementation: construct a Z3 QF_LIA formula and check validity. If Z3 returns UNSAT for the negation, the formula is valid and we produce a Z3Discharge proof term.

Remark E.3. Z3 is used here as a *decision procedure*, not as a sampler. A Z3 UNSAT result is a machine-checked proof. If Z3 times out, omega reports failure—it never returns "probably true."

E.5 The simp Tactic

The simplification tactic applies a set of rewrite rules exhaustively. Given a set of axioms $\mathcal{A} = \{\ell_i \equiv r_i\}$:

1. Normalize both sides (β, δ reduction + constant folding).
2. For each axiom $\ell_i \equiv r_i$, try to match ℓ_i against subterms of the goal. If a match is found, rewrite.
3. Repeat until no more rewrites apply.
4. Check if normalized sides are equal.

The output is a RewriteChain proof term recording each rewrite step.

Remark E.4. Termination of simp is ensured by a maximum iteration count (100). Confluence is not guaranteed for arbitrary axiom sets, but the CCTT axioms are designed to be terminating and confluent on normal forms.

E.6 Tactic Combinators

Tactics compose via combinators:

Definition E.5 (Sequencing)**.** $\mathsf{then}(\tau_1, \tau_2)$: try τ_1; if it fails, try τ_2.

Definition E.6 (Alternative)**.** $\mathsf{orelse}(\tau_1, \ldots, \tau_n)$: try each tactic in order; return the first success.

Definition E.7 (Repetition)**.** $\mathsf{repeat}(\tau, max)$: apply τ repeatedly (up to max times) until it fails or the goal is solved.

These combinators always produce valid proof terms because each constituent tactic produces a valid term, and the combinators compose them via Trans or RewriteChain.

E.7 Integration with Z3

Z3 appears in the proof theory in two distinct roles:

1. **Decision procedure** (via Z3Discharge): For quantifier-free formulas in decidable fragments, Z3 provides a definitive YES/NO answer. This is a *proof*, not sampling.
2. **Tactic backend** (via omega and ring): Tactics use Z3 to discharge arithmetic subgoals. The resulting proof term records the Z3 call; re-verification repeats it.

In both cases, Z3 timeout means proof **rejection**. This is the critical design decision that distinguishes our approach from sampling-based methods.

E.8 What Can and Cannot Be Automated

Task	**Automated?**	**Tactic**
Constant arithmetic	Yes	omega
Polynomial equalities	Yes	ring
Syntactically identical programs	Yes	auto
Programs differing by commutativity	Yes	simp
Same algorithm, different variable names	Yes	refl (after normalization)
Recursive vs. iterative	Skeleton	induction
Different algorithms (same spec)	Skeleton	by_spec
State-space correspondence	Skeleton	by_simulation
Novel algorithmic equivalences	No	Human proof
Concurrent/distributed equivalences	No	Human proof
Amortized complexity arguments	No	Human proof

Remark E.8. The "Skeleton" entries mean the tactic can set up the proof structure (induction base/step, simulation init/step/output, etc.) but the user must fill in the key insights—the invariant, the simulation relation, or the abstraction function. This is inherent: these insights are the creative content of the proof.

E.9 Semi-Automated Proof Construction

The typical workflow for proving a non-trivial algorithmic equivalence:

1. **Choose strategy**: based on the structure of the two programs, select simulation, abstraction refinement, induction, etc.
2. **Identify the key insight**: what is the simulation relation? What is the abstract spec? What is the loop invariant?
3. **Use a strategy tactic** to set up the proof skeleton: `by_simulation(relation)`, `by_spec(spec_name)`, etc.
4. **Fill in sub-proofs**: for each subgoal, use auto, omega, ring, or write explicit proof terms.
5. **Verify**: run the checker on the complete proof. Fix any rejected sub-proofs.
6. **Serialize**: save the verified proof as a `.proof.json` file for permanent record.

This workflow mirrors the practice in interactive proof assistants (Lean, Coq, Isabelle) but is tailored to the domain of Python program equivalence.

E.10 LLM-Assisted Proof Construction

An exciting application: LLMs can assist in proof construction. Given two programs and a goal, an LLM can:

1. Suggest a proof strategy (simulation, abstraction refinement, etc.)
2. Identify the key relation or specification
3. Generate a proof term
4. The CCTT checker verifies the proof mechanically

The LLM output is *untrusted* until the checker verifies it. This is the anti-hallucination principle applied to proofs: the LLM's "proof" might be wrong, but the checker will catch any errors. The LLM is an oracle for proof *search*, not proof *verification*.

Remark E.9. This is a fundamental asymmetry: proof search is hard (creative), but proof verification is easy (mechanical). LLMs are good at the creative part; the checker handles the mechanical part.

Bibliography

[1] C. Cohen, T. Coquand, S. Huber, A. Mörtberg. *Cubical Type Theory: A Constructive Interpretation of the Univalence Axiom.* Journal of Automated Reasoning, 2018.

[2] T. Coquand, S. Huber, A. Mörtberg. *On Higher Inductive Types in Cubical Type Theory.* LICS 2018.

[3] The Univalent Foundations Program. *Homotopy Type Theory: Univalent Foundations of Mathematics.* Institute for Advanced Study, 2013.

[4] C. Angiuli, R. Harper, T. Wilson. *Computational Higher-Dimensional Type Theory.* POPL 2018.

[5] V. Voevodsky. *Univalent Foundations of Mathematics.* WoLLIC 2010.

[6] L. de Moura, N. Bjørner. *Z3: An Efficient SMT Solver.* TACAS 2008.

[7] J. G. Politz et al. *Python: The Full Monty.* OOPSLA 2013.

[8] A. Grothendieck. *Sur quelques points d'algèbre homologique.* Tôhoku Math. J., 1957.

[9] J. C. Reynolds. *Definitional Interpreters for Higher-Order Programming Languages.* ACM Annual Conference, 1972.

[10] A. Pnueli, M. Siegel, E. Singerman. *Translation Validation.* TACAS 1998.

[11] B. Godlin, O. Strichman. *Regression Verification.* DAC 2009.

[12] G. Barthe, J. M. Crespo, C. Kunz. *Relational Verification Using Product Programs.* FM 2011.

[13] P. Cousot, R. Cousot. *Abstract Interpretation.* POPL 1977.

[14] P. M. Rondon, M. Kawaguchi, R. Jhala. *Liquid Types.* PLDI 2008.

[15] P. L. Lumsdaine. *Higher Inductive Types: A Tour.* 2011.

[16] M. Shulman. *Brouwer's Fixed-Point Theorem in Real-Cohesive Homotopy Type Theory.* MSCS 2018.

[17] J. Curry. *Sheaves, Cosheaves and Applications.* PhD thesis, 2014.

[18] S. Abramsky. *Domain Theory in Logical Form.* PhD thesis, 1987.

[19] S. Mac Lane. *Categories for the Working Mathematician.* Springer, 1971.

[20] E. Riehl. *Category Theory in Context.* Dover, 2017.

[21] R. Hartshorne. *Algebraic Geometry.* Springer, 1977.